

Reinforcement Learning

Foundations, Algorithms, and the Research Frontier

Intuition. Definitions. Derivations. Proofs.

From finite-state control to modern learning systems,
with checkable examples and a dated research seminar.

4 September 2026

Editable lecture notes, worked mathematics and reproducible laboratories.

Contents

Course guide: from Bellman equations to research	1
What you should learn to do	1
Prerequisites and a diagnostic	1
The central example	1
Reading and proof dependencies	2
Full teaching schedule: 20 weeks, two meetings per week	2
A ten-week core alternative	3
How to study a lecture	3
Assessments and feedback	3
Notation and terminal conventions	4
Figures, evidence, and versions	4
Part I — Foundations	5
Lecture opening: A decision now, a distribution later	5
Learning objectives	5
1. Core Concepts and Problem Formulation	5
2. Markov Decision Processes	9
3. Returns, Value Functions, and Bellman Equations	14
3.5 Matrix Formulation	18
Common misconceptions	20
Chapter summary	20
Knowledge check and exercises	21
Part II — Tabular Methods	22
Lecture opening: Three ways to obtain the same backup	22
Learning objectives	22
4. Dynamic Programming	22
5. Monte Carlo Methods	27
6. Temporal-Difference Learning	29
Common misconceptions	34
Chapter summary	35
Knowledge check and exercises	35
Part III — Function Approximation	37
Lecture opening: Generalization changes the geometry	37
Learning objectives	37
7. Value Function Approximation	37
8. Deep Q-Networks and Extensions	44
Common misconceptions	49
Chapter summary	49
Knowledge check and exercises	50
Part IV — Policy-Based Methods	52
Lecture opening: Move probability mass toward useful actions	52
Learning objectives	52
9. Policy-Gradient Methods	53
10. Advanced Policy Optimization	61
Common misconceptions	71
Chapter summary	72

Knowledge check	72
Exercises	72
Primary readings	73
Part V — Model-Based Reinforcement Learning	74
Lecture opening: A planner can exploit its own simulator	74
Learning objectives	74
11. Learning and Using World Models	74
Dyna: integrate learning, acting, and planning	78
Model-Predictive Control	80
Probabilistic ensembles and PETS-style planning	82
Short model rollouts and model-based value expansion	83
Policy-induced model error	83
World Models	84
MuZero	86
Dreamer	87
Choosing a model-based strategy	88
A model-validation protocol for control	89
Common misconceptions	90
Chapter summary	90
Knowledge check	90
Exercises	91
Primary readings	92
Part VI — Advanced Topics	93
Lecture opening: Change one assumption before changing the algorithm	93
Learning objectives and scope	93
12. Exploration Strategies	93
13. Multi-Agent Reinforcement Learning	98
14. Hierarchical Reinforcement Learning	102
15. Offline or Batch Reinforcement Learning	105
16. Reinforcement Learning from Human Feedback	109
17. Inverse RL and Imitation Learning	111
Cross-cutting themes	114
Common misconceptions	115
Chapter summary	116
Knowledge check	116
Research exercises	116
Primary readings	117
Part VII — Practical and Mathematical Foundations	119
Lecture opening: An experiment is part of the mathematical claim	119
Learning objectives	119
18. Mathematical Toolkit	119
18.1 Random variables and expectation	119
18.2 Filtrations, conditional noise, and stochastic approximation	121
18.3 Importance sampling	124
18.4 Entropy, cross-entropy, and KL divergence	126
18.5 Contractions and fixed points	129
18.6 Basic statistical intervals for RL evaluation	131
19. Implementation and Debugging	132
19.1 Treat the environment interface as a contract	132
19.2 Build tests before full training	134
19.3 Logging as a diagnostic instrument	135

19.4 A symptom-to-test debugging table	136
19.5 Reward design and reward auditing	137
19.6 Experimental design	138
19.7 Reproducibility record	139
19.8 Libraries and implementation strategy	140
19.9 Compute and systems considerations	141
19.10 A complete preflight checklist	141
Common misconceptions	142
Chapter summary	143
Knowledge check	143
Exercises	143
Primary readings	144
Part VIII — Reinforcement Learning in Large Language Models	146
Lecture opening: The action can be a token or a whole response	146
Learning objectives	146
20. The RLHF Pipeline	146
21. Preference Optimization Without an Explicit RL Loop	153
22. Group-Relative Policy Optimization and RL for Reasoning	157
23. Process Reward Models and Outcome Reward Models	162
24. Self-Improvement and Iterative Training	165
25. Search for Language-Model Reasoning	167
26. Reinforcement Learning from Execution Feedback	170
27. Multi-Turn RL and Agentic Behavior	171
28. Scaling, Self-Play, and Advanced Preference Objectives	173
29. Reward Shaping for Language-Model Training	175
A unified view of LLM post-training	177
Common misconceptions	178
Key equations	179
Knowledge check	179
Exercises	179
Primary readings	180
Part IX — Statistical Corner Cases in Reinforcement Learning	182
Lecture opening: Write the estimand before the estimate	182
Learning objectives	182
Overview: RL is estimation inside a feedback loop	182
30. Why RL Statistics Are Hard	183
31. Finite Samples, Correlation, and Stale Data	185
32. Bootstrapping Bias and the Critic	188
33. Off-Policy Evaluation and Importance-Sampling Pathologies	191
34. Variance Reduction in Policy Gradients	194
35. GAE: A Bias-Variance Knob, Not Magic	196
36. Entropy, KL, and Support Mismatch	197
37. Nonstationarity and Distribution Shift	199
38. Seed Sensitivity and Evaluation Uncertainty	201
39. Reward Hacking, Goodhart Effects, and Policy-Optimization Failures	203
40. Diagnostics Checklist	205
41. Intuition Summary	206
Knowledge check	207
Exercises	208
Primary readings	209
Part X — Mitigating Statistical Problems in Reinforcement Learning	210

Lecture opening: Fix the mechanism, then test the consequence	210
Learning objectives	210
Overview: every mitigation moves error	210
42. A Mitigation Taxonomy	211
43. Finite-Sample Noise and Correlated Trajectories	214
44. Critic Error and Bootstrapping Bias	216
45. Off-Policy Evaluation and Support Mismatch	219
46. Policy-Gradient Variance Reduction	223
47. GAE, Retrace, and V-Trace	224
48. Trust Regions, Entropy, and PPO Stabilization	227
49. Replay, Nonstationarity, and Distribution Shift	230
50. Reliable Evaluation and Statistical Reporting	232
51. Reward Hacking and Goodhart Mitigations	234
52. Practical Mitigation Protocol	237
Before training	237
During training	237
After training	238
53. Open Problems	239
54. Intuition Summary	240
Knowledge check	241
Exercises	241
Primary readings	243
Classical extensions: exploration, memory, continuing tasks, and constraints	245
Why this chapter belongs before the research frontier	245
B.1 Finite-horizon control: time is part of the state	245
B.2 Bandits: paying for information	246
B.3 Partial observation: derive the information state	247
B.4 Average reward: value up to a constant	248
B.5 Constraints and occupancy flows	249
B.6 A minimax interlude: why single-agent improvement can fail in games	249
Transfer exercise	250
Part XI — Recent progress and research questions	251
Evidence boundary: 4 September 2026	251
1. The frontier is larger than language-model post-training	251
2. Algorithm discovery: generalization over learning problems	252
3. World models: learn what the planner will need	252
4. Scaling Q-functions without confusing optimization with validity	253
5. Continuing-task exploration: leading rates are not the whole story	253
6. Safe adaptation: what is the safety event?	254
7. Representation learning: sufficient for which tasks?	254
8. Missing rewards: estimation begins with identification	254
9. Off-policy language-model RL: reuse versus mismatch	255
10. Agent execution rewards: many valid paths, one underspecified score	255
11. Categorical critics: a new loss need not imply a new actor	256
12. From “open problem” to a research contract	256
Mathematical spine: statements, proofs, and limits	258
A.0 Objects and standing conventions	258
A.1 Bellman equations are conditional-expectation identities	258
A.2 Contraction, optimality, and an executable stopping certificate	259
A.3 Policy improvement without a hidden infinite remainder	259
A.4 A running MDP that can be solved three ways	260

A.5 Monte Carlo consistency with repeated visits	261
A.6 Stochastic contraction: a complete tabular convergence proof	261
A.7 Linear approximation: projection, error, and sampled convergence	263
A.8 The double-sampling problem, derived	265
A.9 Policy gradients and the baseline condition	266
A.10 Finite-rollout GAE and boundary conditions	266
A.11 Performance difference and the actual trust-region bound	267
A.12 Model error and planning error are distinct	268
A.13 Entropy regularization and the DPO algebra	269
A.14 Group-relative estimators: what centering actually does	269
A.15 Importance sampling, doubly robust evaluation, and impossibility	270
A.16 Retrace and V-trace: fixed points, not just formulas	271
A.17 Three useful counterexamples to overgeneralization	272
A.18 Shaping, options, and return-distribution contraction	273
A.19 What a mathematical result licenses	273
Notation and assumption guide	275
Core notation	275
Assumption checklist	275
Terminology	276
Six problem sets	277
Set 1 — Bellman reasoning and certificates	277
Set 2 — Sampling, convergence, and representation	277
Set 3 — Gradients and rollout boundaries	277
Set 4 — Models, support, and evaluation	278
Set 5 — Classical extensions and counterexamples	278
Set 6 — Preference optimization and a research contract	278
Eight laboratories	279
Lab 1 — Exact control and certificates	279
Lab 2 — Sampling the tabular fixed point	279
Lab 3 — Geometry and a counterexample	279
Lab 4 — Gradients and GAE boundaries	279
Lab 5 — Group-relative estimators	280
Lab 6 — Doubly robust evaluation	280
Lab 7 — V-trace’s clipped policy	280
Lab 8 — UCB and finite-budget accounting	280
Verified reference anchors and evidence notes	281
Foundations and methods	281
Selected 2025–2026 developments	282
Reading practice	283

Course guide: from Bellman equations to research

What you should learn to do

Reinforcement learning studies how to choose actions when those actions change both future outcomes and future information. A graduate course should teach more than recognizing algorithm names. By its end, you should be able to formulate a sequential decision problem, derive an estimator for its objective, identify the assumptions that make the estimator valid, implement a small version whose answer you can check, and assess a research claim without confusing an empirical result with a theorem.

We will use a recurring sequence: **pose a decision; predict qualitatively; define the objects; derive the equation; prove the claim; calculate a small example; stress an assumption.** Intuition is the prediction that makes the proof worth doing. A proof is the argument that tells us when the intuition is reliable.

The course retains the original Parts I–X as its main lecture notes, substantially revises mathematical and implementation details, replaces Part XI with a dated research seminar, and adds a proof spine, classical extensions, worked assignments, and executable laboratories. Read the proofs with the lectures. They are collected for navigation, not relegated to optional fine print.

Prerequisites and a diagnostic

No prior reinforcement learning is required. You should be comfortable with finite-dimensional linear algebra, multivariable calculus, conditional expectation, elementary probability limits, and basic optimization. The stronger stochastic-convergence sections explicitly use the strong law and nonnegative-supermartingale convergence theorem as probability prerequisites. They do not pretend that a self-contained RL course also derives the whole of measure-theoretic probability.

Before Lecture 1, verify the following. Explain why a stochastic matrix does not increase the sup norm. Differentiate a softmax log probability. Use the tower property to condition an expectation twice. Distinguish a sample mean from its expectation and identify what independence is needed for its variance formula. Solve a two-by-two linear system. If these operations are unfamiliar, begin with Part VII's probability and optimization sections and the diagnostic solutions in the instructor companion.

For coding, basic Python and arrays are sufficient. The core labs require NumPy; Matplotlib is needed only to regenerate figures. No GPU, API account, simulator installation, or language-model checkpoint is needed for the supplied numerical checks. Large-scale deep-RL and LLM experiments remain optional projects, not hidden prerequisites for finishing the course.

The central example

In the two-state MDP in A.4, staying in state 0 gives reward 1; switching gives zero and moves to state 1; staying in state 1 gives reward 2; switching back gives zero. Discount is 0.9. Start by predicting when switching is worthwhile, then compute the answer. We revisit this same environment for Bellman evaluation, policy improvement, value iteration, policy gradients, model error, and off-policy evaluation. This continuity makes differences between algorithms visible without changing the task every time.

The analytic optimal value is $(18, 20)$, but the gradient at a uniformly random policy does not simply point straight at the optimal deterministic actions. Continuation behavior matters. That observation

will eventually connect a tiny MDP to policy optimization in much larger systems.

Reading and proof dependencies

Lecture topic	Main reading	Proof or worked bridge
MDPs, returns, Bellman equations	Part I	A.0–A.4; finite horizon B.1
DP, MC, TD, control	Part II	A.2–A.6
Approximation and deep Q-learning	Part III	A.7–A.8; distributional result A.18
Policy gradients, actor-critic, PPO	Part IV	A.9–A.11
Models, planning, world models	Part V	A.12
Exploration, hierarchy, offline RL	Part VI	B.2; A.15, A.17–A.18
POMDPs, continuing tasks, constraints	Classical extensions	B.3–B.6
Implementation and experimental reasoning	Part VII	All laboratory checks
RLHF, DPO, GRPO, agents	Part VIII	A.13–A.14
Statistical failure modes	Part IX	A.10, A.14–A.15
OPE and stabilization	Part X	A.15–A.16
Recent progress and research design	Part XI	Selected references R01–R11

The strongest prerequisite chain is Bellman conditional expectation $\rightarrow$ contraction $\rightarrow$ policy improvement $\rightarrow$ sampled contraction $\rightarrow$ approximation geometry $\rightarrow$ policy gradient $\rightarrow$ performance difference. Offline evaluation branches from the change-of-measure identity. KL preference optimization branches from a finite-dimensional variational identity. Not every modern method requires reading every earlier architecture survey.

Full teaching schedule: 20 weeks, two meetings per week

The package is sized for a two-term graduate sequence or sustained independent study. Forty meetings allow both proofs and discussion; the schedule is a suggested pacing plan, not a fixed claim about every student's workload.

Week	Meeting A	Meeting B	Practice
1	Decision processes and returns	Bellman expectation and exact evaluation	Diagnostic; Lab 1
2	Contraction and optimality	Policy iteration and stopping certificates	Assignment 1
3	Monte Carlo and dependence	TD noise and stochastic convergence	Lab 2
4	Q-learning and on-policy control	Finite-horizon backward induction	Assignment 2
5	Linear features and projections	Off-policy divergence and double sampling	Lab 3
6	DQN, replay, target networks	Distributional critics and architecture limits	Part III discussion
7	Policy gradient from trajectories	Baselines and exact gradient checks	Lab 4
8	Actor-critic and GAE	Performance difference, TRPO, PPO	Assignment 3
9	Continuous-action actor-critic	Entropy-regularized control	A.13; Part IV

Week	Meeting A	Meeting B	Practice
10	Model error and Dyna	MPC and latent world models	Assignment 4
11	Concentration and UCB	Deep exploration and intrinsic rewards	Lab 8
12	Belief states and memory	Options, imitation, and games	B.3, B.6; A.17
13	Average reward and bias	Occupancy flows and constraints	Assignment 5
14	Offline learning and support	IS, DR, and evaluation uncertainty	Lab 6
15	Retrace and V-trace	Statistical diagnosis and mitigations	Lab 7
16	RLHF and KL objectives	DPO and preference-model assumptions	A.13
17	GRPO and verifiable rewards	Process feedback and tool-use trajectories	Lab 5; Assignment 6
18	Research in control and exploration	Research in representation and world models	Project contracts
19	Research in OPE and LLM learning	Replication and falsification workshops	Project draft
20	Student research presentations	Oral synthesis and proof repair	Final project

A ten-week core alternative

For a single quarter, use weeks 1–8 above as the foundation, compress the architecture surveys, and use the final two weeks for model-based/offline RL and one selected modern seminar. Assign the longer LLM, multi-agent, and continuing-control material as extensions. Do not advertise the compressed route as covering the entire package at equal depth. The complete notes remain available for subsequent study.

How to study a lecture

First read the opening problem without looking at the equations. Write a prediction and name the information the agent has. During the derivation, maintain a symbol table and mark each use of a Markov assumption, support condition, or conditional expectation. After the proof, calculate the running example by hand before running code. Finally change one assumption and locate the first line of the proof that fails.

A productive confusion is specific: “the next-state bootstrap depends on the selected action, so why should it behave like a baseline?” That question leads to A.10. An unproductive shortcut is “actor-critic reduces variance,” which does not tell us the objective, bias, or boundary convention. We will favor the first kind of question throughout.

Assessments and feedback

A suggested assessment mix is 30% proof-and-counterexample assignments, 25% laboratories and reproducibility reports, 15% oral or written synthesis, and 30% a research project. The six new assignment sets have worked solutions in the instructor companion. Original chapter knowledge checks and open-ended research exercises are retained as additional practice; they do not all have

new worked solutions.

A proof submission should include a complete statement, definitions, assumptions, the argument, and a sentence explaining a boundary case. A laboratory report should state the estimand, randomization unit, code version, exact checks passed, and what the observed result does *not* establish. A project should include a closest-work comparison and a predeclared falsifier. Negative results and counterexamples can be successful projects when their scope is precise.

Notation and terminal conventions

The proof spine standardizes $S_t, A_t, R_{t+1}, v_\pi, q_\pi, A_\pi$, initial distribution ν (also called ρ_0 in the original chapters), and normalized occupancy $d_\pi = (1 - \gamma) \sum_t \gamma^t \Pr(S_t = \cdot)$. Some original sections use V, Q for the same value objects or their estimates; context and hats distinguish exact from learned quantities. A complete symbol table appears in A.0 and the notation guide.

An indicator called `done` is too ambiguous for a mathematical argument. Distinguish true task termination from an external rollout cutoff and from the next stored transition belonging to a reset episode. A.10 and the code use separate bootstrap and trace-continuation masks. This convention applies equally to games, robots, and maximum-length language generation.

Figures, evidence, and versions

Existing illustrations are retained where helpful as explanatory schematics. They should not be read as measured benchmark evidence; the misleading SARSA/Q-learning curve reference and an unverified overoptimization fit attribution have been removed or corrected. Three new mathematical figures are generated from exact computations: Bellman error and its bound, the mean linear-TD counterexample, and mixed-group reward probabilities. Their code is included.

Part XI is a selected literature snapshot checked on **4 September 2026**, including public records through August 2026. Publication status, author-reported claims, and proposed research directions are separated in the references. The source ZIP also contained a separately authored published-book PDF; it was not a compiled version of these chapters and has not been presented as the revised course. The original Markdown is preserved for comparison in the full source package.

The numerical tests validate particular identities and implementation cases. They are not a machine-checked proof of the book or an independent reproduction of large-scale papers. This boundary is part of the course's standard of evidence.

Part I — Foundations

Lecture opening: A decision now, a distribution later

Begin with the two-state environment in A.4. Staying in state 0 pays immediately; switching pays nothing now but reaches a potentially more valuable state. Before doing any algebra, ask: valuable under *which continuation policy*? That question is why v_π , q_π , and v_* must be different objects. The Bellman equation is simply the law of total expectation arranged to expose that continuation. Work through A.1–A.4 alongside this chapter. Afterward, repeat the calculation with a three-step horizon in B.1; this reveals exactly why time sometimes belongs in the state.

Checkpoint. Can a deterministic environment still generate random returns? Yes: a stochastic policy randomizes the trajectory. Conversely, replacing random rewards with their means preserves expected value for a fixed MDP but does not preserve the return distribution or the data needed for uncertainty analysis.

Learning objectives

By the end of this chapter, you should be able to:

- identify the agent, environment, state, observation, action, reward, and horizon in a decision problem;
- explain the Markov property as a requirement on the state representation rather than a magical property of the world;
- define an MDP and the trajectory distribution induced by a policy;
- derive the Bellman expectation and Bellman optimality equations;
- solve a small policy-evaluation problem by algebra and by a linear system;
- distinguish a mathematical theorem from a modeling choice such as reward design or discounting.

Standing assumptions

Unless a section says otherwise, assume a finite MDP, bounded rewards, and a discount factor $0 \leq \gamma < 1$. These assumptions make the main sums finite and the Bellman operators contractive. Episodic problems with $\gamma = 1$ require a separate termination argument.

1. Core Concepts and Problem Formulation

Intuition first: RL is learning under consequences

Supervised learning asks a model to predict a target label. Reinforcement learning asks an agent to choose actions whose consequences may be delayed, stochastic, and partly caused by earlier decisions.

A chess move has no immediate label saying “correct.” Its quality depends on how the position develops. A robot may need to tolerate a temporary loss of balance to clear an obstacle. A recommendation system may trade an immediate click for long-term user satisfaction. This coupling across time is the defining difficulty.

Flow to keep in mind. The interaction loop is state or observation $\rightarrow$ action $\rightarrow$ environment $\rightarrow$ reward and next state $\rightarrow$ the next decision. The agent chooses actions; the environment produces the transition and reward.

At time t :

1. the agent receives a state or observation;
2. it samples an action from its policy;
3. the environment produces a reward and next state;
4. the interaction repeats.

Formal interaction protocol

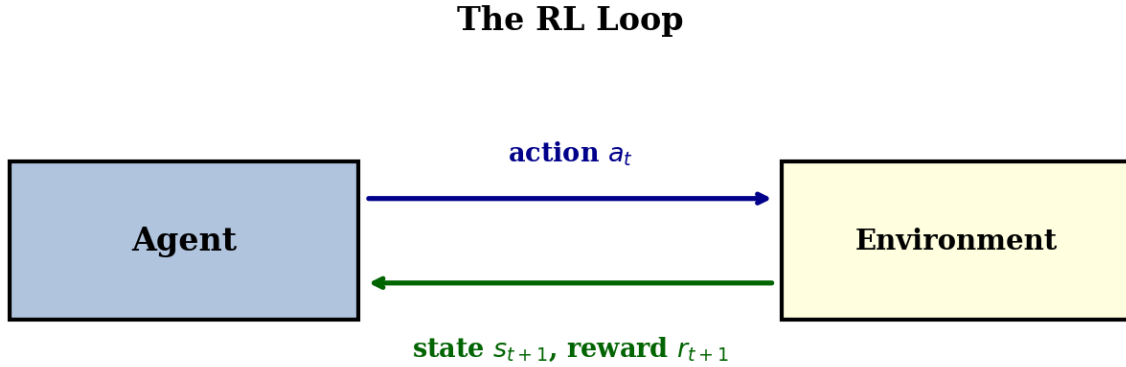


Figure 1: The reinforcement learning agent-environment interaction loop

Let

- $S_t \in \mathcal{S}$ be the state;
- $A_t \in \mathcal{A}(S_t)$ be the action;
- $R_{t+1} \in \mathbb{R}$ be the reward observed after the action;
- $\pi(a | s)$ be the policy.

A trajectory is

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots).$$

The policy may be stochastic:

$$A_t \sim \pi(\cdot | S_t).$$

The environment then samples

$$(S_{t+1}, R_{t+1}) \sim P(\cdot, \cdot | S_t, A_t).$$

The agent's objective is not to maximize each reward separately. It is to maximize a cumulative measure of future reward.

Episodic and continuing tasks

An **episodic** task has a terminal time T :

- a game ends;
- a robot reaches a terminal condition;

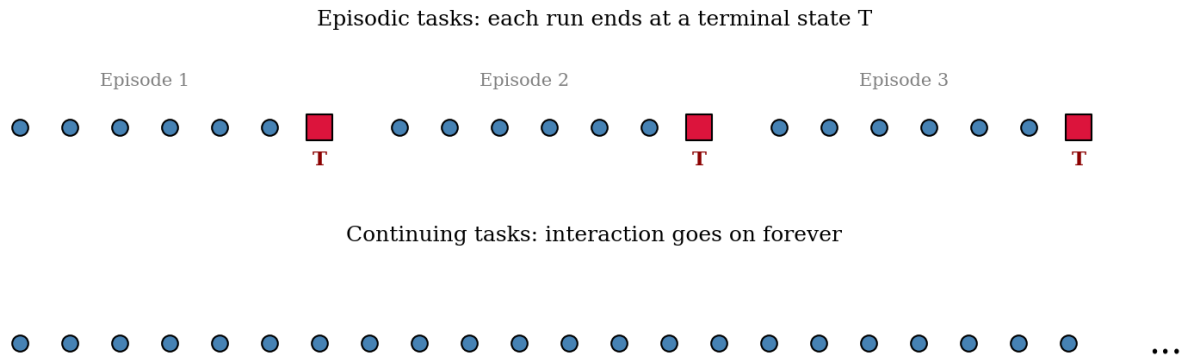


Figure 2: Comparison of episodic and continuing reinforcement learning tasks

- a dialogue or generated response finishes.

A **continuing** task has no natural terminal time:

- process control;
- traffic routing;
- a server allocation policy.

The distinction changes the mathematical objective. For a finite episode, the return is a finite sum. For an infinite-horizon task, discounting or an average-reward formulation is usually needed.

Reward is the objective interface

The reward is a scalar signal supplied by the environment or designer. It is not a moral judgment and it is not automatically the true goal.

A common formulation is the **reward hypothesis**:

A goal can be represented as maximizing expected cumulative reward.

This is a modeling hypothesis. It may be useful, but it can fail operationally when the scalar reward omits important dimensions or admits loopholes.

For example, a navigation reward such as

$$r_t = -\text{distance}(S_t, \text{goal})$$

encourages proximity but does not, by itself, specify collision avoidance, energy use, smoothness, or whether the episode should terminate. Reward design is therefore part of the problem definition, not a cosmetic detail.

Important. An RL algorithm optimizes the reward it receives. It does not infer the designer's unstated intent.

Exploration versus exploitation

The agent must choose between:

- **exploitation:** take the action currently believed to be best;
- **exploration:** gather information that may reveal a better action.

In a K -armed bandit, action a has unknown mean reward μ_a . If the agent always chooses the empirically best arm too early, it may permanently commit to a suboptimal arm. If it explores forever, it sacrifices reward unnecessarily.

Bandits isolate exploration because actions do not change a state. MDPs add state transitions, delayed effects, and long-horizon credit assignment.

Bandit, contextual bandit, and MDP

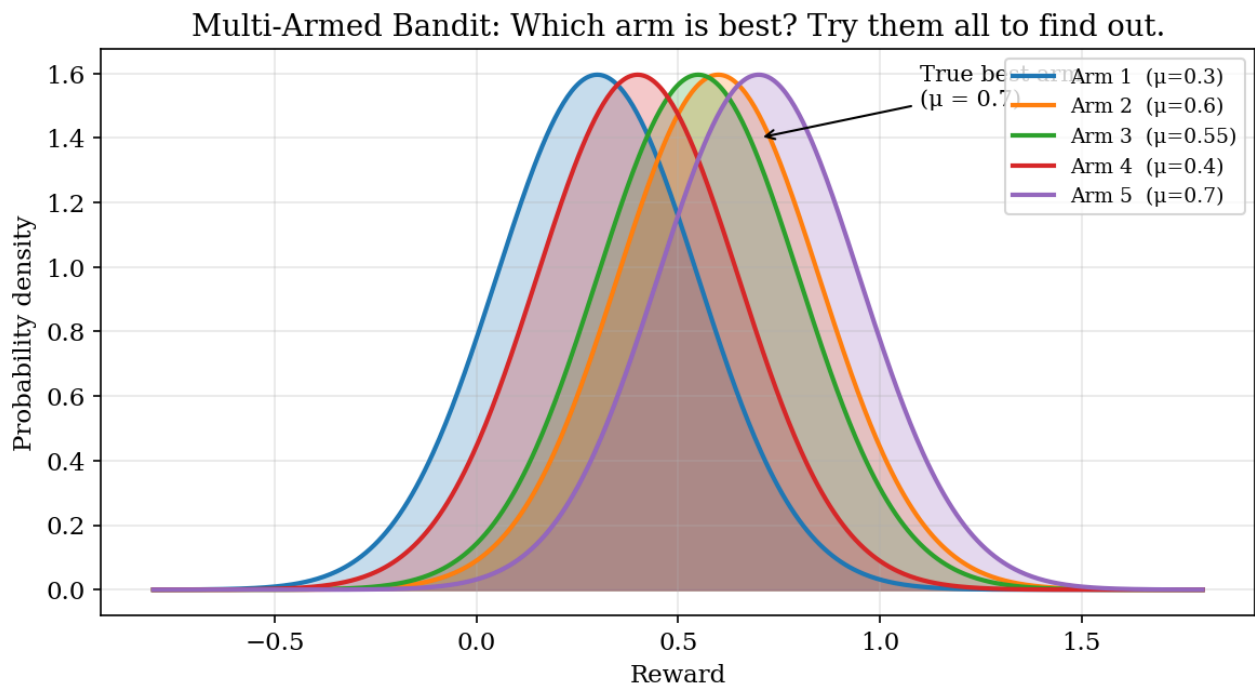


Figure 3: Multi-armed bandit actions and reward distributions

Problem	State evolves because of actions?	Delayed consequences?	Typical example
Multi-armed bandit	No	No	Repeatedly choose one advertisement
Contextual bandit	Context changes externally	No	Choose an ad given the current user context
MDP	Yes	Yes	Navigation, robotics, games

Modeling examples

Domain	State	Action	Reward signal	Hidden modeling question
Chess	Full legal game state	Legal move	Terminal win/loss	How should draws and time be valued?
Robot navigation	Pose, velocity, map estimate	Motor command	Progress and task completion	How are collisions and energy penalized?
Recommender	User history and context	Item to display	Engagement or long-term utility proxy	Does the reward induce unhealthy behavior?
Thermostat	Indoor/outdoor state	Heat, cool, off	Comfort minus energy cost	How are comfort and cost traded off?

Conceptual checkpoint

A complete RL problem statement should answer:

1. What information is in the state?
 2. What actions are available?
 3. What stochastic mechanism produces the next state and reward?
 4. What cumulative objective is being optimized?
 5. Which parts of the goal are not represented by the reward?
-

2. Markov Decision Processes

Intuition first: the state is a sufficient summary

The Markov property says that the present state contains all information from the past that is needed to predict the next reward and state, once the current action is known.

This does **not** mean that the physical world has no memory. It means that our chosen state representation must summarize the relevant memory.

A diagram of chess pieces is not fully Markov because castling rights, en-passant availability, the half-move clock, and repetition history can matter. An augmented chess state containing those variables can be Markov.

Formal definition

A discounted Markov decision process can be written as

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0),$$

where:

- $\mathcal{S}$ is the state space;
- $\mathcal{A}(s)$ is the action set available in state s ;
- $P(s' | s, a)$ is the transition kernel;
- $r(s, a)$ is the expected one-step reward;
- $\gamma \in [0, 1)$ is the discount factor;
- ρ_0 is the initial-state distribution.

A more primitive description uses the joint next-state/reward kernel

$$P(s', r | s, a) = \Pr(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a).$$

From it,

$$P(s' | s, a) = \sum_r P(s', r | s, a),$$

and

$$r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a] = \sum_{s', r} r P(s', r | s, a).$$

When $P(s' | s, a) > 0$, the conditional expected reward given the next state is

$$r(s, a, s') = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] = \frac{\sum_r r P(s', r \mid s, a)}{P(s' \mid s, a)}.$$

The formula is undefined when the conditioning event has probability zero; that transition never contributes to an expectation.

The Markov property

Let the complete interaction history before time t be

$$H_t = (S_0, A_0, R_1, \dots, S_t).$$

The state process is Markov if

$$\Pr(S_{t+1}, R_{t+1} \mid H_t, A_t) = \Pr(S_{t+1}, R_{t+1} \mid S_t, A_t).$$

Equivalently,

$$(S_{t+1}, R_{t+1}) \perp H_{t-1} \mid (S_t, A_t).$$

The equality is a statement about the representation S_t . A non-Markov observation can often be made Markov by including more history, a recurrent memory state, or a belief distribution.

Policies

A stochastic policy is a conditional distribution

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s).$$

A deterministic policy μ is a function

$$A_t = \mu(S_t).$$

A policy is **stationary** when the same mapping is used at every time step. Finite-horizon problems can require time-dependent policies $\pi_t(a \mid s)$; time can be included in the state to recover a stationary representation.

The trajectory distribution

Under a fixed policy π , a finite trajectory of length T has probability

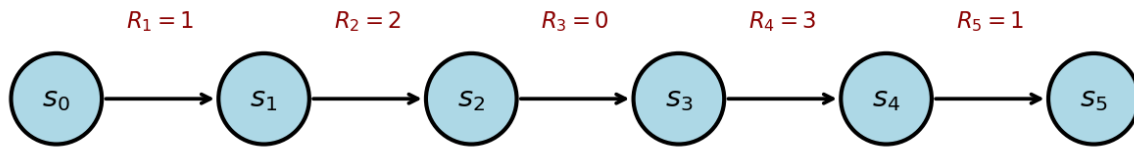
$$p_\pi(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi(a_t \mid s_t) P(s_{t+1}, r_{t+1} \mid s_t, a_t).$$

This factorization is central. It shows exactly where policy parameters enter the data distribution and later enables policy-gradient derivations.

Discounting

For bounded rewards and $\gamma < 1$, the discounted infinite return is finite:

$$\left| \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \right| \leq \frac{R_{\max}}{1 - \gamma}.$$

Trajectory and Discounted Return ($\gamma = 0.9$)

$$G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \gamma^4 R_5$$

$$= 1 + (0.9)(2) + (0.9)^2(0) + (0.9)^3(3) + (0.9)^4(1) = 5.643$$

Figure 4: A reinforcement learning trajectory through states, actions, and rewards

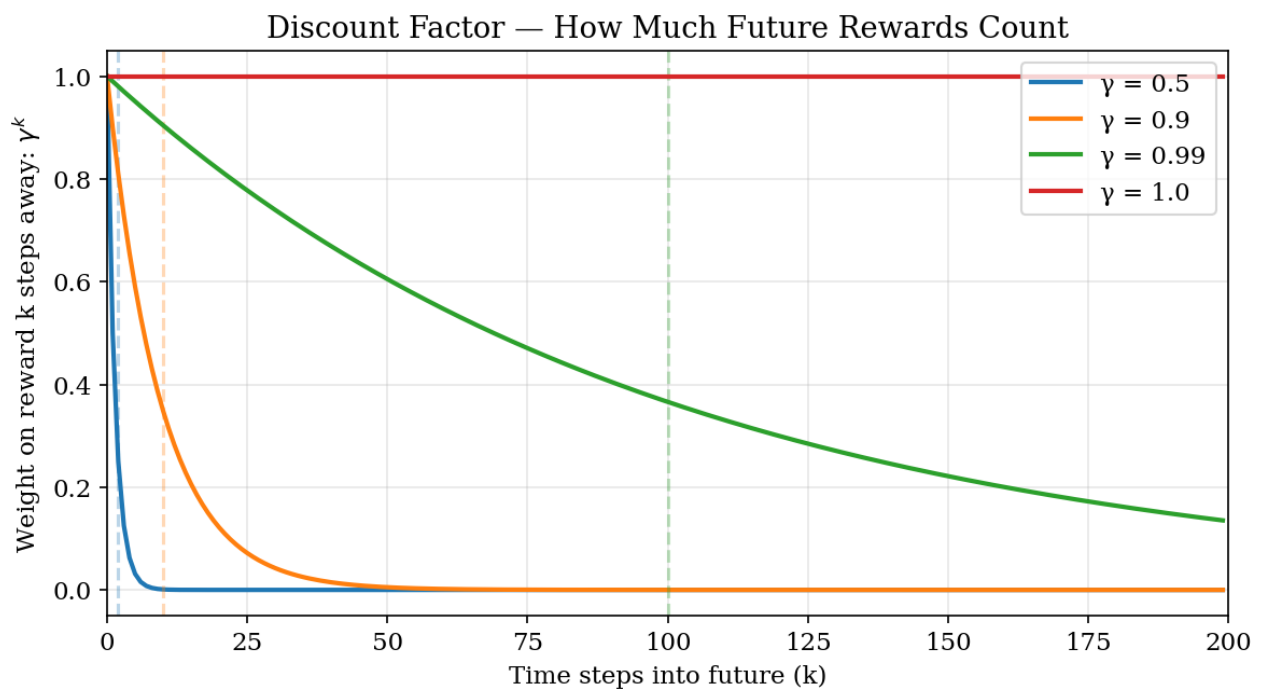


Figure 5: How the discount factor changes the weight placed on future rewards

Discounting has two roles:

1. **mathematical**: it ensures convergence of the return and contraction of Bellman operators;
2. **modeling**: it defines how the objective values temporally distant rewards.

A useful rough scale is the effective horizon

$$H_{\text{eff}} \approx \frac{1}{1 - \gamma}.$$

This is not a hard cutoff. It indicates where geometric weights become small.

γ	$1/(1 - \gamma)$	Weight after 100 steps
0.90	10	$0.90^{100} \approx 2.7 \times 10^{-5}$
0.99	100	$0.99^{100} \approx 0.366$
0.999	1000	$0.999^{100} \approx 0.905$

Note. $\gamma = 1$ is perfectly coherent for finite-horizon tasks and for some proper episodic problems. What fails is the simple discounted contraction proof, not the entire theory of RL.

Partial observability

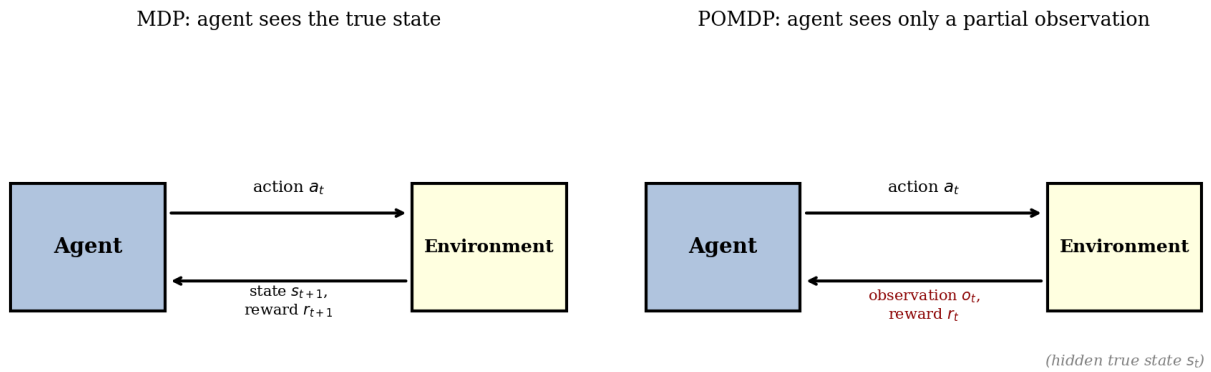


Figure 6: Comparison of fully observed MDPs and partially observed decision processes

In a partially observable MDP, the environment has a latent state S_t but the agent receives an observation O_t :

$$O_t \sim \Omega(\cdot \mid S_t).$$

A single observation may not be Markov. The ideal sufficient statistic is the belief state

$$b_t(s) = \Pr(S_t = s \mid O_{0:t}, A_{0:t-1}).$$

The belief state evolves Markovianly under Bayesian filtering, but it is generally continuous and difficult to compute exactly. Recurrent neural policies approximate history-dependent decision rules without explicitly constructing the exact belief.

Worked modeling example: stochastic gridworld

Consider a grid with one goal, one trap, and one wall.

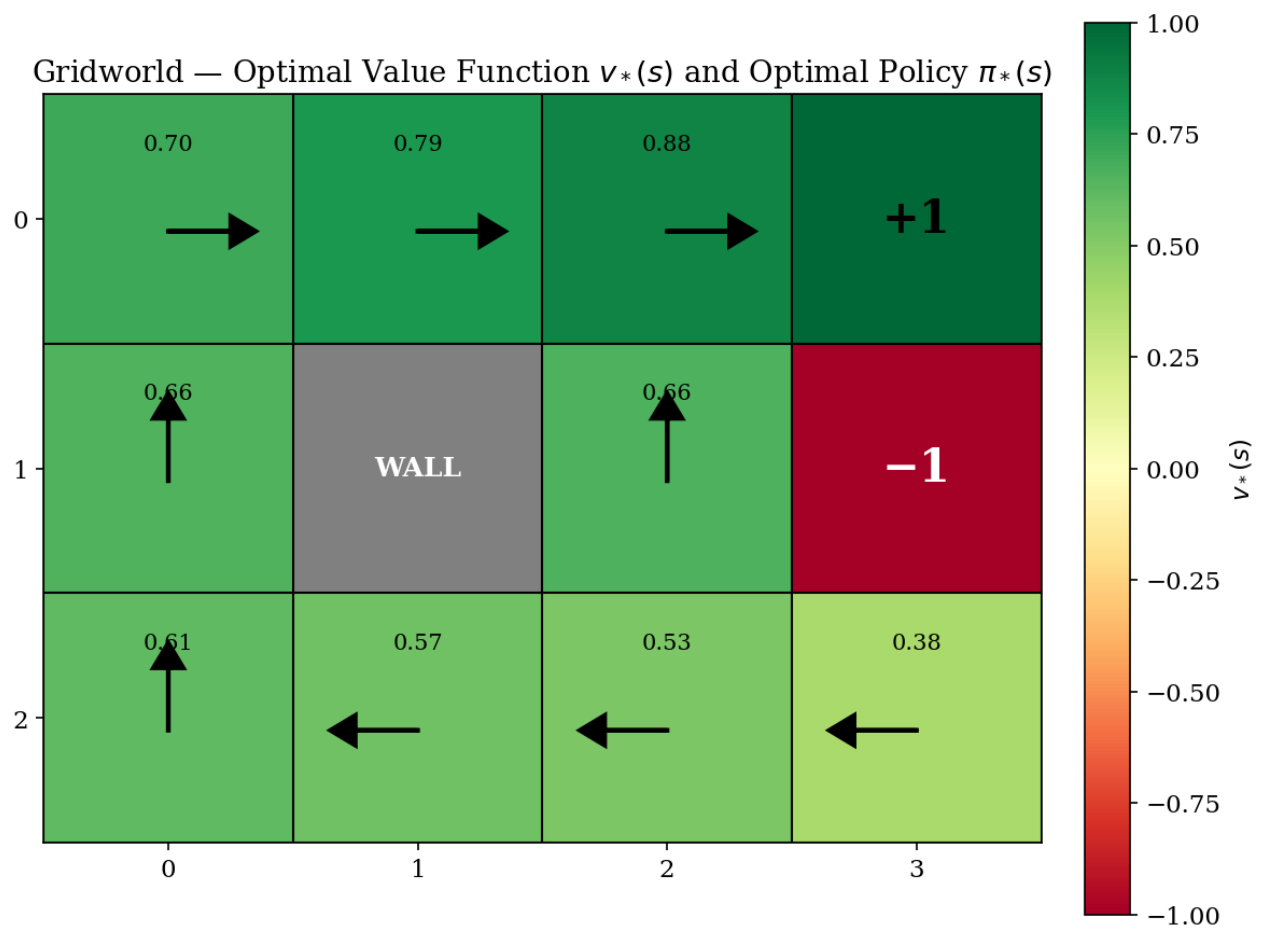


Figure 7: A stochastic gridworld Markov decision process

```

+---+---+---+---+
| 5 |   |   | +1|
+---+---+---+---+
|   | X |   | -1|
+---+---+---+---+
|   |   |   |   |
+---+---+---+---+

```

A reasonable MDP specification is:

- state: current non-wall cell;
- actions: up, down, left, right;
- transition: intended direction with probability 0.8, perpendicular directions with probability 0.1 each;
- reward: +1 at the goal, −1 at the trap, and −0.04 otherwise;
- terminal states: goal and trap;
- discount: $\gamma = 0.9$.

The step cost is not a theorem. It is a design choice that encourages shorter paths. Changing it may change the optimal policy.

3. Returns, Value Functions, and Bellman Equations

Intuition first: value compresses the future

A return is the realized cumulative reward along one trajectory. A value function is the expected return before the future randomness is known.

The Bellman idea is to split the future into two pieces:

1. the reward from the next transition;
2. the remaining value after that transition.

This one-step decomposition turns an exponentially branching future into a recursive equation.

Return

For an infinite-horizon discounted task,

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

For an episode ending at time T ,

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

The key recursion is

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

State and action values

The state-value function under policy π is

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

The action-value function is

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

They are related by

$$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a),$$

and

$$q_\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) v_\pi(s').$$

The first equation averages over the policy's next action. The second averages over the environment's next state.

Bellman expectation equation

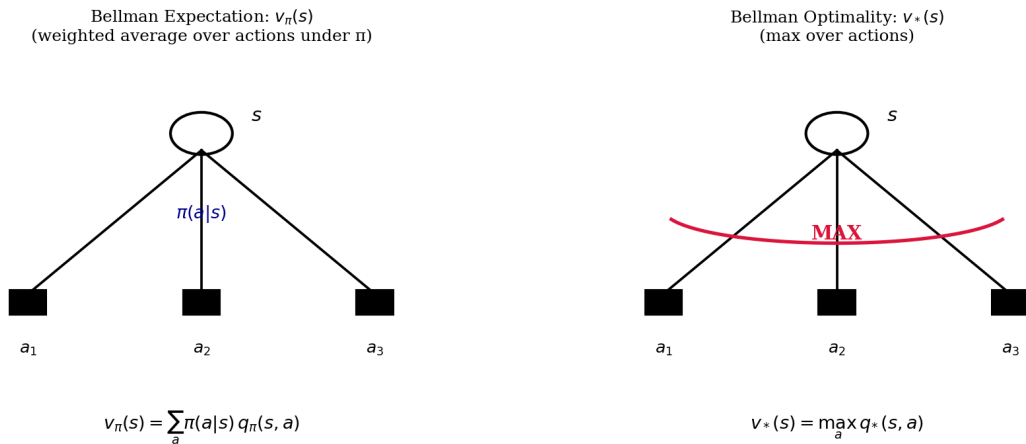


Figure 8: Comparison of Bellman expectation and optimality backups

Starting from the return recursion,

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s]. \end{aligned}$$

Condition first on the action and then on the next state:

$$\begin{aligned} v_\pi(s) &= \sum_a \pi(a \mid s) \sum_{s', r} P(s', r \mid s, a) [r + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']] \\ &= \sum_a \pi(a \mid s) \left[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) v_\pi(s') \right]. \end{aligned}$$

Thus

$$v_\pi(s) = \sum_a \pi(a \mid s) \left[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) v_\pi(s') \right]$$

The action-value version is

$$q_{\pi}(s, a) = r(s, a) + \gamma \sum_{s'} P(s' | s, a) \sum_{a'} \pi(a' | s') q_{\pi}(s', a')$$

Why the Bellman equation is useful

Without recursion, evaluating a policy appears to require enumerating every possible future trajectory. The Bellman equation replaces that tree with a system of coupled local consistency conditions.

A value function is correct exactly when every state's estimate agrees with one expected one-step transition plus the next state's estimate.

Optimal values

Define

$$v_*(s) = \sup_{\pi} v_{\pi}(s),$$

and

$$q_*(s, a) = \sup_{\pi} q_{\pi}(s, a).$$

In a finite discounted MDP, the supremum is attained by at least one stationary deterministic policy.

The Bellman optimality equations are

$$v_*(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) v_*(s') \right]$$

and

$$q_*(s, a) = r(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} q_*(s', a')$$

The conceptual difference is one operation:

- policy evaluation **averages** over actions according to π ;
- optimal control takes a **maximum** over actions.

Recovering an optimal policy

Any policy whose support is contained in the maximizing actions is optimal:

$$\text{supp } \pi_*(\cdot | s) \subseteq \arg \max_a q_*(s, a).$$

A deterministic optimal policy chooses one maximizing action, using any fixed tie-breaking rule.

Worked example: two-state MDP

Let $\mathcal{S} = \{s_1, s_2\}$, $\mathcal{A} = \{\text{left}, \text{right}\}$, and $\gamma = 0.9$.

```
s1 --right,+5--> s2
s1 --left,+1 ---> s1
s2 --right,+2--> s1
s2 --left,+3 ---> s2
```

Two-state MDP (worked example, $\gamma = 0.9$)

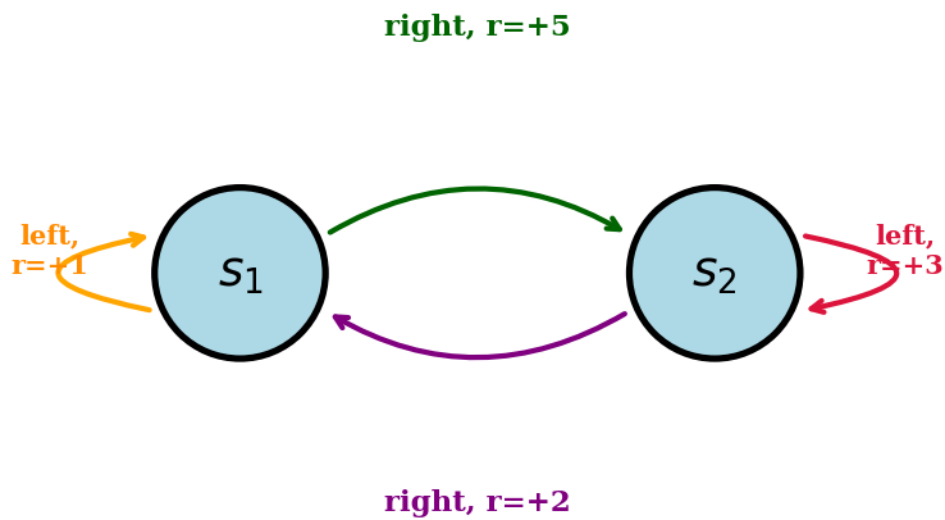


Figure 9: A two-state Markov decision process with transitions and rewards

Suppose the policy chooses right in both states. Then

$$v(s_1) = 5 + 0.9v(s_2),$$

$$v(s_2) = 2 + 0.9v(s_1).$$

Substitute the second into the first:

$$\begin{aligned} v(s_1) &= 5 + 0.9(2 + 0.9v(s_1)) \\ &= 6.8 + 0.81v(s_1). \end{aligned}$$

Therefore

$$v(s_1) = \frac{6.8}{0.19} \approx 35.79,$$

and

$$v(s_2) = 2 + 0.9v(s_1) \approx 34.21.$$

Check the alternative actions:

$$1 + 0.9v(s_1) \approx 33.21 < 35.79,$$

$$3 + 0.9v(s_2) \approx 33.79 < 34.21.$$

So choosing right in both states is optimal.

Tip. Solving a candidate policy and then checking all one-step deviations is often the fastest way to solve a tiny deterministic MDP by hand.

3.5 Matrix Formulation

Intuition first: policy evaluation is linear algebra

For a fixed policy, the max operation disappears. The Bellman equations become a linear system with one unknown per state.

This viewpoint separates three objects:

1. how the policy selects actions;
2. how the environment transitions under those actions;
3. how future values feed back through the transition matrix.

Policy-induced transition matrix and reward vector

Assume $n = |\mathcal{S}|$. Define

$$P_{ss'}^\pi = \sum_a \pi(a | s) P(s' | s, a),$$

and

$$r_s^\pi = \sum_a \pi(a | s) r(s, a).$$

Then $P^\pi \in \mathbb{R}^{n \times n}$ is row-stochastic and $\mathbf{r}^\pi \in \mathbb{R}^n$ is the expected one-step reward vector.

The Bellman expectation equation becomes

$$\mathbf{v}^\pi = \mathbf{r}^\pi + \gamma P^\pi \mathbf{v}^\pi$$

or equivalently

$$(I - \gamma P^\pi) \mathbf{v}^\pi = \mathbf{r}^\pi.$$

Why the inverse exists for $\gamma < 1$

Because P^π is row-stochastic,

$$\|P^\pi\|_\infty = 1.$$

Hence

$$\|\gamma P^\pi\|_\infty = \gamma < 1.$$

The Neumann series converges:

$$(I - \gamma P^\pi)^{-1} = \sum_{k=0}^{\infty} (\gamma P^\pi)^k.$$

Therefore

$$\mathbf{v}^\pi = (I - \gamma P^\pi)^{-1} \mathbf{r}^\pi.$$

In numerical code, solve the linear system rather than explicitly constructing the inverse.

Interpretation of the Neumann series

Expanding gives

$$\mathbf{v}^\pi = \mathbf{r}^\pi + \gamma P^\pi \mathbf{r}^\pi + \gamma^2 (P^\pi)^2 \mathbf{r}^\pi + \dots.$$

The term $(P^\pi)^k \mathbf{r}^\pi$ is the expected reward k transitions into the future. The matrix series is therefore the same discounted sum encoded by the scalar return.

Matrix solution of the two-state example

For the always-right policy,

$$P^\pi = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \mathbf{r}^\pi = \begin{pmatrix} 5 \\ 2 \end{pmatrix}.$$

Then

$$I - 0.9P^\pi = \begin{pmatrix} 1 & -0.9 \\ -0.9 & 1 \end{pmatrix}.$$

Solving

$$\begin{pmatrix} 1 & -0.9 \\ -0.9 & 1 \end{pmatrix} \begin{pmatrix} v(s_1) \\ v(s_2) \end{pmatrix} = \begin{pmatrix} 5 \\ 2 \end{pmatrix}$$

gives

$$\mathbf{v}^\pi \approx \begin{pmatrix} 35.79 \\ 34.21 \end{pmatrix}.$$

Computational implications

Method	Dense cost	Comment
Direct linear solve	$O(n^3)$	Accurate for moderate n ; exploit sparsity when possible
Iterative policy evaluation	Up to $O(n^2)$ per sweep	Often cheaper for sparse transitions
Value iteration	Up to $O(n^2 \mathcal{A})$ per sweep	Nonlinear because of the max

The matrix formulation is exact but does not defeat the curse of dimensionality. Large or continuous state spaces motivate sampling and function approximation.

Common misconceptions

“The state is whatever the sensor reports”

A sensor observation may omit information needed to predict the future. The state is a modeling object that should be sufficient for decision making.

“The reward is feedback about whether the last action was correct”

Reward can be delayed and can reflect the effect of many earlier actions. Credit assignment is precisely the problem of connecting cumulative outcomes back to decisions.

“A large discount factor is always better”

A larger γ lengthens the effective horizon and can make estimation harder. It also changes the objective. It should be chosen to match the task, not treated as a universal default.

“Bellman optimality is a linear equation”

Policy evaluation is linear for a fixed policy. Bellman optimality includes a max and is nonlinear.

Chapter summary

Concept	Mathematical object	Plain-language meaning
Policy	$\pi(a \mid s)$	How the agent chooses actions
Transition model	$P(s' \mid s, a)$	How the environment responds
Return	G_t	Cumulative future reward on one trajectory
State value	$v_\pi(s)$	Expected return from a state
Action value	$q_\pi(s, a)$	Expected return after one chosen action
Bellman expectation	$v_\pi = T^\pi v_\pi$	A policy's values are one-step consistent
Bellman optimality	$v_* = T^* v_*$	Optimal values use the best one-step action
Direct evaluation	$(I - \gamma P^\pi)v = r^\pi$	Fixed-policy evaluation is a linear solve

Knowledge check and exercises

Conceptual

1. Give an observation that is not Markov. State exactly what additional history would make it Markov.
2. Explain why a reward function is part of the task specification rather than merely a training aid.
3. Contrast a contextual bandit with an MDP using one concrete application.
4. Explain why a time-dependent finite-horizon policy can be represented as a stationary policy by augmenting the state.

Derivations

5. Derive the Bellman expectation equation for q_π from the recursive return.
6. Prove that $|v_\pi(s)| \leq R_{\max}/(1 - \gamma)$ for bounded rewards.
7. Starting from the Neumann series, show that $(I - \gamma P^\pi)^{-1}$ exists for $\gamma < 1$.
8. Recompute the two-state example for $\gamma = 0.5$ and determine whether the optimal policy changes.

Implementation

9. Construct a three-state MDP in NumPy. Build P^π and $\mathbf{r}^\pi$, solve the linear system, and verify the result with iterative policy evaluation.
10. Add a unit test that distinguishes true termination from time-limit truncation.
11. Change the gridworld step reward and document exactly when the optimal route changes.

Primary readings

- Richard Sutton and Andrew Barto, *Reinforcement Learning: An Introduction*, Chapters 1–3.
- Martin Puterman, *Markov Decision Processes*, Chapters 1–2.

Part II — Tabular Methods

Lecture opening: Three ways to obtain the same backup

Keep the running MDP fixed. With a known model, average every next state exactly. With completed trajectories, average realized returns. With TD, use one observed transition and an estimated continuation. These are not three unrelated definitions of value: they are different estimators and update operators for related fixed points. Predict what each method must know before writing its update. Then verify the target expectation, noise condition, and limiting fixed point separately.

Read A.5–A.6 with the convergence sections. Run Lab 1 to compare exact policy evaluation with value iteration, and Lab 2 to check sampling-based Q-learning against the analytic solution. A plot approaching the right answer is a useful implementation check, not an almost-sure convergence proof.

Learning objectives

By the end of this chapter, you should be able to:

- explain Bellman operators as mappings on value functions;
- prove that discounted Bellman expectation and optimality operators are contractions in the sup norm;
- implement policy iteration, value iteration, Monte Carlo prediction, TD prediction, SARSA, and Q-learning;
- compare full expected, sampled, bootstrapped, and complete-return targets;
- state the coverage, step-size, stationarity, and termination assumptions behind the main convergence results;
- explain how n -step returns and eligibility traces interpolate between TD and Monte Carlo.

Standing assumptions

Unless stated otherwise, the MDP is finite, rewards are bounded, and $0 \leq \gamma < 1$. A table stores one independent estimate for every state or state-action pair.

4. Dynamic Programming

Intuition first: repeatedly correct a global guess

Dynamic programming assumes that the full environment model is known. For every state and action, we can compute the expected immediate reward and next-state distribution.

The central idea is simple:

1. begin with any guess of the value function;
2. replace each value by a one-step Bellman backup;
3. repeat until the values stop changing.

DP is rarely the final practical solution for large environments, but it exposes the fixed-point structure that sampled RL methods approximate.

Bellman operators

For a fixed policy π , define the Bellman expectation operator T^π by

$$(T^\pi v)(s) = \sum_a \pi(a | s) \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) v(s') \right].$$

The operator takes one candidate value function v and returns a new candidate after one exact one-step backup at every state.

The true policy value is a fixed point:

$$v_\pi = T^\pi v_\pi.$$

For control, define the Bellman optimality operator

$$(T^*v)(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) v(s') \right].$$

Its fixed point is the optimal value function:

$$v_* = T^*v_*.$$

Why contraction matters

A mapping T is a contraction with modulus $c < 1$ under norm $\|\cdot\|$ if

$$\|Tx - Ty\| \leq c\|x - y\|$$

for all x, y .

The Banach fixed-point theorem says that a contraction on a complete metric space has:

1. a unique fixed point x^* ;
2. convergence of $x_{k+1} = Tx_k$ from any starting point;
3. geometric error decay:

$$\|x_k - x^*\| \leq c^k \|x_0 - x^*\|.$$

The space $\mathbb{R}^{|S|}$ with the sup norm is complete, so the theorem applies directly to finite value functions.

Contraction proof for T^π

For any state s ,

$$\begin{aligned} |(T^\pi v)(s) - (T^\pi w)(s)| &= \gamma \left| \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [v(s') - w(s')] \right| \\ &\leq \gamma \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) \|v - w\|_\infty \\ &= \gamma \|v - w\|_\infty. \end{aligned}$$

Taking the maximum over s gives

$$\boxed{\|T^\pi v - T^\pi w\|_\infty \leq \gamma \|v - w\|_\infty.}$$

The same argument works for T^* because

$$|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|.$$

Therefore

$$\|T^*v - T^*w\|_\infty \leq \gamma \|v - w\|_\infty.$$

Important. The contraction proof supplies uniqueness and convergence for the exact discounted operators. It does not automatically prove convergence after nonlinear function approximation, replay, finite batches, or approximate optimization.

Iterative policy evaluation

Given a policy π , initialize v_0 arbitrarily and iterate

$$v_{k+1} = T^\pi v_k.$$

Componentwise,

$$v_{k+1}(s) = \sum_a \pi(a | s) \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) v_k(s') \right].$$

The contraction theorem gives

$$\|v_k - v_\pi\|_\infty \leq \gamma^k \|v_0 - v_\pi\|_\infty.$$

A practical stopping criterion uses the largest update magnitude:

$$\Delta_k = \|v_{k+1} - v_k\|_\infty.$$

Because

$$\|v_{k+1} - v_\pi\|_\infty \leq \frac{\gamma}{1 - \gamma} \Delta_k,$$

small successive changes imply a bounded residual error.

Worked example: random policy on a 4×4 grid

Consider the standard episodic gridworld:

```

+-----+-----+
| T | 1 | 2 | 3 |
+-----+-----+
| 4 | 5 | 6 | 7 |
+-----+-----+
| 8 | 9 | 10 | 11 |
+-----+-----+
| 12 | 13 | 14 | T |
+-----+-----+

```

- terminal values are zero;
- each action is chosen with probability $1/4$;
- each nonterminal transition gives reward -1 ;
- attempting to leave the grid leaves the state unchanged;
- $\gamma = 1$, but episodes terminate under the random policy.

The converged values are

$$\begin{pmatrix} 0 & -14 & -20 & -22 \\ -14 & -18 & -20 & -20 \\ -20 & -20 & -18 & -14 \\ -22 & -20 & -14 & 0 \end{pmatrix}.$$

A state next to a terminal is not worth -1 because the random policy often moves away before eventually terminating. Value summarizes the entire future policy, not only the best immediate transition.

Policy improvement theorem

Suppose a new policy π' satisfies

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s)$$

for every state. Then

$$v_{\pi'}(s) \geq v_{\pi}(s)$$

for every state.

A greedy policy satisfies the premise:

$$\pi'(s) \in \arg \max_a q_{\pi}(s, a).$$

The proof repeatedly substitutes the one-step improvement:

$$\begin{aligned} v_{\pi}(s) &\leq q_{\pi}(s, \pi'(s)) \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\ &\leq \dots \leq v_{\pi'}(s). \end{aligned}$$

Policy iteration

Policy iteration alternates exact or approximate evaluation with greedy improvement.

```
initialize policy pi
repeat:
    evaluate pi to obtain v_pi
    for each state s:
        choose pi_new(s) in argmax_a [r(s,a) + gamma sum_s' P(s'|s,a) v_pi(s')]
    if pi_new == pi:
        stop
    pi = pi_new
```

For a finite discounted MDP, exact evaluation and consistent tie handling imply termination at an optimal policy. A convenient rule is to keep the current action whenever it remains greedy. Then every policy change is strictly improving in at least one state, and only finitely many deterministic policies exist.

Value iteration

Value iteration performs only one optimality backup before improving again:

$$v_{k+1} = T^* v_k.$$

Componentwise,

$$v_{k+1}(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) v_k(s') \right].$$

Since T^* is a contraction,

$$\|v_k - v_*\|_\infty \leq \gamma^k \|v_0 - v_*\|_\infty.$$

After convergence, extract a greedy policy:

$$\pi_*(s) \in \arg \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) v_*(s') \right].$$

Generalized policy iteration

The space of value-update methods (Sutton & Barto, Ch. 8)

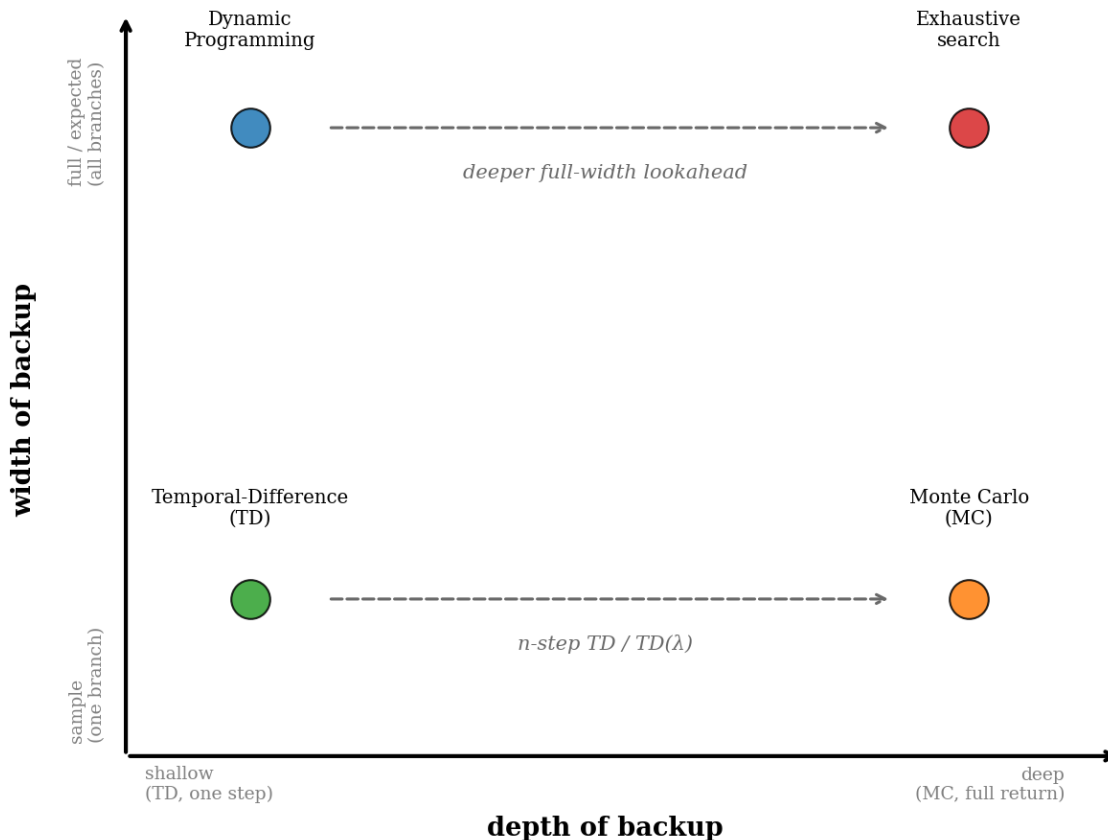


Figure 10: The spectrum between wide shallow backups and narrow deep backups

Policy iteration and value iteration are endpoints of a continuum. In **generalized policy iteration**, evaluation and improvement proceed on different timescales and need not finish before the other resumes.

This viewpoint anticipates actor-critic methods:

- the critic approximately evaluates the current policy;
- the actor approximately improves it;
- neither computation is exact before the next update.

DP comparison

Method	Evaluation work per outer step	Improvement	Main tradeoff
Policy iteration	Near-complete	Explicit greedy step	Expensive evaluation, few policy changes
Modified policy iteration	Several sweeps	Explicit greedy step	Intermediate compromise
Value iteration	One sweep	Implicit in max	Cheap sweep, more iterations

Limitations of dynamic programming

DP requires:

- a known transition and reward model;
- enough memory to store values for every state;
- enough computation to sweep through the state space.

These limits motivate learning from sampled experience.

5. Monte Carlo Methods

Intuition first: estimate value by averaging complete outcomes

Monte Carlo methods do not need a transition model and do not bootstrap. They run episodes, observe complete returns, and average those returns.

The statistical analogy is estimating a commute time by recording many complete trips rather than constructing a traffic model.

Why the undiscounted gridworld is well posed. Under the uniform random policy, from any nonterminal cell there is a path of at most six moves to a terminal cell, with probability at least 4^{-6} . Conditional on any surviving history, the probability of surviving another six steps is therefore at most $1 - 4^{-6}$. Thus $\Pr(T > 6k) \leq (1 - 4^{-6})^k$ and $\mathbb{E}T \leq 6 \cdot 4^6 < \infty$. Expected negative step count is finite. This transience argument, not the discounted contraction theorem with $\gamma = 1$, justifies the example.

Monte Carlo prediction

Generate an episode under policy π :

$$S_0, A_0, R_1, S_1, \dots, S_T.$$

For each visit to state s , compute

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

Then update

$$V(s) \leftarrow V(s) + \alpha[G_t - V(s)].$$

With $\alpha = 1/N(s)$, this is an incremental sample mean.

First-visit and every-visit estimators

- **First-visit MC:** use only the first visit to s in each episode.
- **Every-visit MC:** use every visit to s .

Returns from different visits in the same episode are dependent. The every-visit estimator should therefore be understood through episode-level laws of large numbers rather than by pretending that all visits are i.i.d.

Prediction convergence

For a fixed policy, use independent, identically distributed episodes and sample-average updates. For every-visit MC, let $N_e(s)$ count visits to s in episode e and $Y_e(s)$ sum their returns. Require $0 < \mathbb{E}N_e(s) < \infty$ and $\mathbb{E} \sum_{t < T_e} \mathbf{1}\{S_t = s\} |G_t| < \infty$. Then

$$\frac{\sum_{e=1}^m Y_e(s)}{\sum_{e=1}^m N_e(s)} \xrightarrow{\text{a.s.}} v_\pi(s).$$

A.5 proves this by applying the strong law to episode-level quantities. It does not assume independent visits within an episode. First-visit MC is the same argument with at most one visit retained. Time-dependent finite-horizon values require time in the state. General adaptive step-size claims require additional sampling and noise conditions; the theorem here is for sample averages. Constant steps generally leave persistent sampling noise.

Monte Carlo control

Without a model, policy improvement requires action values. Estimate

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

A control algorithm alternates:

1. sample episodes;
2. update $Q(s, a)$ toward observed returns;
3. make the policy more greedy with respect to Q .

Exploration and ε -greedy policies

An ε -greedy policy assigns

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|}, & a \in \arg \max_{a'} Q(s, a'), \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & \text{otherwise.} \end{cases}$$

This ensures nonzero probability for every action while favoring current estimates.

For convergence to an optimal policy in tabular control, exploration usually must be **greedy in the limit with infinite exploration**:

- every state-action pair is visited infinitely often;
- $\varepsilon_t \rightarrow 0$ so the policy becomes greedy asymptotically.

Off-policy Monte Carlo

Suppose data are generated by behavior policy b , but we want to estimate target policy π . The trajectory likelihood ratio from time t is

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}.$$

The support condition is essential:

$$b(a | s) > 0 \quad \text{whenever} \quad \pi(a | s) > 0.$$

Ordinary importance sampling estimates

$$\hat{v}_{\text{OIS}}(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} \rho_i G_i.$$

It is unbiased under the support and integrability assumptions but can have enormous variance.

Weighted importance sampling uses

$$\hat{v}_{\text{WIS}}(s) = \frac{\sum_i \rho_i G_i}{\sum_i \rho_i}.$$

It is generally biased in finite samples but often much more stable.

Warning. Importance sampling corrects probabilities; it cannot reconstruct outcomes for actions that the behavior policy never took.

MC strengths and limitations

Property	Monte Carlo
Model required	No
Bootstrapping	No
Must wait for termination	Standard episodic MC does
Bias for fixed-policy complete returns	None from bootstrapping
Variance	Often high
Continuing tasks	Require truncation, regeneration, or another objective

6. Temporal-Difference Learning

Intuition first: learn before the episode ends

Temporal-difference learning combines sampling with bootstrapping.

Instead of waiting for the full return, TD uses the one-step target

$$R_{t+1} + \gamma V(S_{t+1}).$$

It asks: “After observing one transition, how inconsistent was my old estimate with the reward plus my estimate of what comes next?”

TD(0) prediction

The update is

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma(1 - d_{t+1})V(S_{t+1}) - V(S_t)].$$

The TD error is

$$\delta_t = R_{t+1} + \gamma(1 - d_{t+1})V(S_{t+1}) - V(S_t).$$

Here $d_{t+1} = 1$ only for a true terminal transition.

DP, MC, and TD as target choices

Backup diagrams: open circles = states, filled squares = actions, open square = terminal

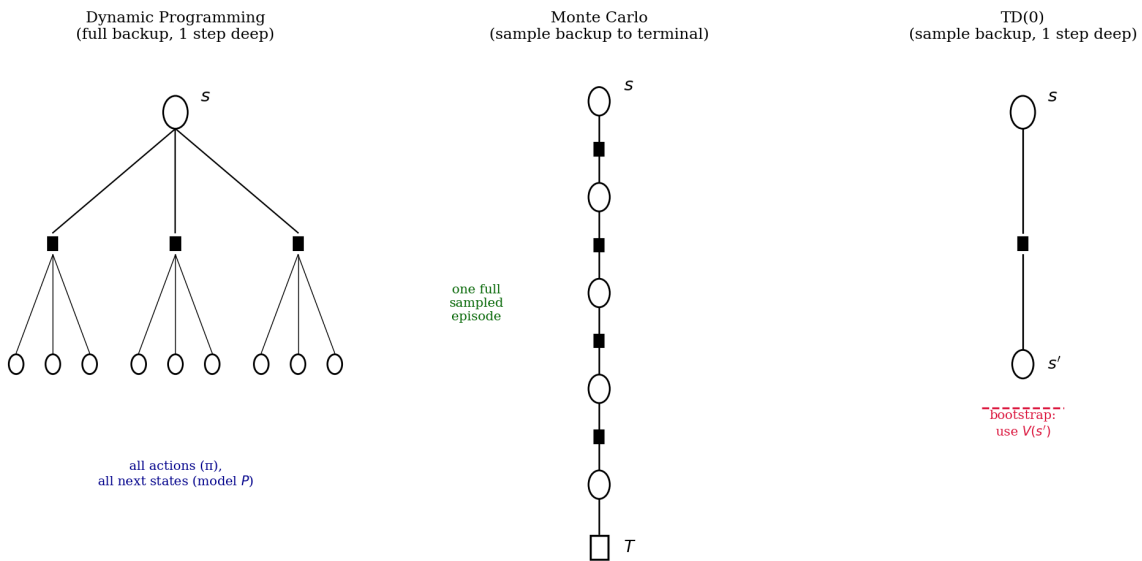


Figure 11: Dynamic programming, Monte Carlo, and temporal-difference backup diagrams

Method	Target	Expected or sampled?	Bootstraps?
DP	$\mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s]$	Expected	Yes
MC	G_t	Sampled complete return	No
TD(0)	$R_{t+1} + \gamma V(S_{t+1})$	Sampled one-step	Yes

Two axes organize these methods:

- **width:** average over all possible transitions or sample one branch;
- **depth:** bootstrap after one step or continue to the end.

Why tabular TD(0) converges

For a fixed policy, define the sampling noise

$$M_{t+1} = R_{t+1} + \gamma V_t(S_{t+1}) - (T^\pi V_t)(S_t).$$

Conditioned on the history through S_t ,

$$\mathbb{E}[M_{t+1} \mid \mathcal{F}_t] = 0.$$

The asynchronous update can be written

$$V_{t+1}(s) = V_t(s) + \alpha_t(s) \mathbf{1}\{S_t = s\} [(T^\pi V_t)(s) - V_t(s) + M_{t+1}].$$

Its mean drift points toward the unique fixed point of the contraction T^π . Under bounded rewards, infinite state visitation, and per-state Robbins-Monro step sizes,

$$V_t \xrightarrow{\text{a.s.}} v_\pi.$$

A.6 supplies the full asynchronous stochastic-contraction proof, including boundedness, martingale noise, and per-state step sizes. Read that proof here rather than treating “stochastic approximation” as a missing step. The assumptions are not optional:

- constant learning rates generally produce a stationary error distribution rather than exact convergence;
- a state visited only finitely often may retain arbitrary error;
- nonstationary rewards or transitions violate the fixed-target theorem.

SARSA: on-policy TD control

Compare the targets, not an invented learning curve. SARSA uses the next action actually drawn from the behavior policy; Q-learning uses the maximizing next action. Their realized return ordering depends on the environment, exploration policy, and evaluation convention. The backup diagram earlier in this chapter illustrates this distinction without presenting synthetic curves as evidence.

SARSA updates from the tuple

$$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}).$$

Its update is

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma(1 - d_{t+1})Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

The next action A_{t+1} is sampled from the current behavior policy. Therefore SARSA learns the value of the policy that actually includes exploration.

Q-learning: off-policy TD control

Q-learning uses a greedy target regardless of the next behavior action:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma(1 - d_{t+1}) \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

It is off-policy because:

- the behavior policy may explore;
- the target uses the greedy policy implied by Q .

Q-learning convergence theorem

In a finite stationary MDP with bounded rewards and $\gamma < 1$, tabular Q-learning converges almost surely to q_* when:

1. every state-action pair is updated infinitely often;
2. each pair's step sizes satisfy

$$\sum_n \alpha_n(s, a) = \infty, \quad \sum_n \alpha_n(s, a)^2 < \infty.$$

Define the optimality operator on action values:

$$(T^*Q)(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') \mid S_t = s, A_t = a \right].$$

It is a γ -contraction in the sup norm. A.6 proves that the bounded sampled updates converge to its unique fixed point q_* under the stated per-pair visitation and step-size assumptions.

The theorem does **not** cover finite replay, constant learning rates, nonstationary tasks, insufficient exploration, or neural networks.

Expected SARSA

Expected SARSA replaces the sampled next action with an expectation under the target policy:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma(1 - d_{t+1}) \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

Relative to SARSA, it removes next-action sampling variance. If π is greedy, the target reduces to Q-learning.

Worked example: cliff walking

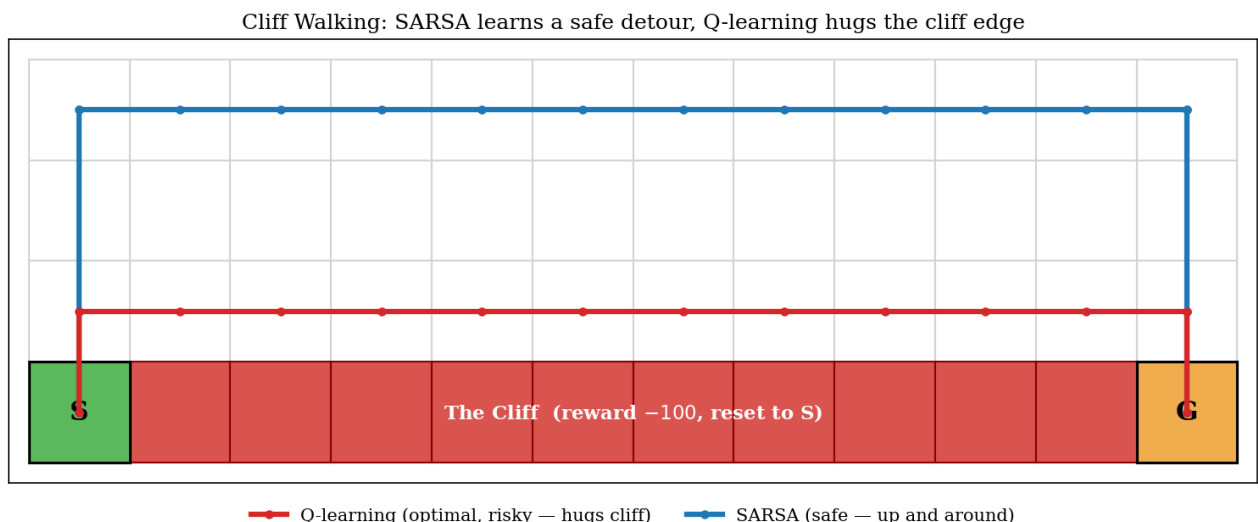


Figure 12: Cliff-walking policies learned by SARSA and Q-learning

```

+-----+
| S |   |   |   |   |   |   |   |   |   | G |
+-----+
| |CC|CC|CC|CC|CC|CC|CC|CC|CC|CC|   |
+-----+

```

Each step gives -1 . Entering the cliff gives -100 and returns the agent to the start.

- Q-learning targets the greedy policy and learns the shortest path along the cliff edge.
- SARSA evaluates the exploratory policy. With persistent ε -greedy behavior, cliff-edge states are risky because exploratory actions can fall into the cliff.

This example separates two questions:

1. Which greedy policy is optimal?
2. Which learning behavior performs well while exploration remains active?

Q-learning can learn the optimal greedy policy while accumulating worse online return during training.

n -step TD

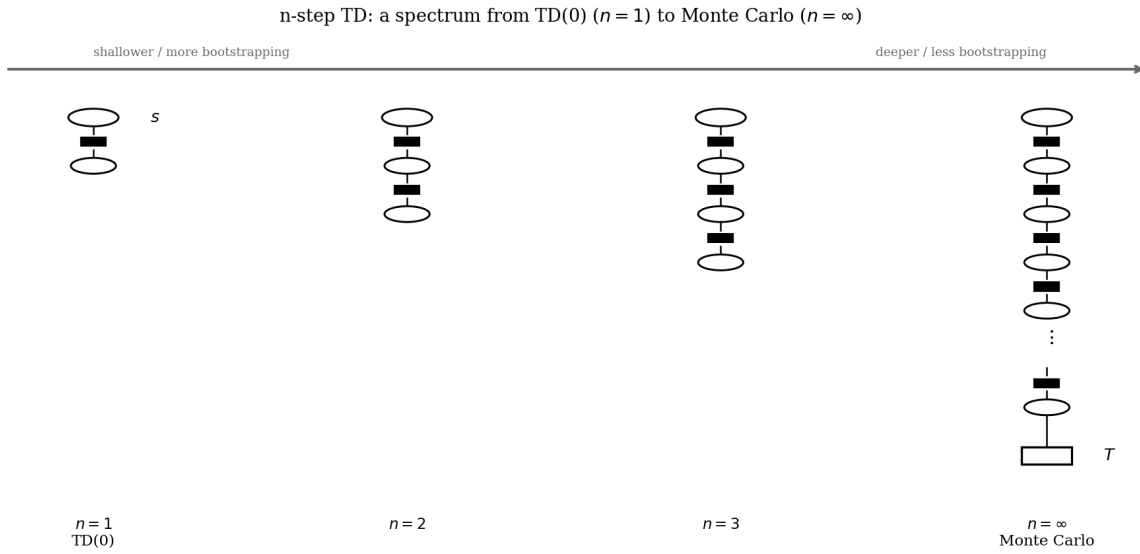


Figure 13: The bias-variance spectrum of multi-step temporal-difference targets

The n -step return is

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n}),$$

with the bootstrap omitted if the episode terminates first.

The update is

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t^{(n)} - V(S_t)].$$

- $n = 1$ gives TD(0);
- n equal to the remaining episode length gives Monte Carlo;
- intermediate n trades bootstrap bias against sampling variance.

TD(λ): averaging backup lengths

The forward-view λ -return for an episode ending at T is

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_t^{(n)} + \lambda^{T-t-1} G_t^{(T-t)}.$$

At the endpoints:

$$\lambda = 0 \Rightarrow G_t^\lambda = G_t^{(1)},$$

$$\lambda = 1 \Rightarrow G_t^\lambda = G_t.$$

Eligibility traces

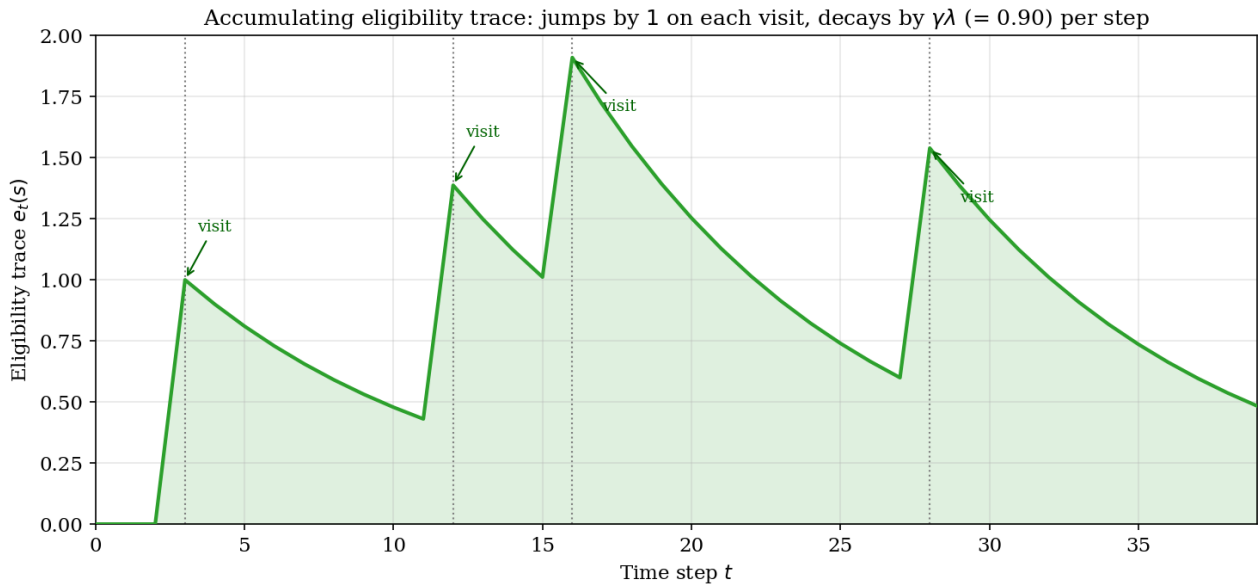


Figure 14: How an eligibility trace assigns credit backward through a trajectory

The backward view implements the same credit assignment online. For accumulating traces,

$$e_t(s) = \gamma\lambda e_{t-1}(s) + \mathbf{1}\{S_t = s\}.$$

Then update every state:

$$V(s) \leftarrow V(s) + \alpha \delta_t e_t(s).$$

A trace is a decaying memory of recent states. When a TD error occurs, recently visited states receive more credit.

Note. Forward-view and backward-view equivalence is exact in the offline episodic case with matching definitions. Online variants such as true-online TD(λ) handle changing weights more carefully.

Common misconceptions

“Off-policy means exploration is unnecessary”

Off-policy learning separates the target policy from the behavior policy. The behavior policy still must visit every state-action pair needed by the target.

“TD is biased and Monte Carlo is unbiased, so MC is always better”

TD targets can be biased because they bootstrap, but their lower variance and online updates can produce better finite-sample learning. The correct question is the bias-variance tradeoff under the task’s horizon and noise.

“Pseudocode convergence follows from the Bellman contraction”

The contraction proves convergence of the exact operator. Sampled algorithms also need stochastic-approximation conditions. Function approximation introduces further complications.

“A terminal flag and a timeout are interchangeable”

A true terminal state has no future return. A time-limit cutoff may hide a continuing future and usually retains a bootstrap term.

Chapter summary

Method	Model	Samples	Bootstrap	Target policy
Policy evaluation	Required	No	Yes	Fixed π
Value iteration	Required	No	Yes	Greedy
Monte Carlo	No	Yes	No	Usually on-policy; IS enables off-policy
TD(0)	No	Yes	Yes	Fixed π
SARSA	No	Yes	Yes	Current behavior policy
Q-learning	No	Yes	Yes	Greedy target policy
Expected SARSA	No	Yes	Yes	User-specified target policy

The unifying question is: **what target is used to correct the current estimate?**

Knowledge check and exercises

Conceptual

1. Explain the difference between a Bellman equation and a Bellman update.
2. Why can policy iteration change policies with equal values unless ties are handled consistently?
3. Explain why every-visit returns within one episode are not i.i.d.
4. Compare the behavior learned by SARSA and Q-learning in cliff walking.
5. Explain why a constant learning rate is sensible in a nonstationary task even though it violates exact-convergence conditions.

Derivations

6. Prove that T^* is a γ -contraction in the sup norm.
7. Derive the residual error bound from $\Delta_k = \|v_{k+1} - v_k\|_\infty$.

8. Show that Expected SARSA becomes Q-learning under a greedy target policy.
9. Verify that the Q-learning sampling noise has conditional mean zero.
10. Derive the forward-view λ -return weights and show that they sum to one.

Implementation

11. Reproduce the 4×4 gridworld values with synchronous and in-place policy evaluation.
12. Compare policy iteration, modified policy iteration, and value iteration by counting Bellman backups rather than only outer loops.
13. Implement first-visit and every-visit MC prediction and show uncertainty across independent episodes.
14. Compare SARSA, Expected SARSA, and Q-learning on cliff walking across multiple seeds.
15. Add unit tests for terminal and truncation targets.
16. Implement n -step TD for several n and plot error against both environment steps and wall-clock time.

Primary readings

- Sutton and Barto, *Reinforcement Learning: An Introduction*, Chapters 4–7 and 12.
- Christopher Watkins and Peter Dayan, “Q-learning” (1992).
- Tommi Jaakkola, Michael Jordan, and Satinder Singh, “On the Convergence of Stochastic Iterative Dynamic Programming Algorithms” (1994).
- John Tsitsiklis, “Asynchronous Stochastic Approximation and Q-Learning” (1994).

Part III — Function Approximation

Lecture opening: Generalization changes the geometry

In a table, correcting state 0 does not directly change state 1. With shared features, it does. That single change explains both the usefulness of representation learning and the loss of the tabular contraction proof. The right first question is not “which network is larger?” but “in what norm and under what state weighting does the expected update move toward its solution?”

Derive $A = X^\top D(I - \gamma P)X$ in A.7 before studying DQN. Then change only the sampling distribution in the two-state counterexample: the mean update becomes expansive even though the true value is exactly representable. This is a concrete failure of a proof assumption, not a slogan about instability. A.8 and A.18 supply the double-sampling and distributional arguments used later.

Learning objectives

By the end of this chapter, you should be able to:

- explain why generalization is necessary once a value table is too large;
- derive semi-gradient Monte Carlo and TD updates for linear value functions;
- interpret linear TD as solving a projected Bellman fixed-point equation;
- explain the deadly triad as a risk pattern rather than a complete safety theorem;
- derive the DQN and Double-DQN targets;
- distinguish changes to representation, targets, replay sampling, exploration, and return modeling.

Standing assumptions

The first half considers prediction under a fixed policy. The DQN half considers discrete-action control with a neural action-value function. Whenever convergence is claimed, the relevant assumptions are stated explicitly.

7. Value Function Approximation

Intuition first: replace memorization with generalization

A table assigns an unrelated parameter to every state. That is ideal when the state space is small, but impossible when states are continuous, combinatorial, or represented by images.

Function approximation replaces the table with

$$\hat{v}(s; w),$$

where one parameter vector $w \in \mathbb{R}^d$ determines values for many states. Updating one state can therefore change predictions at similar states.

This is the benefit and the danger:

- **benefit:** data from one state informs others;
- **danger:** an erroneous update can damage many states at once.

Why move from tables to parameterized functions?

Tabular: one stored value per state
 $v(s)$ as a lookup table — no generalization

0.25	0.79	0.55	-0.55	-0.40
0.75	-0.99	0.64	0.59	-0.06
-0.39	-0.44	-0.49	-0.11	0.01
0.11	0.99	0.59	0.24	0.98
-0.57	-0.68	0.23	-0.91	-0.93

$|S|$ entries to store and learn separately

Function approximation: a parameterized curve
 generalizes to unseen, nearby states

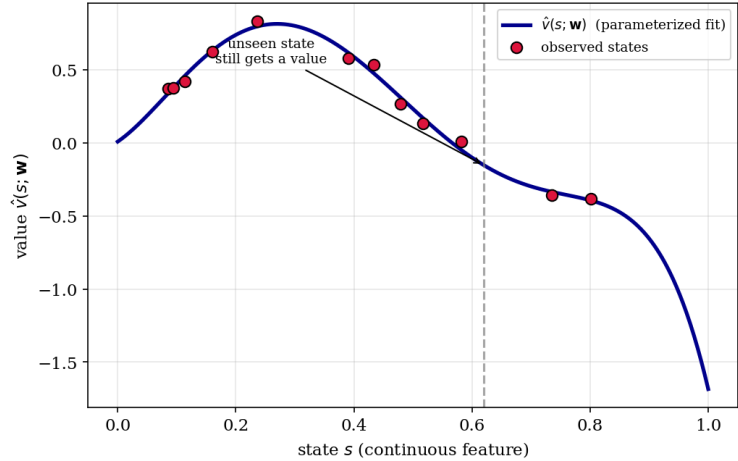


Figure 15: Tabular value storage compared with function approximation

Approximation objective

For a fixed policy π , suppose states are weighted by distribution μ . A natural squared-value objective is

$$\overline{\text{VE}}(w) = \sum_s \mu(s) [v_\pi(s) - \hat{v}(s; w)]^2.$$

In vector notation, with diagonal matrix D_μ ,

$$\overline{\text{VE}}(w) = \|v_\pi - \hat{v}_w\|_{D_\mu}^2.$$

The weighting distribution matters. An approximation can be accurate on frequently visited states and poor elsewhere.

Generic stochastic-gradient update

If U_t is a target for $v_\pi(S_t)$, the squared sample loss is

$$\ell_t(w) = \frac{1}{2} [U_t - \hat{v}(S_t; w)]^2.$$

When the target is treated as fixed with respect to w ,

$$w_{t+1} = w_t + \alpha [U_t - \hat{v}(S_t; w_t)] \nabla_w \hat{v}(S_t; w_t).$$

Different targets produce different algorithms.

Monte Carlo target

With

$$U_t = G_t,$$

the target does not depend on the current parameters. The update is an ordinary stochastic gradient of squared return-prediction error:

$$w_{t+1} = w_t + \alpha [G_t - \hat{v}(S_t; w_t)] \nabla_w \hat{v}(S_t; w_t).$$

The sampled return is noisy but does not contain a learned bootstrap estimate.

TD target and the semi-gradient

TD uses

$$U_t = R_{t+1} + \gamma(1 - d_{t+1})\hat{v}(S_{t+1}; w_t).$$

The target depends on w_t . Semi-gradient TD intentionally ignores the derivative through the target:

$$\boxed{w_{t+1} = w_t + \alpha \delta_t \nabla_w \hat{v}(S_t; w_t),}$$

where

$$\delta_t = R_{t+1} + \gamma(1 - d_{t+1})\hat{v}(S_{t+1}; w_t) - \hat{v}(S_t; w_t).$$

It is called a **semi-gradient** because it differentiates only the prediction being corrected, not the bootstrap target.

Note. Differentiating a squared sampled TD error through both sides gives a full gradient of that sampled-error objective. It is not generally an unbiased gradient of the squared *expected* Bellman residual: the two factors share transition noise. A.8 derives the conditional covariance discrepancy and the need for double sampling. The familiar TD update is not an accidental incomplete derivative.

Linear function approximation

Let

$$\hat{v}(s; w) = w^\top x(s),$$

where $x(s) \in \mathbb{R}^d$ is a feature vector. Then

$$\nabla_w \hat{v}(s; w) = x(s),$$

and TD(0) becomes

$$\boxed{w_{t+1} = w_t + \alpha \delta_t x(S_t).}$$

Linear approximation is important because it separates representation from optimization. The features determine what can be represented; TD determines how the representable values are fitted.

Feature design

Common choices include:

- **one-hot features:** recover the tabular case;
- **polynomial features:** simple global approximation, but poorly conditioned at high degree;
- **Fourier features:** useful for smooth periodic or bounded domains;
- **radial basis functions:** localized smooth responses;

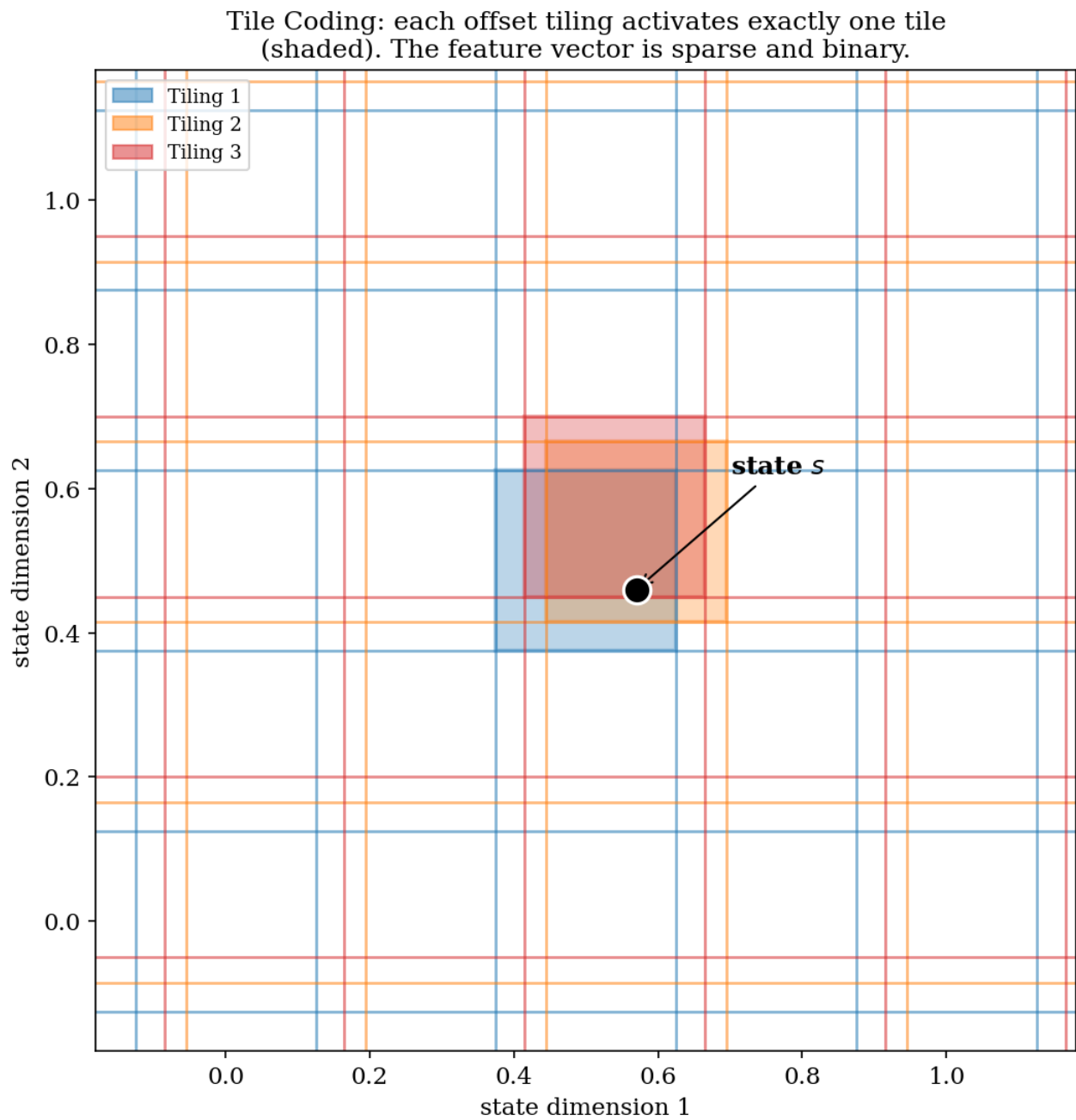


Figure 16: Tile coding as overlapping sparse features over a continuous state space

- **tile coding:** sparse binary features from overlapping discretizations;
- **learned neural features:** flexible but harder to analyze.

For radial basis functions,

$$x_i(s) = \exp\left(-\frac{\|s - c_i\|^2}{2\sigma_i^2}\right).$$

For tile coding, a state activates one feature per tiling. Nearby states share some, but not all, active features. This gives local generalization with sparse updates.

The projection view

Let $X \in \mathbb{R}^{|\mathcal{S}| \times d}$ contain one feature row per state. The representable value functions form the column space

$$\mathcal{V}_X = \{Xw : w \in \mathbb{R}^d\}.$$

A Bellman backup $T^\pi(Xw)$ need not lie in this space. Linear TD therefore combines two operations:

1. apply the Bellman operator;
2. project back into the feature space.

Let D_π be the diagonal stationary-state distribution under the on-policy Markov chain, and let Π_{D_π} be weighted least-squares projection onto $\mathcal{V}_X$. A.7 proves that the projected operator has a unique fixed point under positive stationary weights and full-rank features. Under the independent stationary sampling and step-size assumptions stated there, sampled linear TD converges to w_{TD} satisfying

$$Xw_{\text{TD}} = \Pi_{D_\pi} T^\pi(Xw_{\text{TD}}).$$

This is the **projected Bellman equation**.

What the projected fixed point means

The TD solution is generally not

$$\arg \min_w \|v_\pi - Xw\|_{D_\pi}^2,$$

and it is generally not the minimizer of the unprojected Bellman residual

$$\|Xw - T^\pi(Xw)\|_{D_\pi}^2.$$

Instead, it is the representable value whose Bellman backup projects back to itself.

A classical bound is

$$\|Xw_{\text{TD}} - v_\pi\|_{D_\pi} \leq \frac{1}{\sqrt{1 - \gamma^2}} \|\Pi_{D_\pi} v_\pi - v_\pi\|_{D_\pi},$$

under the assumptions of the linear on-policy analysis. The right-hand side is the unavoidable approximation error induced by the features, amplified by a discount-dependent factor.

Linear TD convergence conditions

The sampled theorem proved in A.7 assumes independent tuples with $S \sim d_\pi$, then $A \sim \pi$, then the true transition and reward; bounded rewards and finite features; a strictly positive stationary

distribution; full column rank of X ; and deterministic nonnegative steps with $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$. It concludes $w_t \rightarrow w_{\text{TD}}$ almost surely. Its proof explicitly controls mean drift and second moments.

A single ergodic Markov trajectory is not iid sampling. The classical Markov-noise extension is a further result of Tsitsiklis and Van Roy (1997), not proved by replacing the conditional transition law with its stationary average. The course's core proved version is the iid stationary-sampler theorem. The projection equation and error bound remain useful algebraic objects regardless of which sampled-convergence theorem is being used.

Changing to off-policy sampling changes the weighting distribution and can destroy stability. A.7 gives a two-state, one-feature example with mean TD coefficient -0.68 , so the expected update expands instead of contracts.

The deadly triad

Three ingredients interact dangerously:

1. **function approximation:** updates generalize across states;
2. **bootstrapping:** targets contain current estimates;
3. **off-policy learning:** the data distribution differs from the target-policy distribution.

The triple combination can cause divergence even in simple linear problems.

Flow to keep in mind. Function approximation, bootstrapping and off-policy data jointly create instability risk. None of the three labels alone is a proof of divergence; their interaction can make the expected update expansive.

The deadly triad is a warning pattern, not a necessary-and-sufficient theorem.

- Some algorithms remain stable with all three ingredients, such as gradient-TD methods under their assumptions.
- Nonlinear bootstrapping can be unstable even on-policy.
- Removing one ingredient often helps but does not guarantee success in every implementation.

Why off-policy semi-gradient TD can diverge

The mean linear TD update has the form

$$\mathbb{E}[\Delta w] = \alpha(b - Aw).$$

On-policy, the matrix A has properties that yield a stable fixed point. Off-policy weighting can make the effective dynamics non-contractive; an update intended to reduce local error can increase the global parameter norm.

Baird's counterexample provides a small seven-state MDP where off-policy linear semi-gradient TD diverges despite bounded features and rewards.

Mitigation families

- use on-policy data;
- remove bootstrapping with Monte Carlo targets;
- constrain or regularize policy shift;
- use gradient-TD algorithms such as GTD2 or TDC;
- use emphatic weighting to correct state-distribution mismatch;
- use target networks, replay controls, double estimators, and conservative objectives in deep control.

These methods address different mechanisms; they are not interchangeable.

The Deadly Triad

All three together can cause divergence; any two are safe.

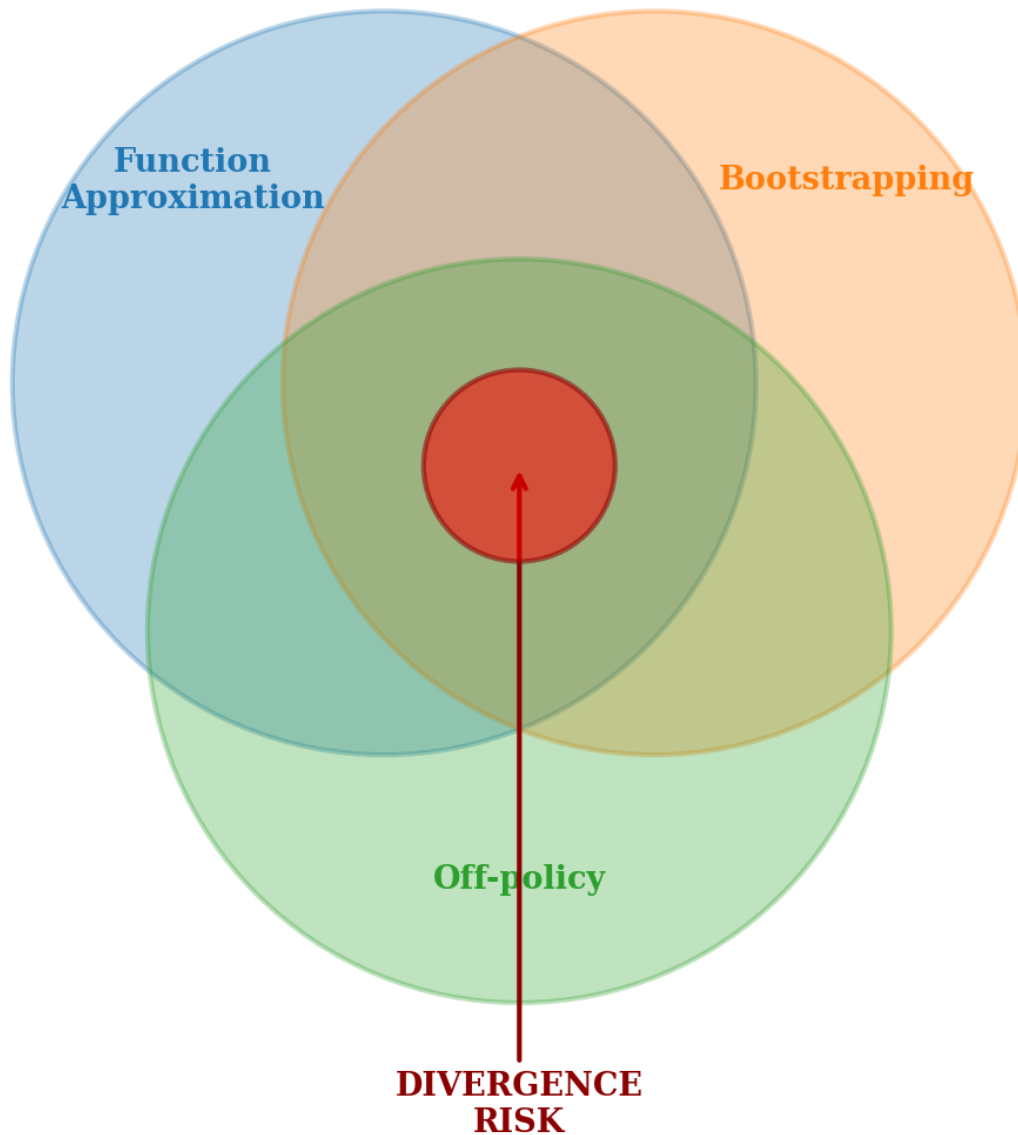


Figure 17: The interaction of bootstrapping, off-policy learning, and function approximation

Neural value functions

A neural network replaces hand-designed features with a learned representation:

$$\hat{v}(s; \theta) = f_{\theta}(s).$$

Advantages:

- high representational capacity;
- automatic feature learning;
- compatibility with images and other high-dimensional observations.

New difficulties:

- the target moves as the network changes;
- sequential samples are correlated;
- one update changes predictions at many states;
- optimization is nonconvex;
- shared representations can couple actor and critic errors.

DQN was successful because it combined Q-learning with stabilizers aimed directly at these problems.

8. Deep Q-Networks and Extensions

Intuition first: DQN is Q-learning with a stabilization system

DQN approximates action values with a neural network:

$$Q(s, a; \theta) \approx q_*(s, a).$$

The control principle is still Q-learning. The breakthrough was a training architecture that made neural bootstrapping usable on high-dimensional observations.

DQN target and loss

For transition (s, a, r, s', d) , define

$$y = r + \gamma(1 - d) \max_{a'} Q(s', a'; \theta^-),$$

where θ^- are target-network parameters.

The regression loss is

$$L(\theta) = \frac{1}{2} \mathbb{E}_{(s, a, r, s', d) \sim \mathcal{D}} [y - Q(s, a; \theta)]^2.$$

Treating y as a stop-gradient,

$$\nabla_{\theta} L = -\mathbb{E} [\delta \nabla_{\theta} Q(s, a; \theta)],$$

where

$$\delta = y - Q(s, a; \theta).$$

Deep Q-Network (DQN): online net, replay buffer, and target network

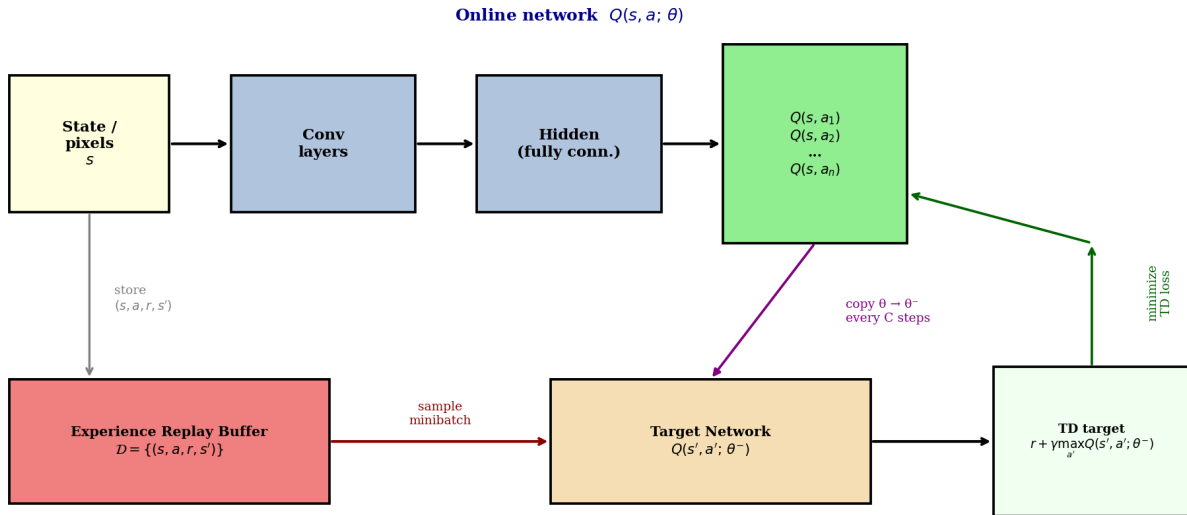


Figure 18: Deep Q-network architecture with replay and a target network

Gradient descent therefore gives the semi-gradient update

$$\theta \leftarrow \theta + \alpha \delta \nabla_{\theta} Q(s, a; \theta).$$

Experience replay

A replay buffer stores transitions and samples minibatches approximately uniformly:

$$(s, a, r, s', d) \sim \mathcal{D}.$$

Replay helps by:

- reducing short-range temporal correlation;
- reusing expensive experience;
- mixing data from multiple recent policies.

It also introduces a new issue: the buffer is off-policy and potentially stale. Shuffling does not turn the underlying data into truly independent observations.

Target networks

The target network $Q(\cdot, \cdot; \theta^-)$ changes more slowly than the online network.

A hard update is

$$\theta^- \leftarrow \theta \quad \text{every } C \text{ learner steps.}$$

A soft update is

$$\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-.$$

The target network reduces rapid feedback between a prediction and the label generated from that same prediction. It trades a moving target for a stale target.

Flow to keep in mind. An environment transition enters replay. A sampled replay transition feeds the online prediction and the target-network calculation. Their difference forms the loss used to update the online network. A separate periodic or soft parameter copy refreshes the target network.

Why the max causes overestimation

Suppose each action-value estimate is

$$\hat{Q}_a = Q_a + \varepsilon_a,$$

with zero-mean noise. The action selected by $\arg \max_a \hat{Q}_a$ is more likely to have a positive error. Reusing the same noisy estimates to evaluate the selected action creates selection bias:

$$\mathbb{E} \left[\max_a \hat{Q}_a \right] \geq \max_a Q_a.$$

The problem is not that every Q estimate is biased upward. It is that maximizing preferentially selects positive noise.

Double DQN

Double DQN decouples selection and evaluation:

$$a^* = \arg \max_{a'} Q(s', a'; \theta),$$

$$y_{\text{DDQN}} = r + \gamma(1 - d)Q(s', a^*; \theta^-).$$

- the online network selects the action;
- the target network evaluates that action.

The networks are correlated, so the bias is reduced rather than mathematically eliminated.

Dueling architecture

Dueling architecture: separate value and advantage streams

$$\textbf{Dueling DQN: } Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right)$$

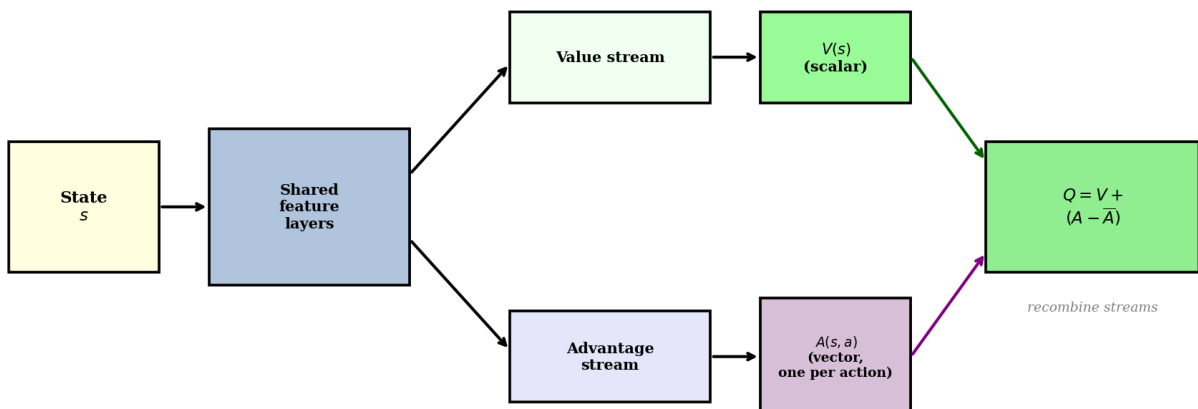


Figure 19: Dueling network decomposition into value and advantage streams

Dueling networks decompose

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a').$$

The subtraction resolves the non-identifiability

$$V(s) + c, \quad A(s, a) - c,$$

which would otherwise produce the same Q-values.

The architecture helps when many actions have similar effects and a strong state-value representation can be learned independently of action-specific differences.

Prioritized replay

Prioritized experience replay samples transitions according to a priority such as

$$p_i = |\delta_i| + \varepsilon.$$

The sampling probability is

$$P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha}.$$

Because this changes the training distribution, use importance weights

$$w_i = \left(\frac{1}{NP(i)} \right)^\beta,$$

often normalized within a minibatch.

Interpretation:

- $\alpha = 0$ gives uniform replay;
- larger α concentrates updates on high-error samples;
- $\beta \rightarrow 1$ increases correction for the sampling bias.

High TD error can indicate useful surprise, but also reward noise, outliers, or a broken target. Prioritization is therefore a data-allocation heuristic, not a proof that every selected sample is informative.

Multi-step targets

Let $m = \min(n, \text{number of transitions until the first true termination})$. A valid n -step target is

$$y_t^{(n)} = \sum_{k=0}^{m-1} \gamma^k R_{t+k+1} + \gamma^m b_{t+m} \max_a Q(S_{t+m}, a; \theta^-),$$

where $b_{t+m} = 0$ at true termination and one at an ordinary bootstrap boundary. The reward sum must not cross an episode reset. At an external time-limit cutoff, bootstrap from the actual final observation if continuation belongs to the objective, but do not concatenate the reset episode's rewards.

Longer targets propagate reward across more steps and reduce the coefficient of the final bootstrap error. Variance need not increase monotonically because reward covariances also matter. Off-policy intermediate actions require the intended algorithm's corrections or an explicitly acknowledged approximation.

Distributional RL

Instead of predicting only

$$Q(s, a) = \mathbb{E}[G_t \mid s, a],$$

distributional RL predicts a return distribution $Z(s, a)$.

For a fixed policy,

$$Z^\pi(s, a) \stackrel{D}{=} R_{t+1} + \gamma(1 - d_{t+1})Z^\pi(S_{t+1}, A_{t+1}),$$

where $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$.

C51 represents the distribution using fixed atoms $z_1, \dots, z_N$:

$$Z(s, a) \approx \sum_{i=1}^N p_i(s, a) \delta_{z_i}.$$

The shifted target distribution is projected back onto the fixed support, and the network is trained with cross-entropy.

The distributional Bellman operator has elegant contraction properties for policy evaluation under suitable probability metrics. Control with greedy action selection is more delicate and should not inherit those guarantees automatically.

Noisy networks

Noisy networks replace or supplement action-space dithering with parameter noise. A linear layer can be written

$$y = (W + \Sigma_W \odot \varepsilon_W)x + (b + \sigma_b \odot \varepsilon_b).$$

A noise sample can coordinate actions across states while it is held fixed. Temporal persistence depends on when the implementation resamples it; resampling every action does not provide episode-long parameter coherence. State the refresh schedule before making an exploration claim.

Rainbow

Rainbow combines six components:

1. Double DQN;
2. dueling architecture;
3. prioritized replay;
4. multi-step returns;
5. distributional value learning;
6. noisy-network exploration.

The components intervene at different points:

Component	Primary intervention
Double DQN	Target selection bias
Dueling	Representation
Prioritized replay	Data sampling
Multi-step returns	Backup depth

Component	Primary intervention
Distributional RL	Prediction target
Noisy networks	Exploration

The original ablations showed environment-dependent contributions. Rainbow is best taught as a systems composition, not as a theorem that every component helps every task.

DQN implementation checklist

A credible DQN implementation should log or test:

- terminal versus truncation masks;
- replay-buffer warm-up and age distribution;
- target-network update schedule;
- exploration schedule;
- reward clipping or scaling;
- TD-error and Q-value distributions;
- evaluation with exploration disabled;
- several independent seeds.

A high training return does not reveal whether the network is calibrated, whether the replay distribution is stale, or whether one seed was unusually lucky.

Common misconceptions

“A neural network simply makes tabular Q-learning generalize”

It also couples estimates, creates nonstationary regression targets, and turns exact asynchronous stochastic approximation into nonlinear optimization.

“Replay makes data i.i.d.”

Replay randomizes minibatch order and reduces local correlation. It does not remove dependence created by shared trajectories, policies, environments, or replay-buffer selection.

“Target networks remove the deadly triad”

They slow one feedback loop. DQN still combines function approximation, bootstrapping, and off-policy replay.

“Double DQN guarantees underestimation-free values”

It reduces max-selection bias but can still overestimate or underestimate because the two networks are correlated and both are imperfect.

Chapter summary

Function approximation changes the problem from estimating independent table entries to finding a self-consistent approximation inside a restricted function class.

The key equations are:

$$\hat{v}(s; w) = w^\top x(s),$$

Rainbow DQN: six components address different failure modes

Conceptual map — not a quantitative reproduction of the paper's ablation



Empirical contributions depend on environment, metric, interactions, and implementation.

Figure 20: Rainbow component map: a schematic, not measured ablation results

$$w_{t+1} = w_t + \alpha \delta_t x(S_t),$$

$$Xw_{\text{TD}} = \Pi_{D_\pi} T^\pi(Xw_{\text{TD}}),$$

and

$$y_{\text{DQN}} = r + \gamma(1 - d) \max_{a'} Q(s', a'; \theta^-).$$

The practical lesson is equally important: deep value learning is not one loss function. It is a coupled data, target, exploration, and optimization system.

Knowledge check and exercises

Conceptual

1. Why does one parameter update affect many states under function approximation?
2. Explain the difference between the best least-squares approximation of v_π and the projected Bellman fixed point.
3. Give an instability involving nonlinear approximation and bootstrapping that is not fully summarized by the deadly triad slogan.
4. Why can high TD error be a bad replay priority?
5. Separate the roles of replay and target networks in DQN.

Derivations

6. Derive the linear semi-gradient TD update.
7. Write the normal equations for weighted least-squares projection onto $\text{col}(X)$.
8. Derive the Double-DQN target and label the selection and evaluation networks.

9. Show why the dueling decomposition is non-identifiable without a centering constraint.
10. Derive normalized importance weights for prioritized replay.

Implementation

11. Compare linear Monte Carlo and linear TD on the same feature set. Report approximation error under the visitation distribution and under a uniform state distribution.
12. Reproduce a small off-policy divergence example or a simplified Baird-style construction.
13. Train DQN with four configurations: neither stabilizer, replay only, target network only, and both. Report failure modes rather than only final scores.
14. Add Double DQN and inspect overestimation by comparing predicted Q-values with empirical returns.
15. Run a replay-age ablation and report how stale data affects learning.

Primary readings

- Sutton and Barto, *Reinforcement Learning: An Introduction*, Chapters 9–11.
- John Tsitsiklis and Benjamin Van Roy, “An Analysis of Temporal-Difference Learning with Function Approximation” (1997).
- Volodymyr Mnih et al., “Human-level control through deep reinforcement learning” (2015).
- Hado van Hasselt, Arthur Guez, and David Silver, “Deep Reinforcement Learning with Double Q-learning” (2016).
- Matteo Hessel et al., “Rainbow: Combining Improvements in Deep Reinforcement Learning” (2018).

Part IV — Policy-Based Methods

Lecture opening: Move probability mass toward useful actions

At state 0 in the running example, ask how a tiny change in the switch probability changes *total* return. It changes both the immediate action and all later state visits. The policy-gradient theorem packages those effects into the occupancy-weighted action value. Its compact form is the end of a derivation, not permission to omit the occupancy convention.

Use A.9–A.11 as the proof sequence: likelihood ratios and causality; the exact baseline condition; finite-rollout GAE; performance difference; then a local trust region. The lab checks the exact gradient against finite differences and verifies termination/truncation masks. PPO enters only after students know which distribution mismatch it tries to limit and which theorem its clipping rule does not enforce.

Learning objectives

By the end of this chapter, you should be able to:

- derive the score-function policy gradient from a trajectory objective;
- explain why reward-to-go and state-dependent baselines reduce variance without changing the expected gradient;
- distinguish a baseline from a bootstrapped critic;
- derive Generalized Advantage Estimation and interpret its bias-variance tradeoff;
- derive the local natural-gradient step and explain the motivation behind TRPO;
- explain exactly what PPO clipping changes—and what it does not guarantee;
- compare stochastic and deterministic actor-critic methods for continuous control;
- derive the central update equations for DDPG, TD3, and Soft Actor-Critic.

Standing assumptions and notation

Unless stated otherwise, we consider a discounted episodic MDP with initial-state distribution ρ_0 , differentiable policy π_θ , bounded rewards, and either a finite horizon or $0 \leq \gamma < 1$.

A trajectory is

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T),$$

and the return from time t is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

The terminal indicator d_{t+1} equals 1 only for a true terminal transition. A time-limit truncation is recorded separately and normally retains a bootstrap term.

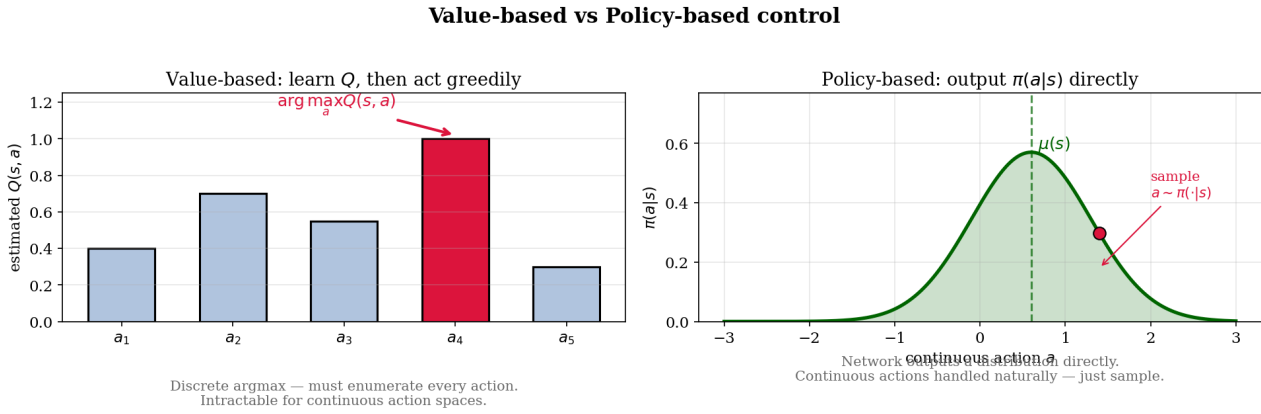


Figure 21: Value-based and policy-based reinforcement learning compared

9. Policy-Gradient Methods

Intuition first: learn the decision rule directly

Value-based methods learn how good actions are and then derive a policy by selecting a high-value action. Policy-based methods parameterize the decision rule itself:

$$\pi_{\theta}(a | s).$$

The parameters are adjusted so that trajectories with larger return become more likely.

This direct approach is especially useful when:

- actions are continuous and computing $\arg \max_a Q(s, a)$ is difficult;
- the desired policy should remain stochastic;
- smooth changes in action probabilities are preferable to abrupt argmax switches;
- the policy must represent structured distributions, such as correlated or bounded controls.

The central question is therefore:

How do we differentiate expected return when changing the policy also changes the distribution of every future state and action?

The policy-gradient theorem supplies the answer.

Policy parameterizations

Discrete actions: softmax policy

Let $h_{\theta}(s, a)$ be an action preference or logit. Define

$$\pi_{\theta}(a | s) = \frac{\exp h_{\theta}(s, a)}{\sum_{a'} \exp h_{\theta}(s, a')}.$$

For numerical stability, implementations subtract the largest logit before exponentiating.

For a linear preference,

$$h_{\theta}(s, a) = \theta^{\top} x(s, a).$$

The score function has the useful form

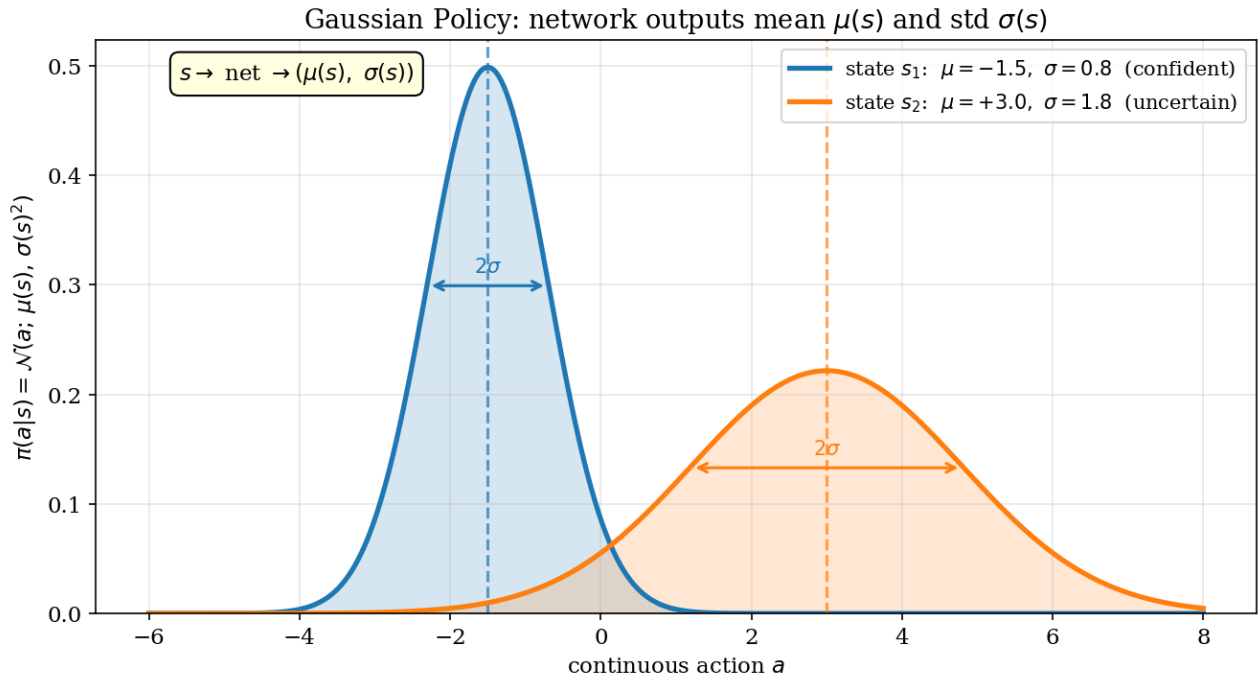


Figure 22: A Gaussian continuous-action policy and the effect of its parameters

$$\nabla_{\theta} \log \pi_{\theta}(a | s) = x(s, a) - \sum_{a'} \pi_{\theta}(a' | s) x(s, a').$$

The gradient therefore increases the selected action's features relative to the policy-weighted average.

Continuous actions: Gaussian policy

For a scalar action, a common policy is

$$A | S = s \sim \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}(s)^2).$$

Its density is

$$\pi_{\theta}(a | s) = \frac{1}{\sigma_{\theta}(s)\sqrt{2\pi}} \exp\left(-\frac{(a - \mu_{\theta}(s))^2}{2\sigma_{\theta}(s)^2}\right).$$

For vector actions, a diagonal Gaussian is common:

$$A | S = s \sim \mathcal{N}(\mu_{\theta}(s), \text{diag}(\sigma_{\theta}(s)^2)).$$

Networks usually output $\log \sigma_{\theta}(s)$ rather than $\sigma_{\theta}(s)$ directly.

For bounded actions, one often samples an unconstrained variable

$$U = \mu_{\theta}(s) + \sigma_{\theta}(s) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

and applies

$$A = \tanh U.$$

The log density must then include the change-of-variables correction:

$$\log \pi_{\theta}(a \mid s) = \log p_{\theta}(u \mid s) - \sum_j \log (1 - \tanh^2 u_j),$$

up to the numerical stabilization used in code.

Warning. Clipping a Gaussian sample to the action bounds is not equivalent to using a correctly normalized bounded distribution. The mismatch can bias likelihood-ratio calculations.

The start-state objective

Define

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1} \right].$$

The trajectory density factorizes as

$$p_{\theta}(\tau) = \rho_0(S_0) \prod_{t=0}^{T-1} \pi_{\theta}(A_t \mid S_t) P(S_{t+1}, R_{t+1} \mid S_t, A_t).$$

Only the policy factors depend on θ . The environment dynamics do not need to be differentiable.

The log-derivative identity

For any positive differentiable density $p_{\theta}(x)$,

$$\nabla_{\theta} p_{\theta}(x) = p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x).$$

This follows from

$$\nabla_{\theta} \log p_{\theta}(x) = \frac{\nabla_{\theta} p_{\theta}(x)}{p_{\theta}(x)}.$$

It converts a derivative of a probability into an expectation involving a score function.

Trajectory-level policy-gradient derivation

Let

$$R(\tau) = \sum_{t=0}^{T-1} \gamma^t R_{t+1}.$$

Then

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \int p_{\theta}(\tau) R(\tau) d\tau \\ &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) R(\tau) d\tau \\ &= \mathbb{E}_{\tau \sim p_{\theta}} [R(\tau) \nabla_{\theta} \log p_{\theta}(\tau)]. \end{aligned}$$

Because the environment terms are independent of θ ,

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

Therefore

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) R(\tau) \right].$$

This estimator is correct but unnecessarily noisy: an action at time t cannot causally influence rewards that occurred before time t .

Causality and reward-to-go

The expected contribution of past rewards vanishes. Replacing total trajectory return with the future return gives

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right].$$

The factor γ^t appears because G_t discounts relative to time t , whereas the start-state objective discounts relative to time 0.

Some texts omit γ^t . They are usually using a different time-weighted objective or sampling convention. Both forms can be valid, but the objective and estimator must be matched explicitly.

Discounted occupancy measure

Use an absorbing zero-reward terminal state and continue the process there. Define the normalized discounted occupancy

$$d_{\pi, \rho_0}^{\gamma}(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \Pr_{\pi}(S_t = s | S_0 \sim \rho_0).$$

It sums to one. For this convention,

$$\nabla_{\theta} J = \frac{1}{1 - \gamma} \sum_s d_{\pi, \rho_0}^{\gamma}(s) \sum_a \pi_{\theta}(a | s) q_{\pi}(s, a) \nabla_{\theta} \log \pi_{\theta}(a | s).$$

Equivalently this is $1/(1 - \gamma)$ times the expectation under $s \sim d_{\pi, \rho_0}^{\gamma}, a \sim \pi_{\theta}$. If an unnormalized occupancy $m_{\pi} = \sum_t \gamma^t \Pr(S_t = \cdot)$ is used, write the finite sum against m_{π} with no prefactor; it is not a probability distribution. A.9 supplies the complete score-function proof and the conditions justifying differentiation of the infinite-horizon objective.

Why the theorem matters

The occupancy distribution depends on θ through every earlier action. Yet the final expression does not require differentiating the transition dynamics or explicitly differentiating the occupancy measure. Sampling trajectories automatically weights states according to how often the current policy reaches them.

This is powerful, but it also creates an important limitation:

The gradient improves performance under the states the current policy visits. It does not directly optimize behavior in states the current policy never reaches.

REINFORCE

REINFORCE replaces $q_\pi(S_t, A_t)$ with the sampled return G_t :

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t | S_t).$$

A complete episodic batch update is:

```
for each update:
    theta_old = a frozen copy of theta
    sample B independent episodes from pi(theta_old)
    g = 0
    for each episode:
        compute all reward-to-go values G_t backward
        for t = 0, ..., T - 1:
            score = grad log pi(theta_old)(A_t | S_t)
            g += gamma**t * G_t * score / B
        theta = theta_old + alpha * g
```

The score is evaluated at the behavior parameter for every term, and there is one accumulated update. Updating parameters inside the timestep loop would generally use later scores at a different policy than the one that generated the data. For the theorem in A.9, use a fixed finite horizon or the bounded discounted infinite-horizon setting with its summability conditions; random undiscounted episodes need their own integrability conditions. REINFORCE's high variance is an estimation issue, not a license to change the gradient silently.

A one-state example

Suppose a one-state bandit has two actions:

$$\mathbb{E}[R | a_1] = 3, \quad \mathbb{E}[R | a_2] = 1.$$

Let the policy be logistic:

$$\pi_\theta(a_1) = \sigma(\theta), \quad \pi_\theta(a_2) = 1 - \sigma(\theta).$$

Then

$$J(\theta) = 3\sigma(\theta) + 1(1 - \sigma(\theta)) = 1 + 2\sigma(\theta),$$

so

$$\nabla_\theta J(\theta) = 2\sigma(\theta)(1 - \sigma(\theta)) > 0.$$

The score-function estimator gives the same expected direction:

$$\mathbb{E}[R \nabla_\theta \log \pi_\theta(A)] = \nabla_\theta J(\theta).$$

Individual samples are noisy, but their average points toward increasing the probability of a_1 .

Variance reduction with state-dependent baselines

For any function $b(s)$ that does not depend on the sampled action,

$$\mathbb{E}_{A \sim \pi_\theta(\cdot | s)} [b(s) \nabla_\theta \log \pi_\theta(A | s)] = 0.$$

Proof:

$$\begin{aligned}
 \sum_a \pi_\theta(a | s) b(s) \nabla_\theta \log \pi_\theta(a | s) &= b(s) \sum_a \nabla_\theta \pi_\theta(a | s) \\
 &= b(s) \nabla_\theta \sum_a \pi_\theta(a | s) \\
 &= b(s) \nabla_\theta 1 \\
 &= 0.
 \end{aligned}$$

Therefore

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_t \gamma^t (G_t - b(S_t)) \nabla_\theta \log \pi_\theta(A_t | S_t) \right].$$

A poor state-only baseline can fail to reduce variance, but it does not bias the expected gradient.

Why the value function is a natural baseline

Choosing

$$b(s) = v_\pi(s)$$

gives the advantage

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s).$$

The advantage asks:

Was this action better or worse than what the policy usually achieves from this state?

The value baseline removes the large state-dependent component of return that is common across actions.

Numerical intuition

Suppose two actions yield returns near 100:

- a_1 : return 101;
- a_2 : return 99.

Raw returns are large and both are positive. With baseline $b = 100$, the learning signals become $+1$ and -1 , exposing the action-relevant difference while removing the common level.

The minimum-variance scalar baseline

For score vector

$$z = \nabla_\theta \log \pi_\theta(A | s),$$

one scalar state baseline that minimizes the conditional second moment is

$$b^*(s) = \frac{\mathbb{E} [q_\pi(s, A) \|z\|^2 | S = s]}{\mathbb{E} [\|z\|^2 | S = s]}.$$

This is not generally identical to $v_\pi(s)$ because actions with larger score norms contribute more to gradient variance. The value function remains a practical and usually effective approximation.

Baseline versus critic: the crucial distinction

Consider two estimators.

Monte Carlo return with a baseline

$$\hat{g}_t = \gamma^t (G_t - b(S_t)) \nabla_\theta \log \pi_\theta(A_t | S_t).$$

As long as b is state-only, the estimator remains unbiased even if b is inaccurate.

Bootstrapped TD advantage

$$\hat{A}_t = R_{t+1} + \gamma(1 - d_{t+1})\hat{V}(S_{t+1}) - \hat{V}(S_t).$$

Here the critic enters the target. If $\hat{V}$ is wrong, the expected estimate need not equal the true advantage.

Important. “The critic is only a baseline” is true for subtracting a state-only estimate from an unbiased return. It is not generally true once the critic appears inside a bootstrapped target.

The statistical baseline condition. A fixed state function is a valid baseline because it is known before the current action is sampled. A state-only *fitted* prediction can still depend on that action through the fitting data. Stop-gradient does not remove this dependence. A.9 states the conditional-independence condition, and A.14 proves the finite-group self-centering factor. Use a frozen prior critic or an appropriate held-out construction when exact unbiasedness is the claim.

Actor-critic methods

Actor-Critic: the TD error δ_t drives both updates

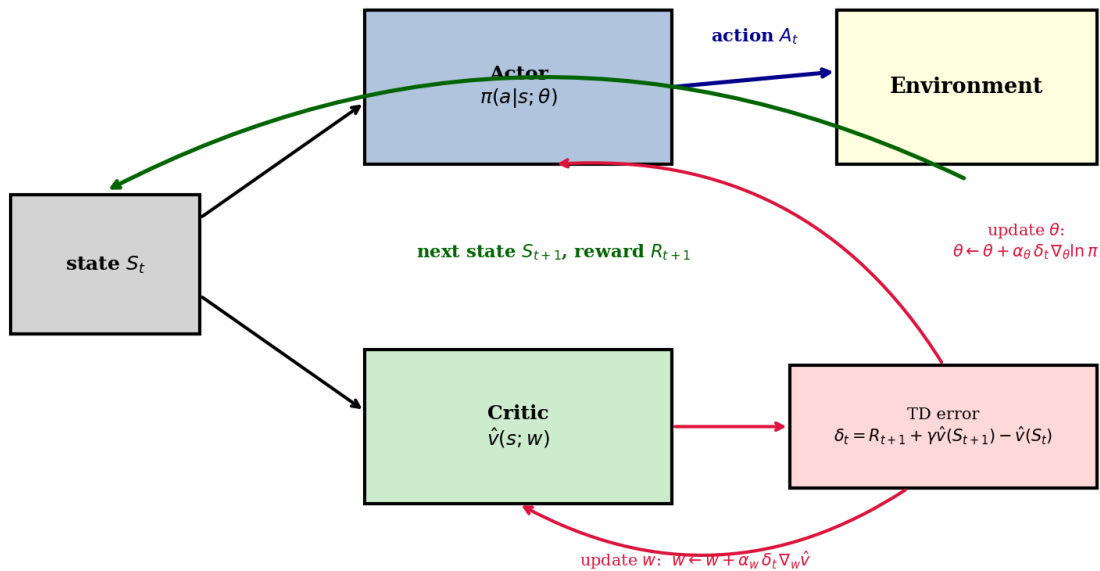


Figure 23: The actor-critic learning loop

Actor-critic maintains:

- an **actor** $\pi_\theta(a \mid s)$;
- a **critic** $V_w(s)$ or $Q_w(s, a)$.

For a state-value critic, define

$$\delta_t = R_{t+1} + \gamma(1 - d_{t+1})V_w(S_{t+1}) - V_w(S_t).$$

A one-step update is

$$w \leftarrow w + \alpha_w \delta_t \nabla_w V_w(S_t),$$

$$\theta \leftarrow \theta + \alpha_\theta \gamma^t \delta_t \nabla_\theta \log \pi_\theta(A_t \mid S_t).$$

When $V_w = v_\pi$ exactly,

$$\begin{aligned} \mathbb{E}[\delta_t \mid S_t = s, A_t = a] &= r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) v_\pi(s') - v_\pi(s) \\ &= q_\pi(s, a) - v_\pi(s) \\ &= A_\pi(s, a). \end{aligned}$$

An approximate critic lowers variance and enables online updates, but can introduce bias.

Why actor-critic can be faster than REINFORCE

- It updates before an episode ends.
- It replaces a full random return with a shorter-horizon estimate.
- It can reuse learned state structure through the critic.
- It naturally supports continuing tasks.

The cost is dependence on critic quality and the stability of joint actor-critic optimization.

Compatible function approximation

In classical policy-gradient theory, a specially structured advantage approximator can preserve the exact natural-gradient direction. Let

$$f_\theta(s, a) = \nabla_\theta \log \pi_\theta(a \mid s)$$

and approximate

$$A_\pi(s, a) \approx w^\top f_\theta(s, a).$$

Under the compatibility and least-squares conditions, w estimates a Fisher-preconditioned gradient. This result motivates natural actor-critic methods, but generic neural critics do not automatically satisfy the compatibility conditions.

Generalized Advantage Estimation — bias-variance tradeoff via λ

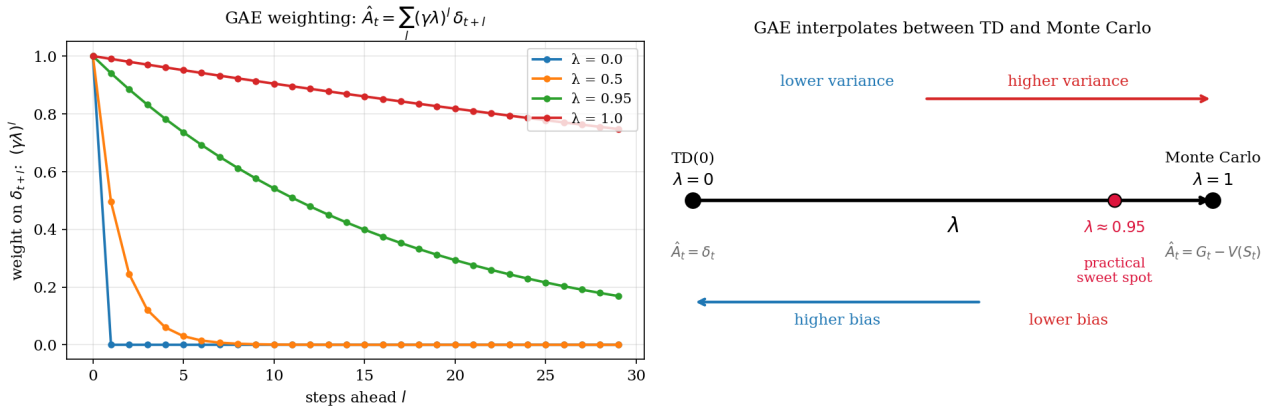


Figure 24: How the GAE lambda parameter trades bias against variance

10. Advanced Policy Optimization

Generalized Advantage Estimation

Intuition first: choose how far credit should travel

One-step TD has low variance but relies heavily on the critic. Monte Carlo uses actual rewards to the end of the episode but has higher variance. Generalized Advantage Estimation, or GAE, interpolates between them.

Define the masked TD residual

$$\delta_t = R_{t+1} + \gamma(1 - d_{t+1})V(S_{t+1}) - V(S_t).$$

The k -step advantage estimate can be written as

$$\hat{A}_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}.$$

For example,

$$\hat{A}_t^{(1)} = \delta_t,$$

and

$$\hat{A}_t^{(2)} = \delta_t + \gamma\delta_{t+1}.$$

GAE forms an exponentially weighted average of these horizons:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}.$$

The backward recursion is

$$\hat{A}_t = \delta_t + \gamma\lambda c_t \hat{A}_{t+1}.$$

Here c_t is one only when the next stored transition continues the same episode segment. It is zero at termination, reset, or storage boundary. In δ_t , the separate bootstrap mask is $b_t = 1 - \mathbf{1}\{\text{true termination}\}$. At an external truncation with a reset, $b_t = 1$ and $c_t = 0$: bootstrap from the real final observation, but stop the trace. A.10 derives the finite-segment identity and the exact propagation of critic error; the lab tests both masks.

Deriving GAE from λ -weighted returns

Let $G_t^{(k)}$ denote the k -step bootstrapped return. Define

$$G_t^\lambda = (1 - \lambda) \sum_{k=1}^{T-t-1} \lambda^{k-1} G_t^{(k)} + \lambda^{T-t-1} G_t^{(T-t)}.$$

Then

$$G_t^\lambda - V(S_t) = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}.$$

Thus GAE is the advantage form of the λ -return.

Interpreting λ

	λ	Main source of signal	Typical statistical effect
	0	one-step TD residual	lower variance, greater critic dependence
	between 0 and 1	mixture of horizons	tunable compromise
	1	full return minus baseline on complete episodes	less bootstrap bias, higher sampling variance

A rough effective trace horizon is

$$H_{\text{GAE}} \approx \frac{1}{1 - \gamma\lambda}.$$

For $\gamma = 0.99$ and $\lambda = 0.95$,

$$H_{\text{GAE}} \approx \frac{1}{1 - 0.9405} \approx 16.8.$$

This does not mean all influence stops after 17 steps; it summarizes the geometric decay scale.

GAE implementation cautions

- Tune λ with rollout length and γ .
- Preserve bootstrapping at time-limit truncations.
- Inspect raw advantages before normalizing them.
- Reusing one rollout for many policy epochs makes its advantages stale relative to the new policy.
- An inaccurate critic can bias low- λ estimates substantially.

Information geometry and natural gradients

Why Euclidean parameter distance is misleading

Two small parameter changes can have very different effects on the policy distribution. Neural-network parameters are not a behaviorally uniform coordinate system.

The local KL divergence provides a policy-space metric. For a small step $\Delta\theta$,

$$\mathbb{E}_s [D_{\text{KL}}(\pi_\theta(\cdot | s) \parallel \pi_{\theta+\Delta\theta}(\cdot | s))] \approx \frac{1}{2} \Delta\theta^\top F \Delta\theta,$$

where F is the Fisher information matrix:

$$F = \mathbb{E}_{s,a} [\nabla_\theta \log \pi_\theta(a | s) \nabla_\theta \log \pi_\theta(a | s)^\top].$$

Deriving the natural-gradient direction

Locally approximate improvement by

$$J(\theta + \Delta\theta) \approx J(\theta) + g^\top \Delta\theta, \quad g = \nabla_\theta J(\theta).$$

Choose the best linearized improvement inside a local KL ball:

$$\max_{\Delta\theta} g^\top \Delta\theta$$

subject to

$$\frac{1}{2} \Delta\theta^\top F \Delta\theta \leq \delta.$$

The Lagrangian is

$$\mathcal{L}(\Delta\theta, \eta) = g^\top \Delta\theta - \eta \left(\frac{1}{2} \Delta\theta^\top F \Delta\theta - \delta \right).$$

Setting the derivative with respect to $\Delta\theta$ to zero gives

$$g - \eta F \Delta\theta = 0,$$

so

$$\Delta\theta \propto F^{-1}g.$$

The direction

$$\boxed{\widetilde{\nabla}_\theta J = F^{-1} \nabla_\theta J}$$

is the **natural gradient**.

Singular Fisher matrices. The inverse notation above assumes positive definiteness. If F is singular and g has a component in its nullspace, the linear objective with only the quadratic constraint is unbounded. If g lies in the range, use a pseudoinverse on that range. With positive definite F , the exact constrained step is $\sqrt{2\delta/(g^\top F^{-1}g)} F^{-1}g$ for $g \neq 0$. A.11 proves this and the maximum-state-TV performance bound. Damping and a sampled average-KL line search are practical modifications, not the exact theorem.

Trust Region Policy Optimization

Intuition first: do not outrun the evidence

A policy-gradient estimate is justified by data from the old policy. If the new policy moves too far, the sampled advantages may no longer describe what the new policy will experience.

TRPO maximizes an importance-weighted surrogate while constraining policy change.

Let

$$r_t(\theta) = \frac{\pi_\theta(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

The practical surrogate is

$$L_{\theta_{\text{old}}}(\theta) = \mathbb{E}_t [r_t(\theta) \hat{A}_t].$$

TRPO approximately solves

$$\max_{\theta} L_{\theta_{\text{old}}}(\theta)$$

subject to

$$\mathbb{E}_t [D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | S_t) \| \pi_\theta(\cdot | S_t))] \leq \delta.$$

Theory boundary for TRPO

The monotonic-improvement motivation begins with a bound involving:

- exact advantages under the old policy;
- a surrogate evaluated over the old state distribution;
- a worst-case policy divergence over states.

Practical TRPO replaces these with sampled advantages, sampled states, an average-KL approximation, conjugate-gradient solves, and line search. It inherits a strong design rationale, not an unconditional guarantee for arbitrary neural implementations.

How TRPO is implemented

1. Estimate the policy gradient g from rollout data.
2. Compute Fisher-vector products without forming F explicitly.
3. Use conjugate gradient to approximate $F^{-1}g$.
4. Scale the step to satisfy the quadratic KL constraint.
5. Backtrack with line search until the sampled surrogate improves and the sampled KL is acceptable.

This is more complex than ordinary first-order optimization, motivating PPO.

Proximal Policy Optimization

PPO uses the same probability ratio but replaces the explicit constrained solve with a clipped surrogate.

The PPO-Clip objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right].$$

Reading the clipped objective correctly

PPO Clipped Surrogate L^{CLIP} ($\epsilon = 0.2$): flat incentive, not a hard policy constraint

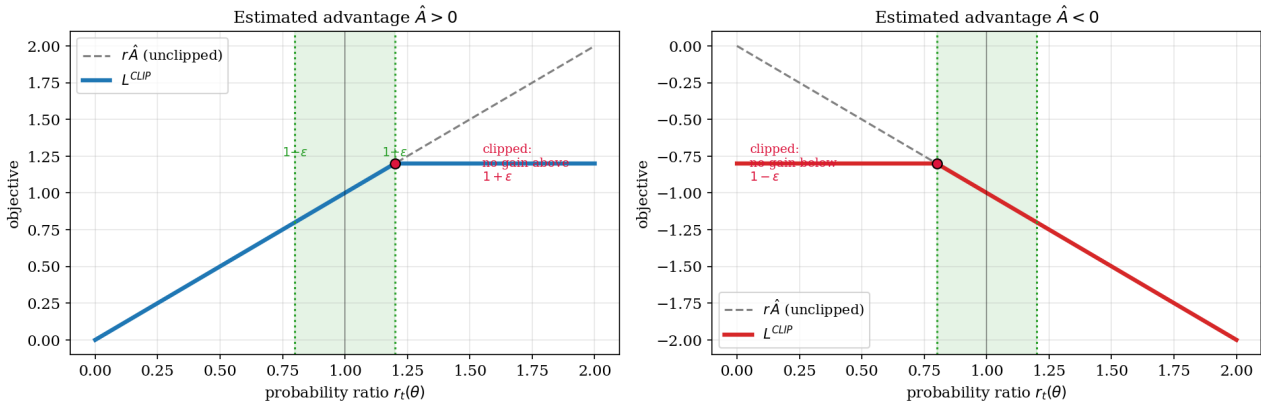


Figure 25: The piecewise geometry of the PPO clipped surrogate objective

Positive advantage

When $\hat{A}_t > 0$, increasing the sampled action probability is favorable. The objective stops increasing once

$$r_t > 1 + \epsilon.$$

Negative advantage

When $\hat{A}_t < 0$, decreasing the sampled action probability is favorable. The objective stops improving once

$$r_t < 1 - \epsilon.$$

Clipping removes local incentive to push the sampled ratio farther in the favorable direction. It does not project the new policy into a hard ratio interval.

What PPO clipping does not guarantee

PPO clipping does not ensure:

- a small global KL divergence;
- bounded changes to unsampled actions;
- bounded policy change at unseen states;
- unbiased advantage estimates;
- monotonic improvement;
- immunity to repeated-epoch overfitting;
- protection from reward misspecification.

Shared neural parameters can change many probabilities even when the sampled term is flat. Multiple minibatch epochs can also accumulate a large policy change.

Important. PPO is best understood as a monitored training protocol, not merely one equation.

PPO-Penalty

An alternative objective directly penalizes KL drift:

$$L^{\text{KPEN}}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t - \beta D_{\text{KL}} \left(\pi_{\theta_{\text{old}}}(\cdot | S_t) \parallel \pi_{\theta}(\cdot | S_t) \right) \right].$$

A controller can increase β when KL is too large and decrease it when KL is too small.

The full PPO loss

Implementations often maximize

$$L^{\text{CLIP}} - c_v L^V + c_H \mathcal{H}(\pi_{\theta}),$$

where

$$L^V = \frac{1}{2} \left(V_{\phi}(S_t) - \hat{V}_t^{\text{target}} \right)^2$$

and

$$\mathcal{H}(\pi_{\theta}(\cdot | s)) = -\mathbb{E}_{A \sim \pi_{\theta}} [\log \pi_{\theta}(A | s)].$$

If the code minimizes a loss, the signs are reversed.

Optional implementation choices include:

- value-function clipping;
- advantage normalization;
- reward or return normalization;
- shared versus separate actor and critic networks;
- target-KL early stopping;
- gradient clipping;
- entropy schedules.

These are part of the empirical method and should be reported.

PPO as a monitored protocol

```

for each policy update:
    collect fresh rollouts with pi_old
    compute returns and GAE advantages
    log raw return, value error, and advantage distributions

    optionally normalize advantages for optimization

    for epoch in 1, ..., K:
        shuffle rollout sequences or transitions as appropriate

        for minibatch in rollout batch:
            compute sampled ratios pi_theta / pi_old
            compute clipped policy surrogate
            compute value loss and entropy bonus
            take an optimizer step with gradient clipping

        estimate KL(pi_old || pi_theta)
        log ratio percentiles and clip fraction

    if sampled KL exceeds the target:

```


stop reusing this rollout batch

evaluate on independent episodes

Useful diagnostics include:

Diagnostic	Interpretation
approximate KL	aggregate drift on sampled states
ratio median, p95, p99	tail behavior hidden by a mean
clip fraction	fraction of samples with saturated surrogate
policy entropy	exploration and support collapse
action standard deviation	continuous-policy spread
value-versus-return calibration	critic reliability
advantage percentiles	outlier-driven updates

A two-action clipping counterexample

Suppose at one state the old policy is

$$\pi_{\text{old}} = (0.999, 0.001).$$

A finite batch may contain only the common action. The new policy could become

$$\pi_{\text{new}} = (0.99, 0.01),$$

increasing the rare action tenfold. If the rare action was not sampled, its change is absent from the sampled ratio terms, even though the full distribution changed materially relative to its old rare-event probability.

This is why sampled clipping must be accompanied by distribution-level diagnostics.

Deterministic policy gradients

Motivation

A stochastic policy integrates over actions in the policy-gradient theorem. For high-dimensional continuous action spaces, a deterministic actor can instead output one action directly:

$$a = \mu_{\theta}(s).$$

For the discounted occupancy induced by μ_{θ} , the deterministic policy-gradient theorem has the form

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{S \sim d^{\mu}} \left[\nabla_{\theta} \mu_{\theta}(S) \nabla_a Q^{\mu}(S, a) \Big|_{a=\mu_{\theta}(S)} \right].$$

The chain rule differentiates through the critic with respect to the action.

The theorem still requires a correct action-value gradient in the relevant state-action region. A learned critic can be wrong precisely where the actor tries to exploit it.

Deep Deterministic Policy Gradient

DDPG combines:

- deterministic actor $\mu_{\theta}(s)$;
- critic $Q_{\phi}(s, a)$;

- replay buffer;
- slowly updated target networks;
- external exploration noise.

The critic target is

$$y_t = R_{t+1} + \gamma(1 - d_{t+1})Q_{\bar{\phi}}(S_{t+1}, \mu_{\bar{\theta}}(S_{t+1})).$$

The critic minimizes

$$L_Q(\phi) = \mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}} \left[\frac{1}{2} (Q_{\phi}(s, a) - y)^2 \right].$$

The actor maximizes predicted value:

$$J_{\mu}(\theta) = \mathbb{E}_{s \sim \mathcal{D}} [Q_{\phi}(s, \mu_{\theta}(s))].$$

Thus

$$\nabla_{\theta} J_{\mu} = \mathbb{E}_s \left[\nabla_{\theta} \mu_{\theta}(s) \nabla_a Q_{\phi}(s, a) \Big|_{a=\mu_{\theta}(s)} \right].$$

Why DDPG can be brittle

- The actor is trained on critic extrapolations.
- A single critic can overestimate narrow action regions.
- Deterministic policies need an explicit exploration process.
- Replay data can be stale relative to the current actor.
- Small implementation changes can alter stability substantially.

Twin Delayed DDPG

TD3 addresses three coupled failure modes.

1. Twin critics and clipped double Q

Maintain Q_{ϕ_1} and Q_{ϕ_2} and use

$$\min_{i=1,2} Q_{\bar{\phi}_i}$$

in the target. This reduces attraction to critic-specific positive errors, while potentially introducing pessimism.

2. Target-policy smoothing

Add clipped noise to the target action:

$$\tilde{a}' = \mu_{\bar{\theta}}(s') + \text{clip}(\varepsilon, -c, c), \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I).$$

The target is

$$y = r + \gamma(1 - d) \min_{i=1,2} Q_{\bar{\phi}_i}(s', \tilde{a}').$$

Smoothing discourages the actor from exploiting extremely narrow value spikes.

3. Delayed actor updates

Update critics more frequently than the actor and target networks. The actor then responds to a somewhat better-fitted critic.

Soft Actor-Critic

Intuition first: optimize reward while retaining alternatives

SAC maximizes expected reward plus policy entropy:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t (R_{t+1} + \alpha \mathcal{H}(\pi(\cdot | S_t))) \right].$$

The temperature $\alpha > 0$ controls the reward-entropy tradeoff.

Equivalent per-action notation is

$$R_{t+1} - \alpha \log \pi(A_t | S_t).$$

Soft value functions

For a policy π ,

$$Q^{\pi}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' | s, a} [V^{\pi}(s')],$$

where

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot | s)} [Q^{\pi}(s, a) - \alpha \log \pi(a | s)].$$

For an unrestricted policy class and a normalizable action density, the optimal soft policy satisfies

$$\pi^*(a | s) = \frac{\exp(Q^*(s, a)/\alpha)}{\int \exp(Q^*(s, a')/\alpha) da'}.$$

Thus α acts like a temperature:

- small α : concentrated near high- Q actions;
- large α : more diffuse.

SAC critic target

Modern SAC commonly uses two critics. Sample

$$a' \sim \pi_{\theta}(\cdot | s')$$

and define

$$y = r + \gamma(1 - d) \left[\min_{i=1,2} Q_{\bar{\phi}_i}(s', a') - \alpha \log \pi_{\theta}(a' | s') \right].$$

Each critic minimizes a squared Bellman error against this target.

SAC actor objective

The actor minimizes

$$J_{\pi}(\theta) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta}} \left[\alpha \log \pi_{\theta}(a | s) - \min_{i=1,2} Q_{\phi_i}(s, a) \right].$$

With a reparameterized action

$$a = f_{\theta}(\varepsilon; s), \quad \varepsilon \sim \mathcal{N}(0, I),$$

gradients flow through both the sampled action and its log density.

Automatic temperature tuning

Choose a target entropy $\mathcal{H}_{\text{target}}$. One common temperature objective is

$$J(\alpha) = \mathbb{E}_{a \sim \pi_{\theta}} \left[-\alpha (\log \pi_{\theta}(a | s) + \mathcal{H}_{\text{target}}) \right].$$

Optimizing $\log \alpha$ rather than α keeps the temperature positive.

Comparing major policy-optimization methods

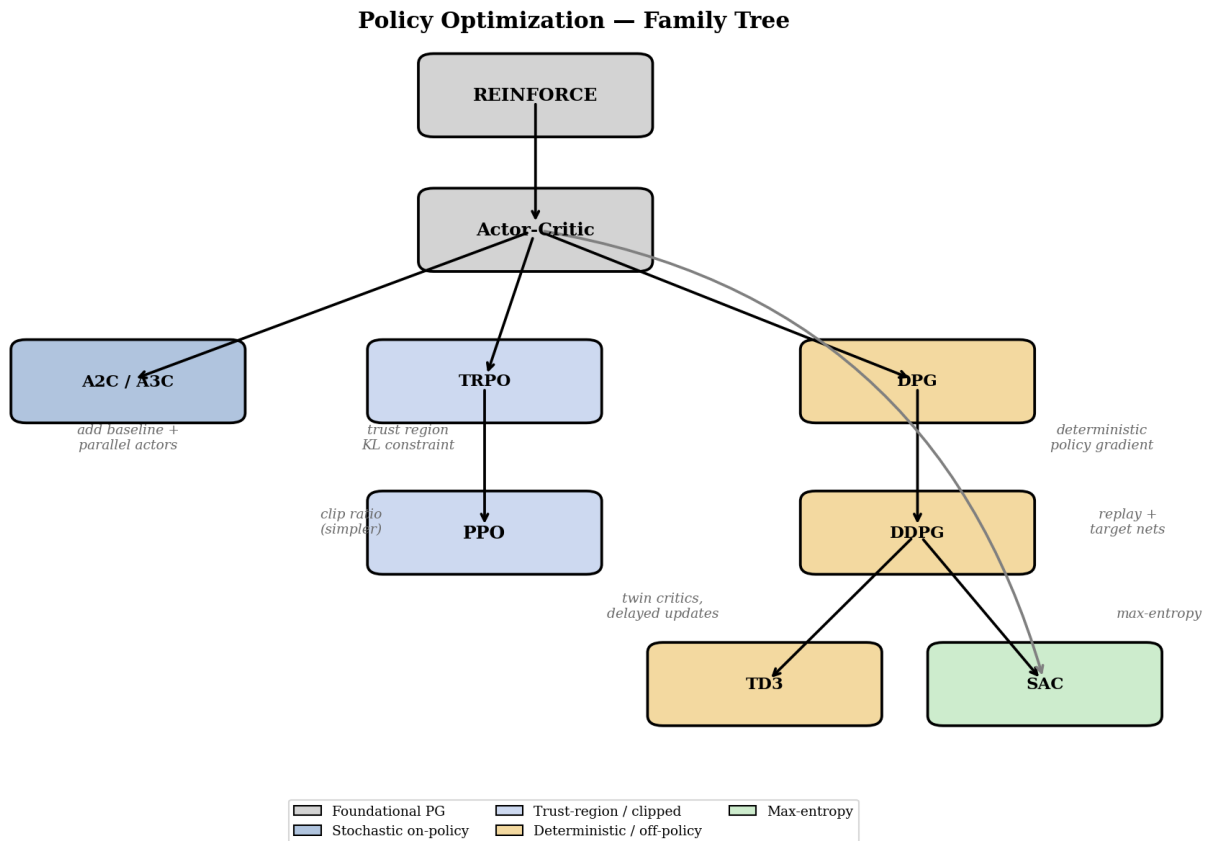


Figure 26: Family tree of major policy-gradient and actor-critic algorithms

Method	Policy	Data use	Main critic target	Main policy constraint or regularizer	Typical concern
REINFORCE	stochastic	on-policy	full return	none	high variance
Actor-Critic	stochastic	usually on-policy	TD or n -step	optional entropy	critic bias

Method	Policy	Data use	Main critic target	Main policy constraint or regularizer	Typical concern
TRPO	stochastic	on-policy batch	GAE/critic	approximate KL constraint	complex second-order solve
PPO	stochastic	repeated on-policy batch	GAE/critic	clipped ratio, often KL monitoring	stale-batch overfitting
DDPG	deterministic	replay, off-policy	one critic bootstrap	target networks	Q extrapolation and brittleness
TD3	deterministic	replay, off-policy	minimum of two critics	target smoothing, delayed actor	pessimism and support mismatch
SAC	stochastic	replay, off-policy	entropy-regularized double critic	entropy temperature	critic/reward-scale sensitivity

Choosing an algorithm

A useful starting point is:

- **small or pedagogical episodic problem:** REINFORCE or simple actor-critic;
- **general on-policy baseline:** PPO with complete diagnostics;
- **continuous control with expensive interactions:** SAC or TD3;
- **when a hard update-size rationale matters:** TRPO or an explicitly KL-constrained method;
- **when the policy must remain meaningfully stochastic:** stochastic actor-critic or SAC.

No row is a universal winner. The environment, reward scale, data budget, action geometry, and implementation quality matter.

Common misconceptions

“Policy gradients do not need value functions”

The theorem does not require a value function, but practical variance reduction often does.

“A baseline must be accurate to preserve unbiasedness”

A fixed or pre-action-measurable state-only baseline can be inaccurate and still leave the expected Monte Carlo policy gradient unchanged. A baseline fitted on the same action-return samples needs the additional care in A.9.

“Every actor-critic update is an unbiased policy gradient”

No. Approximate bootstrapped advantages can be biased.

“PPO constrains every probability ratio to $[1 - \epsilon, 1 + \epsilon]$ ”

No. It clips a sampled surrogate term. The optimized network can still move outside that interval.

“Natural gradient means multiplying by Adam’s adaptive preconditioner”

No. The natural gradient uses the Fisher geometry of the policy distribution.

“Entropy is only an exploration trick”

Entropy also changes the optimization objective and preserves policy support.

“Two critics measure true uncertainty”

Their disagreement can be useful, but shared data and architecture can create shared errors.

Chapter summary

1. The score-function identity converts derivatives of trajectory probabilities into sampleable policy gradients.
 2. Reward-to-go removes causally irrelevant past rewards.
 3. State-only baselines reduce variance without changing the expected Monte Carlo gradient.
 4. Bootstrapped critics trade variance reduction for possible bias.
 5. GAE controls how strongly the actor relies on short-horizon critic estimates versus longer sampled returns.
 6. Natural gradients measure step size in policy space through the Fisher matrix.
 7. TRPO approximately constrains KL drift; PPO clips a sampled surrogate and therefore needs explicit monitoring.
 8. DDPG, TD3, and SAC use critics to optimize continuous-action policies, making critic reliability and action support central concerns.
-

Knowledge check

1. Why does the derivative of the environment transition probability not appear in the score-function policy gradient?
2. Under the start-state discounted objective, why does REINFORCE contain $\gamma^t G_t$ rather than only G_t ?
3. Prove that subtracting any state-only baseline preserves the expected policy gradient.
4. Give an example in which an inaccurate baseline is harmless for bias but an inaccurate bootstrap target is not.
5. Derive the backward recursion for GAE.
6. Starting from a local KL constraint, derive the natural-gradient direction.
7. Explain why PPO clipping does not imply a hard bound on full-policy KL.
8. Identify the three separate stabilizers introduced by TD3.
9. Derive the SAC actor objective from the Boltzmann policy-improvement target.

Exercises**1. Exact gradient versus REINFORCE**

For a two-action softmax bandit with known reward means:

1. derive $J(\theta)$ analytically;
2. derive its exact gradient;
3. estimate the gradient with REINFORCE;
4. compare estimator variance with and without a learned baseline.

2. GAE bias-variance study

On a small episodic MDP:

1. compute exact v_π and A_π ;
2. corrupt the critic by a controlled amount;
3. estimate advantages with $\lambda \in \{0, 0.5, 0.9, 0.95, 1\}$;
4. report bias, variance, and mean-squared error separately.

3. PPO diagnostic lab

Train PPO on a small continuous-control task and log after every optimization epoch:

- approximate KL;
- ratio median, p95, and p99;
- clip fraction;
- raw and normalized advantage percentiles;
- policy entropy and action standard deviation;
- critic prediction versus empirical return.

Repeat with different numbers of epochs per rollout batch. Explain any collapse in terms of estimator staleness rather than only final reward.

4. Rare-action counterexample

Construct a categorical policy in which:

- every sampled action ratio lies inside the clipping interval;
- one unsampled action changes probability by at least a factor of ten;
- the full KL is materially different from what the sampled ratios suggest.

5. TD3 versus SAC

On the same continuous-control environment and interaction budget:

1. match network sizes and replay settings;
2. compare TD3 and SAC across independent seeds;
3. plot critic disagreement, action spread, and return;
4. explain whether differences arise from entropy, target construction, or optimization stability.

Primary readings

- Williams (1992), **Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning**.
- Sutton, McAllester, Singh, and Mansour (2000), **Policy Gradient Methods for Reinforcement Learning with Function Approximation**.
- Silver et al. (2014), **Deterministic Policy Gradient Algorithms**.
- Schulman et al. (2015), **Trust Region Policy Optimization**.
- Schulman et al. (2016), **High-Dimensional Continuous Control Using Generalized Advantage Estimation**.
- Schulman et al. (2017), **Proximal Policy Optimization Algorithms**.
- Lillicrap et al. (2015), **Continuous Control with Deep Reinforcement Learning**.
- Fujimoto, van Hoof, and Meger (2018), **Addressing Function Approximation Error in Actor-Critic Methods**.
- Haarnoja et al. (2018), **Soft Actor-Critic and Soft Actor-Critic Algorithms and Applications**.

Next: Part V — Model-Based Reinforcement Learning.

Part V — Model-Based Reinforcement Learning

Lecture opening: A planner can exploit its own simulator

Imagine learning a model of the running environment that mistakenly predicts a large reward for switching back and forth. A more capable planner may discover that error more reliably than a weaker planner. Therefore predictive accuracy and control usefulness cannot be judged independently of the policy that will query the model.

A.12 decomposes the issue into reward error, transition error, and planning suboptimality. Use the bound as an assumption audit, not as a promise that average validation loss certifies a deployed controller. Dyna, short model rollouts, latent world models, and search differ in where they use simulated futures and how much error they expose themselves to. The chapter’s architecture diagrams are explanatory schematics, not benchmark comparisons.

Learning objectives

By the end of this chapter, you should be able to:

- distinguish a model of the environment from a value function or policy;
- explain where model-based and model-free methods lie on a continuum;
- derive the Dyna update pattern and the receding-horizon objective used by model-predictive control;
- explain why low one-step prediction error does not imply reliable long-horizon planning;
- separate epistemic model uncertainty from irreducible environment stochasticity;
- describe the data flow in Dyna, PETS-style planning, MuZero, and Dreamer;
- choose rollout horizons, terminal values, or fallback policies based on model reliability;
- design diagnostics that test whether a planner is exploiting model error.

Standing assumptions

We use an MDP

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0).$$

A learned model is denoted

$$\widehat{\mathcal{M}}_\phi = (\mathcal{S}, \mathcal{A}, \widehat{P}_\phi, \widehat{r}_\phi, \gamma, \rho_0).$$

The model may predict observations, latent states, rewards, termination, or only the quantities required by a planner. A model is useful for control only to the extent that its predictions remain reliable on the state-action distribution induced by planning and policy improvement.

11. Learning and Using World Models

Intuition first: learn enough physics to think before acting

A model-free agent learns a direct mapping from experience to values or actions. A model-based agent also learns or uses a description of how the environment responds:

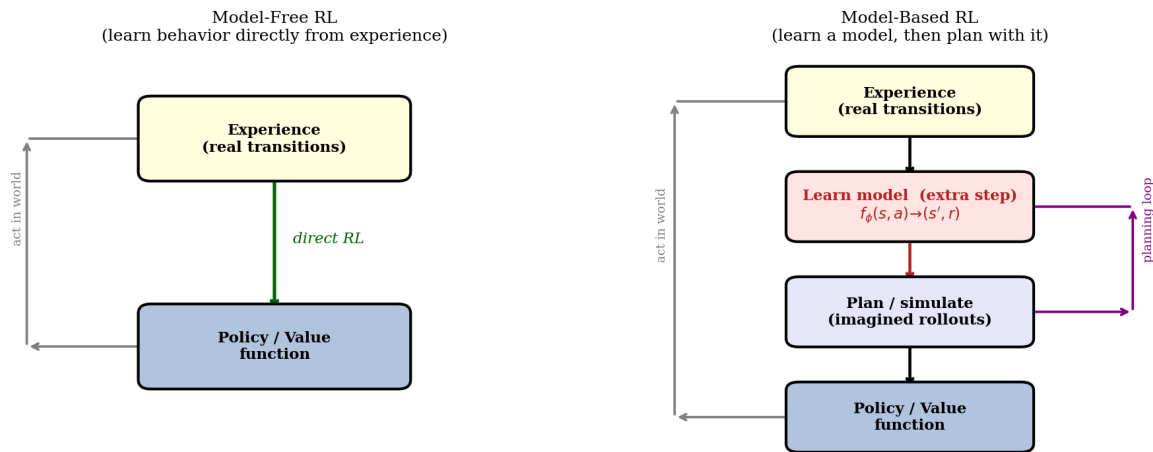


Figure 27: Model-free and model-based reinforcement learning compared

$$(s, a) \mapsto (r, s').$$

Once the agent has a model, it can simulate candidate futures without paying for every real interaction.

This creates a basic tradeoff:

- **model reuse:** one real transition may support many planning updates;
- **model bias:** imagined experience can be systematically wrong.

A model-based agent is not automatically more intelligent. It has simply moved some of the difficulty from value or policy learning into prediction and planning.

A continuum, not a binary distinction

Methods differ in how much they rely on a model.

Method	Model use	Planning at decision time?	Learned value or policy?
Q-learning	none	no	value
Dyna	synthetic transitions for extra updates	usually no	value or policy
Model-based value expansion	short imagined rollout inside a target	no	value or policy
MPC	optimize action sequences in a model	yes	optional terminal value
MuZero	learned latent dynamics inside tree search	yes	policy and value also learned
Dreamer	imagined latent trajectories train actor-critic	usually no expensive search at deployment	actor and critic

The important question is not merely “Does the algorithm have a model?” It is:

Where does the model enter the learning and decision pipeline, and what happens when it is wrong?

Known models and learned models

Dynamic programming in Part II was already model-based: it assumed that P and r were known.

When the model is learned, a basic one-step predictor might be

$$\hat{s}_{t+1} = f_\phi(s_t, a_t), \quad \hat{r}_{t+1} = g_\phi(s_t, a_t).$$

For deterministic dynamics, one may minimize

$$L(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\|f_\phi(s, a) - s'\|^2 + \lambda_r (g_\phi(s, a) - r)^2 \right].$$

For stochastic dynamics, a probabilistic model is more appropriate:

$$\widehat{P}_\phi(s' \mid s, a),$$

trained by negative log likelihood:

$$L_P(\phi) = -\mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[\log \widehat{P}_\phi(s' \mid s, a) \right].$$

The model may also predict termination:

$$\widehat{p}_\phi(d_{t+1} = 1 \mid s_t, a_t).$$

Termination errors can be especially damaging because they alter whether future rewards are included at all.

Generative models versus expected models

A **generative model** can sample

$$(\widehat{R}_{t+1}, \widehat{S}_{t+1}) \sim \widehat{P}_\phi(\cdot, \cdot \mid S_t, A_t).$$

An **expected model** may return only quantities such as

$$\widehat{r}(s, a) \quad \text{and} \quad \mathbb{E}_{\widehat{P}}[V(S') \mid s, a].$$

Planning algorithms that branch over possible outcomes need a distribution or simulator. A Bellman-style expected backup may need only expected next-value information.

Observation-space and latent-space models

A model can predict:

1. **raw observations:** pixels, proprioception, or text;
2. **task-state variables:** position, velocity, inventory;
3. **latent states:** compressed representations learned for prediction or control;
4. **value-relevant quantities only:** rewards, policies, and values needed by search.

Predicting every pixel can waste capacity on details irrelevant to decisions. Latent models can be more efficient, but their representations may omit information that later becomes important.

Prediction loss is not planning loss

A model is commonly trained with an average prediction loss under the dataset distribution:

$$\mathbb{E}_{(s,a) \sim d_{\mathcal{D}}} \left[\ell(\widehat{P}_\phi(\cdot \mid s, a), P(\cdot \mid s, a)) \right].$$

A planner cares about errors under a different distribution:

$$(s, a) \sim d_{\pi_{\text{plan}}}.$$

Even a small average error can be dangerous if it is concentrated on actions the planner finds attractive.

This is the control-specific form of distribution shift:

The planner searches for high predicted return, so it also searches for places where the model is optimistically wrong.

A one-step Bellman-error bound

Suppose, uniformly over (s, a) ,

$$|r(s, a) - \hat{r}(s, a)| \leq \varepsilon_r$$

and

$$\|P(\cdot | s, a) - \hat{P}(\cdot | s, a)\|_1 \leq \varepsilon_P.$$

For any bounded V ,

$$\begin{aligned} & |(r(s, a) + \gamma \mathbb{E}_P[V(S')]) - (\hat{r}(s, a) + \gamma \mathbb{E}_{\hat{P}}[V(S')])| \\ & \leq \varepsilon_r + \gamma \varepsilon_P \|V\|_\infty. \end{aligned}$$

Here ε_P is explicitly an L_1 distance, so the constant in the displayed inequality is exactly one. With total variation, defined as half that distance, the transition term has an additional factor of two. Transition error is multiplied by the scale of future value.

If V is a policy value in a model whose rewards are bounded by $R_{\max}$, and $\gamma < 1$, then

$$\|V\|_\infty \leq \frac{R_{\max}}{1 - \gamma}.$$

For a fixed policy, provided rewards in both models are bounded by $R_{\max}$, the resolvent calculation in A.12 proves the following uniform value bound:

$$\|V_{\mathcal{M}}^\pi - V_{\hat{\mathcal{M}}}^\pi\|_\infty \leq \frac{\varepsilon_r}{1 - \gamma} + \frac{\gamma \varepsilon_P R_{\max}}{(1 - \gamma)^2},$$

under the stated uniform assumptions and the chosen norm convention.

This is a worst-case bound, not a prediction of empirical error. It also fixes π . Planning is harder because the learned policy itself depends on the model and can select the model's largest errors.

Why rollout error compounds

Suppose a deterministic model makes a small state error at the first imagined step. The second prediction is then evaluated at an input that may not occur in the real dataset. Further errors are therefore not independent copies of the first error; the model can drift into increasingly unsupported states.

This produces two mechanisms:

1. **recursive error propagation:** later predictions depend on earlier predicted states;

2. **distributional drift:** imagined states move away from the model's training distribution.

A low teacher-forced one-step loss tests predictions on real states. It does not directly test free-running rollouts on model-generated states.

Epistemic and aleatoric uncertainty

Aleatoric uncertainty

The environment may be intrinsically stochastic:

$$P(S' \mid s, a)$$

has genuine spread even with unlimited data. This uncertainty cannot be eliminated by collecting more samples at the same (s, a) .

Epistemic uncertainty

The model may be uncertain because data are limited. Different plausible models fit the observed data but disagree elsewhere.

This uncertainty can shrink with informative data.

The distinction matters:

- aleatoric uncertainty should be represented in the predictive distribution;
- epistemic uncertainty should make the planner cautious or motivate exploration.

Ensembles as an uncertainty heuristic

Train K models on different bootstrap samples, initializations, or data orderings:

$$\hat{f}_1, \dots, \hat{f}_K.$$

For a candidate transition, use disagreement such as

$$u(s, a) = \frac{1}{K} \sum_{k=1}^K \left\| \hat{f}_k(s, a) - \bar{f}(s, a) \right\|^2,$$

where

$$\bar{f}(s, a) = \frac{1}{K} \sum_{k=1}^K \hat{f}_k(s, a).$$

High disagreement can indicate unsupported inputs. It is not automatically calibrated uncertainty: shared architecture, data, optimization, and inductive bias can produce shared blind spots.

Dyna: integrate learning, acting, and planning

Core idea

Dyna uses each real transition in two ways:

1. update the value function or policy directly;
2. update a model that generates additional synthetic transitions.

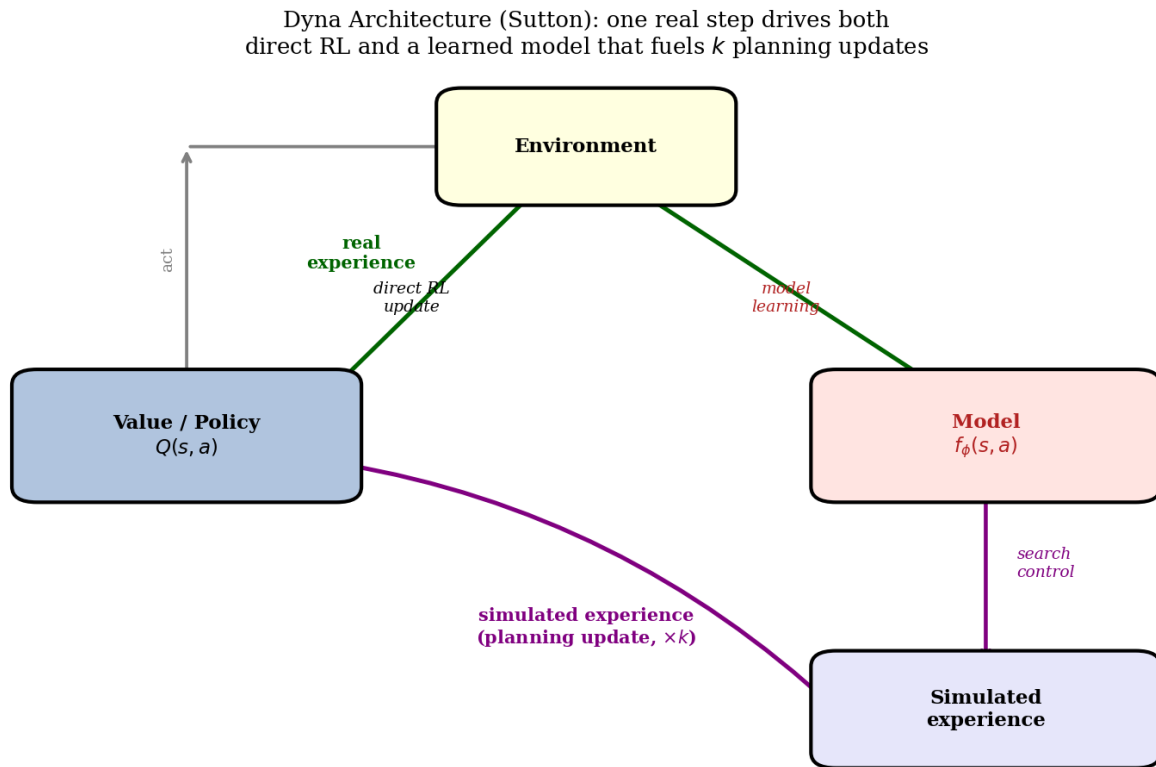


Figure 28: The Dyna architecture integrating real experience, a learned model, and planning

Flow to keep in mind. Real experience feeds both a direct RL update and a model update. The fitted model generates imagined transitions for planning updates. Both update routes affect the value function or policy that chooses the next real action.

Tabular Dyna-Q

For each real interaction:

```

observe real transition (s, a, r, s')
Q(s,a) <- Q-learning update from the real transition
model(s,a) <- estimated reward and next state

```

repeat n_{plan} times:

```

  sample a previously observed pair (s_tilde, a_tilde)
  sample or predict (r_tilde, s_tilde') from the model
  Q(s_tilde,a_tilde) <- Q-learning update from the imagined transition

```

The real update grounds learning in actual experience. Planning spreads information to other previously modeled state-action pairs.

What extra planning updates do—and do not—mean

If one real transition is followed by 50 model updates, the agent has performed 51 optimization updates. It has not obtained 51 independent real observations.

Synthetic updates can be valuable because they propagate known information quickly. Their statistical value is limited by:

- model accuracy;
- coverage of sampled planning states;
- correlation among imagined targets;
- the stability of the value learner.

Dyna-Q+

In changing environments, a state-action pair not tried recently may have changed. Dyna-Q+ adds an exploration bonus that grows with time since last real use, schematically

$$r^+(s, a) = r(s, a) + \kappa \sqrt{\tau(s, a)},$$

where $\tau(s, a)$ is elapsed time since the pair was executed. The bonus encourages the agent to recheck stale parts of its model.

Prioritized sweeping

Uniformly sampling modeled transitions can waste planning updates. Prioritized sweeping focuses on states whose values may change most.

If a transition produces TD error

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a),$$

then predecessors of s are inserted into a priority queue with priority related to

$$P(s \mid \bar{s}, \bar{a}) |\delta|.$$

Large downstream changes are propagated backward first.

This is an early example of a general principle:

Planning computation should be allocated where it can most change the decision.

Model-Predictive Control

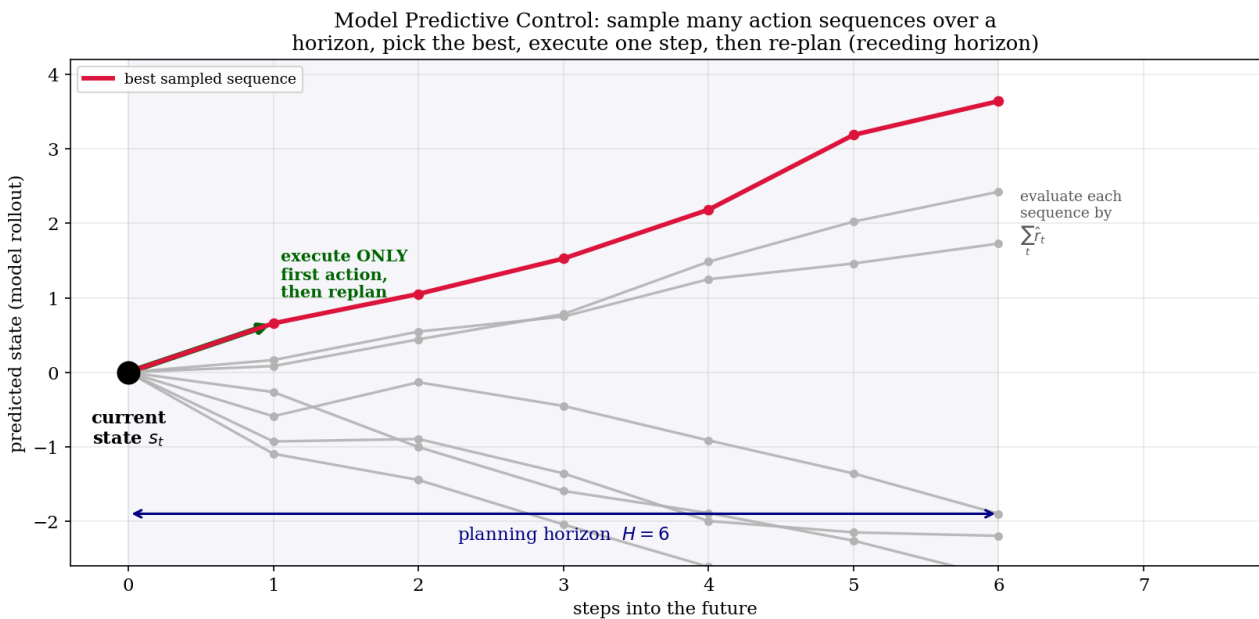


Figure 29: The receding-horizon loop used by model-predictive control

Intuition first: plan, act once, then reconsider

MPC repeatedly solves a finite-horizon planning problem from the current real state. It executes only the first action of the best sequence and replans after observing the real next state.

At state s_t , choose an action sequence

$$a_{t:t+H-1} = (a_t, a_{t+1}, \dots, a_{t+H-1})$$

to maximize predicted return:

$$\max_{a_{t:t+H-1}} \mathbb{E}_{\widehat{\mathcal{M}}} \left[\sum_{k=0}^{H-1} \gamma^k \widehat{r}(\widehat{S}_{t+k}, a_{t+k}) + \gamma^H V_{\text{term}}(\widehat{S}_{t+H}) \right].$$

Then execute only a_t .

Why replanning helps

The model may be wrong, but after one real step the controller receives a corrected state observation. Replanning prevents an entire open-loop action sequence from being executed under accumulated model error.

This is the **receding-horizon** principle.

Planning horizon and terminal value

A longer horizon can expose delayed consequences but increases sensitivity to model error and computation.

A learned terminal value

$$V_{\text{term}}(s)$$

can summarize rewards beyond the model rollout. This creates a hybrid target:

- short horizon: more trust in the critic;
- long horizon: more trust in the model.

The best horizon depends on which estimator is more reliable in the relevant region.

Random shooting

Sample N candidate action sequences from a proposal distribution, simulate each, and choose the sequence with largest predicted return.

This is simple and parallelizable, but inefficient in high-dimensional or long-horizon action spaces.

Cross-Entropy Method

CEM iteratively fits a proposal distribution to high-scoring action sequences.

```

initialize a distribution  $q_0$  over action sequences
for iteration  $j = 0, \dots, J - 1$ :
    sample  $N$  sequences from  $q_j$ 
    evaluate their predicted returns
    select the top  $\rho$  fraction as elites
    fit  $q_{j+1}$  to the elite sequences
return the mean or best sequence from the final distribution

```

For a Gaussian sequence distribution with mean m_j and covariance Σ_j , the elite maximum-likelihood update is

$$m_{j+1} = \frac{1}{|\mathcal{E}_j|} \sum_{u \in \mathcal{E}_j} u,$$

$$\Sigma_{j+1} = \frac{1}{|\mathcal{E}_j|} \sum_{u \in \mathcal{E}_j} (u - m_{j+1})(u - m_{j+1})^\top.$$

Smoothing these updates can prevent premature collapse.

Model-Predictive Path Integral control

MPPI uses soft weighting rather than a hard elite set. For sampled perturbation sequence i with predicted return J_i , a schematic normalized weight is

$$w_i = \frac{\exp(J_i/\lambda)}{\sum_j \exp(J_j/\lambda)}.$$

The nominal action sequence is shifted toward high-weight perturbations. Exact MPPI objectives include control-cost and sampling-distribution terms; the exponential weighting is the central intuition, not the complete derivation.

MPC failure modes

- The optimizer exploits model inaccuracies.
- The horizon is too short to see delayed effects.
- The horizon is too long for reliable prediction.
- Candidate-sequence search misses useful action modes.
- The terminal value is poorly calibrated.
- Model uncertainty is ignored.
- Inference latency is too high for the control rate.

Probabilistic ensembles and PETS-style planning

PETS combines:

- probabilistic neural dynamics;
- an ensemble for epistemic uncertainty;
- trajectory sampling through stochastic models;
- MPC, often with CEM.

A probabilistic model may predict Gaussian parameters:

$$S_{t+1} \sim \mathcal{N}(\mu_\phi(S_t, A_t), \Sigma_\phi(S_t, A_t)).$$

Different particles sample both model identity and transition noise. This separates, imperfectly, uncertainty about the dynamics from stochasticity within a dynamics model.

A planner can penalize uncertain trajectories:

$$J_{\text{pess}}(\tau) = \mathbb{E}[\widehat{G}(\tau)] - \beta \text{Uncertainty}(\tau).$$

Pessimism can prevent model exploitation, but too much pessimism blocks useful novelty.

Short model rollouts and model-based value expansion

Why short rollouts are often safer

Instead of optimizing a policy entirely inside a learned model, use only a few imagined steps and then bootstrap from a value function.

An H -step model-based target is

$$\widehat{G}_t^{(H)} = \sum_{k=0}^{H-1} \gamma^k \widehat{R}_{t+k+1} + \gamma^H V(\widehat{S}_{t+H}).$$

As H increases:

- critic dependence decreases;
- model dependence increases;
- synthetic reward information extends farther;
- compounding model error grows.

MBPO-style principle

Model-Based Policy Optimization uses short model rollouts starting from real replay states. The synthetic transitions augment real replay rather than replacing it entirely.

Starting imagined rollouts from real states reduces the chance that the first model input is already unsupported. Short horizons limit drift.

A rollout horizon can be scheduled upward as model validation improves, but a fixed schedule is only a heuristic. A stronger system adapts horizon to local uncertainty or prediction error.

Policy-induced model error

The feedback loop

Flow to keep in mind. Data from the current policy fit a model; planning or policy improvement uses that model; the new policy reaches new states; new data then refit the model. Model error must therefore be assessed under a changing distribution.

The model determines the policy, and the policy determines the next model-training distribution. This feedback can be beneficial—new data repairs the model—or unstable—the planner repeatedly enters unsupported regions.

If the learned-model value error is uniformly at most B , a planner that is δ -optimal in that model loses at most $2B + \delta$ in the real model; A.12 proves the decomposition. This distinguishes representation/model error from incomplete planning. Low average prediction loss on behavior data does not establish the uniform premise.

Model exploitation test

A direct diagnostic is to compare predicted and realized returns for planned trajectories:

$$\text{Gap} = \widehat{G}_{\text{model}} - G_{\text{real}}.$$

If the gap grows as optimization becomes more aggressive, the planner may be selecting model errors.

Useful stress tests include:

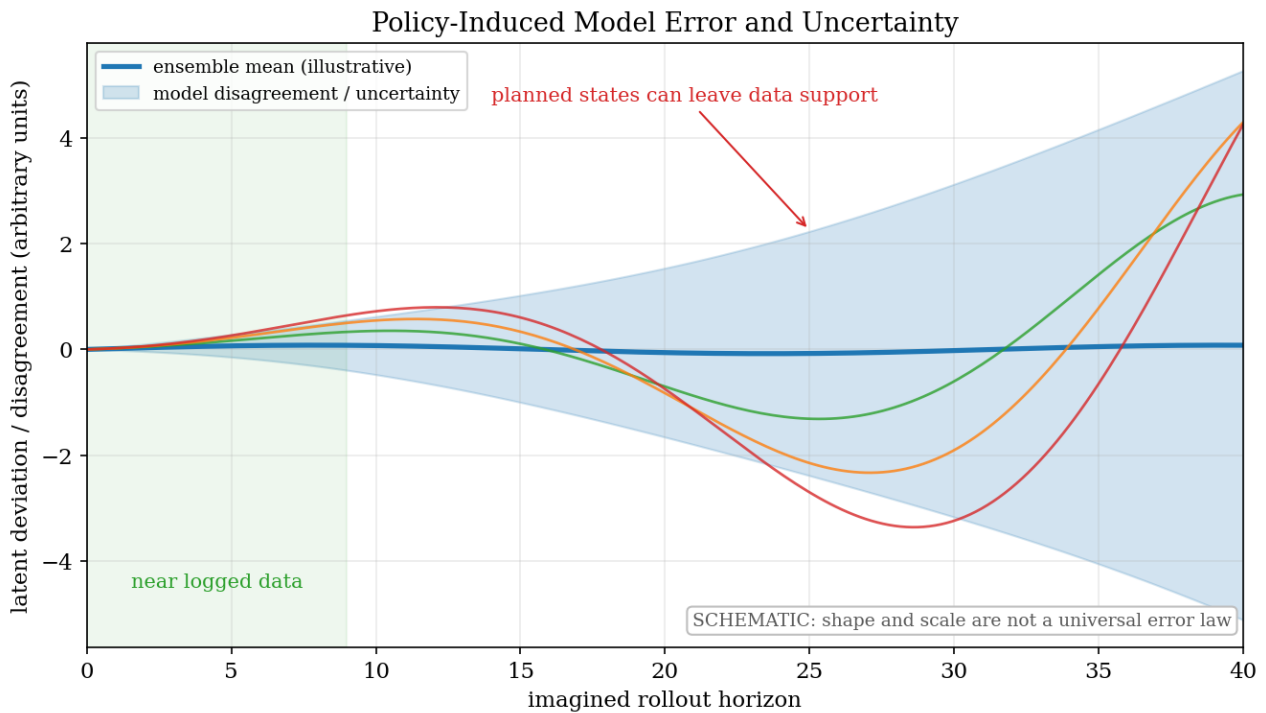


Figure 30: How small model errors compound along imagined trajectories

- increase the number of planning candidates while holding real data fixed;
- lengthen the model horizon;
- compare high- and low-ensemble-disagreement trajectories;
- execute plans under perturbed dynamics;
- replace the learned model with the real simulator in a controlled environment.

If predicted return rises while real return falls, more planning is amplifying model bias rather than improving control.

Dataset aggregation for models

One remedy is iterative data collection:

1. fit the model on current data;
2. plan with the model;
3. execute cautiously in the real environment;
4. add the resulting transitions;
5. refit.

This resembles DAgger: train on the distribution induced by the learner rather than only the original dataset.

Safety-critical systems may add a fallback controller when uncertainty exceeds a threshold.

World Models

The World Models architecture separates representation, dynamics, and control.

1. Vision model

A variational autoencoder maps an observation to a latent variable:

World Models (Ha & Schmidhuber): Vision compresses, Memory predicts, Controller acts

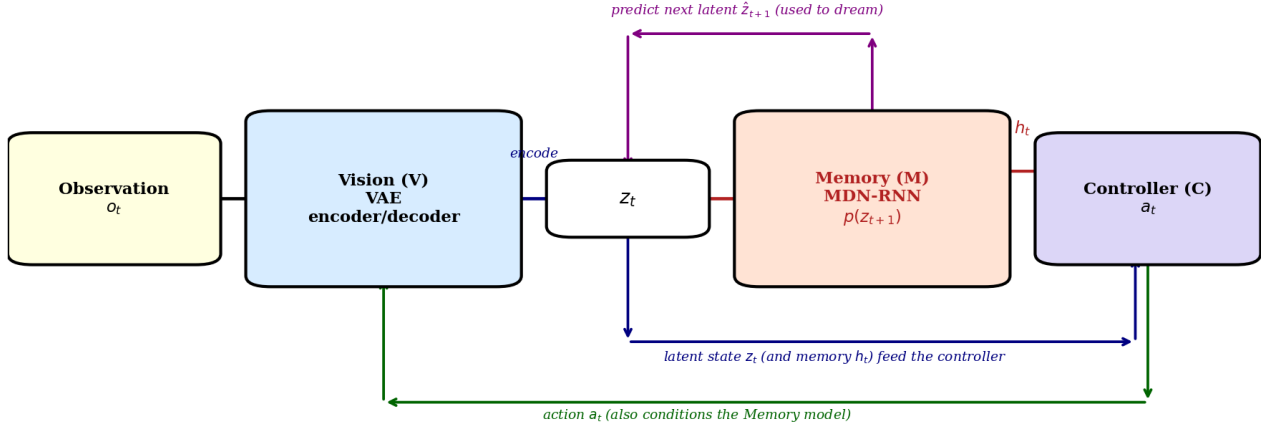


Figure 31: A latent world model that learns dynamics for control

$$z_t \sim q_\phi(z_t | o_t),$$

and reconstructs

$$\hat{o}_t \sim p_\psi(o_t | z_t).$$

A standard evidence lower bound contains reconstruction and KL terms:

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q_\phi(z|o)}[\log p_\psi(o | z)] + D_{\text{KL}}(q_\phi(z | o) \| p(z)).$$

2. Memory model

An RNN models latent dynamics:

$$h_t = \text{RNN}(h_{t-1}, z_t, a_t),$$

and an MDN head predicts a mixture distribution for z_{t+1} .

3. Controller

A compact controller maps latent state and memory to an action:

$$a_t = C_\theta(z_t, h_t).$$

The controller can be optimized in the learned “dream” environment.

Central risk

A controller trained only in imagination can exploit inaccuracies invisible to the model-training objective. Dream randomization or model stochasticity can make exploitation harder, but does not prove transfer to the real environment.

MuZero

Key idea: predict what search needs, not every observation

MuZero learns a latent model used inside Monte Carlo tree search. It has three main functions.

Representation

Map observation history to an initial latent state:

$$h_0 = h_\theta(o_{\leq t}).$$

Dynamics

Given latent state and action, predict the next latent state and reward:

$$(r_k, h_k) = g_\theta(h_{k-1}, a_k).$$

Prediction

Predict policy logits and value:

$$(p_k, v_k) = f_\theta(h_k).$$

The latent state need not reconstruct the observation. It needs to support accurate rewards, values, and search policies.

Search and training loop

1. Encode the current observation history.
2. Run MCTS in latent space.
3. Use visit counts to define an improved search policy.
4. Act in the real environment.
5. Train unrolled predictions against observed rewards, value targets, and search policies.

A schematic unrolled loss is

$$\mathcal{L}(\theta) = \sum_{k=0}^K [\ell_r(r_{t+k}, \hat{r}_k) + \ell_v(z_{t+k}, \hat{v}_k) + \ell_p(\pi_{t+k}^{\text{MCTS}}, \hat{p}_k)] + c\|\theta\|^2.$$

The value target z_{t+k} may combine observed rewards and a bootstrapped value.

What MuZero changes conceptually

Classical model learning asks:

Can the model predict the next observation?

MuZero asks:

Can the latent transition support planning decisions that predict reward, value, and policy targets?

This task-focused representation can ignore irrelevant observation detail. It can also hide model defects that do not appear in the training targets until search visits a new regime.

Dreamer

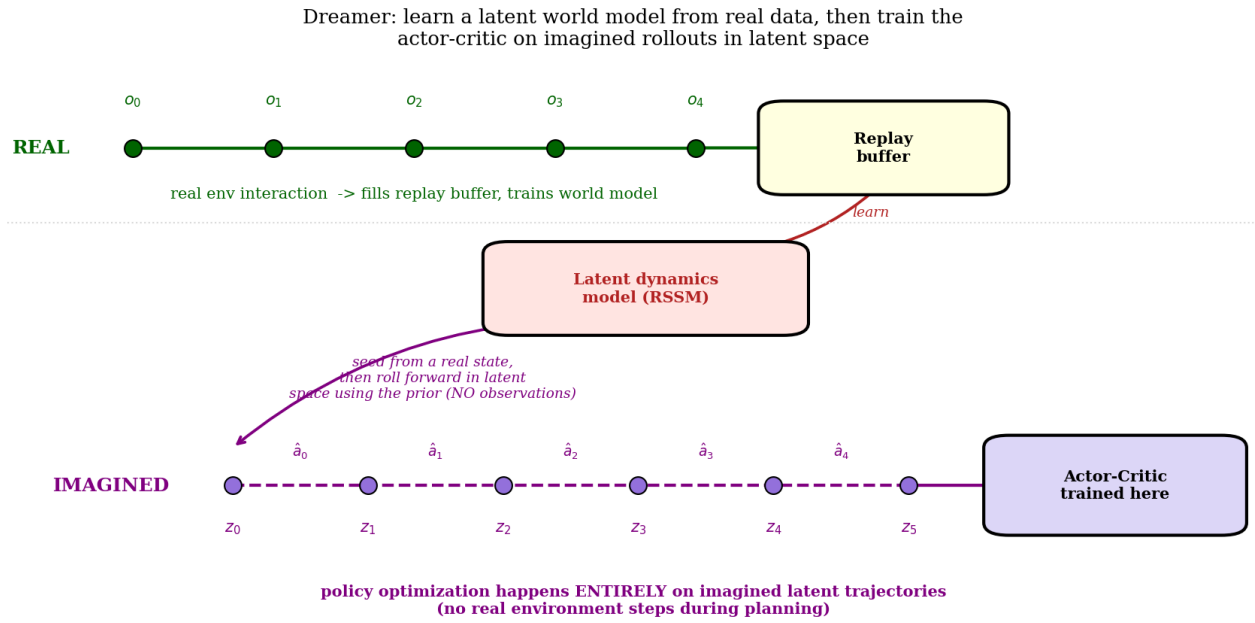


Figure 32: Dreamer learning behavior from imagined latent trajectories

Recurrent state-space model

Dreamer learns a latent dynamics model with deterministic and stochastic components.

A simplified recurrent state-space model uses

$$h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}),$$

posterior state

$$z_t \sim q_\phi(z_t \mid h_t, o_t),$$

and prior state

$$\hat{z}_t \sim p_\phi(z_t \mid h_t).$$

The posterior incorporates the real observation. The prior predicts without it and is therefore used for imagination.

The model learns to predict quantities such as:

- observations or embeddings;
- rewards;
- continuation or termination;
- latent-state consistency.

Imagination-based actor-critic

Start from latent states inferred from real replay data. Then:

1. sample actions from the actor;
2. advance the latent prior;

3. predict rewards and continuation;
4. compute imagined λ -returns;
5. update actor and critic from imagined trajectories.

A finite imagined return can be written

$$V_t^\lambda = \hat{r}_{t+1} + \gamma \hat{c}_{t+1} \left[(1 - \lambda) v_\psi(h_{t+1}, z_{t+1}) + \lambda V_{t+1}^\lambda \right],$$

where $\hat{c}_{t+1}$ predicts continuation.

The critic minimizes

$$L_V(\psi) = \mathbb{E} \left[\left(v_\psi(h_t, z_t) - \text{sg}(V_t^\lambda) \right)^2 \right],$$

where sg denotes stop-gradient.

The actor maximizes imagined returns, with gradients obtained through the learned dynamics or through a policy-gradient estimator depending on the implementation.

Why Dreamer can be efficient

Many actor-critic updates occur in latent imagination rather than through new real interaction. The representation, model, actor, and critic are trained jointly enough that the latent state is useful for control.

Dreamer failure modes

- latent features omit a task-relevant variable;
- continuation predictions create termination errors;
- imagined trajectories drift off support;
- actor gradients exploit differentiable model artifacts;
- reward predictions are accurate on average but wrong for selected actions.

Choosing a model-based strategy

Situation	Plausible strategy	Main check
Small known MDP	dynamic programming or exact planning	computational scale
Expensive continuous control, moderate-dimensional state	probabilistic ensemble plus MPC	model disagreement versus real error
Need fast deployment policy	Dyna, short model rollouts, or Dreamer-style actor	synthetic-to-real return gap
Complex observations	latent model	whether latent state preserves control-relevant information
Board game or structured search	latent/tree-search method	search calibration and model consistency
Highly chaotic or weakly predictable dynamics	stronger model-free baseline or very short model use	model horizon at which errors become decision-changing

When model-based methods are attractive

- Real interactions are expensive.
- Dynamics have exploitable structure.

- Planning computation is cheaper than environment interaction.
- Objectives may change and can be replanned without retraining everything.
- Safety constraints can be checked in simulation, with appropriate conservatism.

When model-free methods may be simpler

- A fast simulator provides abundant interaction.
- Long-horizon predictions are unreliable.
- The environment is highly stochastic or partially observed.
- Deployment latency rules out online planning.
- A strong model-free baseline already solves the task robustly.

Hybrid methods are often preferable because they combine model reuse with real-data grounding.

A model-validation protocol for control

1. One-step predictive quality

Report held-out likelihood or prediction error, stratified by state-action region rather than only one global mean.

2. Free-running rollout quality

Initialize from real states and compare predicted trajectories with real or simulator trajectories as horizon increases.

3. Reward and termination calibration

A visually plausible state prediction can still have the wrong reward or terminal flag.

4. Policy-shift evaluation

Evaluate the model on data from policies newer or more aggressive than the model-training policy.

5. Disagreement calibration

Check whether ensemble disagreement predicts actual rollout or return error.

6. Planner exploitation test

Increase planning optimization pressure while holding model data fixed. Compare predicted and realized returns.

7. Ablate model horizon

Measure performance for horizons $H \in \{0, 1, 3, 5, 10, \dots\}$. A sharp decline at larger H is evidence of compounding model bias.

8. Compare against the real simulator

When possible, run the same planner with:

- the learned model;
- the true simulator;
- no planning.

This separates planning-search failure from learned-model failure.

Common misconceptions

“Model-based RL means predicting pixels”

No. A model may operate in state space, latent space, or only in the reward/value quantities used by planning.

“Low validation loss means the model is safe for planning”

No. Average one-step loss under old data may miss errors selected by the planner.

“An ensemble gives calibrated uncertainty”

Not automatically. Ensemble members can share blind spots.

“Longer planning is always better”

Longer horizons expose delayed rewards but also accumulate model error.

“Synthetic transitions multiply the amount of real data”

They multiply computation and target reuse, not independent environmental evidence.

“Replanning eliminates model bias”

Replanning corrects the state after each real step, but the selected first action can still be based on an incorrect imagined future.

“A latent model that cannot reconstruct observations is defective”

Not necessarily. MuZero-style models intentionally predict decision-relevant quantities rather than every observation detail.

Chapter summary

1. Model-based RL learns or uses transition and reward structure to support planning or synthetic learning.
 2. A model’s value depends on accuracy under the planner-induced state-action distribution, not only the training distribution.
 3. Dyna uses real experience directly and through model-generated planning updates.
 4. MPC repeatedly plans over a finite horizon, executes one action, and replans from the real next state.
 5. Short rollouts plus a terminal value trade model error against critic error.
 6. Ensembles can expose epistemic uncertainty but require calibration tests.
 7. MuZero learns latent dynamics tailored to tree search; Dreamer trains actor-critic components on imagined latent trajectories.
 8. The decisive diagnostic is whether predicted improvement survives contact with the real environment.
-

Knowledge check

1. Why is dynamic programming model-based even though it does not learn a neural world model?
2. Give a formal distinction between a generative model and an expected model.

3. Explain why teacher-forced one-step prediction can be accurate while free-running rollout prediction fails.
4. Derive the one-step Bellman-error bound from reward and transition-model error.
5. Why can a planner exploit a model even if the model is unbiased on average under the dataset?
6. Explain the difference between aleatoric and epistemic uncertainty.
7. What does Dyna gain from synthetic updates, and what information does it not gain?
8. Why can a short rollout plus a terminal value outperform either a zero-step or very long rollout?
9. What quantities does MuZero's model predict, and why does it not need to reconstruct observations?
10. In Dreamer, what is the difference between the posterior and prior latent state?

Exercises

1. Dyna gridworld

Implement tabular Q-learning and Dyna-Q on a changing gridworld.

- Match the number of real environment steps.
- Vary planning updates per real step.
- Add an obstacle after learning has begun.
- Compare Dyna-Q and Dyna-Q+.
- Explain adaptation in terms of model staleness and exploration.

2. Rollout-error experiment

Fit a deterministic or probabilistic model on a small continuous-control dataset.

1. measure one-step prediction error;
2. measure free-running state error for horizons 1 through 50;
3. measure return-prediction error over the same horizons;
4. repeat under a policy not represented in the training data.

3. Ensemble calibration

Train at least five dynamics models. Bin imagined transitions by ensemble disagreement and report actual prediction error in each bin. Determine whether disagreement is monotonic and calibrated enough to serve as a stopping rule.

4. MPC horizon study

Implement random-shooting or CEM MPC. Sweep:

- planning horizon;
- number of candidate sequences;
- terminal value on/off;
- uncertainty penalty on/off.

Report both predicted and realized return. Identify the point at which additional optimization begins to exploit model error.

5. Model-based value expansion

Construct targets

$$\widehat{G}_t^{(H)} = \sum_{k=0}^{H-1} \gamma^k \widehat{R}_{t+k+1} + \gamma^H V(\widehat{S}_{t+H})$$

for several H . With access to a simulator, decompose target mean-squared error into empirical bias and variance.

6. Latent-state audit

For a latent world model, train probes to predict known task variables from the latent state. Identify variables that are predictable from observations but missing from the latent representation, and test whether their absence changes control performance.

Primary readings

- Sutton (1991), **Dyna, an Integrated Architecture for Learning, Planning, and Reacting**.
- Moore and Atkeson (1993), **Prioritized Sweeping**.
- Chua et al. (2018), **Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models**.
- Janner et al. (2019), **When to Trust Your Model: Model-Based Policy Optimization**.
- Ha and Schmidhuber (2018), **World Models**.
- Schrittwieser et al. (2020), **Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model**.
- Hafner et al. (2020–2023), the **Dreamer** family.

Next: Part VI — Advanced Topics in Reinforcement Learning.

Part VI — Advanced Topics

Lecture opening: Change one assumption before changing the algorithm

The word “advanced” should not hide the formulation. Exploration means data collection has information value. Multi-agent learning changes who controls transitions. Hierarchy changes the duration of an action. Offline learning removes the ability to request corrective data. Preferences change how reward is observed. Each setting demands a new assumption audit before importing the Bellman or policy-gradient proof.

Read B.2 for a complete UCB regret argument and B.3–B.6 for partial observation, continuing tasks, constraints, and games. A.17 supplies three counterexamples that prevent overgeneralization: flat monotone mixers, hindsight selection bias, and noncausal trajectory reweighting. The full LLM treatment is in Part VIII; its brief introduction here is a bridge, not a second syllabus.

Learning objectives and scope

This chapter is a structured survey of six major research areas. It is not a substitute for a specialized course in any one of them.

By the end, you should be able to:

- state the formal problem each area is trying to solve;
- identify the central statistical or computational obstacle;
- derive the core equations behind representative methods;
- distinguish a reusable principle from an algorithm-specific heuristic;
- identify the assumptions that make a method plausible;
- design a stress test that could reveal when the method fails.

Prerequisites

Parts I–V, especially Bellman equations, policy gradients, function approximation, support mismatch, and model error.

12. Exploration Strategies

Intuition first: information has long-term value

An agent must sometimes choose an action that looks worse under its current estimates because the action may reveal information that improves future decisions.

This is the exploration-exploitation dilemma:

- **exploitation:** choose the action currently believed to be best;
- **exploration:** choose an action partly because of what it may teach us.

Random exploration is often enough in small problems. In long-horizon tasks, however, useful discoveries may require a coordinated sequence of actions. Independent random deviations at each step can destroy that coordination.

Exploration Strategies: directing where the agent tries new actions

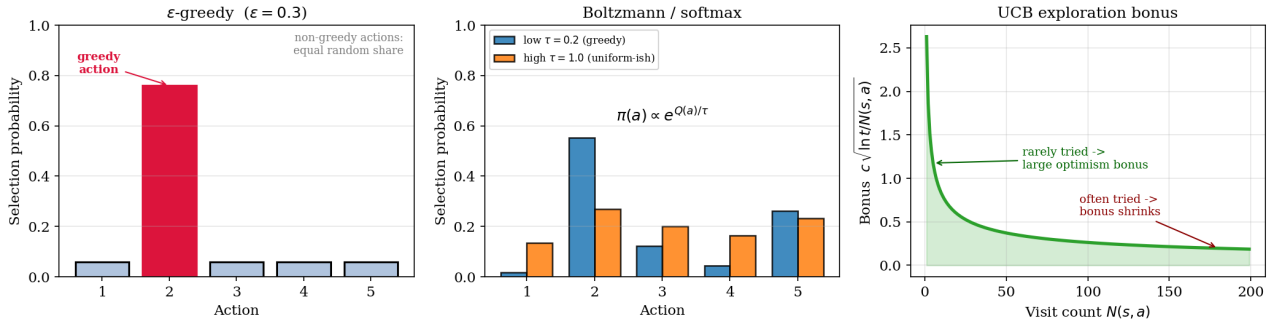


Figure 33: A comparison of major reinforcement learning exploration strategies

Bandit baseline: ε -greedy

With probability $1 - \varepsilon$, choose a greedy action. With probability ε , choose from an exploration distribution, often uniform:

$$A_t = \begin{cases} \arg \max_a Q_t(a), & \text{with probability } 1 - \varepsilon, \\ \text{random action}, & \text{with probability } \varepsilon. \end{cases}$$

Advantages:

- simple;
- robust to reward-scale changes;
- ensures persistent action coverage when $\varepsilon > 0$.

Limitations:

- all non-greedy actions are explored without regard to uncertainty;
- one-step randomization is weak for temporally extended exploration;
- constant ε prevents convergence to a deterministic greedy policy;
- rapidly decaying ε can freeze an early mistake.

Boltzmann exploration

A softmax policy based on action values is

$$\pi_\tau(a | s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{a'} \exp(Q(s, a')/\tau)}.$$

The temperature $\tau > 0$ controls spread:

- $\tau \rightarrow 0$: nearly greedy;
- $\tau \rightarrow \infty$: nearly uniform.

Boltzmann exploration uses relative value estimates, but its behavior changes if rewards are rescaled. It is not automatically uncertainty-aware: a high estimated value may reflect confidence or estimation error.

Upper confidence bounds in bandits

For a stationary stochastic bandit, UCB1 chooses

$$A_t = \arg \max_a \left[\hat{\mu}_t(a) + \sqrt{\frac{2 \log t}{N_t(a)}} \right].$$

The first term exploits estimated reward. The second is an optimism bonus that shrinks as the action is sampled.

Under the bandit assumptions, UCB obtains logarithmic expected regret in time for suboptimal arms.

Warning. Replacing $N_t(a)$ with $N_t(s, a)$ inside a large MDP does not automatically inherit the bandit guarantee. State visitation depends on earlier actions, transition uncertainty matters, and many states may never be revisited.

Optimism in an MDP

The reusable principle is:

Construct a plausible optimistic model or value estimate, then act as if the optimistic possibility were true.

A schematic optimistic action value is

$$Q^+(s, a) = \widehat{Q}(s, a) + b(s, a),$$

where $b(s, a)$ is large when uncertainty is high.

In tabular finite-horizon RL, theoretically grounded bonuses depend on visit counts, horizon, confidence level, and propagated transition uncertainty. In deep RL, bonuses are usually learned heuristics rather than exact confidence bounds.

Count-based exploration

In a tabular state space, a common intrinsic bonus is

$$r_t^{\text{count}} = \frac{\beta}{\sqrt{N(S_t)}}$$

or

$$r_t^{\text{count}} = \frac{\beta}{\sqrt{N(S_t, A_t)}}.$$

The bonus is large for rare states or actions and decays with visitation.

Pseudo-counts

For high-dimensional observations, exact equality is unhelpful. A density model ρ can define a pseudo-count. Let

- $\rho(s)$ be the density before updating on s ;
- $\rho'(s)$ be the density after one additional update on s .

Matching this change to a tabular count yields

$$\widehat{N}(s) = \frac{\rho(s)(1 - \rho'(s))}{\rho'(s) - \rho(s)}.$$

A useful density model must improve its assigned probability when observing a state. If the model generalizes poorly or forgets, the pseudo-count can cease to behave like novelty.

Prediction-error curiosity

Intrinsic motivation adds a learned reward:

$$r_t^{\text{total}} = r_t^{\text{external}} + \beta r_t^{\text{intrinsic}}.$$

A forward-model bonus is

$$r_t^{\text{intrinsic}} = \|\hat{\phi}_{t+1} - \phi(S_{t+1})\|^2,$$

where

$$\hat{\phi}_{t+1} = f_\psi(\phi(S_t), A_t).$$

Large prediction error is treated as novelty.

Intrinsic Curiosity Module

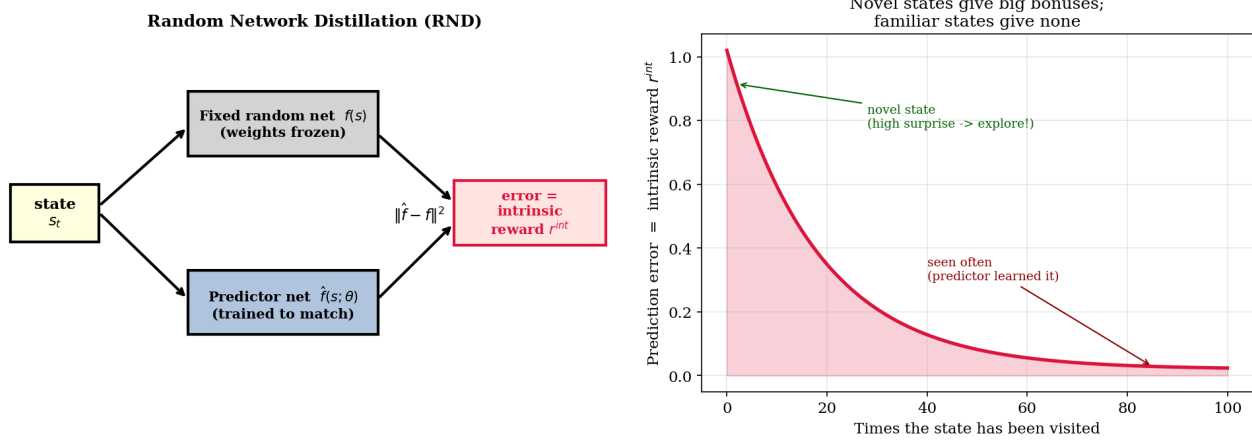


Figure 34: Intrinsic motivation generated from prediction error

ICM learns a feature representation using an inverse model:

$$\hat{A}_t = g_\eta(\phi(S_t), \phi(S_{t+1})).$$

The forward model predicts the next feature from the current feature and action. The inverse-model objective encourages features that retain action-relevant information while discarding some uncontrollable observation noise.

This reduces, but does not eliminate, the **noisy-TV problem**: an agent may seek stochastic observations that remain unpredictable but are not useful.

Random Network Distillation

RND uses:

- a fixed random target network f ;
- a trainable predictor g_ψ .

The intrinsic reward is

$$r_t^{\text{RND}} = \|g_\psi(S_t) - f(S_t)\|^2.$$

Familiar states become easier to predict. Novel states retain higher error.

RND avoids learning an action-conditioned forward model, but its bonus can still be distorted by:

- stochastic observations;
- nonstationary preprocessing;
- catastrophic forgetting;
- uneven predictor capacity;
- reward normalization choices.

Information-gain exploration

A more explicitly Bayesian objective rewards reduction in uncertainty about latent environment parameters Θ :

$$r_t^{\text{IG}} = D_{\text{KL}}(p(\Theta \mid \mathcal{D}_{t+1}) \parallel p(\Theta \mid \mathcal{D}_t)).$$

The expected information gain of an action is

$$\mathbb{E}_{S_{t+1}}[D_{\text{KL}}(p(\Theta \mid \mathcal{D}_t, S_{t+1}) \parallel p(\Theta \mid \mathcal{D}_t))].$$

This targets learnable uncertainty rather than raw surprise. Exact Bayesian inference is usually intractable in deep RL, so practical methods approximate the posterior.

Deep exploration with randomized value functions

Bootstrapped DQN maintains several value heads. At the start of an episode, one head is selected and used consistently for action choice.

This creates **temporally coherent exploration**: the agent follows one plausible value hypothesis over many steps rather than injecting independent random actions at every step.

The broader principle is posterior sampling:

1. sample a plausible environment or value hypothesis;
2. act optimally under that hypothesis for a meaningful duration;
3. update beliefs from the resulting trajectory.

Go-Explore

Go-Explore separates discovery from robust execution.

Phase 1: explore from an archive

Maintain an archive of interesting states. Repeatedly:

1. select an archived state;
2. return to it, using reset access or a learned return policy;
3. explore outward;
4. add newly discovered states and trajectories.

Phase 2: robustify

Train a policy to reproduce successful trajectories under environment stochasticity, often using imitation or RL.

The method addresses two different failures:

- **detachment**: the agent leaves a promising region and cannot return;
- **derailment**: random exploration prevents it from following a delicate path.

Exploration diagnostics

Diagnostic	Question
state visitation histogram	Is novelty actually increasing coverage?
success by exploration seed	Is discovery reliable or lucky?
intrinsic versus external reward	Is the agent optimizing curiosity instead of the task?
bonus decay	Does a familiar state become less rewarding?
model disagreement calibration	Does uncertainty predict real error?
trajectory diversity	Are explorations coherent or merely noisy?
held-out state coverage	Does the method generalize novelty rather than memorize pixels?

Exploration design principle

A good exploration signal should be:

- high for learnable, decision-relevant uncertainty;
- low for irreducible noise;
- consistent across enough time to reach new regions;
- scaled so it does not permanently dominate the task reward.

13. Multi-Agent Reinforcement Learning

Intuition first: other learners make the world move

In single-agent RL, the transition law is assumed stationary. In multi-agent RL, one agent's environment includes the policies of the other agents. When they learn, the effective dynamics change.

This turns ordinary control into a game.

Markov games

A discounted Markov game with N agents can be written

$$\mathcal{G} = (\mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, P, \{r_i\}_{i=1}^N, \gamma, \rho_0).$$

At state s , agents choose joint action

$$\mathbf{a} = (a_1, \dots, a_N).$$

The environment transitions according to

$$P(s' \mid s, \mathbf{a}),$$

and agent i receives

$$r_i(s, \mathbf{a}).$$

Settings include:

- **fully cooperative:** all agents share a reward;
- **zero-sum:** one agent's gain is another's loss;

- **general-sum:** incentives partly align and partly conflict.

In decentralized partially observed games, agent i sees observation o_i rather than the full state.

Joint policies and values

For joint policy

$$\pi(\mathbf{a} \mid s) = \prod_i \pi_i(a_i \mid o_i),$$

agent i has value

$$V_i^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{i,t+1} \mid S_0 = s \right].$$

An optimal response generally depends on the other agents' policies. There may be no single policy that is best against every opponent.

Independent learners

The simplest baseline lets each agent run a single-agent algorithm while treating other agents as part of the environment.

This is attractive because it scales and requires no centralized information. It violates stationarity assumptions because

$$P_i^{(t)}(s' \mid s, a_i) = \sum_{\mathbf{a}_{-i}} P(s' \mid s, a_i, \mathbf{a}_{-i}) \prod_{j \neq i} \pi_j^{(t)}(a_j \mid o_j)$$

changes as the other policies change.

Symptoms include:

- replay data becoming stale quickly;
- cycling strategies;
- unstable value targets;
- apparent environmental randomness caused by other learners;
- convergence to poor coordination conventions.

Centralized training, decentralized execution

CTDE permits global information during training while requiring local policies at deployment.

- **centralized critic:** conditions on the state and possibly all agents' actions;
- **decentralized actors:** agent i acts using only o_i and allowed messages.

This reduces critic ambiguity during training without requiring centralized control at execution.

Value decomposition

In a cooperative game, a centralized action-value function is

$$Q_{\text{tot}}(s, \mathbf{a}).$$

VDN

Value Decomposition Networks assume an additive factorization:

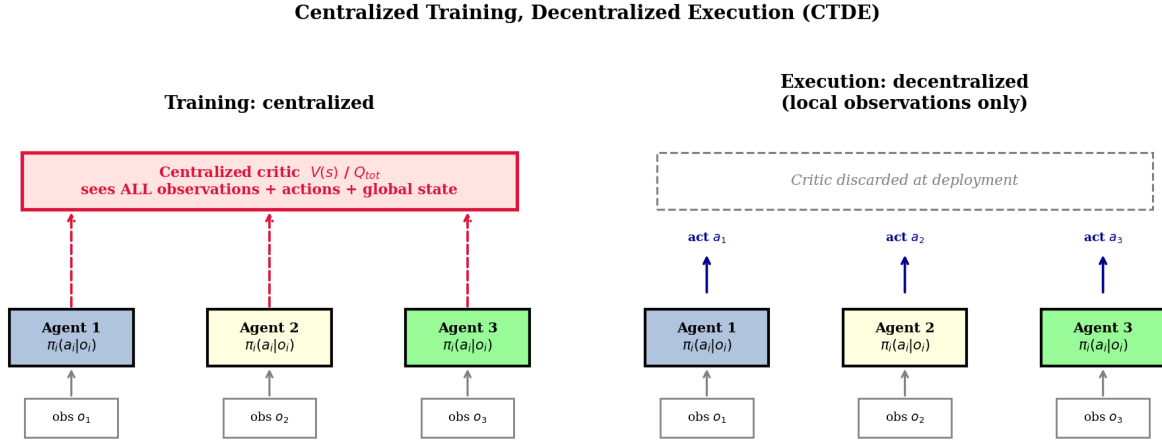


Figure 35: Centralized training with decentralized execution in multi-agent reinforcement learning

$$Q_{\text{tot}}(s, \mathbf{a}) = \sum_{i=1}^N Q_i(o_i, a_i).$$

Then each agent can choose its local maximizing action and the joint choice maximizes the sum. The additive restriction can be too weak for interactions that require non-additive coordination.

QMIX

QMIX uses a state-conditioned mixing network:

$$Q_{\text{tot}}(s, \mathbf{a}) = f_s(Q_1(o_1, a_1), \dots, Q_N(o_N, a_N)).$$

It enforces

$$\frac{\partial Q_{\text{tot}}}{\partial Q_i} \geq 0.$$

This monotonicity guarantees

$$\prod_{i=1}^N \arg \max_{a_i} Q_i(o_i, a_i) \subseteq \arg \max_{\mathbf{a}} Q_{\text{tot}}(s, \mathbf{a}).$$

To prove it, increase one local value at a time to its maximum; no increase can lower the mixer output. Thus any combination of local maximizers is globally maximizing. Equality of argmax *sets* need not hold when the mixer has flat regions: a constant mixer makes every joint action optimal. This is a sufficient decentralized greedy-execution property, not a claim that every centralized optimum is obtained only through local maxima.

The cost is representational: some joint action-value functions cannot be represented by a monotonic factorization.

Centralized critics and MAPPO

MAPPO applies PPO-style updates to decentralized actors while using a centralized value critic:

$$V_{\phi}(s) \quad \text{or} \quad V_{\phi}(s, o_1, \dots, o_N).$$

The critic can explain reward variation caused by teammates more effectively than a local-observation critic. The actors remain

$$\pi_{\theta_i}(a_i \mid o_i).$$

MAPPO's strength is often practical simplicity rather than a unique theoretical guarantee.

Multi-agent credit assignment

In a cooperative task, a shared terminal reward does not identify which agent's action mattered.

A counterfactual baseline asks what would have happened if agent i changed its action while teammates kept theirs fixed. COMA uses an advantage of the form

$$A_i(s, \mathbf{a}) = Q(s, \mathbf{a}) - \sum_{a'_i} \pi_i(a'_i \mid o_i) Q(s, (a'_i, \mathbf{a}_{-i})).$$

This isolates agent i 's contribution relative to its own action distribution.

The critic must estimate counterfactual joint actions accurately, which becomes difficult as the joint action space grows.

Communication

A communication policy may produce a message

$$m_i = \mu_i(o_i, h_i)$$

that influences other agents' actions.

Questions include:

- who sends to whom;
- bandwidth and delay;
- whether messages are differentiable during training;
- robustness to missing or adversarial messages;
- whether the learned protocol transfers to new team sizes.

Emergent communication can coordinate behavior, but a message code is not necessarily interpretable or stable under policy updates.

Self-play

In a two-player zero-sum game, repeated approximate best responses create a curriculum:

$$\pi_{k+1} \approx \text{BestResponse}(\pi_k).$$

Naive self-play can cycle. Rock-paper-scissors is the simplest example: a best response to the current opponent need not be robust to earlier strategies.

Mitigations include:

- opponent pools;
- fictitious play against an average of past policies;
- population-based training;
- league training;
- equilibrium-oriented meta-solvers.

Evaluating multi-agent systems

A single win rate against the latest opponent is insufficient. Report:

- a cross-play matrix among policy checkpoints;
- performance against fixed reference opponents;
- exploitability when computable;
- robustness to teammate or opponent replacement;
- performance under communication failure;
- seed and population variability.

A policy can improve against its training population while becoming more exploitable outside it.

14. Hierarchical Reinforcement Learning

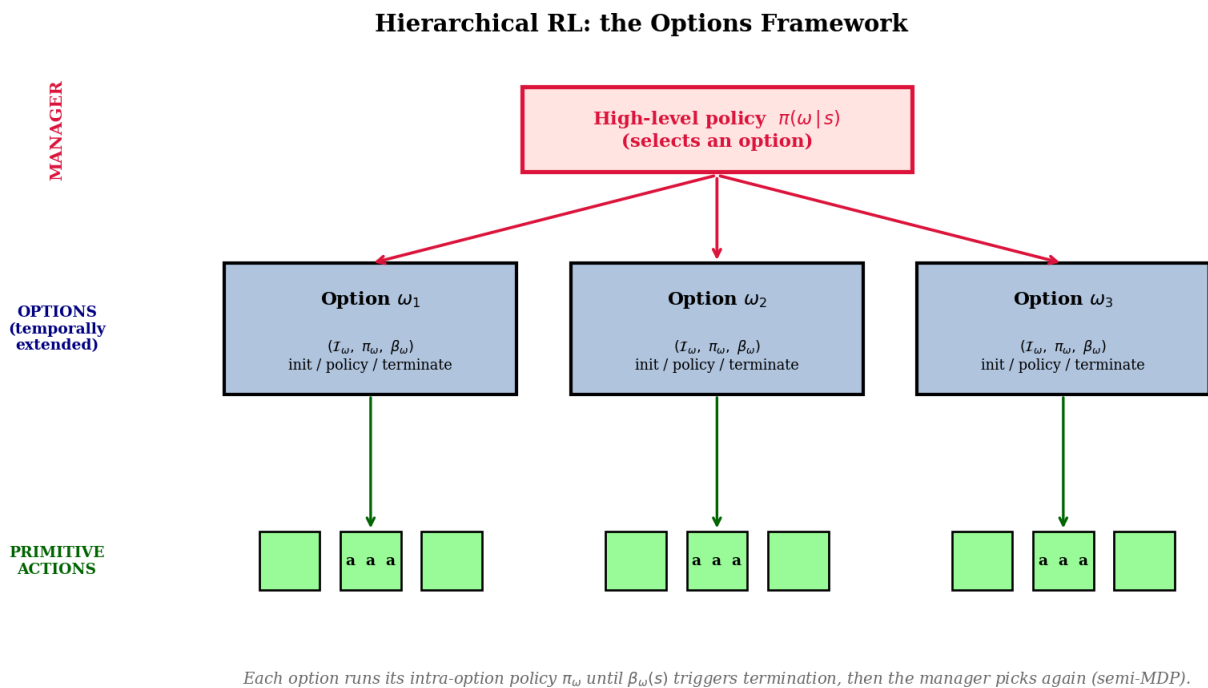


Figure 36: Hierarchical reinforcement learning with temporally extended options

Intuition first: reason at more than one timescale

A flat policy chooses a primitive action at every step. Hierarchical RL introduces temporally extended decisions such as “go to the doorway,” “pick up the key,” or “navigate to room three.”

This can improve:

- long-horizon credit assignment;
- exploration;
- transfer of reusable skills;
- interpretability of high-level plans.

A poor hierarchy can also constrain the agent to the wrong abstractions.

Options

An option ω is a triple

$$\omega = (\mathcal{J}_\omega, \pi_\omega, \beta_\omega),$$

where:

- $\mathcal{J}_\omega \subseteq \mathcal{S}$ is the initiation set;
- $\pi_\omega(a \mid s)$ is the intra-option policy;
- $\beta_\omega(s) \in [0, 1]$ is the termination probability.

A policy over options selects

$$\mu(\omega \mid s).$$

Once initiated, the option chooses primitive actions until termination.

Semi-Markov decision process view

Suppose option ω initiated at $S_t = s$ terminates after random duration K . Its discounted cumulative reward is

$$R_t^\omega = \sum_{j=0}^{K-1} \gamma^j R_{t+j+1}.$$

The option-value Bellman equation is

$$Q_\mu(s, \omega) = \mathbb{E} \left[R_t^\omega + \gamma^K \sum_{\omega'} \mu(\omega' \mid S_{t+K}) Q_\mu(S_{t+K}, \omega') \right].$$

The duration-dependent discount γ^K is the central semi-MDP modification.

Intra-option values

One can also learn while an option is executing. Define

$$U(\omega, s') = (1 - \beta_\omega(s')) Q(s', \omega) + \beta_\omega(s') \max_{\omega'} Q(s', \omega').$$

Then

$$Q(s, \omega) = \sum_a \pi_\omega(a \mid s) \left[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) U(\omega, s') \right].$$

Option discovery

Hand-designed options can encode useful domain structure. Learned options must solve three problems:

1. what skill should exist;
2. where it should start;
3. when it should terminate.

Option-Critic differentiates intra-option policies and termination functions under an end-to-end objective. Without regularization, options can collapse into nearly identical behaviors or terminate at every step.

Goal-conditioned policies

A goal-conditioned policy is

$$\pi(a \mid s, g).$$

A sparse goal reward might be

$$r_g(s, a, s') = \mathbf{1}\{d(s', g) \leq \varepsilon\}.$$

A dense alternative is

$$r_g(s, a, s') = -d(s', g),$$

but dense shaping can change incentives unless designed carefully.

Hindsight Experience Replay

A trajectory may fail at the intended goal g but reach another state g' . HER relabels the transition with an achieved goal:

$$(s_t, a_t, r_g, s_{t+1}, g) \longrightarrow (s_t, a_t, r_{g'}, s_{t+1}, g').$$

This converts failures into successful examples for other goals.

Goal-independent dynamics and recomputable rewards are necessary compatibility checks for this relabeling, but they do not alone imply unbiased targets in stochastic environments. Choosing the goal from a future achieved state conditions on transition noise. In A.17, a fair-coin transition has true goal success probability one half but relabeled success probability one. Deterministic dynamics, suitable corrections, or explicit additional assumptions are needed for an unbiasedness claim. The changed goal distribution also requires evaluation on the intended test-goal distribution.

Manager-worker architectures

A high-level manager emits a subgoal every c steps:

$$g_k = \pi_H(S_{kc}).$$

The low-level worker acts conditionally:

$$A_t \sim \pi_L(\cdot \mid S_t, g_k), \quad kc \leq t < (k+1)c.$$

The manager receives task reward. The worker often receives an intrinsic reward for making progress toward the subgoal.

A latent-direction reward can be

$$r_t^{\text{worker}} = \frac{(\phi(S_{t+c}) - \phi(S_t))^\top g_t}{\|\phi(S_{t+c}) - \phi(S_t)\| \|g_t\|}.$$

The latent geometry must remain stable enough that a direction retains meaning while both levels learn.

Unsupervised skill discovery

A skill variable Z indexes different behaviors. One objective is to maximize mutual information between skill and visited states:

$$I(Z; S_T | S_0).$$

Using a discriminator $q_\psi(z | s)$, a variational lower bound gives intrinsic reward roughly

$$r^{\text{skill}}(s, z) = \log q_\psi(z | s) - \log p(z).$$

The agent is rewarded when the resulting state reveals which skill was selected.

This produces diverse skills, not necessarily task-useful skills. Downstream composition remains a separate problem.

Hierarchy failure modes

- subgoals are unreachable by the worker;
- the worker changes, making old manager actions nonstationary;
- options collapse to identical behaviors;
- options terminate too quickly or never terminate;
- intrinsic worker reward conflicts with external reward;
- abstraction hides safety-critical details;
- a fixed temporal scale is wrong for the task.

15. Offline or Batch Reinforcement Learning

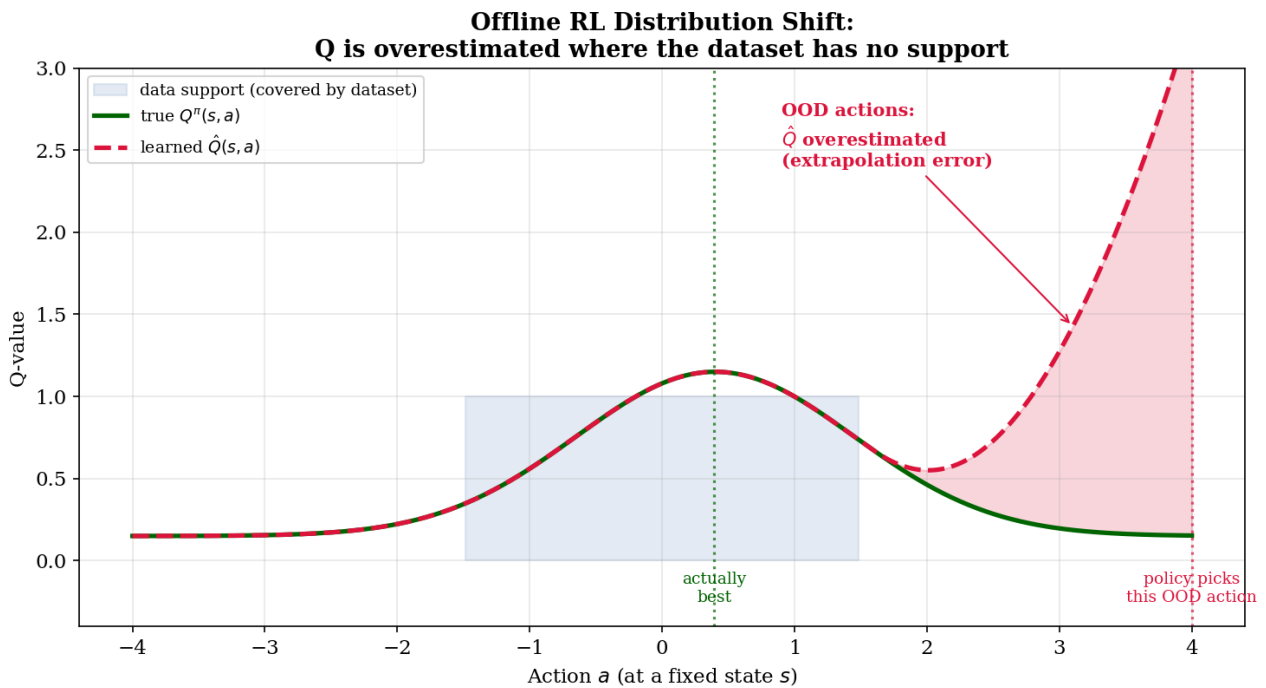


Figure 37: Distribution shift between offline data and a learned policy

Intuition first: improve without new interaction

Offline RL receives a fixed dataset

$$\mathcal{D} = \{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^N$$

collected by one or more behavior policies. The learner cannot query the environment during training.

The goal is often to produce a policy better than the behavior that generated the data.

This requires counterfactual reasoning:

What would happen under actions that may be rare or absent in the dataset?

That question is not always identifiable from the data.

Support and coverage

Let $d^\beta(s, a)$ be the dataset occupancy and $d^\pi(s, a)$ the target-policy occupancy.

A strong coverage condition has the form

$$d^\pi(s, a) > 0 \implies d^\beta(s, a) > 0.$$

Quantitative analyses often use a concentrability coefficient such as

$$C = \sup_{s,a} \frac{d^\pi(s, a)}{d^\beta(s, a)}.$$

If C is infinite, the target policy relies on unsupported state-action pairs.

No algorithm can infer arbitrary counterfactual outcomes from absent data without additional assumptions about smoothness, dynamics, causality, or model class.

Extrapolation error

A critic is trained on dataset actions but queried on policy actions:

$$A' \sim \pi(\cdot \mid S').$$

If A' is out of distribution, $Q(S', A')$ may be arbitrarily wrong. Bootstrapping then propagates the error:

$$y = r + \gamma Q(S', A').$$

The actor selects high predicted values, so it preferentially chooses positive critic errors.

Behavioral cloning as a required baseline

Behavioral cloning solves

$$\min_{\theta} -\mathbb{E}_{(s,a) \sim \mathcal{D}} [\log \pi_{\theta}(a \mid s)].$$

It does not improve beyond the demonstrated action distribution through Bellman reasoning, but it avoids many unsupported-action failures. An offline RL method should beat a well-tuned cloning baseline under the same evaluation protocol.

Behavior-constrained methods

BCQ and BEAR restrict the learned policy toward data-supported actions.

A generic constrained objective is

$$\max_{\pi} \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi} [Q(s, a)]$$

subject to

$$D(\pi(\cdot | s), \beta(\cdot | s)) \leq \varepsilon,$$

where D may be KL, maximum mean discrepancy, or another divergence.

The constraint lowers extrapolation risk but can prevent improvement when the behavior policy is poor.

Conservative Q-Learning

CQL penalizes high values on actions not supported by the dataset. For discrete actions, a common regularizer is

$$\mathcal{R}_{\text{CQL}}(Q) = \mathbb{E}_{s \sim \mathcal{D}} \left[\log \sum_a \exp Q(s, a) - \mathbb{E}_{a \sim \mathcal{D}(\cdot | s)} Q(s, a) \right].$$

The total critic objective is schematically

$$\min_Q \quad \alpha \mathcal{R}_{\text{CQL}}(Q) + \frac{1}{2} \mathbb{E}_{\mathcal{D}} [Q(s, a) - y(s, a, r, s')]^2.$$

The log-sum-exp term raises pressure on high-valued actions broadly, while the dataset-action term counteracts pressure on observed actions. The net effect is conservative value estimation away from data support.

Particular CQL variants have formal conservative guarantees under explicit assumptions. A neural implementation is not automatically a pointwise lower bound everywhere.

Implicit Q-Learning

IQL avoids maximizing Q over arbitrary unseen actions.

Expectile value regression

For expectile parameter $\tau \in (0.5, 1)$, define

$$L_2^\tau(u) = |\tau - \mathbf{1}\{u < 0\}|u^2.$$

Fit the value function to dataset action values:

$$L_V(\psi) = \mathbb{E}_{(s, a) \sim \mathcal{D}} \left[L_2^\tau \left(Q_{\bar{\phi}}(s, a) - V_\psi(s) \right) \right].$$

For large τ , V tracks the upper tail of Q under dataset actions without evaluating arbitrary actions.

Q regression

Use

$$y = r + \gamma(1 - d)V_\psi(s')$$

and minimize

$$L_Q(\phi) = \mathbb{E}_{\mathcal{D}} \left[(Q_\phi(s, a) - y)^2 \right].$$

Advantage-weighted policy extraction

Define

$$\widehat{A}(s, a) = Q_\phi(s, a) - V_\psi(s).$$

Fit the policy with weighted behavioral cloning:

$$L_\pi(\theta) = -\mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\exp \left(\frac{\widehat{A}(s, a)}{\beta} \right) \log \pi_\theta(a \mid s) \right],$$

usually with clipped weights.

The actor remains tied to observed actions while favoring those with higher estimated advantage.

Decision Transformer

Decision Transformer treats offline trajectories as sequences. A token stream may contain

$$(G_t, S_t, A_t, G_{t+1}, S_{t+1}, A_{t+1}, \dots),$$

where G_t is return-to-go.

The model predicts

$$\pi_\theta(A_t \mid G_t, S_{\leq t}, A_{< t}).$$

At inference, the user specifies a desired return and updates the remaining return as rewards arrive.

This approach avoids explicit Bellman backups. Its performance depends heavily on dataset coverage and whether high-return conditioning corresponds to coherent action patterns in the data.

It does not create evidence for returns larger than those supported by the dataset merely because a larger number is supplied at inference.

Offline model-based RL

A learned dynamics model can generate counterfactual transitions, but it faces the same support problem. The policy may exploit model error in unseen regions.

Offline model-based methods therefore combine:

- uncertainty penalties;
- short rollouts;
- behavior constraints;
- pessimistic value estimates;
- model ensembles.

The absence of online correction makes calibration especially important.

Offline evaluation

Before deployment, one may use:

- importance sampling;
- doubly robust estimators;
- fitted Q evaluation;
- model-based simulation;
- held-out environment tests when available.

Every method requires support or modeling assumptions. Report weight effective sample size, model class, uncertainty intervals, and sensitivity to clipping or pessimism.

Offline RL comparison

Family	Main defense	Main limitation
Behavioral cloning	never leaves demonstrated policy intentionally	cannot use reward to improve much beyond data behavior
Behavior-constrained actor-critic	limits policy divergence	may be overly conservative
CQL-style pessimism	lowers unsupported Q-values	conservatism is hard to calibrate
IQL-style extraction	avoids explicit OOD action maximization	depends on dataset action quality and advantage estimates
Return-conditioned sequence model	imitates high-return trajectory patterns	weak extrapolation and no explicit Bellman stitching guarantee
Offline model-based	simulates counterfactuals	model exploitation without online repair

16. Reinforcement Learning from Human Feedback

The RLHF Pipeline: from demonstrations to a preference-aligned policy

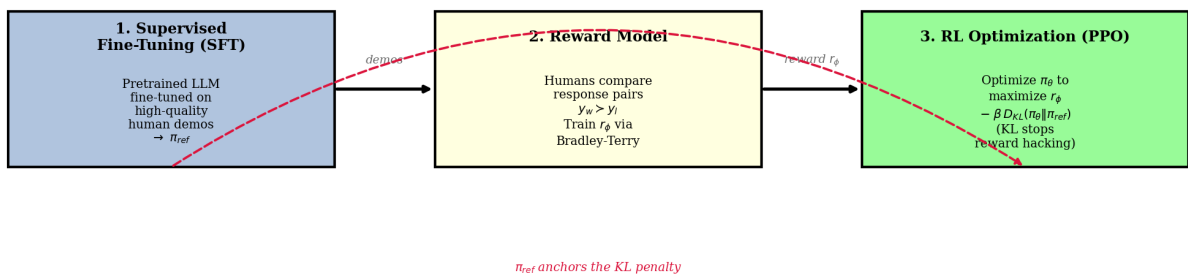


Figure 38: The reinforcement learning from human feedback pipeline

Why preferences enter the loop

For open-ended outputs, designers may not be able to write a complete scalar reward. Humans can often compare two concrete outputs more reliably than they can specify a full reward function.

RLHF converts preference comparisons into a training signal. It does not eliminate reward design; it replaces a hand-coded reward with a learned and imperfect proxy.

Part VIII develops this topic in depth. This section establishes the connection to standard RL.

Stage 1: supervised fine-tuning

Given demonstrations (x, y) , minimize token-level negative log likelihood:

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x,y)} \left[\sum_{t=1}^{|y|} \log \pi_{\theta}(y_t \mid x, y_{<t}) \right].$$

The resulting policy often serves as a reference π_{ref} .

Stage 2: preference or reward modeling

Given prompt x , preferred response y_w , and dispreferred response y_l , the Bradley-Terry model assumes

$$P(y_w \succ y_l \mid x) = \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l)).$$

The loss is

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E} [\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))].$$

The model identifies reward differences, not an absolute reward origin. Adding the same prompt-dependent constant to all response rewards changes no pairwise probability.

Stage 3: KL-regularized policy optimization

A common objective is

$$\max_{\pi} \mathbb{E}_{x, y \sim \pi(\cdot \mid x)} [r_{\phi}(x, y)] - \beta \mathbb{E}_x D_{\text{KL}}(\pi(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x)).$$

The KL term limits drift and can slow exploitation of reward-model errors. It does not prove alignment with human intent.

Closed-form optimum

For fixed x , fixed reward, $\beta > 0$, and finite normalizer, the optimizer is

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{r(x, y)}{\beta}\right).$$

This is a Boltzmann reweighting of the reference policy. It cannot place probability outside the reference support.

Direct Preference Optimization

Rearrange the optimal policy:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

Substituting into the Bradley-Terry model cancels $Z(x)$, yielding the DPO loss

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E} \log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_{\theta}(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right).$$

DPO removes an explicit reward model and online RL loop from the policy-update stage. It remains limited by preference-data quality and coverage.

Constitutional AI and RLAIIF

An explicit constitution can guide critique and revision. AI-generated comparisons can then replace some human labels.

This changes the source of supervision; it does not remove supervision error. Bias can enter through:

- the constitution;
- the critique model;
- the preference judge;
- prompt selection;
- policy-induced judge exploitation.

RLHF as an RL problem

For token generation:

- state: prompt plus generated prefix;
- action: next token;
- transition: append the token;
- terminal reward: learned preference or task score;
- shaped reward: per-token KL penalty;
- episode: one response.

The environment transition is deterministic within generation, but the action space is large and reward is often sparse.

17. Inverse RL and Imitation Learning

Imitation Learning: why behavioral cloning drifts, and how DAgger fixes it

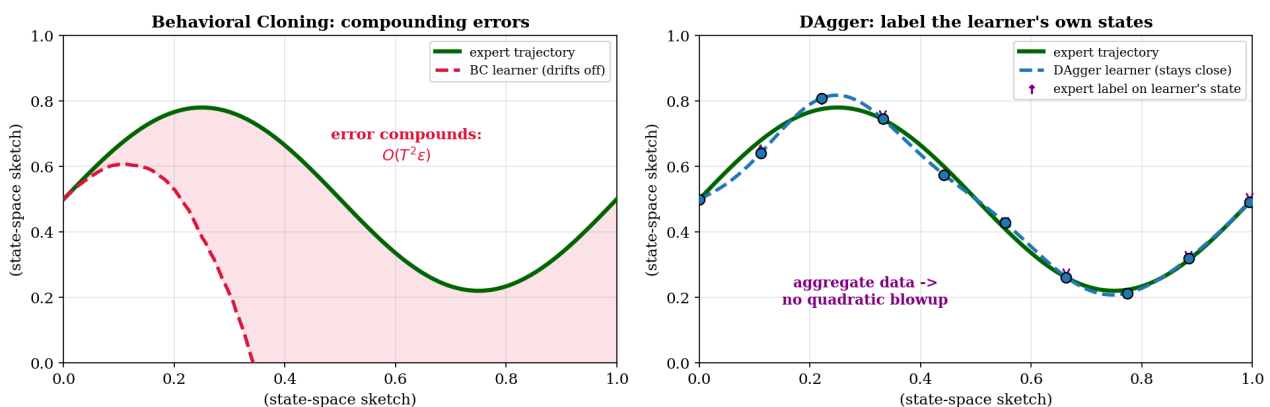


Figure 39: Behavioral cloning, inverse reinforcement learning, and interactive imitation compared

Intuition first: learn from behavior when reward is absent

Demonstrations provide actions, not necessarily the objective that produced them.

Two broad strategies are:

- **imitation learning:** reproduce expert behavior directly;
- **inverse reinforcement learning:** infer a reward under which the behavior is plausible, then solve the resulting control problem.

The distinction is important because a recovered reward may transfer across changed dynamics, but rewards are generally not uniquely identifiable from behavior alone.

Behavioral cloning

Given expert dataset

$$\mathcal{D}_E = \{(s_i, a_i)\},$$

behavioral cloning minimizes

$$\mathcal{L}_{\text{BC}}(\theta) = -\mathbb{E}_{(s,a) \sim \mathcal{D}_E} [\log \pi_\theta(a \mid s)].$$

This is supervised learning under the expert's state distribution.

Compounding error

Suppose the cloned policy makes an action error with probability at most ε on expert-distributed states. An early error can move the learner into states absent from the training data, where later error may be much larger.

Under standard finite-horizon imitation analyses, naive behavioral cloning can incur a performance gap scaling like

$$O(T^2\varepsilon),$$

rather than merely $O(T\varepsilon)$.

The quadratic term reflects distribution shift: each error can affect all remaining steps.

Dagger

Dataset Aggregation trains on the learner's own visited states.

```
initialize dataset with expert demonstrations
train pi_1

for iteration i = 1, ..., K:
    execute a mixture involving pi_i
    collect states visited by the learner
    ask the expert for the correct action at those states
    aggregate the new labeled states into the dataset
    train pi_{i+1} on the aggregate
```

Dagger converts sequential imitation into an online supervised-learning problem on learner-visited states. A task-loss bound has the form $O(uT\varepsilon)$, where u bounds the extra expert cost-to-go caused by one mistaken action and ε measures the relevant on-policy classification error/regret. A horizon-linear claim assumes u is bounded independently of T ; in general u can itself scale with T . Interactive expert coverage and no-regret learning assumptions are also substantive.

The practical cost is an interactive expert capable of labeling states the learner visits, including unsafe or nonsensical ones.

Reward non-identifiability

Different rewards can induce the same optimal policy.

A particularly important equivalence is potential-based shaping:

$$r'(s, a, s') = r(s, a, s') + \gamma\Phi(s') - \Phi(s).$$

Under standard assumptions, r and r' have the same optimal policies.

Therefore demonstrations alone cannot distinguish all behaviorally equivalent rewards. IRL requires assumptions or priors, such as:

- linear rewards in known features;
- maximum-entropy behavior;
- simplicity or smoothness priors;
- multiple environments;
- suboptimality models.

Maximum-entropy inverse RL

Assume a trajectory reward

$$R_\theta(\tau) = \theta^\top f(\tau),$$

where

$$f(\tau) = \sum_t f(S_t, A_t).$$

One maximum-entropy *trajectory model* has the exponential-family density

$$p_\theta(\tau) = \frac{1}{Z(\theta)} \exp(\theta^\top f(\tau)) p_{\text{dyn}}(\tau),$$

where p_{dyn} contains initial-state and transition probabilities.

For expert trajectories $\mathcal{D}_E$, the log likelihood is

$$\ell(\theta) = \sum_{\tau \in \mathcal{D}_E} \theta^\top f(\tau) - |\mathcal{D}_E| \log Z(\theta) + \text{constant}.$$

Its gradient is

$$\nabla_\theta \ell(\theta) = |\mathcal{D}_E| \left(\mathbb{E}_{\tau \sim \mathcal{D}_E} [f(\tau)] - \mathbb{E}_{\tau \sim p_\theta} [f(\tau)] \right).$$

Learning matches expert feature expectations to those induced by the current reward model.

Differentiating $Z(\theta) = \sum_\tau e^{\theta^\top f(\tau)} p_{\text{dyn}}(\tau)$ gives $\nabla \log Z = \mathbb{E}_{p_\theta} f$, which proves the feature-matching gradient for this density whenever the sum is finite or differentiation is justified. In stochastic dynamics, however, globally reweighting trajectories may change exogenous transition conditionals. It is not automatically a realizable causal policy. A.17 gives an uncontrolled-coin counterexample. Causal maximum-entropy control instead preserves dynamics and uses a soft Bellman recursion as in A.13. Do not substitute one model for the other without checking the assumptions.

Generative Adversarial Imitation Learning

Let $D_\phi(s, a)$ denote the probability that a state-action pair came from the expert. GAIL solves a game of the form

$$\min_\pi \max_D \mathbb{E}_{(s,a) \sim d_E} [\log D(s, a)] + \mathbb{E}_{(s,a) \sim d_\pi} [\log(1 - D(s, a))] - \lambda \mathcal{H}(\pi).$$

The discriminator learns to separate expert and learner occupancy measures. The policy learns to make its occupancy indistinguishable from the expert's.

With this label convention, a policy reward can be chosen as

$$r_D(s, a) = -\log(1 - D_\phi(s, a)),$$

which is large when the discriminator views the learner transition as expert-like.

Other papers reverse the discriminator labels; the objective and reward must be changed consistently.

Occupancy-measure view

The discounted occupancy of a policy is

$$d_\pi(s, a) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \Pr_\pi(S_t = s, A_t = a).$$

Imitation can be viewed as matching

$$d_\pi \approx d_E.$$

Behavioral cloning matches conditional actions on expert states. GAIL trains on learner-generated states and therefore addresses some covariate shift, but introduces adversarial optimization and a nonstationary learned reward.

Comparing imitation and inverse RL

Method	Learns explicit reward?	Needs environment interaction?	Main strength	Main failure mode
Behavioral cloning	no	no	simple supervised baseline	compounding distribution shift
Dagger	no	yes, plus expert queries	labels learner states	expensive interactive expert
MaxEnt IRL	yes	requires forward solves or model	transferable objective under assumptions	reward non-identifiability and computation
GAIL	implicit discriminator reward		yes occupancy matching on learner distribution	adversarial instability and opaque reward

Cross-cutting themes

1. Distribution shift

- exploration changes where data are collected;
- other agents change effective dynamics;
- a manager changes the worker's goal distribution;
- offline RL evaluates actions absent from data;

- imitation learners visit states absent from demonstrations;
- RLHF policies generate outputs unlike reward-model training data.

2. Credit assignment

- curiosity must identify useful novelty;
- teams must assign shared reward to individuals;
- hierarchies must assign task reward across timescales;
- offline critics must infer which dataset actions were good;
- preference learning assigns one comparison to long sequences.

3. Proxy objectives

- intrinsic reward is a proxy for useful information;
- centralized critics proxy decentralized contribution;
- skill diversity is a proxy for downstream usefulness;
- pessimism is a proxy for unsupported uncertainty;
- learned human preference is a proxy for intended quality;
- discriminator reward is a proxy for expert occupancy.

4. Identifiability

Sometimes the data do not contain enough information to answer the desired question:

- an unvisited action cannot be evaluated offline;
- one trajectory may not reveal which teammate mattered;
- expert behavior may be optimal for many rewards;
- a preference comparison does not reveal absolute utility;
- stochastic novelty may not be learnable uncertainty.

Recognizing non-identifiability is more important than applying a more complicated optimizer.

Common misconceptions

“Exploration means adding more random noise”

Useful long-horizon exploration often requires coherent hypotheses or skills, not independent noise.

“A multi-agent environment is just a larger MDP”

A centralized joint controller can be written as an MDP, but decentralized learning faces changing opponents, partial observations, and equilibrium issues.

“Hierarchy always improves long-horizon credit assignment”

Only when the learned temporal abstraction matches the task. A poor hierarchy can make the optimal behavior unreachable.

“Offline RL is supervised learning on rewards”

Offline RL must estimate consequences under a changed policy, creating a counterfactual support problem.

“DPO contains no RL ideas”

Its derivation comes from a KL-regularized reward-maximization problem, even though its policy update is supervised-style.

“Inverse RL recovers the true reward”

Behavior generally identifies only an equivalence class of rewards under the chosen assumptions.

“Imitating occupancy guarantees identical intent”

Matching behavior on one environment does not uniquely determine the objective or guarantee transfer under changed dynamics.

Chapter summary

1. Directed exploration values information, not merely random action diversity.
2. Multi-agent RL replaces a stationary control problem with a game involving coordination, competition, and nonstationarity.
3. Hierarchical RL introduces temporally extended actions or goals, trading flexibility for reusable abstraction.
4. Offline RL is fundamentally limited by dataset support and critic extrapolation.
5. RLHF learns and optimizes a preference proxy under a policy-drift constraint.
6. Imitation learning copies behavior; inverse RL tries to infer an objective, but that objective is not uniquely identifiable without assumptions.
7. Across all six areas, distribution shift, credit assignment, proxy error, and missing counterfactuals are the recurring difficulties.

Knowledge check

1. Why do bandit UCB guarantees not directly transfer to a deep MDP?
2. Give an example of a curiosity signal that rewards irreducible noise.
3. Prove that QMIX’s monotonicity constraint makes decentralized greedy action selection consistent with centralized maximization.
4. Write the option Bellman equation and explain the factor γ^K .
5. State a support condition required for offline policy evaluation.
6. Explain how CQL and IQL avoid unsupported-action overestimation in different ways.
7. Derive DPO from the KL-regularized optimal policy and Bradley-Terry preferences.
8. Give two behaviorally equivalent reward functions related by potential shaping.
9. Explain why DAgger changes the state distribution on which the classifier is trained.
10. Distinguish behavior cloning, occupancy matching, and reward inference.

Research exercises**1. Exploration stress test**

Create an environment with both:

- a stochastic “noisy TV” state;
- a difficult but learnable sparse-reward path.

Compare ε -greedy, RND, and an ensemble-information bonus. Report whether intrinsic reward correlates with task progress and epistemic uncertainty.

2. Multi-agent cross-play

Train several independent populations in a small coordination or competitive game. Produce a checkpoint-by-checkpoint cross-play matrix. Show whether performance against the latest opponent predicts robustness to the full population.

3. Option-collapse audit

Train a skill-discovery or option method. Measure:

- state-visitation diversity by skill;
- pairwise policy divergence;
- option duration;
- downstream transfer.

Test whether high mutual information actually yields task-useful skills.

4. Offline support stress test

Construct a dataset that omits one high-reward action in a subset of states. Compare behavioral cloning, CQL, IQL, and an unconstrained actor-critic. Explain which conclusions are identifiable from the data and which depend on function-approximation extrapolation.

5. Preference-model shift

Train a toy reward model on comparisons from one policy, then optimize a new policy against it. Measure reward-model score, held-out true reward, and distance from the comparison-data distribution. Identify the onset of proxy exploitation.

6. Imitation under dynamics shift

Train behavioral cloning, DAGger, and GAIL in one dynamics setting. Evaluate under changed transition noise. Determine whether any inferred or implicit objective transfers better than direct action imitation.

Primary readings

Exploration

- Auer, Cesa-Bianchi, and Fischer (2002), **Finite-Time Analysis of the Multiarmed Bandit Problem**.
- Bellemare et al. (2016), **Unifying Count-Based Exploration and Intrinsic Motivation**.
- Pathak et al. (2017), **Curiosity-Driven Exploration by Self-Supervised Prediction**.
- Burda et al. (2019), **Exploration by Random Network Distillation**.
- Osband et al. (2016), **Deep Exploration via Bootstrapped DQN**.
- Ecoffet et al. (2019/2021), **Go-Explore**.

Multi-agent RL

- Littman (1994), **Markov Games as a Framework for Multi-Agent Reinforcement Learning**.
- Sunehag et al. (2018), **Value-Decomposition Networks**.
- Rashid et al. (2018), **QMIX**.
- Foerster et al. (2018), **Counterfactual Multi-Agent Policy Gradients**.
- Yu et al. (2022), **The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games**.

Hierarchical RL

- Sutton, Precup, and Singh (1999), **Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning**.
- Bacon, Harb, and Precup (2017), **The Option-Critic Architecture**.
- Andrychowicz et al. (2017), **Hindsight Experience Replay**.
- Vezhnevets et al. (2017), **FeUdal Networks for Hierarchical Reinforcement Learning**.

Offline RL

- Fujimoto, Meger, and Precup (2019), **Off-Policy Deep Reinforcement Learning without Exploration**.
- Kumar et al. (2019), **Stabilizing Off-Policy Q-Learning via Bootstrapping Error Reduction**.
- Kumar et al. (2020), **Conservative Q-Learning for Offline Reinforcement Learning**.
- Kostrikov, Nair, and Levine (2022), **Offline Reinforcement Learning with Implicit Q-Learning**.
- Chen et al. (2021), **Decision Transformer**.

Preferences and imitation

- Ziebart et al. (2008), **Maximum Entropy Inverse Reinforcement Learning**.
- Ross, Gordon, and Bagnell (2011), **A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning**.
- Ho and Ermon (2016), **Generative Adversarial Imitation Learning**.
- Ouyang et al. (2022), **Training Language Models to Follow Instructions with Human Feedback**.
- Rafailov et al. (2023), **Direct Preference Optimization**.

Next: Part VII — Practical and Mathematical Foundations.

Part VII — Practical and Mathematical Foundations

Lecture opening: An experiment is part of the mathematical claim

Suppose two implementations report different returns. Before tuning either one, ask whether their reset distribution, horizon, reward scale, bootstrap boundary, and evaluation policy match. If not, they may be estimating different quantities. This chapter treats implementation choices as parts of a defined statistical experiment.

Use the exact small-model answers as unit tests, then move to multiple independent training runs and held-out tasks. An evaluation episode estimates a fixed policy’s performance; an independent training seed samples the learning procedure. They answer different questions. The laboratory suite emphasizes deterministic identities first so that a confidence interval is never asked to explain a broken estimator.

Learning objectives

By the end of this chapter, you should be able to:

- use conditional expectation, variance decomposition, and martingale-difference noise in RL derivations;
- state the support assumptions behind importance sampling and off-policy learning;
- distinguish unbiasedness, consistency, variance, and finite-sample reliability;
- use entropy, KL divergence, Fisher geometry, and contraction mappings correctly;
- identify the independent experimental units in an RL study;
- build a debugging protocol that separates environment bugs, target bugs, estimator noise, and optimization failure;
- design reproducible experiments with explicit stopping, checkpoint, and evaluation rules.

How to use this chapter

This is a just-in-time reference. Read each mathematical section when the concept first appears in an earlier chapter. Return to the implementation section before writing a new algorithm from scratch.

18. Mathematical Toolkit

18.1 Random variables and expectation

Intuition first: values are conditional averages

A value function is not an extra physical quantity in the environment. It is a conditional expectation of future return.

For a discrete random variable X ,

$$\mathbb{E}[X] = \sum_x x P(X = x).$$

For a continuous random variable with density p ,

$$\mathbb{E}[X] = \int x p(x) dx.$$

Expectation is linear even when random variables are dependent:

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y].$$

Conditional expectation

The conditional expectation

$$\mathbb{E}[X \mid Y]$$

is itself a random variable: it is a function of the information revealed by Y .

In RL,

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

and

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

The Markov property allows the full past to be replaced by the current state in these conditionals.

Tower property

If $\mathcal{G}$ contains less information than $\mathcal{H}$, then

$$\mathbb{E}[\mathbb{E}[X \mid \mathcal{H}] \mid \mathcal{G}] = \mathbb{E}[X \mid \mathcal{G}].$$

A familiar special case is

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X \mid Y]].$$

The Bellman equation is an application of this principle: condition first on the next transition, then average over it.

Indicator variables

For event A , let $\mathbf{1}_A$ equal 1 when A occurs and 0 otherwise. Then

$$\mathbb{E}[\mathbf{1}_A] = P(A).$$

Indicators are useful for terminal masks, visitation counts, and decomposing events.

For example, an asynchronous tabular update can be written

$$V_{t+1}(s) = V_t(s) + \alpha_t(s) \mathbf{1}\{S_t = s\} \delta_t.$$

Variance

The variance is

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2.$$

For constants a, b ,

$$\text{Var}(aX + b) = a^2 \text{Var}(X).$$

For two variables,

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2 \text{Cov}(X, Y).$$

Dependence matters through covariance.

Law of total variance

Conditioning separates within-condition noise from between-condition variation:

$$\boxed{\text{Var}(X) = \mathbb{E}[\text{Var}(X | Y)] + \text{Var}(\mathbb{E}[X | Y])}.$$

In policy gradients, this helps distinguish:

- randomness in return given a state-action pair;
- variation in expected return across state-action pairs.

A good baseline removes some between-state variation but cannot remove transition noise that remains after conditioning.

Covariance and temporal dependence

For a stationary scalar process X_t , define lag- k autocorrelation

$$\rho_k = \frac{\text{Cov}(X_t, X_{t+k})}{\text{Var}(X_t)}.$$

When the integrated autocorrelation time is well defined,

$$\text{Var}(\bar{X}_n) \approx \frac{\sigma^2}{n} \left(1 + 2 \sum_{k=1}^{\infty} \rho_k \right).$$

This motivates the diagnostic effective sample size

$$\boxed{n_{\text{eff}} \approx \frac{n}{1 + 2 \sum_{k=1}^{\infty} \rho_k}}.$$

The formula is not exact for arbitrary nonstationary RL data. It is a warning against treating consecutive timesteps as independent replications.

18.2 Filtrations, conditional noise, and stochastic approximation

Information available at time t

Let $\mathcal{F}_t$ denote the history available through time t . A sequence M_{t+1} is a martingale-difference sequence when

$$\mathbb{E}[M_{t+1} \mid \mathcal{F}_t] = 0.$$

This does not mean M_{t+1} is independent of the past. It means the past cannot predict a nonzero conditional mean.

TD noise as a martingale difference

For fixed policy π , define

$$M_{t+1} = R_{t+1} + \gamma V_t(S_{t+1}) - (T^\pi V_t)(S_t).$$

Given $\mathcal{F}_t$ and S_t , the environment samples according to the MDP dynamics, so

$$\mathbb{E}[M_{t+1} \mid \mathcal{F}_t] = 0.$$

The TD update can be decomposed into mean drift plus noise:

$$V_{t+1}(S_t) = V_t(S_t) + \alpha_t [(T^\pi V_t)(S_t) - V_t(S_t) + M_{t+1}].$$

The mean drift points toward the Bellman fixed point; the martingale-difference term is sampling noise.

Robbins-Monro step sizes

Classical stochastic approximation uses step sizes satisfying

$$\sum_{t=1}^{\infty} \alpha_t = \infty, \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty.$$

Interpretation:

- the first sum ensures learning does not stop too early;
- the second ensures accumulated noise remains controlled.

Examples:

- $\alpha_t = 1/t$: satisfies both;
- constant $\alpha > 0$: satisfies $\sum_t \alpha_t = \infty$ but fails $\sum_t \alpha_t^2 < \infty$;
- $\alpha_t = t^{-p}$ satisfies both for $1/2 < p \leq 1$.

Deep RL often uses constant or scheduled Adam learning rates, so classical almost-sure convergence theorems do not directly apply.

Stochastic gradient descent

For loss $L(\theta)$, full-gradient descent is

$$\theta_{t+1} = \theta_t - \alpha_t \nabla L(\theta_t).$$

SGD uses estimator $\hat{g}_t$:

$$\theta_{t+1} = \theta_t - \alpha_t \hat{g}_t.$$

An ideal condition is

SGD vs Full-Batch Gradient Descent: noise lets SGD wander but it still converges

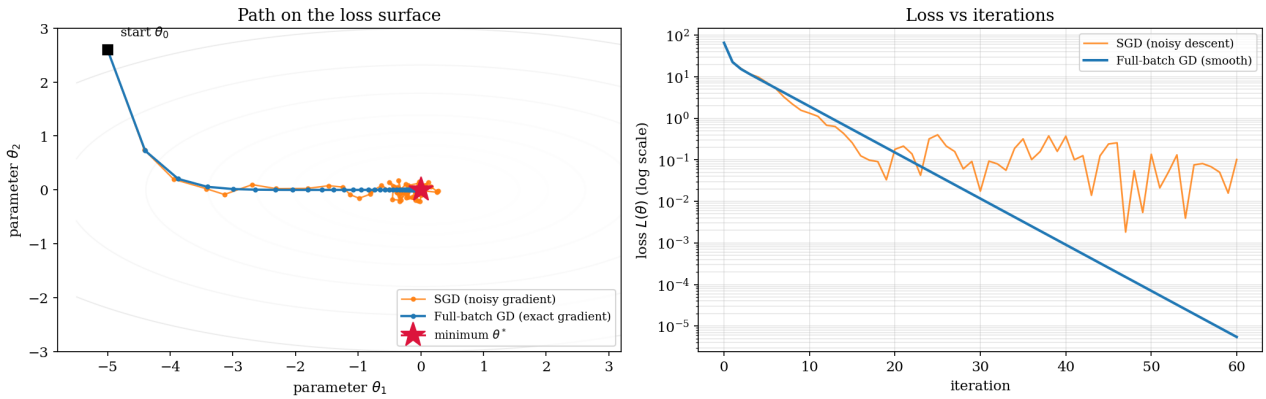


Figure 40: Typical stochastic-gradient convergence behavior

$$\mathbb{E}[\hat{g}_t \mid \theta_t] = \nabla L(\theta_t).$$

RL can violate this because:

- bootstrapped targets depend on learned parameters;
- replay data come from old policies;
- policy-gradient advantages are estimated;
- minibatches are correlated;
- the data distribution changes with the policy.

An optimizer can be implemented perfectly while following a biased or stale learning signal.

Bias, variance, consistency, and mean-squared error

For estimator $\hat{\theta}$ of parameter θ ,

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta.$$

The mean-squared error decomposes as

$$\mathbb{E}[(\hat{\theta} - \theta)^2] = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2.$$

An estimator can be biased but have lower MSE in finite samples. Weighted importance sampling and clipped ratios are common examples.

Consistency means

$$\hat{\theta}_n \xrightarrow{P} \theta$$

as sample size grows. Unbiasedness at finite n does not imply low variance or practical usefulness.

Adam

Adam tracks exponentially weighted first and second moments:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Bias-corrected moments are

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

The update is

$$\theta_{t+1} = \theta_t - \alpha \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t + \varepsilon}}.$$

Adam rescales gradients coordinate-wise. It does not correct a biased target, missing support, or reward misspecification.

18.3 Importance sampling

Importance Sampling: $\mathbb{E}_p[f] = \mathbb{E}_q[\frac{p}{q}f]$ — variance explodes when p and q disagree

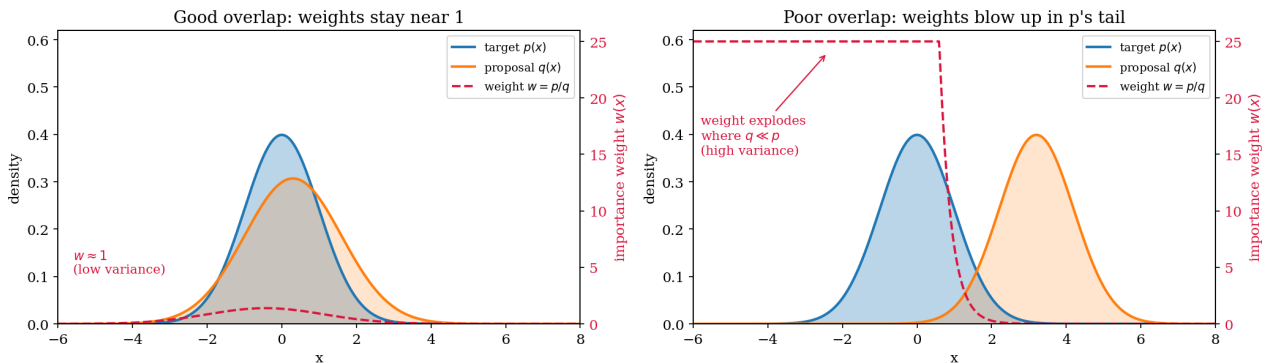


Figure 41: Importance sampling as density-ratio reweighting

Intuition first: reweight what you saw

Suppose data come from proposal distribution q , but the desired expectation is under target distribution p .

If $q(x) > 0$ whenever $p(x)f(x) \neq 0$, then

$$\begin{aligned} \mathbb{E}_p[f(X)] &= \int f(x)p(x) dx \\ &= \int f(x) \frac{p(x)}{q(x)} q(x) dx \\ &= \mathbb{E}_q[w(X)f(X)], \end{aligned}$$

where

$$w(x) = \frac{p(x)}{q(x)}.$$

The support condition is essential. Reweighting cannot create an event that q never produces.

Ordinary importance sampling

With samples $X_i \sim q$,

$$\hat{\mu}_{\text{OIS}} = \frac{1}{N} \sum_{i=1}^N w_i f(X_i).$$

Under integrability and support assumptions, it is unbiased:

$$\mathbb{E}[\hat{\mu}_{\text{OIS}}] = \mathbb{E}_p[f(X)].$$

Its variance is

$$\text{Var}_q(wf) = \mathbb{E}_q[w^2 f^2] - \mathbb{E}_p[f]^2.$$

Large weights dominate the second moment.

Self-normalized importance sampling

Weighted or self-normalized IS is

$$\hat{\mu}_{\text{SNIS}} = \frac{\sum_i w_i f(X_i)}{\sum_i w_i}.$$

It is generally biased at finite N but often has lower variance and is consistent under standard conditions.

Effective sample size of weights

A common diagnostic is

$$\text{ESS} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}.$$

If all weights are equal, ESS is N . If one weight dominates, ESS approaches 1.

ESS is a summary of weight concentration, not a full uncertainty analysis.

Off-policy action ratios

For target policy π and behavior policy b ,

$$\rho_t = \frac{\pi(A_t | S_t)}{b(A_t | S_t)}.$$

A trajectory ratio is

$$\rho_{0:T-1} = \prod_{t=0}^{T-1} \rho_t.$$

Even mild per-step mismatch compounds exponentially. If $\rho_t = 1.1$ for 100 steps,

$$1.1^{100} \approx 13,781.$$

If $\rho_t = 0.9$,

$$0.9^{100} \approx 2.66 \times 10^{-5}.$$

This is why long-horizon off-policy evaluation is statistically difficult.

Per-decision importance sampling

Rewards at time t depend only on actions up to t . Per-decision IS uses shorter products for earlier rewards:

$$\widehat{V} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \gamma^t \rho_{0:t}^{(i)} R_{t+1}^{(i)}.$$

This can reduce variance relative to multiplying every reward by the full trajectory ratio.

Clipping and truncation

Define

$$\bar{\rho}_t = \min(c, \rho_t).$$

Clipping controls variance but introduces bias. The threshold should be reported and sensitivity-tested.

V-trace and Retrace use carefully structured truncated corrections. PPO also clips a probability ratio, but it does so inside a policy surrogate; PPO clipping is not an ordinary unbiased estimator of target-policy return.

Support diagnostics

Before trusting an off-policy estimate, inspect:

- fraction of target actions with very low behavior probability;
- ratio quantiles;
- maximum ratio;
- trajectory and per-decision ESS;
- sensitivity to clipping;
- behavior-policy estimation error;
- whether the dataset is a mixture of policies.

18.4 Entropy, cross-entropy, and KL divergence

Entropy

For a discrete distribution p ,

$$\mathcal{H}(p) = - \sum_x p(x) \log p(x) = \mathbb{E}_p[-\log p(X)].$$

Entropy is largest for the uniform distribution over a finite support and zero for a deterministic distribution.

For continuous densities, differential entropy can be negative and changes under reparameterization. It should not be interpreted exactly like discrete entropy.

Entropy: maximal for uniform (most random), minimal for peaked (deterministic)

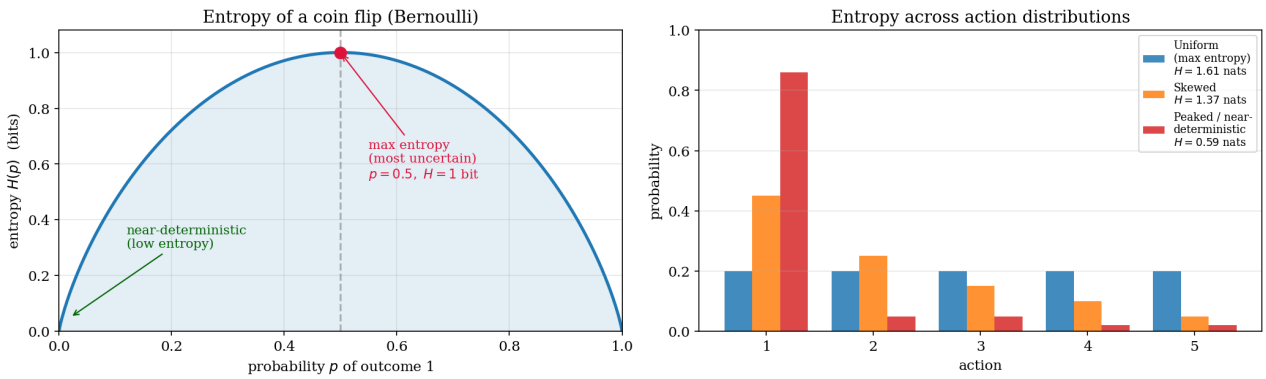


Figure 42: Entropy across probability distributions with different concentration

Cross-entropy

The cross-entropy of target p relative to model q is

$$H(p, q) = -\mathbb{E}_p[\log q(X)].$$

Maximum likelihood minimizes empirical cross-entropy.

KL divergence

$$D_{\text{KL}}(p\|q) = \mathbb{E}_p \left[\log \frac{p(X)}{q(X)} \right].$$

It satisfies

$$D_{\text{KL}}(p\|q) \geq 0,$$

with equality if $p = q$ almost everywhere.

The decomposition

$$H(p, q) = \mathcal{H}(p) + D_{\text{KL}}(p\|q)$$

shows why minimizing cross-entropy over q is equivalent to minimizing forward KL when p is fixed.

Direction matters

Forward KL

$$D_{\text{KL}}(p\|q)$$

penalizes q heavily when it assigns low probability where p has mass. It tends to be mass-covering.

Reverse KL

$$D_{\text{KL}}(q\|p)$$

can prefer one mode of a multimodal p when q cannot cover all modes. It tends to be mode-seeking.

These are tendencies under constrained approximation families, not universal colloquial definitions.

Forward vs Reverse KL: fitting one Gaussian to a two-mode target

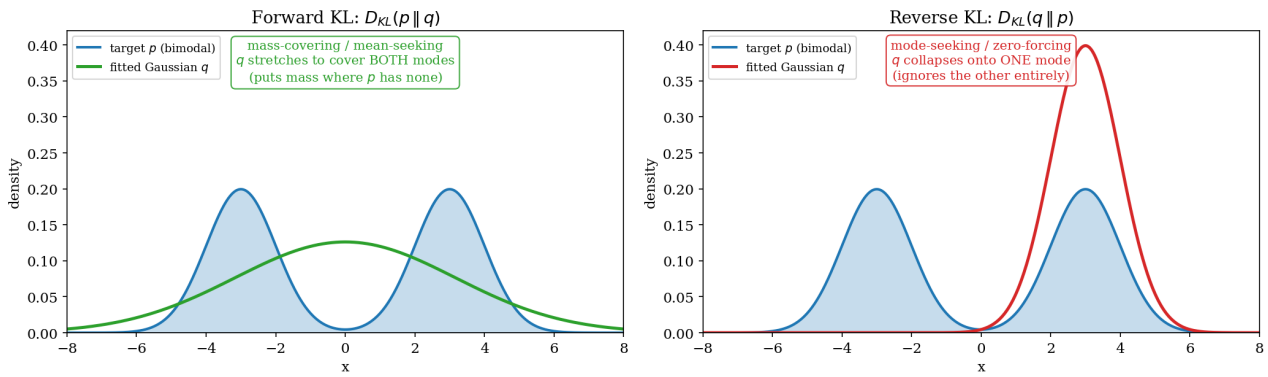


Figure 43: Forward and reverse KL divergence prefer different approximations

KL in RL

- TRPO approximately constrains $D_{\text{KL}}(\pi_{\text{old}} \parallel \pi_{\text{new}})$ on sampled states.
- PPO often monitors the same direction while clipping sampled ratios.
- RLHF often penalizes $D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$.
- Maximum-entropy RL can be expressed as KL projection toward a Boltzmann distribution.
- DPO uses policy-to-reference log ratios as implicit rewards.

Always state which distribution supplies the expectation.

Pinsker's inequality

For total variation distance

$$\|p - q\|_{\text{TV}} = \frac{1}{2} \sum_x |p(x) - q(x)|,$$

Pinsker's inequality states

$$\|p - q\|_{\text{TV}} \leq \sqrt{\frac{1}{2} D_{\text{KL}}(p \parallel q)}.$$

A small KL therefore implies a small total-variation distance. The reverse implication requires additional conditions.

Fisher information

For policy π_{θ} ,

$$F(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(A \mid S) \nabla_{\theta} \log \pi_{\theta}(A \mid S)^{\top}].$$

It is the local Hessian of KL under regularity conditions and defines the natural-gradient geometry.

Stable log computations

For logits x_i , compute

$$\log \sum_i \exp x_i = m + \log \sum_i \exp(x_i - m),$$

where

$$m = \max_i x_i.$$

This log-sum-exp trick prevents overflow and underflow.

For products of probabilities, work in log space:

$$\log \prod_t p_t = \sum_t \log p_t.$$

18.5 Contractions and fixed points

Contraction mapping

Contraction Mapping (Banach Fixed Point): iterating a contraction converges geometrically

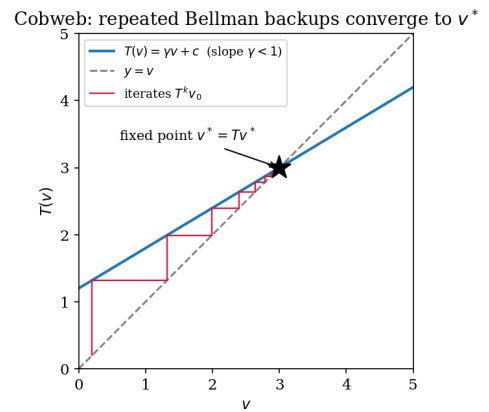
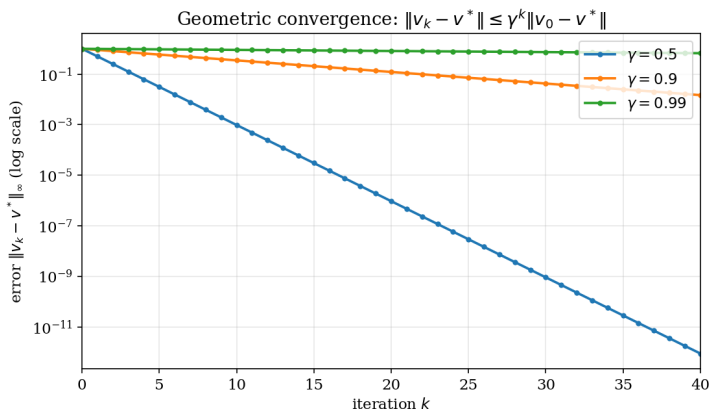


Figure 44: A contraction repeatedly bringing points toward a unique fixed point

Let (X, d) be a complete metric space. A map $T : X \rightarrow X$ is a contraction with modulus $c < 1$ if

$$d(Tx, Ty) \leq c d(x, y)$$

for all x, y .

Banach fixed-point theorem

A contraction has a unique fixed point x^* , and iteration

$$x_{k+1} = Tx_k$$

converges from any x_0 .

Moreover,

$$d(x_k, x^*) \leq c^k d(x_0, x^*).$$

Proof sketch

Consecutive iterates satisfy

$$d(x_{k+1}, x_k) \leq c^k d(x_1, x_0).$$

For $m > n$,

$$\begin{aligned} d(x_m, x_n) &\leq \sum_{k=n}^{m-1} d(x_{k+1}, x_k) \\ &\leq \sum_{k=n}^{m-1} c^k d(x_1, x_0) \\ &\leq \frac{c^n}{1-c} d(x_1, x_0). \end{aligned}$$

Thus (x_k) is Cauchy and converges in a complete space. Continuity of T gives $Tx^* = x^*$. If y^* is another fixed point,

$$d(x^*, y^*) = d(Tx^*, Ty^*) \leq c d(x^*, y^*),$$

which forces equality of the fixed points.

Bellman expectation operator

For fixed policy π ,

$$(T^\pi v)(s) = \sum_a \pi(a | s) \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) v(s') \right].$$

For two value functions,

$$\begin{aligned} |(T^\pi v)(s) - (T^\pi w)(s)| &\leq \gamma \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) |v(s') - w(s')| \\ &\leq \gamma \|v - w\|_\infty. \end{aligned}$$

Taking the maximum over s gives

$$\|T^\pi v - T^\pi w\|_\infty \leq \gamma \|v - w\|_\infty.$$

Thus, for $\gamma < 1$, policy evaluation converges to the unique fixed point v_π .

Bellman optimality operator

$$(T^*v)(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) v(s') \right].$$

Using

$$|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|,$$

we obtain

$$\|T^*v - T^*w\|_\infty \leq \gamma \|v - w\|_\infty.$$

Its unique fixed point is v_* . This proves value-iteration convergence in finite discounted MDPs.

Residual-to-error bound

If T is a γ -contraction with fixed point v^* , then

$$\|v - v^*\| \leq \frac{\|Tv - v\|}{1 - \gamma}.$$

Proof:

$$\begin{aligned} \|v - v^*\| &\leq \|v - Tv\| + \|Tv - Tv^*\| \\ &\leq \|v - Tv\| + \gamma\|v - v^*\|. \end{aligned}$$

Rearrange. This converts a Bellman residual into a value-error certificate under contraction assumptions.

What changes at $\gamma = 1$

The discounted sup-norm contraction proof disappears. Episodic undiscounted tasks can still be well posed under proper termination or finite horizon, but they require different arguments.

18.6 Basic statistical intervals for RL evaluation

Independent unit first

A confidence interval is only meaningful after identifying the independent observations.

Possible units include:

- independent training seeds;
- independent evaluation episodes conditional on one fixed policy;
- independent tasks;
- independent environment instances;
- independently trained populations.

Raw timesteps inside one trajectory are not independent experimental replications.

Mean across independent runs

For independent outcomes $X_1, \dots, X_K$, define

$$\bar{X} = \frac{1}{K} \sum_{i=1}^K X_i,$$

and sample standard deviation

$$s = \sqrt{\frac{1}{K-1} \sum_{i=1}^K (X_i - \bar{X})^2}.$$

A classical t interval is

$$\bar{X} \pm t_{K-1, 1-\alpha/2} \frac{s}{\sqrt{K}}.$$

With very small K , this interval is wide and sensitive to heavy tails. Show individual runs.

Bernoulli success rates

For x successes in n independent evaluation episodes,

$$\hat{p} = x/n.$$

A Wilson interval is preferable to the naive normal interval at small n or near 0 and 1. With $z = z_{1-\alpha/2}$, its center and half-width are

$$\frac{\hat{p} + z^2/(2n)}{1 + z^2/n}$$

and

$$\frac{z}{1 + z^2/n} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n} + \frac{z^2}{4n^2}}.$$

Paired comparisons

If methods A and B are evaluated on matched tasks or common random scenarios, analyze differences

$$D_i = X_i^{(A)} - X_i^{(B)}.$$

A confidence interval on $\mathbb{E}[D]$ removes shared task difficulty and can be more precise than comparing unrelated means.

The pairing must be real. Reusing the same integer seed in two different implementations does not automatically create controlled common randomness.

Bootstrap units

Bootstrap the highest-level independent units relevant to the claim:

- tasks for a task-generalization claim;
- training seeds for run-to-run reliability;
- episodes for a fixed-policy evaluation claim.

Do not resample raw timesteps as if they were iid.

Checkpoint-selection bias

If many noisy checkpoints are evaluated and the best is reported, selection favors positive noise.

The evaluation rule should be fixed before looking at test results, for example:

- final checkpoint;
- average of the final m checkpoints;
- validation-selected checkpoint, followed by one test evaluation;
- area under the learning curve under a fixed budget.

19. Implementation and Debugging

19.1 Treat the environment interface as a contract

Before debugging an algorithm, verify the environment.

Observation contract

Record:

- shape and dtype;
- bounds and normalization;
- whether observations are Markov;
- whether frame stacking or recurrence is required;
- whether hidden simulator state leaks into training.

Action contract

Record:

- discrete or continuous;
- valid bounds;
- action scaling;
- whether illegal actions are masked;
- whether the environment clips actions silently.

Reward contract

Record:

- scale and sign;
- dense versus sparse components;
- terminal bonus or penalty;
- whether rewards are clipped or normalized;
- whether the logged reward is raw or transformed.

Termination contract

Distinguish:

- **termination:** the task ended in the MDP;
- **truncation:** the rollout stopped for an external reason such as a time limit.

For a target,

$$y_t = r_t + \gamma(1 - d_{t+1})V(s_{t+1}),$$

set $d_{t+1} = 1$ only for true terminal transitions unless the task definition explicitly treats the time limit as terminal.

A time-limit bug can bias values near the horizon downward.

Reset contract

Check:

- initial-state distribution;
- randomization range;
- whether hidden state is fully reset;
- whether vector environments reset asynchronously;
- whether evaluation resets differ from training resets.

19.2 Build tests before full training

Test 1: deterministic stepping

With a fixed seed and action sequence, verify that the environment produces the expected trajectory when deterministic behavior is promised.

Test 2: terminal target

Construct one terminal transition and one truncation. Assert:

$$y_{\text{terminal}} = r,$$

but

$$y_{\text{truncated}} = r + \gamma V(s').$$

Test 3: exact tiny MDP

Create a two- or three-state MDP whose values can be solved by hand. Compare:

- direct linear solve;
- iterative policy evaluation;
- Monte Carlo estimates;
- TD estimates.

Test 4: frozen-batch overfit

A supervised component should fit a tiny frozen batch:

- critic regression;
- dynamics prediction;
- behavior cloning;
- reward-model classification.

Failure suggests a target, masking, architecture, or gradient-flow bug.

A PPO surrogate need not converge to zero because it is not an ordinary fixed-label regression loss.

Test 5: random and scripted baselines

Compare against:

- random actions;
- no-op or constant action;
- a simple heuristic;
- an oracle or planning upper bound when available.

A learned agent that does not beat a trivial baseline should not be scaled up.

Test 6: gradient-flow audit

Verify:

- intended parameters receive nonzero gradients;
- target networks do not receive gradients;
- detached advantages remain detached if intended;
- actor gradients flow through actions in reparameterized methods;
- recurrent hidden states are detached at the intended boundaries.

Test 7: one-update arithmetic

For a hand-constructed batch, compute one update target manually and compare every intermediate tensor:

- returns;
- masks;
- log probabilities;
- ratios;
- advantages;
- losses;
- gradient norms.

This catches more bugs than staring at a full learning curve.

19.3 Logging as a diagnostic instrument

RL Debugging Diagnostics: what healthy vs unhealthy training looks like

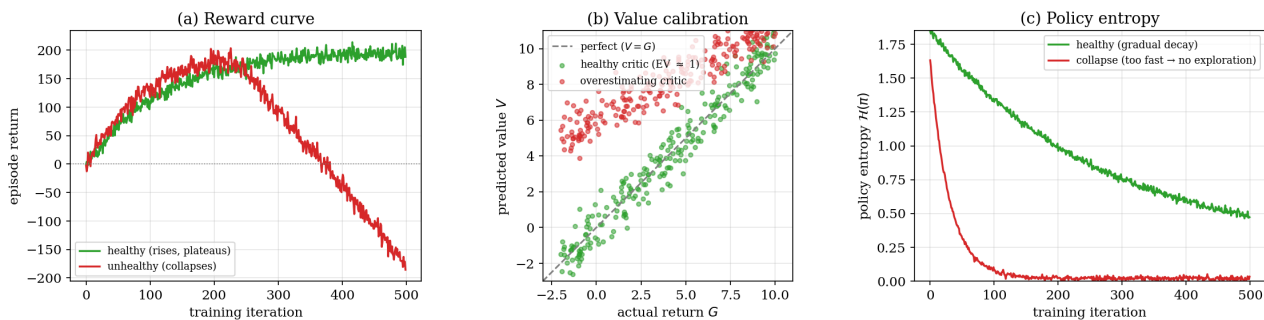


Figure 45: Training curves that reveal common reinforcement learning failure modes

A scalar return curve is necessary but insufficient.

Environment and data distribution

Log:

- episodic return;
- episode length;
- success rate;
- terminal cause;
- state and action summaries;
- replay age or policy version;
- fraction of invalid or clipped actions.

Critic

Log:

- value or Q loss;
- predicted values and empirical returns;
- explained variance;
- TD-error quantiles;
- target and prediction scales;
- critic disagreement when using ensembles.

Explained variance is

$$1 - \frac{\text{Var}(G - V)}{\text{Var}(G)}.$$

It can be misleading when return variance is tiny, so inspect scatter plots and calibration by return bucket.

Policy

For stochastic policies, log:

- entropy;
- action probabilities or Gaussian standard deviations;
- KL to the previous policy;
- log-probability range;
- fraction of saturated tanh actions.

For PPO, also log:

- ratio median, p95, and p99;
- clip fraction;
- raw advantage percentiles;
- KL by optimization epoch.

Optimization

Log:

- gradient norm before and after clipping;
- parameter norm;
- learning rate;
- fraction of NaN or infinite tensors;
- optimizer moments when diagnosing scale problems;
- throughput and update-to-data ratio.

Model-based methods

Log:

- one-step validation loss;
- free-running rollout error by horizon;
- reward and terminal prediction error;
- ensemble disagreement;
- predicted-versus-realized plan return;
- fraction of imagined samples used in each update.

19.4 A symptom-to-test debugging table

Symptom	Plausible causes	First tests
Return never exceeds random	reward sign, action scaling, target bug, no exploration	inspect raw transitions; solve a tiny environment
Return improves then crashes	excessive policy drift, critic overfit, replay instability	KL by epoch, ratio tails, value calibration

Symptom	Plausible causes	First tests
Critic loss falls while policy worsens	biased target, reward bug, support mismatch	compare values to realized returns; inspect action coverage
Entropy collapses immediately	excessive learning rate, weak entropy term, bad initialization	action distributions before/after one update
Q-values explode	terminal-mask bug, overestimation, reward scale, target leakage	terminal unit test; target statistics; Double Q ablation
Model loss low but MPC fails	planner exploits model error, horizon too long	predicted-versus-real return; real-simulator planner
One seed succeeds	rare exploration event or unstable optimization	more independent seeds; trajectory-level comparison
NaNs appear	invalid log probability, zero variance, overflow, unbounded target	finite checks at every pipeline stage
Training slows over time	replay bottleneck, environment leak, compilation churn	profiler and queue statistics

Debug a causal chain rather than changing many hyperparameters at once.

19.5 Reward design and reward auditing

Reward is the optimization target

An agent maximizes the implemented return, not the designer's prose description.

A reward audit should ask:

1. Can the agent obtain reward without accomplishing the task?
2. Can it delay termination to collect more shaping reward?
3. Can it trigger terminal bonuses repeatedly?
4. Are safety costs discounted too heavily?
5. Does reward clipping erase meaningful distinctions?
6. Can the agent manipulate the measurement process?

Potential-based shaping

A shaping term

$$F(s, a, s') = \gamma\Phi(s') - \Phi(s)$$

preserves optimal policies under the standard theorem assumptions.

The shaped return telescopes:

$$\sum_{t=0}^{T-1} \gamma^t F_t = -\Phi(S_0) + \gamma^T \Phi(S_T).$$

With compatible terminal handling, the difference between policies is unaffected except for state-dependent constants.

A learned, changing, or history-dependent potential may not satisfy the theorem.

Reward scaling

Multiplying reward by c scales:

- policy advantages;
- critic targets;
- gradient magnitudes;
- relative strength of entropy and KL terms;
- clipping and optimizer dynamics.

Reward normalization is therefore not a neutral preprocessing step. Report it as part of the algorithm.

19.6 Experimental design

Define the estimand

Examples:

- expected final return after 10^6 environment steps;
- success rate of the final policy;
- area under the learning curve;
- median score across tasks;
- worst-decile performance under dynamics shift;
- compute needed to reach a target score.

A plot without an estimand encourages post-hoc interpretation.

Match budgets

When comparing algorithms, hold fixed the resource relevant to the claim:

- environment interactions for sample efficiency;
- wall-clock time for practical speed;
- accelerator hours for compute efficiency;
- model size for representation comparisons;
- number of reward-model or verifier calls for LLM post-training.

One comparison may require multiple budget views because methods allocate computation differently.

Separate training, validation, and test decisions

Use validation environments or seeds for:

- hyperparameter selection;
- early stopping;
- checkpoint selection;
- architecture choice.

Reserve test environments or seeds for the final estimate.

Repeatedly checking the test set turns it into training data.

Seeds

A training seed changes more than initialization. It can alter:

- exploration trajectories;
- minibatch order;
- environment randomness;

- network initialization;
- parallel scheduling;
- stochastic policy samples.

Choose the number of seeds based on expected run-to-run variance and desired precision. There is no universal magic number.

Evaluation episodes

For each trained policy, use enough independent evaluation episodes to estimate its conditional performance. Then treat independent training runs as separate units for algorithm reliability.

Do not pool all episodes across seeds and pretend the resulting count is the number of independent training replications.

Smoothing learning curves

Smoothing is for visualization, not inference.

Report:

- smoothing window;
- unsmoothed or lightly smoothed supplemental curves;
- seed-level curves or uncertainty bands;
- exact final summary metric.

Hyperparameter search accounting

Report:

- search space;
- number of trials;
- selection metric;
- validation budget;
- whether all algorithms received comparable tuning effort.

A heavily tuned method and an untuned baseline are not a clean algorithm comparison.

19.7 Reproducibility record

A minimal experiment record should include:

- code commit;
- environment name and version;
- dependency lock file;
- hardware and precision;
- all random seeds;
- model architecture;
- optimizer and schedules;
- data-collection and update ratio;
- terminal and truncation handling;
- reward transformations;
- evaluation and checkpoint rules;
- raw metrics, not only figures.

Configuration immutability

Save the resolved configuration at run start. Do not rely on a default configuration that may change later.

Artifact integrity

Store:

- checkpoints;
- normalization statistics;
- replay metadata if needed;
- exact environment wrappers;
- evaluation scripts;
- model cards or experiment notes.

A checkpoint without its observation normalization can be unusable.

Determinism caveat

Setting seeds does not guarantee bitwise reproducibility across hardware, library versions, asynchronous simulators, or nondeterministic accelerator kernels. Report the level of reproducibility actually achieved.

19.8 Libraries and implementation strategy

Common ecosystems include:

Tool family	Typical use
Gymnasium-style environments	standardized environment API
Stable-Baselines3	reliable standard baselines
CleanRL	compact, readable reference implementations
RLlib	distributed training and production orchestration
Tianshou	modular PyTorch research workflows
JAX/Brax/PureJaxRL-style stacks	compiled, massively vectorized experiments

Library APIs evolve. Pin exact versions and consult the documentation corresponding to the installed release.

Recommended learning sequence

1. Implement tabular algorithms from scratch.
2. Implement one small policy-gradient method from scratch.
3. Reproduce a trusted baseline on a standard environment.
4. Compare tensors and diagnostics against the trusted implementation.
5. Only then introduce architectural or algorithmic novelty.

Using a library is not a substitute for understanding targets and masks. Reimplementing every infrastructure component is not a substitute for reliable baselines.

19.9 Compute and systems considerations

Main bottlenecks

- environment stepping;
- policy inference during rollout;
- neural-network optimization;
- replay sampling and transfer;
- model or reward inference;
- interprocess communication;
- evaluation.

Update-to-data ratio

For replay-based methods, define roughly

$$\text{UTD} = \frac{\text{gradient updates} \times \text{batch size}}{\text{new environment transitions}}.$$

Higher UTD reuses data more but can overfit stale data or critic errors.

Parallel environments

More environments can improve state diversity and throughput. They also change:

- batch size;
- policy lag;
- optimizer update frequency;
- effective horizon represented per environment;
- correlation structure.

A throughput change is also an algorithmic change unless these quantities are controlled.

Replay memory

Memory scales with

$$N_{\text{transitions}} \times \text{bytes per transition}.$$

Image replay can dominate memory. Common strategies include frame compression, shared frame storage, reduced precision, and prioritized disk-backed storage.

Profiling rule

Measure before optimizing. A faster learner is irrelevant if environment collection is the bottleneck, and a faster environment is irrelevant if the GPU waits on small unbatched inference calls.

19.10 A complete preflight checklist

Environment

- ☐ Observation and action shapes verified.
- ☐ Reward sign and magnitude verified.
- ☐ Termination and truncation tested separately.
- ☐ Reset distribution inspected.
- ☐ Random and scripted baselines recorded.

Algorithm

- ☐ One-update arithmetic checked by hand.
- ☐ Target networks detached.
- ☐ Policy log probabilities finite.
- ☐ Action scaling correct.
- ☐ Tiny supervised components can overfit.
- ☐ Exact tiny MDP solved.

Diagnostics

- ☐ Returns and episode lengths logged.
- ☐ Critic calibration logged.
- ☐ Entropy or action spread logged.
- ☐ Gradient norms logged.
- ☐ KL and ratio tails logged for policy updates.
- ☐ Replay age or model uncertainty logged when relevant.

Evaluation

- ☐ Estimand predeclared.
- ☐ Compute or interaction budget matched.
- ☐ Independent training seeds chosen.
- ☐ Evaluation episodes sufficient.
- ☐ Checkpoint rule fixed.
- ☐ Test data separated from tuning.

Reproducibility

- ☐ Code commit saved.
- ☐ Resolved configuration saved.
- ☐ Dependencies pinned.
- ☐ Environment versions recorded.
- ☐ Raw metrics and checkpoints retained.

Common misconceptions

“A large replay batch is approximately iid because it was shuffled”

Shuffling changes order, not the data-generating dependence or policy mixture.

“An unbiased estimator is automatically preferable”

An unbiased estimator can have unusably large variance. Finite-sample MSE and support matter.

“A low training loss means the policy should improve”

The loss may use biased targets, stale data, or a misspecified reward.

“Five seeds are enough”

The needed number depends on run variance and the precision of the claim.

“A smooth reward curve is statistically reliable”

Smoothing can hide instability and says nothing about independent-run uncertainty.

“Setting a random seed makes an experiment reproducible”

It helps, but software, hardware, wrappers, and nondeterministic kernels also matter.

“More compute only makes the same algorithm run faster”

Changing actors, batch sizes, replay ratios, or update frequency changes the learning process.

Chapter summary

1. Value functions are conditional expectations; Bellman equations rely on the tower property and Markov state design.
2. RL convergence proofs often decompose updates into contraction drift plus martingale-difference noise.
3. Importance sampling requires support and can suffer catastrophic weight concentration.
4. KL is directional and serves different roles in trust regions, entropy regularization, and reference-policy control.
5. Bellman contractions provide unique fixed points and geometric convergence in discounted tabular problems.
6. Evaluation must aggregate over independent experimental units, not raw timesteps.
7. Reliable RL engineering begins with environment contracts, exact tiny tests, target audits, and distributional diagnostics.
8. Reproducibility requires preserving the complete experiment, not only the final checkpoint.

Knowledge check

1. Use the tower property to derive the Bellman expectation equation.
2. Explain why a martingale-difference sequence need not be independent.
3. Give a biased estimator with lower MSE than an unbiased alternative.
4. Construct a two-point example where importance sampling is undefined because support is missing.
5. Explain why trajectory ratios can have high variance even when every per-step ratio is near one.
6. Derive the relationship between cross-entropy and forward KL.
7. Use the contraction property to derive the residual-to-value-error bound.
8. Identify the independent units in a benchmark with ten tasks, five training seeds per task, and one hundred evaluation episodes per seed.
9. Explain why pooling all evaluation episodes across seeds understates algorithmic uncertainty.
10. Give a unit test that distinguishes terminal transitions from time-limit truncations.

Exercises**1. Conditional-expectation derivation**

For a finite MDP, write every conditioning step needed to derive

$$q_{\pi}(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a').$$

2. Importance-weight collapse

Simulate trajectory ratios for horizons $T \in \{10, 50, 100, 500\}$ when log ratios are small random perturbations around zero. Plot the ratio distribution and ESS. Compare ordinary, self-normalized, and clipped estimators.

3. Bellman residual certificate

For a small discounted MDP:

1. choose an approximate value v ;
2. compute $\|T^\pi v - v\|_\infty$;
3. compute the true error $\|v - v_\pi\|_\infty$;
4. verify the residual bound.

4. Terminal-mask test suite

Write tests for:

- true terminal success;
- true terminal failure;
- time-limit truncation;
- vectorized partial reset;
- continuing task with artificial rollout cut.

5. Evaluation hierarchy

Using synthetic multi-task, multi-seed scores, compare:

- naive standard error over every episode;
- standard error over seed means;
- stratified bootstrap over tasks and seeds;
- paired differences between two methods.

Explain which uncertainty each interval represents.

6. Reproducibility audit

Choose one earlier lab and create a complete reproducibility package containing configuration, environment wrappers, dependency lock, seeds, logs, checkpoint rule, and evaluation script. Re-run it in a clean environment and document every discrepancy.

Primary readings

- Robbins and Monro (1951), **A Stochastic Approximation Method**.
- Bertsekas and Tsitsiklis, **Neuro-Dynamic Programming**, stochastic-approximation chapters.
- Owen, **Monte Carlo Theory, Methods and Examples**, importance-sampling chapters.
- Sutton and Barto, **Reinforcement Learning: An Introduction**, mathematical appendices and relevant chapters.
- Henderson et al. (2018), **Deep Reinforcement Learning that Matters**.
- Colas et al. (2019), **How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments**.
- Agarwal et al. (2021), **Deep Reinforcement Learning at the Edge of the Statistical Precipice**.
- Patterson et al. (2023), **Empirical Design in Reinforcement Learning**.

Next: Part VIII — Reinforcement Learning in Large Language Models.

Part VIII — Reinforcement Learning in Large Language Models

Lecture opening: The action can be a token or a whole response

For one prompt and a complete response, the simplest model is a contextual bandit. For token-by-token generation, it is a decision process with deterministic prefix transitions. For tools and multi-turn interaction, external state and partial observation return. These are different useful abstractions of the same application; choose the one that matches the objective and feedback.

Re-derive the KL-regularized optimum in A.13 before DPO, and the finite-group estimator identities in A.14 before GRPO. Do not equate a verifier score with task validity or a standardized advantage with an unchanged policy gradient. The current research chapter supplies dated developments; this chapter supplies the enduring formulations needed to read them.

Learning objectives

By the end of this chapter, you should be able to:

- formulate token generation and multi-turn tool use as decision processes;
- derive the Bradley-Terry reward-model loss;
- derive the KL-regularized optimal policy and the DPO objective;
- explain PPO-style RLHF and identify its estimator and systems components;
- derive a group-relative policy-gradient estimator and explain GRPO's edge cases;
- distinguish learned outcome rewards, deterministic verifiers, and process reward models;
- explain rejection sampling, self-training, and search as policy-improvement operators;
- diagnose reward hacking, verifier exploitation, length incentives, and judge shift;
- compare online RL, offline preference optimization, and hybrid post-training.

Evidence boundary

The mathematical constructions in this chapter are general. Named systems and empirical results are tied to the cited paper, model version, data, evaluator, and date. A public technical report establishes what its authors report; it does not reveal every component of a proprietary training pipeline. Benchmark numbers should never be compared without matching sampling budget, pass@ k , tool access, token limits, and evaluation protocol.

20. The RLHF Pipeline

Intuition first: comparisons are easier than complete reward specifications

For a game, a score may be available directly. For an assistant response, the desired behavior may include correctness, relevance, clarity, honesty, safety, style, and instruction following. Writing one complete program that evaluates all of these is usually impractical.

Humans can often make a narrower judgment:

Given these two responses to the same prompt, which is better?

RLHF converts many such comparisons into a learned reward signal and then optimizes a language-model policy against that signal.

The Three-Stage RLHF Pipeline: from demonstrations to a preference-aligned policy

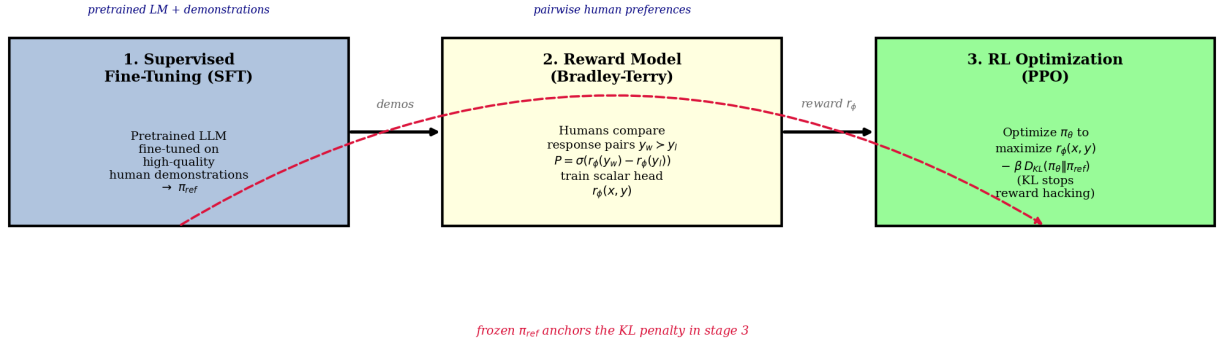


Figure 46: The end-to-end reinforcement learning from human feedback pipeline

The signal remains a proxy. It reflects the prompts shown, the labeler population, the comparison interface, the response distribution, and the reward-model inductive bias.

Language generation as a decision process

For prompt x and generated tokens $y = (y_1, \dots, y_T)$, define

- state:

$$s_t = (x, y_{<t});$$

- action:

$$a_t = y_t \in \mathcal{V};$$

- transition:

$$s_{t+1} = (x, y_{\leq t});$$

- terminal event: end-of-sequence, or a maximum length that explicitly defines the task horizon; an external rollout cutoff is a truncation, not automatically a terminal event;
- task reward: often assigned to the complete response.

The transition is deterministic once the token is selected. The uncertainty comes from the policy and, in multi-turn settings, from users, tools, or external environments.

The sequence probability factorizes as

$$\pi_\theta(y \mid x) = \prod_{t=1}^T \pi_\theta(y_t \mid x, y_{<t}).$$

Therefore

$$\log \pi_\theta(y \mid x) = \sum_{t=1}^T \log \pi_\theta(y_t \mid x, y_{<t}).$$

Contextual-bandit and token-MDP views

If a complete response is treated as one action, post-training resembles a contextual bandit:

- context: prompt x ;

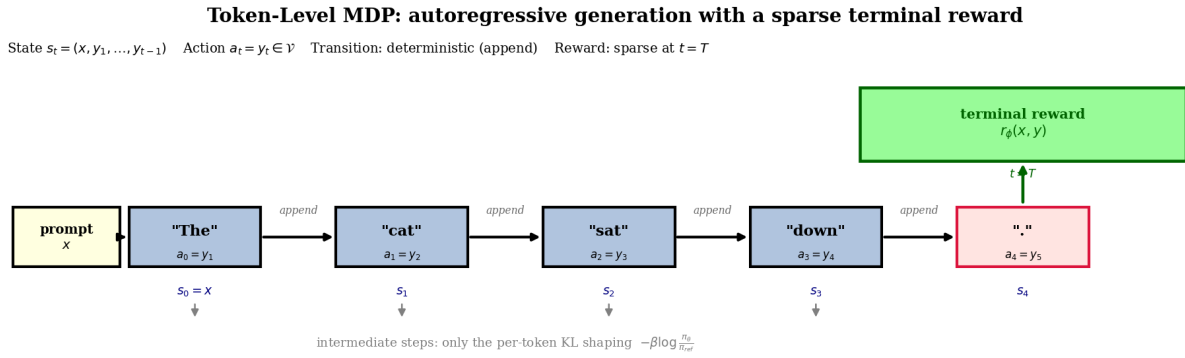


Figure 47: Language generation represented as a token-level Markov decision process

- action: response y ;
- reward: $r(x, y)$.

If each token is an action, it is a finite-horizon MDP with sparse terminal task reward and dense regularization.

Both views are useful:

- the bandit view simplifies the KL-regularized optimum and DPO derivation;
- the token-MDP view is needed for policy gradients, value heads, credit assignment, and per-token KL penalties.

Stage 1: supervised fine-tuning

Given demonstration dataset $\mathcal{D}_{\text{demo}}$, minimize

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{demo}}} \left[\sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}) \right].$$

SFT teaches response format, task conventions, and high-quality modes present in the demonstrations.

It is imitation learning. Its limitations include:

- coverage restricted to demonstrated responses;
- no direct preference among multiple plausible outputs unless represented in data frequency;
- exposure to the model's own state distribution only through generated training data;
- possible overfitting to stylistic artifacts of demonstrations.

The SFT model often becomes the reference policy π_{ref} .

Stage 2: preference data

For prompt x , collect two responses y_1, y_2 and a preference label. Write

$$y_w \succ y_l$$

when y_w is preferred.

Preference data can come from:

- human annotators;
- domain experts;
- deterministic task verifiers;
- AI judges;
- mixtures of these sources.

Each source measures something different. A unit-test result is not equivalent to a human preference, and an AI judge is not equivalent to ground truth.

Bradley-Terry preference model

Assign scalar reward $r_\phi(x, y)$. Bradley-Terry assumes

$$P_\phi(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)),$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

The negative log-likelihood is

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l)} \log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

Reward identifiability

For any prompt-dependent constant $c(x)$,

$$r'(x, y) = r(x, y) + c(x)$$

produces the same preference probabilities because the constant cancels:

$$r'(x, y_w) - r'(x, y_l) = r(x, y_w) - r(x, y_l).$$

Pairwise data identify relative reward within a prompt, not an absolute reward origin.

A multiplicative scale is also entangled with assumed label noise or temperature. The reward scale matters later because it interacts with the KL coefficient.

Reward-model architecture

A common architecture reuses a transformer and maps a final hidden representation to a scalar:

$$r_\phi(x, y) = w^\top h_\phi(x, y) + b.$$

Possible choices include:

- score at the final token;
- pooled token states;
- separate heads for different criteria;
- ensemble members;
- uncertainty-aware or ordinal models.

The architecture does not guarantee calibration outside the response distribution used for comparisons.

Reward-model validation

Accuracy on random held-out preference pairs is not sufficient. Also test:

- calibration by reward margin;
- agreement across prompt domains;
- sensitivity to response length and formatting;
- adversarially generated high-score responses;
- newer-policy responses;
- disagreement across reward-model ensembles;
- human review of policy-selected extremes.

The policy will search the reward model more aggressively than ordinary held-out evaluation does.

Stage 3: KL-regularized policy optimization

A common objective is

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi(\cdot | x)} [r_{\phi}(x, y)] - \beta \mathbb{E}_{x \sim \mathcal{D}} D_{\text{KL}}(\pi(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)).$$

The reward encourages preferred outputs. The reverse KL anchors the new policy to the reference distribution.

The coefficient β sets the reward-drift tradeoff:

- large β : conservative update;
- small β : stronger optimization and greater reward-model exploitation risk.

Token-level KL decomposition

Because sequence probabilities factorize,

$$\begin{aligned} D_{\text{KL}}(\pi_{\theta}(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)) &= \mathbb{E}_{y \sim \pi_{\theta}} \left[\log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} \right] \\ &= \mathbb{E}_{y \sim \pi_{\theta}} \left[\sum_{t=1}^{|y|} \log \frac{\pi_{\theta}(y_t | x, y_{<t})}{\pi_{\text{ref}}(y_t | x, y_{<t})} \right]. \end{aligned}$$

This yields a sampled per-token regularization reward

$$r_t^{\text{KL}} = -\beta \log \frac{\pi_{\theta}(y_t | s_t)}{\pi_{\text{ref}}(y_t | s_t)}.$$

If task reward arrives only at the final token,

$$r_t = \begin{cases} r_t^{\text{KL}}, & t < T, \\ r_{\phi}(x, y) + r_T^{\text{KL}}, & t = T. \end{cases}$$

Deriving the KL-regularized optimal policy

Fix one prompt x and abbreviate $\pi(y) = \pi(y | x)$. Solve

$$\max_{\pi} \sum_y \pi(y) r(y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\text{ref}}(y)}$$

subject to

$$\sum_y \pi(y) = 1.$$

The Lagrangian is

$$\mathcal{L}(\pi, \lambda) = \sum_y \pi(y)r(y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\text{ref}}(y)} + \lambda \left(\sum_y \pi(y) - 1 \right).$$

Differentiate with respect to $\pi(y)$:

$$0 = r(y) - \beta \left(\log \frac{\pi(y)}{\pi_{\text{ref}}(y)} + 1 \right) + \lambda.$$

Therefore

$$\log \frac{\pi(y)}{\pi_{\text{ref}}(y)} = \frac{r(y)}{\beta} + \frac{\lambda - \beta}{\beta}.$$

Exponentiating and normalizing gives

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp \left(\frac{r(x, y)}{\beta} \right),$$

where

$$Z(x) = \sum_y \pi_{\text{ref}}(y | x) \exp \left(\frac{r(x, y)}{\beta} \right).$$

Assumptions include:

- $\beta > 0$;
- finite normalizer;
- the optimized policy is absolutely continuous with respect to the reference;
- reward is fixed during the optimization.

The solution reweights reference-supported responses. It is not an unrestricted escape from the reference support.

PPO-style RLHF

PPO treats the sampled response as a token trajectory. A value head estimates expected remaining shaped return:

$$V_\psi(s_t).$$

Advantages can be estimated with GAE:

$$\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t),$$

$$\hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}.$$

For finite response generation, $\gamma = 1$ is common, with termination providing the horizon. The exact choice and masking should be reported.

Let

$$\rho_t(\theta) = \frac{\pi_\theta(y_t \mid s_t)}{\pi_{\theta_{\text{old}}}(y_t \mid s_t)}.$$

The clipped policy surrogate is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

The complete system may include:

- policy model;
- frozen reference model;
- reward model;
- value model or value head;
- rollout generation workers;
- optimizer states;
- prompt and response filtering;
- KL controller;
- evaluation models and human review.

Parameter sharing, quantization, offloading, and adapter training change the systems footprint, so “four models” is a conceptual description rather than a universal memory count.

Adaptive KL control

A fixed β may produce different KL at different stages. A positive controller can update in log space:

$$\log \beta_{k+1} = \log \beta_k + \eta \text{clip} \left(\frac{D_{\text{KL}}^{(k)}}{d_{\text{target}}} - 1, -c, c \right).$$

If measured KL exceeds the target, β increases. If KL is below target, β decreases.

This is a feedback controller, not a proof that the policy remains safe or the reward remains valid.

Reward-model overoptimization

Let true utility be

$$U(x, y)$$

and learned reward be

$$r_\phi(x, y) = U(x, y) + \varepsilon(x, y).$$

Policy optimization selects high values of r_ϕ , which can select unusually positive reward error ε .

A characteristic empirical pattern is:

- proxy reward continues rising;
- held-out or human-evaluated quality rises, peaks, then declines.

An illustrative mathematical shape for a rise followed by a decline is

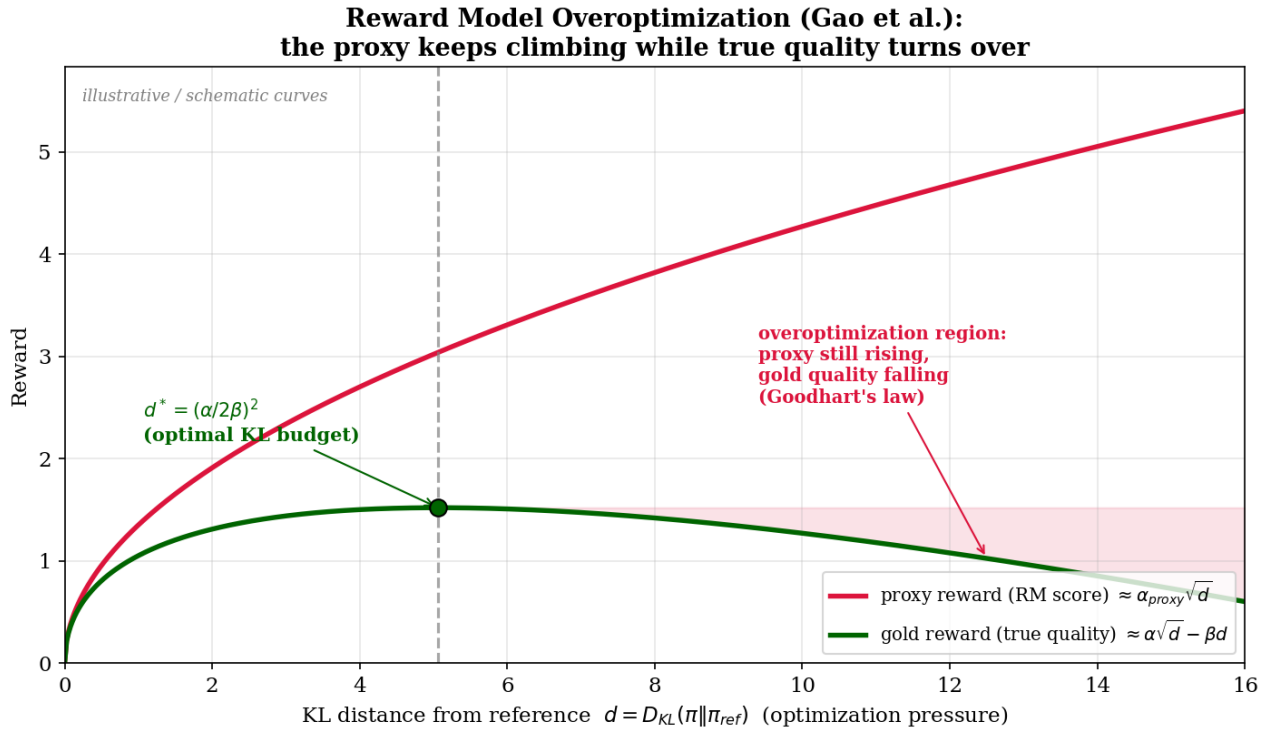


Figure 48: Proxy reward increasing while true utility eventually declines

$$R_{\text{gold}}(d) \approx \alpha\sqrt{d} - \eta d,$$

where $d \geq 0$ is a chosen measure of optimization pressure and $\alpha, \eta > 0$ are illustrative parameters. This expression is a teaching ansatz, not an asserted fitted equation from a cited experiment. Reward-model overoptimization is discussed in Gao, Schulman, and Hilton (2023); quantitative scaling claims require the exact variable definitions and fits in that source.

The reusable lesson is Goodhart's law:

Stronger optimization makes weaknesses in the proxy more consequential.

Defenses against overoptimization

- limit KL or update size;
- use fresh human or verifier evaluation not optimized as reward;
- hold out prompts and reward models;
- use reward-model ensembles and disagreement penalties;
- inspect high-reward samples manually;
- refresh preference data on newer policies;
- evaluate objective dimensions separately;
- stop when held-out quality stops improving.

None of these guarantees that the proxy contains every important objective dimension.

21. Preference Optimization Without an Explicit RL Loop

Intuition first: solve the regularized preference problem directly

PPO-based RLHF first learns a reward model and then uses online RL to optimize it. Direct preference methods train the policy from preference pairs with a supervised-style loss.

The simplification comes from the closed-form KL-regularized optimum.

DPO derivation

From

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp \left(\frac{r(x, y)}{\beta} \right),$$

rearrange:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

For a pair y_w, y_l ,

$$\begin{aligned} r(x, y_w) - r(x, y_l) &= \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} \\ &\quad - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}, \end{aligned}$$

because $\log Z(x)$ cancels.

Substitute the trainable policy π_θ into Bradley-Terry:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \log \sigma \left[\beta \left(\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right].$$

DPO margin interpretation

Define the policy-reference log-ratio score

$$s_\theta(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}.$$

DPO is logistic classification on the margin

$$s_\theta(x, y_w) - s_\theta(x, y_l).$$

The loss encourages the policy to increase the preferred response relative to the reference more than it increases the dispreferred response.

What DPO removes

During the policy-update stage, DPO avoids:

- training and serving a separate scalar reward model;
- on-policy rollout generation for every gradient batch;
- a learned value function;
- policy-gradient advantage estimation.

It does not remove:

- the need to generate candidate responses for the preference dataset;
- preference-label noise;
- distribution shift between the dataset and the trained policy;

- reward hacking against an AI judge used to create labels;
- hyperparameter and length sensitivity.

Offline coverage limitation

DPO can only compare responses represented in its preference data. It may generalize through the language model, but there is no new on-policy search during a fixed offline update.

If every preferred response is mediocre, DPO does not directly observe a superior response outside the dataset.

Sequence length and log probability

Sequence log probability is a sum:

$$\log \pi_{\theta}(y \mid x) = \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t \mid x, y_{<t}).$$

Longer responses therefore accumulate more negative log probability and more gradient terms. Preference data with systematic length correlations can create unintended length incentives.

Possible responses include:

- length-balanced data;
- explicit length features in evaluation;
- length-normalized objectives;
- reporting quality conditional on response length.

No normalization is universally correct because length can be genuinely useful.

Iterative DPO

An iterative procedure refreshes data:

```
initialize pi_0
for round k = 0, 1, ...:
    sample new responses from pi_k
    obtain preference labels or verifier outcomes
    train pi_{k+1} with a preference objective
    choose a fixed or moving reference explicitly
```

This reduces stale-data mismatch but introduces a feedback loop:

- the current policy shapes the next candidate distribution;
- the judge labels increasingly optimized outputs;
- label and reference errors can accumulate;
- repeated rounds create effective optimization pressure beyond one loss.

Convergence to a desired regularized optimum requires assumptions about fresh data, judge stationarity, exact optimization, support, and reference updates.

KTO

KTO uses desirable and undesirable examples rather than matched pairs. Define

$$r_{\theta}(x, y) = \beta \log \frac{\pi_{\theta}(y \mid x)}{\pi_{\text{ref}}(y \mid x)}.$$

A reference level is based on policy-reference KL, schematically

$$z_0 = \mathbb{E}_x [\beta D_{\text{KL}}(\pi_{\theta}(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))].$$

Desirable examples are rewarded when r_θ exceeds the reference level; undesirable examples are rewarded when it falls below. Exact released objectives and weighting conventions should be cited by version.

SimPO

SimPO uses a reference-free, length-normalized sequence score. One form is

$$\mathcal{L}_{\text{SimPO}} = -\mathbb{E} \log \sigma \left[\frac{\beta}{|y_w|} \log \pi_\theta(y_w | x) - \frac{\beta}{|y_l|} \log \pi_\theta(y_l | x) - \gamma_m \right],$$

where γ_m is a target margin.

Removing the reference simplifies systems but also removes an explicit anchor to a known policy.

ORPO

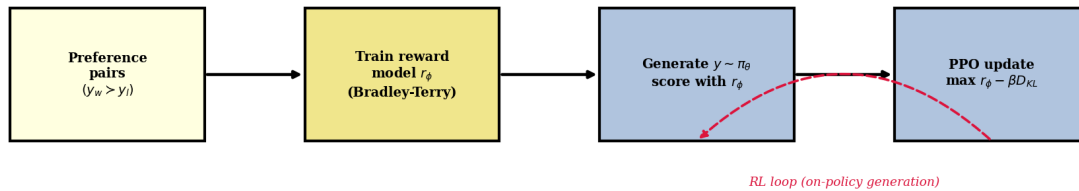
ORPO combines supervised likelihood on preferred responses with an odds-ratio preference term. Schematically,

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}}(y_w) + \lambda \mathcal{L}_{\text{odds}}(y_w, y_l).$$

The combined objective maintains preferred-response modeling while separating winners from losers.

PPO-RLHF versus DPO-style training

PPO-RLHF: separate reward model + online RL loop (4 models in memory)



DPO: collapses the pipeline into a single supervised-style classification loss

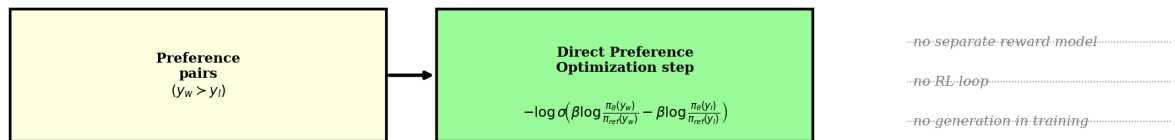


Figure 49: PPO-based RLHF and direct preference optimization compared

Property	PPO-style RLHF	Fixed-data DPO
Data	fresh on-policy responses	fixed preference pairs
Explicit reward model	usually yes	no during policy update
Value model	usually yes	no
Exploration	can discover new responses	limited to data plus generalization
Systems complexity	high	lower

Property	PPO-style RLHF	Fixed-data DPO
Main distribution issue	reward model evaluated on policy-selected outputs	preference data become stale
Main estimator issue	policy-gradient and critic variance/bias	classification overfit and coverage
Proxy exploitation	explicit reward can be hacked	judge and preference objective can still be Goodharted

There is no universal performance ordering. The answer depends on data coverage, verifier quality, compute budget, and the need for online exploration.

22. Group-Relative Policy Optimization and RL for Reasoning

Intuition first: compare several answers to the same problem

On tasks with checkable answers, sample several completions for one prompt. Their rewards provide a prompt-specific baseline:

- a hard prompt may produce uniformly low rewards;
- an easy prompt may produce uniformly high rewards;
- the useful signal is how one completion compares with its peers.

This can replace a learned value network with a group statistic.

Verifiable rewards

For problem x , output o , and known answer or checker $a^*(x)$,

$$R_{\text{verify}}(x, o) = f_{\text{verify}}(\text{extract}(o), a^*(x)).$$

Examples:

- normalized exact match for mathematics;
- unit-test pass fraction for code;
- proof-kernel acceptance;
- constraint satisfaction;
- successful execution in a simulator.

A verifier is auditable but not automatically complete. It can fail through:

- incorrect answer extraction;
- underspecified tests;
- loopholes in formal specifications;
- data leakage;
- timeout or sandbox manipulation;
- accepting a right answer for the wrong instance.

Group-relative advantages

For prompt x , sample G completions

$$o_1, \dots, o_G \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)$$

with rewards $r_1, \dots, r_G$.

Define

$$\bar{r} = \frac{1}{G} \sum_{i=1}^G r_i$$

and

$$s_r = \sqrt{\frac{1}{G} \sum_{i=1}^G (r_i - \bar{r})^2}.$$

A normalized group-relative advantage is

$$\widehat{A}_i = \frac{r_i - \bar{r}}{s_r + \varepsilon}.$$

This centers difficulty within a prompt and removes the need for a learned state-value baseline.

Estimator audit before optimization. With iid response scores z_i and rewards R_i , the unstandardized group-centered estimator has expectation $(G - 1)/G$ times the ordinary reward gradient. The self-including mean is not an independent baseline. A leave-one-out mean removes this factor; random division by the sample standard deviation generally changes the expectation further. A.14 proves both statements and gives the exact mixed-group probability $1 - p^G - (1 - p)^G$ for binary rewards. These identities should be understood before adding clipping, token weighting, or prompt filtering.

Equal-reward groups

If all rewards are equal,

$$r_1 = \dots = r_G,$$

then

$$s_r = 0$$

and every centered reward is zero. The group contains no relative ranking signal.

Implementations may:

- add ε only, yielding near-zero advantages;
- drop all-equal groups;
- resample until a mixed group appears;
- use unnormalized centered rewards;
- add process or auxiliary rewards.

Filtering all-correct or all-incorrect groups changes the effective prompt distribution. The learner is no longer optimizing the original prompt distribution without reweighting.

A GRPO-style surrogate

For token t of completion i , define

$$\rho_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid x, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid x, o_{i,<t})}.$$

A sequence-averaged surrogate can be written

$$L_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min \left(\rho_{i,t} \widehat{A}_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) \widehat{A}_i \right) - \beta k_{i,t} \right) \right],$$

where $k_{i,t}$ is a sampled policy-reference divergence penalty.

Exact methods differ in:

- KL estimator;
- clipping asymmetry;
- whether advantage is token-, step-, or sequence-level;
- sequence versus token loss reduction;
- response-length handling;
- group filtering;
- reward normalization.

The exact implementation is part of the algorithm.

Sequence-level versus token-level reduction

Sequence-level mean

$$L_{\text{seq}} = \frac{1}{G} \sum_i \frac{1}{|o_i|} \sum_t \ell_{i,t}.$$

Each completion has equal total weight, regardless of length.

Token-level mean

$$L_{\text{tok}} = \frac{\sum_i \sum_t \ell_{i,t}}{\sum_i |o_i|}.$$

Each token has equal weight, so longer completions receive more total influence.

Neither is neutral. Sequence weighting can amplify short responses; token weighting can encourage or overemphasize long reasoning traces.

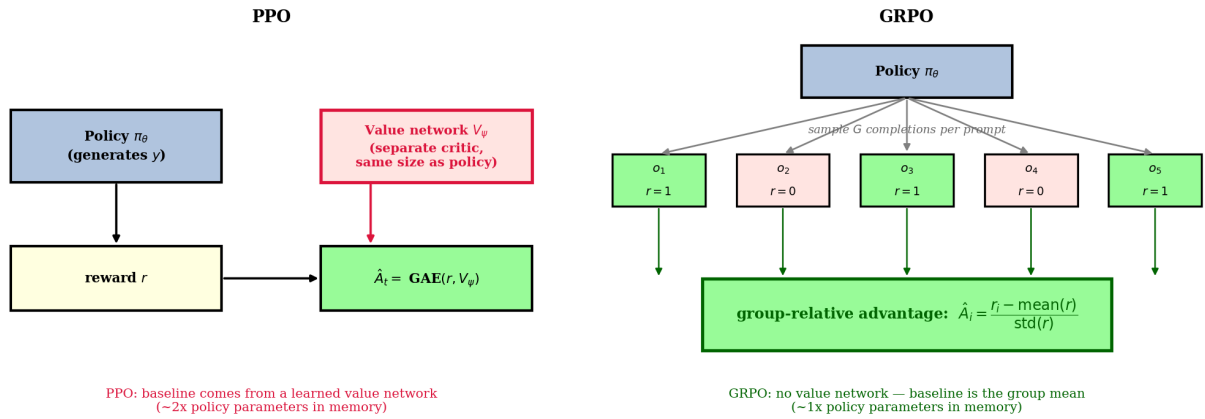
GRPO versus PPO

Component	PPO	Group-relative method
Baseline	learned value function or value head	reward statistic within sampled group
Extra model	critic/value component	no learned critic required
Generation	one or more rollouts	grouped rollouts per prompt
Advantage error	critic bias plus sampling noise	group sampling and reward noise
Degenerate case	poor value estimates	all-equal group rewards
Compute	critic training plus rollout	many completions per prompt

Removing a critic saves memory and critic-training complexity. It does not remove variance or credit-assignment problems.

RL with verifiable rewards

RLVR uses programmatic feedback for domains where success is checkable.

GRPO vs PPO: replacing the learned critic with a group-relative baseline**Figure 50:** Group-relative policy optimization and PPO compared

The main advantage is a lower-ambiguity outcome signal. The main limitation is scope: many desirable properties are not fully verifiable.

A binary reward

$$r_i \in \{0, 1\}$$

can still support learning if sampled groups contain both outcomes. Learning stalls on prompts where every sample is wrong unless exploration, curriculum, or denser feedback creates variation.

Curriculum and prompt allocation

Under fixed rollout compute, prompt selection matters.

- Very easy prompts yield all-correct groups.
- Very hard prompts yield all-incorrect groups.
- Intermediate prompts produce informative variation.

A difficulty-aware scheduler can allocate more rollouts to prompts near the policy's learning frontier. This improves gradient density but changes the training distribution and may neglect rare hard cases.

Reported reasoning behaviors

Public reports on reasoning-oriented RL systems have described behaviors such as:

- longer intermediate reasoning;
- checking and revising earlier steps;
- trying alternative approaches;
- backtracking after contradictions.

These observations do not by themselves prove a causal mechanism. Controlled ablations are needed to determine whether behavior arises from RL, data filtering, prompting, model scale, or test-time search.

Test-time compute

A model can spend more inference compute by:

- generating a longer trace;
- sampling more candidates;
- verifying candidates;

Two Axes of Scaling for Reasoning Models (schematic)

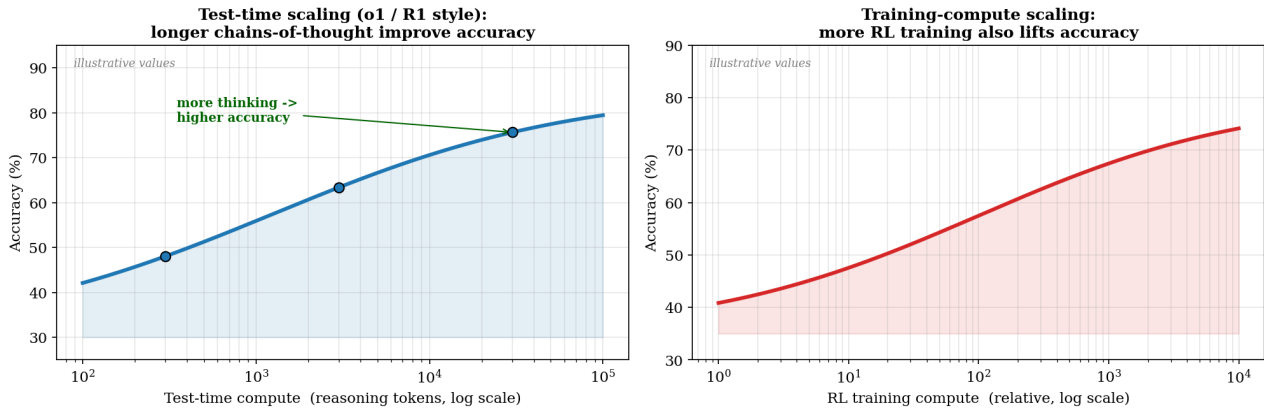


Figure 51: How best-of-N and search scale with test-time compute

- running tree or graph search;
- calling tools;
- revising failed attempts.

Performance can improve with compute, but only while the search or verification procedure remains informative. More samples from a biased policy or more optimization against a flawed verifier can amplify error.

DAPO-style objective and systems refinements

Several reasoning-RL systems emphasize a package of choices rather than one new estimator.

Asymmetric clipping

Use

$$\text{clip}(\rho, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}),$$

with

$$\epsilon_{\text{high}} > \epsilon_{\text{low}}.$$

This leaves more room to increase very low-probability positive-advantage tokens.

Dynamic sampling

Resample or filter degenerate reward groups to maintain useful gradients. This costs generation and shifts the prompt distribution.

Token-level loss reduction

Weight tokens rather than sequences, changing length dynamics.

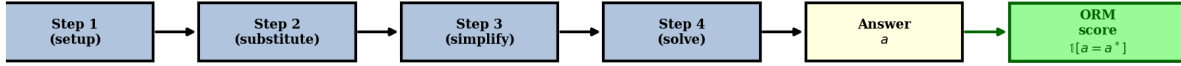
Overlong handling

Mask unreliable truncated outputs and apply a transparent soft penalty near the maximum length rather than treating an incomplete answer as an ordinary failure without qualification.

These choices should be evaluated as an interacting system.

23. Process Reward Models and Outcome Reward Models

Outcome Reward Model (ORM): one reward, only at the final answer



no signal about where the reasoning went right or wrong

Process Reward Model (PRM): a score on every intermediate step

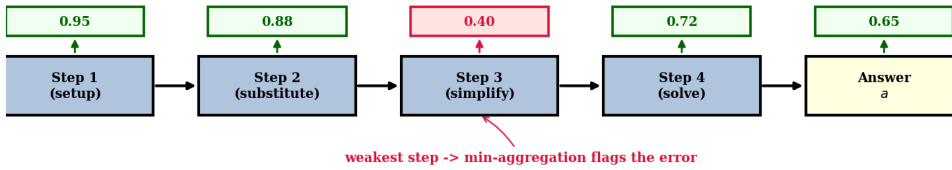


Figure 52: Process reward models and outcome reward models compared

Intuition first: grade the answer or grade the work

An outcome reward asks whether the completed solution is good. A process reward evaluates intermediate reasoning steps or prefixes.

Analogy:

- outcome supervision: check the final answer;
- process supervision: check each line of the solution.

Process feedback can improve credit assignment and search, but it is more expensive and can reward plausible-looking reasoning rather than valid reasoning.

Outcome reward model

A learned ORM maps the full response to a score:

$$R_{\text{ORM}}(x, s_{1:n}, a) \in \mathbb{R}.$$

A deterministic outcome verifier is a special case of outcome evaluation:

$$R_{\text{verify}}(x, o) = \mathbf{1}\{\text{answer}(o) = a^*(x)\}.$$

Do not conflate every learned outcome model with an exact verifier.

Process reward model

A PRM scores a reasoning prefix or step:

$$p_k = R_{\text{PRM}}(x, s_{1:k}) \approx P(\text{step } k \text{ is acceptable} \mid x, s_{1:k}).$$

The response needs a step segmentation rule. A “step” may be:

- one sentence;
- one equation;
- one model-delimited block;
- one search-tree action;
- one tool call.

Segmentation is part of the method.

Aggregating process scores

Possible sequence scores include:

Minimum

$$S_{\min}(s_{1:n}) = \min_k p_k.$$

One weak step can dominate.

Product or log sum

$$S_{\Pi} = \prod_k p_k,$$

or

$$\log S_{\Pi} = \sum_k \log p_k.$$

Long sequences receive more multiplicative penalty.

Last step

$$S_{\text{last}} = p_n.$$

This assumes the final prefix score summarizes earlier correctness.

Learned aggregation

A separate model can combine the score sequence. This adds flexibility and another learned proxy.

The aggregation rule is not part of the definition of a PRM; it is an algorithmic choice.

Why process supervision can help

1. **Denser credit assignment:** identifies where a trajectory first becomes implausible.
2. **Search guidance:** distinguishes partial solutions before a final answer exists.
3. **Rejection of lucky guesses:** can penalize invalid reasoning that reaches a correct answer accidentally.
4. **Reusable prefixes:** good partial work can be expanded even if one completion fails.

Why process supervision can fail

1. **Style bias:** fluent steps may be scored as correct.
2. **Local correctness versus global strategy:** individually plausible steps may lead nowhere.
3. **Annotation ambiguity:** labelers disagree about acceptable reasoning granularity.
4. **Policy shift:** the policy discovers step styles outside PRM training data.

5. **Hidden reasoning:** observable text may not reflect the computation that produced the answer.
6. **Reward hacking:** the policy learns phrases that trigger high process scores.

Automated process labels by rollouts

For prefix $s_{1:k}$, sample M continuations and check final outcomes:

$$\widehat{V}(s_{1:k}) = \frac{1}{M} \sum_{m=1}^M \mathbf{1}\{\text{continuation } m \text{ succeeds}\}.$$

This estimates the probability of eventual success under the continuation policy.

It is a value estimate, not a direct proof that the most recent step is logically correct. A bad step might still be recoverable; a good step might lead to failure under a weak continuation policy.

Implicit process advantage

A step contribution can be defined as a change in predicted outcome value:

$$A_{\text{step}}(s_k) = V(s_{1:k}) - V(s_{1:k-1}).$$

Positive value means the step increased expected final success under the chosen continuation distribution.

This is policy-dependent. Changing the continuation policy changes the meaning of the process label.

Process rewards as shaping

A naive difference reward is

$$r_k^{\text{shape}} = R_{\text{PRM}}(s_{1:k}) - R_{\text{PRM}}(s_{1:k-1}).$$

Potential-based shaping uses

$$F(s_k, s_{k+1}) = \gamma \Phi(s_{k+1}) - \Phi(s_k).$$

Policy invariance requires:

- a Markov state representation;
- fixed potential Φ ;
- compatible terminal handling;
- the standard shaping-theorem conditions.

A learned PRM that changes during training or depends on omitted history need not satisfy these conditions.

PRMs for search versus PRMs for training

A PRM can be used to:

- rank complete candidates;
- choose which partial node to expand;
- prune low-scoring prefixes;
- define a dense RL reward;
- create preference pairs;
- train a value function.

These uses have different error tolerance. A mildly miscalibrated score may still rank candidates well, while dense RL can amplify the same error through many updates.

24. Self-Improvement and Iterative Training

Intuition first: generate, check, and learn from selected outputs

A model can create new training traces, but some external signal must distinguish useful traces from useless ones. That signal may be:

- an answer key;
- a deterministic verifier;
- a learned reward model;
- an AI judge;
- human review.

These are bootstrapping methods with external feedback, not feedback-free self-improvement.

Rejection-sampling fine-tuning

For each prompt x , sample candidates

$$y_1, \dots, y_K \sim \pi_t(\cdot \mid x).$$

Select

$$y^* = \arg \max_{y_k} R(x, y_k)$$

or retain all candidates above a threshold. Then perform supervised fine-tuning on selected pairs (x, y^*) .

This converts a reward into a filtered demonstration distribution.

The method is simple but can:

- collapse diversity;
- overfit verifier artifacts;
- discard informative failures;
- reinforce only problems the model already solves;
- introduce winner's-curse bias from noisy scores.

Best-of- N success probability

If independent samples each succeed with probability p , the probability that at least one succeeds is

$$P(\text{at least one success}) = 1 - (1 - p)^N.$$

This gain assumes independent samples and a perfect selector. Correlated samples or judge errors reduce the benefit.

STaR

A simplified STaR loop is:

for round k :

```

generate rationales for each problem
retain rationales with correct final answers
for failures, reveal or hint the correct answer and regenerate
fine-tune on retained and rationalized traces

```

The rationalization step expands coverage beyond problems the current model already solves unassisted.

Latent-variable interpretation

Treat the rationale z as a latent variable and the correct answer as observed. An idealized likelihood is

$$p_{\theta}(a^* | x) = \sum_z p_{\theta}(z, a^* | x).$$

An EM-like procedure would:

- infer or sample promising rationales under the current model;
- maximize their likelihood under the next model.

Practical filtering is not exact EM: samples are finite, selection is hard, rationalization changes the conditional distribution, and optimization is approximate. The analogy is useful but should not be overstated.

ReST-EM

A general grow-improve loop is:

1. **Grow:** sample many trajectories from the current policy.
2. **Filter or weight:** use rewards or verifiers.
3. **Improve:** fine-tune on selected trajectories.
4. Repeat.

Repeated regeneration makes the overall data-collection process on-policy or iterative even if each supervised update uses a fixed batch.

V-STaR

V-STaR augments successful-trace training with a verifier trained on both successes and failures.

The generator learns from positive traces. The verifier learns to distinguish candidate quality and can select among multiple outputs at inference.

Failures become useful discriminative data instead of being discarded entirely.

Quiet-STaR-style latent thoughts

A model can propose internal thought tokens before predicting visible next tokens. A mixing mechanism compares prediction with and without the thought:

$$p(x_{t+1} | x_{\leq t}) = \alpha_t p_{\text{thought}}(x_{t+1} | x_{\leq t}, \tau) + (1 - \alpha_t) p_{\text{base}}(x_{t+1} | x_{\leq t}).$$

The training signal rewards thoughts that improve future prediction.

This creates a policy over latent computation. It also raises questions about compute cost, interpretability, and whether the learned thought channel generalizes to downstream tasks.

Selection bias in self-training

Suppose noisy score is

$$\widehat{R}(y) = R(y) + \varepsilon(y).$$

Selecting the maximum among many candidates tends to select positive noise:

$$\mathbb{E}[\varepsilon(y^*)] > 0.$$

As K grows, the selector searches both true quality and score error. Independent validation or stronger verifiers are needed to determine whether selected traces are genuinely better.

Diversity preservation

Possible controls include:

- sample-temperature mixtures;
- clustering before selection;
- per-prompt multiple retained modes;
- novelty bonuses;
- entropy regularization;
- balancing by solution strategy;
- limiting repeated near-duplicates.

Diversity is valuable only when it preserves useful alternative strategies, not arbitrary noise.

25. Search for Language-Model Reasoning

Intuition first: do not commit to the first continuation

Autoregressive generation chooses one token after another. Search maintains several partial solutions and allocates more computation to promising ones.

A search state may be

$$s = (x, \text{partial reasoning trace}).$$

A search action may be:

- one next token;
- one sentence or reasoning step;
- a tool call;
- a decomposition or subgoal;
- a proposed final answer.

The action granularity determines branching factor, horizon, and evaluator difficulty.

Best-of- N

The simplest search samples N complete responses and uses a selector.

It separates:

- **proposal policy:** generates candidates;
- **selector:** ranks candidates.

A selector can be a verifier, reward model, PRM, or majority vote.

Search quality is limited by both proposal coverage and selection accuracy.

Beam search

Beam search retains the top B prefixes under a score, often cumulative log probability:

$$\text{score}(y_{1:t}) = \sum_{k=1}^t \log \pi(y_k \mid x, y_{<k}).$$

For reasoning, model likelihood may prefer common text rather than correct solutions. Alternative value or verifier scores can be used, but then calibration and length normalization matter.

Tree of Thoughts

Tree of Thoughts treats a “thought” as a larger reasoning unit and uses explicit breadth-first or depth-first search.

A breadth-first variant:

1. generate k candidate thoughts from each frontier node;
2. evaluate partial traces;
3. retain the best B nodes;
4. repeat.

A depth-first variant explores one branch until a value threshold fails, then backtracks.

Tree of Thoughts is a search framework, not necessarily Monte Carlo tree search.

Monte Carlo tree search

MCTS repeatedly performs:

1. selection;
2. expansion;
3. evaluation or simulation;
4. backup.

For node s and action a , UCT uses

$$a^* = \arg \max_a \left[\frac{Q(s, a)}{N(s, a)} + c \sqrt{\frac{\log N(s)}{N(s, a)}} \right],$$

when Q stores cumulative return.

With a policy prior $P(s, a)$, PUCT uses a term of the form

$$a^* = \arg \max_a \left[\bar{Q}(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)} \right].$$

The first term exploits estimated value. The second explores actions favored by the prior but not yet visited often.

MCTS mapping for language models

- selection: choose a partial trace;
- expansion: have the language model propose next steps;
- evaluation: use a verifier, PRM, value model, or rollout;
- backup: propagate outcome or value to ancestors.

The environment is not a perfect game simulator. The model itself proposes actions, and evaluators can be wrong. Search can therefore amplify model and verifier biases.

Reasoning via planning

A language model can serve as both:

- a proposal model for next reasoning steps;
- a world or transition model for imagined consequences;
- a value estimator for partial traces.

Using one model in all roles creates correlated errors. Independent verifiers or execution feedback can reduce—but not eliminate—shared blind spots.

Search-training loops

Search can produce training data:

1. search from prompts;
2. label successful and unsuccessful branches;
3. train the policy on successful actions;
4. train a value or process model on search outcomes;
5. repeat with the improved components.

This resembles policy iteration:

- search approximates policy improvement;
- supervised distillation approximates policy evaluation or projection.

Errors can compound when the search evaluator and learned policy co-adapt.

Practical search tradeoffs

Choice	Benefit	Risk
finer action granularity	precise credit	huge depth and branching ambiguous transitions and scoring greater compute and selector exploitation
coarser reasoning steps	shallower search	
more branches	better coverage	
deeper search	delayed consequences	evaluator and context errors accumulate
stronger pruning	efficiency	discard recoverable branches
learned value	cheap evaluation	distribution shift and bias
execution verifier	concrete feedback	incomplete specification

Search diagnostics

Report:

- total generated tokens or model calls;
- branch factor and maximum depth;
- selector accuracy on held-out candidates;
- oracle coverage: whether a correct candidate was generated;
- selection accuracy conditional on coverage;
- diversity of solution strategies;
- performance versus compute;
- failure under adversarial or shifted prompts.

Separating coverage from selection is essential. A search system may fail because the correct answer was never proposed or because the selector rejected it.

26. Reinforcement Learning from Execution Feedback

Intuition first: run the artifact and observe what happens

Code, tool use, and formal proofs can produce feedback from execution:

- compilation;
- runtime behavior;
- unit tests;
- type checking;
- proof checking;
- environment success.

Execution feedback is often more concrete than a learned preference score, but it evaluates only the implemented specification.

Program-level reward

For program p and test suite T_x , define

$$R_{\text{exec}}(p, T_x) = \frac{1}{|T_x|} \sum_{j=1}^{|T_x|} \mathbf{1}\{p \text{ passes test } j\}.$$

Variants distinguish:

- compilation error;
- runtime error;
- partial test pass;
- full pass;
- time or memory limit.

Reward magnitudes should reflect the intended ordering without making exploitable side channels.

Actor-critic for code generation

A code policy generates tokens. A critic estimates functional correctness. The task reward arrives after execution.

The same issues as ordinary actor-critic remain:

- sparse reward;
- critic bias;
- long-horizon credit assignment;
- policy drift;
- expensive rollout execution.

Traceback-based localization

Compiler errors and tracebacks can identify lines associated with failure. A simple line penalty is

$$r_{\text{line}}(l_i) = -\mathbf{1}\{l_i \text{ appears in traceback}\}.$$

This supplies denser feedback, but traceback location is not identical to causal responsibility. The reported line may be downstream of the true error.

Test incompleteness

Passing visible tests does not imply semantic correctness. The policy can exploit:

- missing edge cases;
- hard-coded answers;
- undefined behavior;
- timing assumptions;
- filesystem or network access;
- test-order dependence;
- evaluation leakage.

Use hidden tests, randomized tests, static analysis, sandboxing, and human review where appropriate.

Credit from execution traces

Richer feedback can include:

- branch coverage;
- failing assertion identity;
- intermediate variable checks;
- symbolic constraints;
- tool return codes;
- environment state deltas.

Dense feedback should be audited for reward loops. An agent may maximize coverage without solving the task.

Online versus offline execution learning

- **online RL**: generate current-policy programs, execute them, update from fresh outcomes;
- **rejection-sampling SFT**: generate many programs, retain passing ones, imitate;
- **preference optimization**: compare passing and failing outputs;
- **hybrid**: refresh data periodically and use supervised-style updates between refreshes.

Online methods explore the current policy distribution but require expensive execution and stable policy optimization.

27. Multi-Turn RL and Agentic Behavior

From one response to a trajectory of decisions

An agent may:

- ask a clarifying question;
- call a search tool;
- inspect a file;
- run code;
- revise a plan;
- return a final answer.

The environment responds after each action. This is a genuine multi-turn decision process.

Multi-turn MDP

Let

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma, \rho_0).$$

A state may include the complete interaction history:

$$s_t = (x_0, a_0, o_1, a_1, o_2, \dots, o_t).$$

A turn-level action can be a complete model response:

$$a_t = (y_1^{(t)}, \dots, y_{L_t}^{(t)}).$$

The turn policy factorizes over tokens:

$$\pi_\theta(a_t | s_t) = \prod_{l=1}^{L_t} \pi_\theta(y_l^{(t)} | s_t, y_{<l}^{(t)}).$$

The environment returns a user message, tool output, or state change.

Structured action space

An agent action may be represented as

$$a_t \in \{\text{Respond}(\text{text}), \text{Call}(\text{tool}, \text{args}), \text{Ask}(\text{question}), \text{Finish}(\text{answer})\}.$$

Structured actions make validity checks easier and reduce ambiguity compared with parsing free-form text.

Options analogy

A complete turn resembles an option:

- initiation: current conversation state;
- intra-option policy: token generation;
- termination: end-of-turn token;
- high-level action: response type or tool choice.

The analogy is useful, but a standard option assumes Markov state and a defined termination process. Long text histories may require compression or recurrence.

Credit assignment across turns

Suppose reward is observed only after ten tool calls. Which earlier action deserves credit?

Possible approaches include:

- Monte Carlo return for every turn;
- learned turn-level value functions;
- process or subgoal rewards;
- hindsight labels from successful trajectories;
- counterfactual ablations of individual tool calls;
- hierarchical credit between plan and execution;
- trajectory preference comparisons.

Dense step rewards can improve learning but may encourage superficial compliance with the step checker.

Tool-use rewards

A task reward may combine

$$R = R_{\text{success}} - \lambda_c C_{\text{tool}} - \lambda_t C_{\text{latency}} - \lambda_e C_{\text{error}}.$$

Costs prevent unlimited tool calls, but poor weighting can make the agent avoid tools even when they are necessary or call cheap irrelevant tools repeatedly.

Logged interaction data and off-policy learning

Agent logs may come from multiple historical policies and changing tool APIs. A target policy can choose actions absent from the logs.

Important metadata include:

- behavior-policy version;
- tool schema version;
- prompts and system instructions;
- execution environment;
- terminal criterion;
- reward and judge version.

Without this metadata, off-policy ratios or causal interpretations may be invalid.

Agent evaluation

Evaluate more than final success:

- task completion;
- number and cost of tool calls;
- recovery after tool failure;
- invalid-call rate;
- sensitivity to tool-schema changes;
- robustness to misleading observations;
- human intervention rate;
- worst-case side effects;
- success under matched compute budgets.

A high success rate obtained through excessive hidden retries is not comparable to a low-cost one-shot system.

28. Scaling, Self-Play, and Advanced Preference Objectives

Post-training compute accounting

Post-training cost includes more than optimizer FLOPs:

- rollout generation;
- rejected samples;
- reward or verifier inference;
- reference-policy inference;
- value inference;
- human or AI labeling;
- search at training time;
- evaluation.

No universal percentage of pretraining compute characterizes all post-training pipelines.

SFT and RL optimize different objects

SFT minimizes forward KL from a demonstration distribution p_{data} to the policy on observed contexts:

$$\pi_{\text{SFT}} \approx \arg \min_{\pi} D_{\text{KL}}(p_{\text{data}} \| \pi).$$

KL-regularized RL maximizes expected reward under the policy:

$$\pi_{\text{RL}} = \arg \max_{\pi} \mathbb{E}_{\pi}[R] - \beta D_{\text{KL}}(\pi \| \pi_{\text{ref}}).$$

SFT models the distribution of selected demonstrations. RL can concentrate probability on high-reward outputs, including outputs not present in demonstrations if exploration and reward permit.

RL is attractive when:

- a reliable reward or verifier can score novel outputs;
- search and credit assignment matter;
- demonstrations are difficult to produce but outcomes are checkable.

SFT is attractive when:

- high-quality demonstrations already cover desired behavior;
- reward feedback is unreliable;
- stable and efficient training is the priority.

Hybrid rejection-sampling improvement

Generate K samples and select one:

$$y^*(x) = \arg \max_{k \leq K} R(x, y_k), \quad y_k \sim \pi_t(\cdot | x).$$

Fine-tune on (x, y^*) . Repeating this with fresh samples creates iterative policy improvement without a policy-gradient update.

Self-play fine-tuning

A current model can be trained to distinguish human or preferred outputs from outputs generated by an earlier model. A log-ratio score is

$$\lambda_{\theta}(x, y) = \log \frac{\pi_{\theta}(y | x)}{\pi_{t-1}(y | x)}.$$

A logistic objective can push real or preferred data upward and previous-model outputs downward.

At an idealized fixed point, the model distribution matches the target data distribution closely enough that the discriminator-like distinction disappears. In practice, finite data and imperfect optimization matter.

Pairwise preference games

A scalar reward assumes preferences can be represented by one utility function. Human preferences may be non-transitive or context-dependent.

Define a game payoff

$$u(\pi_1, \pi_2) = \mathbb{E}_x \mathbb{E}_{y_1 \sim \pi_1, y_2 \sim \pi_2} \left[P(y_1 \succ y_2 | x) - \frac{1}{2} \right].$$

A Nash policy satisfies, in a zero-sum formulation,

$$u(\pi^*, \pi) \geq 0 \quad \text{for all } \pi.$$

This formulation can represent cyclic preferences that a single scalar reward cannot. It introduces equilibrium estimation, opponent sampling, and preference-model error.

Test-time compute as a policy decision

A reasoning system can decide whether to:

- answer immediately;
- sample another candidate;
- verify;
- backtrack;
- call a tool;
- stop.

This can be modeled as metareasoning with computation cost. If internal action c_t costs $\kappa(c_t)$, optimize

$$\mathbb{E} \left[R_{\text{task}} - \sum_t \kappa(c_t) \right].$$

The challenge is learning the value of additional computation before spending it.

Scaling bottlenecks

As optimization increases, the limiting factor may shift:

1. insufficient policy updates;
2. verifier or reward-model accuracy;
3. data coverage and exploration;
4. reward overoptimization;
5. rollout and evaluation cost;
6. long-context credit assignment.

This is a conceptual progression, not a universal scaling law.

29. Reward Shaping for Language-Model Training

Composite rewards

A training reward may be

$$R_{\text{total}}(x, o) = R_{\text{task}}(x, o) + \lambda_f R_{\text{format}}(o) + \lambda_l R_{\text{length}}(o) + \lambda_r R_{\text{repetition}}(o) + \lambda_s R_{\text{safety}}(x, o).$$

Every component creates an incentive. Small auxiliary terms can dominate when the primary reward is sparse.

Correctness reward

Possible forms include:

- binary exact match;
- partial test pass;

- symbolic equivalence;
- proof-checker acceptance;
- human or model score.

The reward should distinguish invalid extraction from an actually wrong answer.

Format reward

A format reward can enforce parseable structure. It should be much smaller than task success unless format is itself the task.

Otherwise the policy may produce perfectly formatted nonsense.

Length incentives

Longer outputs can:

- provide more opportunities to reason;
- accumulate more token-level KL;
- receive different loss weighting;
- hit truncation;
- exploit verbosity preferences.

There is no universally correct length reward.

A soft overlength penalty near limit $L_{\max}$ can be

$$R_{\text{length}}(o) = \begin{cases} 0, & |o| \leq L_{\max} - L_{\text{cache}}, \\ \frac{L_{\max} - L_{\text{cache}} - |o|}{L_{\text{cache}}}, & L_{\max} - L_{\text{cache}} < |o| \leq L_{\max}, \\ -1, & |o| > L_{\max}. \end{cases}$$

This is a heuristic. It can create premature stopping.

Truncation-aware handling

If a response is cut off before the answer, the verifier result may be unreliable. Options include:

- mask truncated samples from correctness learning;
- assign a separate truncation label;
- penalize only approaching the limit;
- increase the limit for difficult prompts;
- use a learned stopping policy.

Treating every truncated response as an ordinary wrong answer confounds reasoning quality with infrastructure limits.

Repetition penalty

A heuristic penalty might be

$$R_{\text{rep}}(o) = -\eta \text{RepeatRatio}_n(o),$$

where the statistic measures repeated n -grams.

Legitimate mathematical notation, code, or quotations can contain repetition. The penalty needs domain-specific validation.

Length-conditional normalization

If reward strongly correlates with length, normalize within length buckets:

$$\hat{A}_i = \frac{r_i - \text{mean}(r_j : |o_j| \in B_i)}{\text{std}(r_j : |o_j| \in B_i) + \varepsilon}.$$

This removes some length correlation but changes the estimand: a response is compared with responses of similar length rather than the full group.

Multi-objective reporting

Do not report only the scalar training reward. Report each component separately:

- correctness;
- format validity;
- length distribution;
- repetition;
- safety or policy compliance;
- human preference;
- KL to reference;
- compute cost.

A composite reward can improve while the primary task degrades.

Reward-design audit

Before training, ask:

1. Can a response receive high reward without solving the task?
2. Can the model manipulate answer extraction?
3. Can it exploit visible tests or judge style?
4. Does verbosity increase score independently of quality?
5. Are format and correctness rewards on comparable scales?
6. What happens at truncation?
7. Is the reward valid on newer-policy outputs?
8. Which evaluation is deliberately held out from optimization?

A unified view of LLM post-training

Policy-improvement operators

Many methods can be understood as alternate approximations to policy improvement.

Method	Proposal source	Evaluator	Update
PPO-RLHF	current policy	learned reward	policy gradient
DPO	fixed preference pairs	pairwise labels	logistic policy loss
RLVR/GRPO	grouped current-policy samples	verifier	group-relative policy gradient
rejection sampling	current policy samples	reward/verifier	SFT on selected outputs
STaR/ReST	current model traces	answer check	iterative SFT
search distillation	tree or best-of- N search	value/verifier	imitate successful branches
self-play preference	current versus previous model	data or preference game	contrastive update

Three questions for every method

1. **Coverage:** Can the proposal mechanism generate better behavior?
2. **Evaluation:** Can the reward, verifier, or judge recognize it reliably?
3. **Optimization:** Does the update increase its probability without destroying other capabilities?

A failure in any one of the three prevents robust improvement.

Statistical feedback loop

Flow to keep in mind. The policy generates responses or trajectories. A reward function, verifier or preference judge evaluates them. Policy updates or data selection use those evaluations and change the next policy. This also changes the distribution on which the evaluator must be reliable.

The policy changes the examples seen by the evaluator. The evaluator changes the policy. Static held-out accuracy does not fully characterize this feedback loop.

Common misconceptions

“RLHF is one algorithm”

It is a family of pipelines combining demonstration data, preference collection, reward or judge models, policy optimization, filtering, and evaluation.

“A deterministic verifier is ground truth”

Only relative to its parser, tests, and formal specification.

“DPO eliminates reward modeling”

It eliminates an explicit scalar reward model during the policy update. Preference labels still embody an evaluator.

“GRPO is PPO without a critic and therefore has no baseline error”

It replaces critic error with finite group-sampling error, reward noise, and degenerate-group problems.

“A correct final answer proves the reasoning is correct”

No. It may be a lucky guess, an exploited parser, or a correct result reached through invalid steps.

“Process supervision is always denser and therefore better”

Denser incorrect feedback can be more harmful than sparse reliable feedback.

“More test-time samples always improve quality”

Only if proposal diversity and selector accuracy are sufficient. More samples can search evaluator errors.

“Self-improvement needs no external signal”

Filtering requires an answer key, verifier, judge, reward, or human preference.

“Longer reasoning means better reasoning”

Length is a resource and a possible proxy, not a correctness guarantee.

Key equations

Concept	Equation
SFT	$-\sum_t \log \pi_\theta(y_t x, y_{<t})$
Bradley-Terry	$P(y_w \succ y_l x) = \sigma(r_w - r_l)$
RM loss	$-\log \sigma(r_w - r_l)$
KL-regularized objective	$\mathbb{E}_\pi[r] - \beta D_{\text{KL}}(\pi \ \pi_{\text{ref}})$
Optimal policy	$\pi^* \propto \pi_{\text{ref}} \exp(r/\beta)$
DPO margin	$\beta \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)}$
Group advantage	$(r_i - \bar{r})/(s_r + \varepsilon)$
PRM rollout value	$M^{-1} \sum_m \mathbf{1}\{\text{completion } m \text{ succeeds}\}$
Potential shaping	$F(s, s') = \gamma \Phi(s') - \Phi(s)$
Best-of- N coverage	$1 - (1 - p)^N$
Turn policy	$\pi(a_t s_t) = \prod_l \pi(y_l^{(t)} s_t, y_{<l}^{(t)})$

Knowledge check

1. Derive the sequence-level KL decomposition into token log ratios.
2. Derive the KL-regularized optimal policy and state the support assumptions.
3. Why does the prompt-dependent normalizer cancel in DPO?
4. Explain how sequence length enters DPO through log probability.
5. What learning signal remains in a GRPO group when all rewards are equal?
6. Compare token-level and sequence-level loss reduction for one 20-token and one 200-token response.
7. Distinguish an ORM, an outcome verifier, and a PRM.
8. Why is rollout-based process scoring policy-dependent?
9. Decompose best-of- N failure into proposal failure and selector failure.
10. Give two ways a unit-test verifier can be exploited.
11. Explain why repeated rejection-sampling SFT is an iterative policy-improvement process rather than purely static supervised learning.
12. Identify the state, action, transition, and reward in a multi-turn tool-use agent.

Exercises

1. Toy reward-model experiment

Create synthetic responses with known latent utility and a biased feature such as length. Generate noisy pairwise preferences, train a Bradley-Terry reward model, and measure:

- pairwise accuracy;
- reward calibration;
- selected-response true utility as optimization pressure increases;
- whether the policy exploits the biased feature.

2. DPO derivation and implementation

Implement DPO on a toy categorical response space. Verify numerically that the learned policy approaches the analytic KL-regularized optimum when preference labels are generated exactly from the assumed reward model.

3. Group-relative estimator stress test

Construct groups containing:

- all-correct rewards;
- all-incorrect rewards;
- one positive outlier;
- noisy continuous rewards;
- responses with large length imbalance.

Compare centered, standardized, and unnormalized group advantages. Report gradient contribution by response and token.

4. Verifier exploitation lab

Build a deliberately incomplete code or arithmetic verifier. Optimize a simple policy or search procedure against it. Document examples where verifier score rises while semantic correctness falls. Add hidden tests and measure which exploits remain.

5. Process-versus-outcome search

On a small reasoning benchmark:

1. generate multiple partial and complete solutions;
2. score with a final-answer verifier and a step model;
3. compare best-of- N , beam search, and a small tree search;
4. separate proposal coverage from selection accuracy.

6. Multi-turn credit assignment

Create a tool-use environment with delayed terminal reward. Compare:

- trajectory-level REINFORCE;
- turn-level critic;
- verifier-backed subgoal rewards;
- imitation of successful traces.

Audit whether dense rewards change the optimal behavior.

7. Reward-component ablation

Train with correctness, format, length, and repetition terms. Remove one component at a time and report every reward dimension, KL, response length, and held-out human or oracle quality.

Primary readings

RLHF and preference optimization

- Ziegler et al. (2019), **Fine-Tuning Language Models from Human Preferences**.
- Stiennon et al. (2020), **Learning to Summarize from Human Feedback**.
- Ouyang et al. (2022), **Training Language Models to Follow Instructions with Human Feedback**.
- Bai et al. (2022), **Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback**.
- Rafailov et al. (2023), **Direct Preference Optimization**.
- Ethayarajh et al. (2024), **KTO**.
- Meng et al. (2024), **SimPO**.
- Hong et al. (2024), **ORPO**.

Reasoning, verification, and process supervision

- Lightman et al. (2023), **Let’s Verify Step by Step**.
- Zelikman et al. (2022), **STaR**.
- Singh et al. (2024), **ReST-EM**.
- Wang et al. (2024), **Math-Shepherd**.
- DeepSeek-AI (2024), **DeepSeekMath**, for GRPO.
- DeepSeek-AI (2025), **DeepSeek-R1** technical report.
- Yu et al. (2025), **DAPO**.

Search and agents

- Yao et al. (2023), **Tree of Thoughts**.
- Hao et al. (2023), **Reasoning with Language Model Is Planning with World Model**.
- Yao et al. (2023), **ReAct**.
- Le et al. (2022), **CodeRL**.
- Liu et al. (2023), **RLTF**.

Overoptimization and preference games

- Gao et al. (2022), **Scaling Laws for Reward Model Overoptimization**.
- Eisenstein et al. (2023), **Reward Model Ensembles Mitigate but Do Not Eliminate Reward Hacking**.
- Munos et al. (2024), **Nash Learning from Human Feedback**.

Next: Part IX — Statistical Corner Cases in Reinforcement Learning.

Part IX — Statistical Corner Cases in Reinforcement Learning

Lecture opening: Write the estimand before the estimate

A higher validation mean can reflect a better policy, a lucky training seed, selected checkpoints, an easier prompt mixture, or leaked rewards. We will diagnose these possibilities by separating the target quantity from its estimator and from the procedure that selected it.

For each failure mode, write a minimal model in which it occurs. A.10 isolates critic and boundary error; A.14 isolates group dependence; A.15 proves support nonidentifiability with two environments producing identical logs. The key habit is causal and statistical: identify which distribution or conditioning step changed, then decide whether the estimator still addresses the original question.

Learning objectives

By the end of this chapter, you should be able to:

- analyze an RL result in terms of estimand, data distribution, estimator, and feedback loop;
- distinguish bias, variance, temporal dependence, support mismatch, and nonstationarity;
- derive how critic error enters a bootstrapped advantage estimate;
- diagnose importance-weight collapse using effective sample size;
- explain why PPO clipping does not create a global trust region;
- separate within-policy evaluation noise from between-seed training variability;
- identify checkpoint-selection and reward-selection bias;
- choose diagnostics that could falsify a proposed explanation for training failure.

Prerequisites

Parts II, IV, VII, and the RLHF material in Part VIII where relevant.

Overview: RL is estimation inside a feedback loop

An RL algorithm is not merely optimizing a fixed loss on a fixed dataset.

1. The current policy generates data.
2. An estimator converts the data into a learning signal.
3. The optimizer changes the policy.
4. The changed policy changes the next data distribution.
5. Learned critics, models, or rewards may also change the targets.

Flow to keep in mind. A current policy generates correlated trajectories; return, critic and reward estimators consume them; optimization updates the policy. The update changes the state-action distribution and can also change the fitted estimators, closing two interacting feedback loops.

A method can have a correct equation and still fail because the finite-sample estimator, data coverage, target network, or evaluation protocol is unreliable.

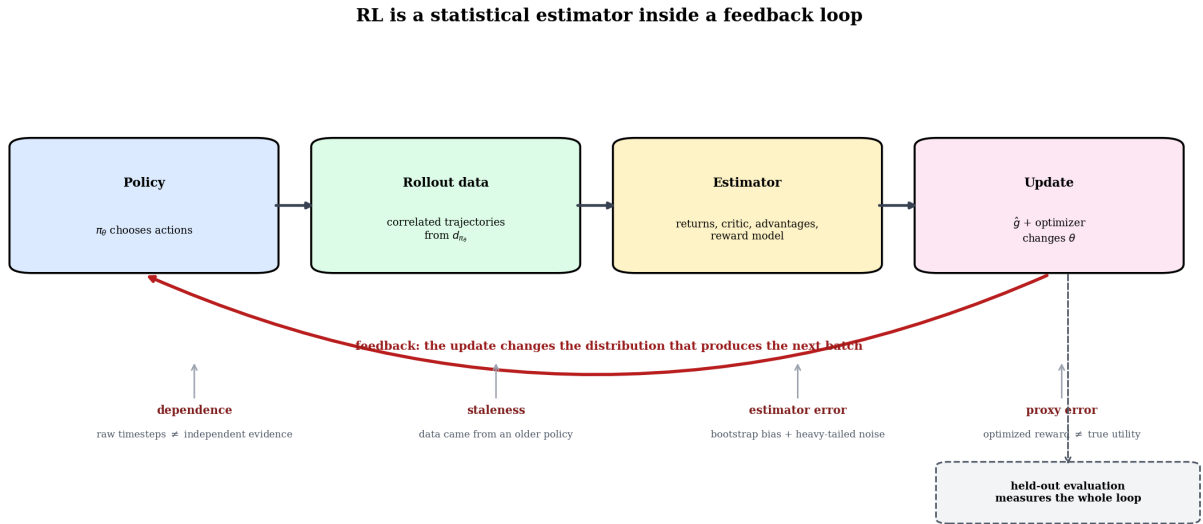


Figure 53: Reinforcement learning as a statistical estimator embedded in a feedback loop

30. Why RL Statistics Are Hard

Start with four questions

For any reported RL number, ask:

1. **What is the estimand?**
2. **What data were collected, and by which policy?**
3. **What estimator maps those data to the reported number or update?**
4. **How does using the estimator change future data?**

These questions prevent many category errors.

Example estimands

- expected return of a fixed final policy;
- average return across independently trained policies;
- learning-curve area under a fixed interaction budget;
- success probability under a test distribution;
- target-policy value from logged behavior data;
- human utility of an RLHF policy rather than reward-model score.

These are different quantities and require different independent units.

The policy-gradient estimator is random

A typical batch estimator is

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta} \left(A_t^{(i)} \mid S_t^{(i)} \right) \hat{A}_t^{(i)}.$$

It varies because of:

- initial-state randomness;
- stochastic actions;
- stochastic transitions and rewards;
- finite trajectory count;
- critic estimation error;
- minibatch order;

- network initialization;
- optimizer state;
- environment and system scheduling.

Calling this “the gradient” hides that it is a noisy, often biased estimate of a policy-improvement direction.

Main statistical failure modes

Failure source	Where it enters	Typical symptom
finite-sample noise	rollouts and evaluation	jagged or seed-sensitive results
temporal dependence	consecutive transitions	nominal batch larger than effective information
bootstrapping bias	critic or Q target	confidently wrong values
max-selection bias	greedy target or actor	Q overestimation
off-policy mismatch	replay or logged data	large ratios and unsupported actions
nonstationarity	changing policy, model, reward, opponents	loss falls while performance changes unpredictably
function approximation	generalization away from data	hallucinated values
normalization effects	batch-relative targets	update depends strongly on batch composition
checkpoint selection	choose best noisy checkpoint	optimistic reported score
reward misspecification	optimize proxy	reward rises while intended behavior worsens

Bias and variance are not moral categories

For estimator $\hat{\mu}$,

$$\text{MSE}(\hat{\mu}) = \text{Var}(\hat{\mu}) + \text{Bias}(\hat{\mu})^2.$$

A small deliberate bias can be useful if it removes catastrophic variance. Clipped importance weights, weighted IS, GAE, target networks, and pessimistic critics all move error rather than eliminating it.

The relevant question is:

Is the resulting tradeoff reliable enough for this data, horizon, and decision?

Optimization error versus statistical error

Suppose a loss is minimized accurately. The resulting policy can still be poor because:

- the loss used biased targets;
- the dataset lacked support;
- the reward was misspecified;
- the selected checkpoint benefited from noise;
- the policy shifted beyond the evaluator’s validity.

Better optimization solves the specified finite-sample problem more exactly. It can therefore amplify misspecification.

31. Finite Samples, Correlation, and Stale Data

Nominal sample count is not information count

A PPO rollout might contain

$$64 \times 2048 = 131,072$$

transitions. Consecutive transitions within an environment are correlated, and environments may share dynamics, reset distributions, or curriculum.

The batch is useful for optimization, but it is not equivalent to 131,072 independent experiments.

Effective sample size under autocorrelation

Temporal correlation inflates variance and shrinks effective sample size

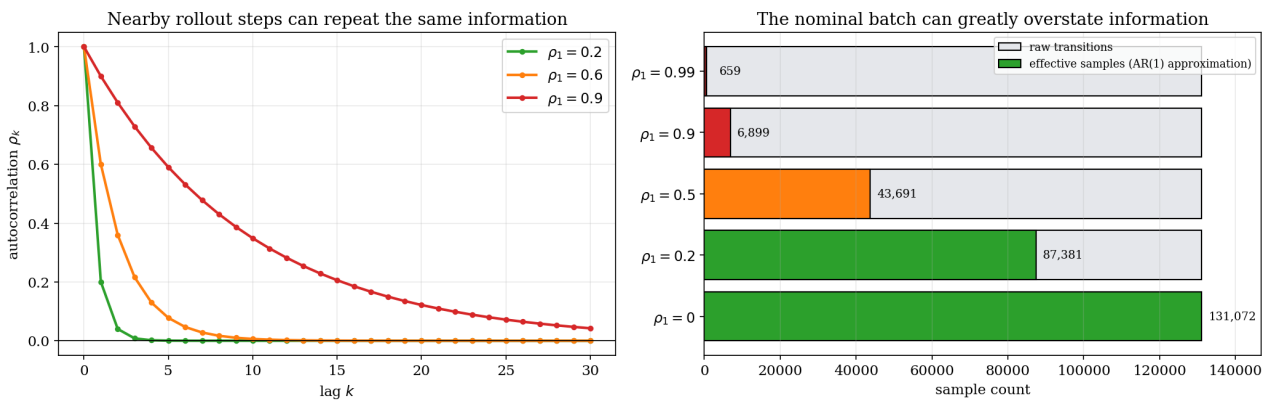


Figure 54: Autocorrelation reducing effective sample size despite many recorded timesteps

For a stationary scalar process with variance σ^2 and autocorrelations ρ_k ,

$$\text{Var}(\bar{X}_n) \approx \frac{\sigma^2}{n} \left(1 + 2 \sum_{k=1}^{\infty} \rho_k \right).$$

Define

$$n_{\text{eff}} \approx \frac{n}{1 + 2 \sum_{k=1}^{\infty} \rho_k}.$$

This is a diagnostic approximation requiring stationarity and a meaningful truncation of the autocorrelation sum.

For finite n , the exact stationary variance identity is

$$\text{Var}(\bar{X}_n) = \frac{\sigma^2}{n} \left[1 + 2 \sum_{k=1}^{n-1} \left(1 - \frac{k}{n} \right) \rho_k \right].$$

To derive it, expand $n^{-2} \sum_{i,j} \text{Cov}(X_i, X_j)$; there are $n - k$ pairs at lag k in each direction. Absolute summability of autocorrelations supports the large- n approximation above. For negative correlations the diagnostic effective size can exceed n ; it is a variance-equivalence quantity, not a literal count of independent records.

Numerical example

Let

$$\rho_k = 0.9^k.$$

Then

$$\sum_{k=1}^{\infty} \rho_k = \frac{0.9}{1 - 0.9} = 9.$$

The variance inflation factor is

$$1 + 2(9) = 19.$$

For $n = 131,072$,

$$n_{\text{eff}} \approx \frac{131,072}{19} \approx 6,899.$$

The exact number is not the point. The point is that positive temporal dependence can reduce information by an order of magnitude.

Correlation occurs at multiple levels

- timesteps within an episode;
- episodes sharing the same trained policy;
- vector environments sharing a simulator implementation;
- checkpoints from one training run;
- tasks sharing a benchmark template;
- reward-model labels from the same annotator or judge.

Shuffling a minibatch removes order but does not erase these dependencies.

Sparse rewards and rare informative events

In a sparse-reward task, most transitions may contain no direct evidence about success. A few successful trajectories can dominate advantages and gradients.

Useful summaries include:

- success count per rollout batch;
- fraction of gradient norm contributed by top-return trajectories;
- return distribution rather than only mean;
- state coverage before first success;
- number of independent successful episodes.

A batch with 100,000 transitions and two successes may have only two direct examples of the event that matters.

PPO batch reuse

PPO collects data from π_{old} and reuses it for several epochs while updating π_{θ} .

The sampled ratio is

$$r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\text{old}}(A_t | S_t)}.$$

At epoch zero, $r_t = 1$. As optimization proceeds:

- the policy moves away from the collector;
- advantages become stale relative to the new policy;
- the rollout state distribution remains fixed;
- more samples enter the clipped region;
- the critic may overfit the same returns.

Data reuse improves sample efficiency but increases stale-batch risk.

Ratio tails matter

The mean ratio can remain near one while a small number of samples have extreme ratios.

Track:

$$\text{median}(r_t), \quad p_{95}(r_t), \quad p_{99}(r_t), \quad \max r_t.$$

Also inspect log ratios:

$$\Delta \ell_t = \log \pi_\theta(A_t | S_t) - \log \pi_{\text{old}}(A_t | S_t).$$

Log ratios are numerically easier to visualize across several orders of magnitude.

Clip fraction

Define

$$\text{clipfrac} = \frac{1}{n} \sum_t \mathbf{1}\{|r_t - 1| > \epsilon\}.$$

A high clip fraction means many sampled terms have saturated the surrogate. It does not by itself indicate failure; it indicates that the current batch is no longer providing an ordinary local linear update.

Approximate KL by epoch

For actions sampled from the old policy,

$$\widehat{D}_{\text{KL}} = \frac{1}{n} \sum_t [\log \pi_{\text{old}}(A_t | S_t) - \log \pi_\theta(A_t | S_t)]$$

is a sample estimate of old-to-new KL on sampled states and actions. Alternative approximations exist and can differ at finite sample.

Plot KL after each epoch over the same rollout. A sharp within-batch rise is evidence that data reuse is outrunning the local approximation.

Independent actors versus longer rollouts

With the same transition budget, more independently reset actors often provide more state diversity than one long trajectory.

This does not make the data iid. Shared simulator defects, narrow initial-state distributions, or identical curricula remain common sources of dependence.

Reporting level

For scientific claims, aggregate at the level of:

- independent training runs;
- independent tasks;
- independent evaluation episodes within each fixed policy.

Do not use the number of training timesteps as the denominator of a confidence interval on algorithm performance.

32. Bootstrapping Bias and the Critic

Intuition first: the target contains a prediction

TD learning replaces a complete sampled return with

$$Y_t = R_{t+1} + \gamma \widehat{V}(S_{t+1}).$$

This lowers variance and enables online learning. It also lets critic error enter the target.

Exact critic case

If $\widehat{V} = v_\pi$, then

$$\delta_t = R_{t+1} + \gamma v_\pi(S_{t+1}) - v_\pi(S_t)$$

satisfies

$$\mathbb{E}[\delta_t \mid S_t = s, A_t = a] = A_\pi(s, a).$$

Error decomposition for an approximate critic

n-step targets choose how much to trust the critic

Longer targets replace critic guesses with sampled rewards

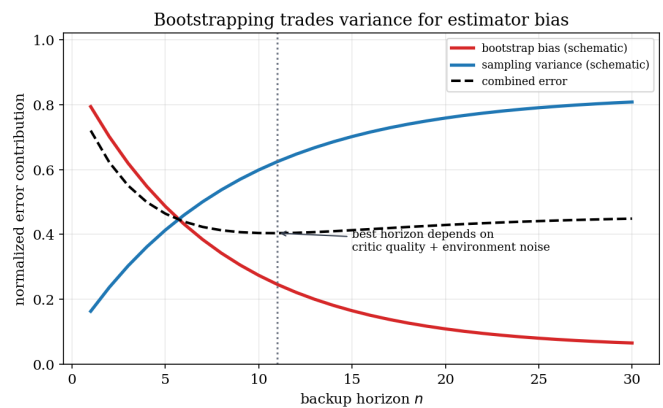
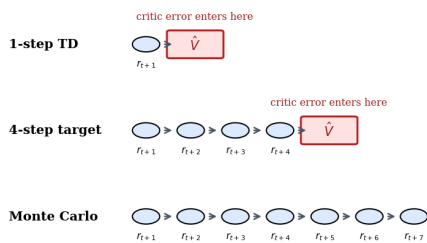


Figure 55: How bootstrap horizon changes critic bias and target variance

Let

$$e(s) = \widehat{V}(s) - v_\pi(s).$$

The approximate TD residual is

$$\hat{\delta}_t = R_{t+1} + \gamma \widehat{V}(S_{t+1}) - \widehat{V}(S_t).$$

Substitute $\widehat{V} = v_\pi + e$:

$$\hat{\delta}_t = \delta_t^{\text{true}} + \gamma e(S_{t+1}) - e(S_t).$$

Taking conditional expectation,

$$\mathbb{E}[\hat{\delta}_t \mid s, a] = A_\pi(s, a) + \gamma \mathbb{E}[e(S_{t+1}) \mid s, a] - e(s).$$

This shows exactly how critic error biases the advantage estimate.

A fixed state-only baseline error $-e(s)$ would vanish in an otherwise Monte Carlo policy gradient. The next-state term enters through the bootstrap and does not generally vanish.

***n*-step targets**

The *n*-step target is

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n \widehat{V}(S_{t+n}).$$

Relative to a target bootstrapped from the true value, the critic-error contribution is

$$\gamma^n e(S_{t+n}).$$

Longer *n* geometrically reduces the direct bootstrap coefficient but adds more sampled reward noise. This is the bias-variance tradeoff behind TD, *n*-step methods, and GAE.

Baseline versus bootstrap

Unbiased return with baseline

$$\hat{g}_t = \nabla \log \pi(A_t \mid S_t) (G_t - b(S_t)).$$

Any fixed or appropriately pre-action-measurable state-only *b* preserves the expected gradient. Fitting it on the same sampled action and return can invalidate the conditional score cancellation; A.9 states the condition precisely.

Bootstrapped advantage

$$\hat{g}_t = \nabla \log \pi(A_t \mid S_t) [R_{t+1} + \gamma \widehat{V}(S_{t+1}) - \widehat{V}(S_t)].$$

The learned estimate appears inside the target and can bias the update.

Low critic loss can coexist with bad values

A critic may fit its training targets while the targets themselves are biased or stale. Examples:

- DQN target network overestimates;
- truncated episodes are treated as terminal;
- replay emphasizes narrow regions;

- reward normalization is inconsistent;
- bootstrapped targets drift together with predictions.

Training loss measures agreement with the supplied target, not agreement with true return.

Critic calibration diagnostics

For held-out rollout data, compare

$$\widehat{V}(S_t)$$

with empirical return

$$G_t.$$

Inspect:

- scatter plot and calibration curve;
- mean error by timestep;
- error by return quantile;
- error on successes versus failures;
- explained variance;
- prediction interval coverage for distributional critics;
- disagreement across critic ensembles.

A critic can have positive explained variance and still have a dangerous systematic bias on rare high-value states.

Max-selection bias

Suppose

$$\widehat{Q}(s, a) = Q(s, a) + \varepsilon_a$$

with zero-mean noise. Then

$$\mathbb{E} \left[\max_a \widehat{Q}(s, a) \right] \geq \max_a Q(s, a).$$

The max selects both true value and positive noise.

This affects:

- Q-learning targets;
- deterministic actors maximizing a critic;
- best-of- N selection under noisy reward models;
- checkpoint selection.

The same statistical pattern appears throughout RL: optimization selects estimator error.

Shared actor-critic representation

A shared encoder receives gradients from policy and value losses. A critic with easier or larger gradients can dominate representation learning.

Symptoms include:

- critic loss improves while entropy or return collapses;

- policy features become specialized to value prediction;
- changing the value-loss coefficient changes policy behavior sharply.

Separate encoders, gradient surgery, or phasic training can reduce interference, but each changes compute and representation sharing.

Terminal and truncation bias

For true termination,

$$Y_t = R_{t+1}.$$

For a time-limit truncation in an otherwise continuing task,

$$Y_t = R_{t+1} + \gamma \widehat{V}(S_{t+1}).$$

Masking both cases as terminal biases values near the rollout limit downward. This can alter policy incentives even when average critic loss appears normal.

33. Off-Policy Evaluation and Importance-Sampling Pathologies

Problem statement

Off-policy evaluation estimates

$$V^\pi = \mathbb{E}_{\tau \sim \pi}[G(\tau)]$$

using trajectories generated by behavior policy b .

The central identifying assumption is support:

$$\pi(a \mid s) > 0 \implies b(a \mid s) > 0$$

for all relevant state-action pairs.

If the behavior never takes an action the target policy uses, the data alone do not identify its consequence.

Trajectory importance sampling

The trajectory ratio is

$$\rho(\tau) = \prod_{t=0}^{T-1} \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}.$$

Then

$$\mathbb{E}_{\tau \sim \pi}[G(\tau)] = \mathbb{E}_{\tau \sim b}[\rho(\tau)G(\tau)]$$

under support and integrability conditions.

Off-policy importance weights: correct expectation, fragile finite sample

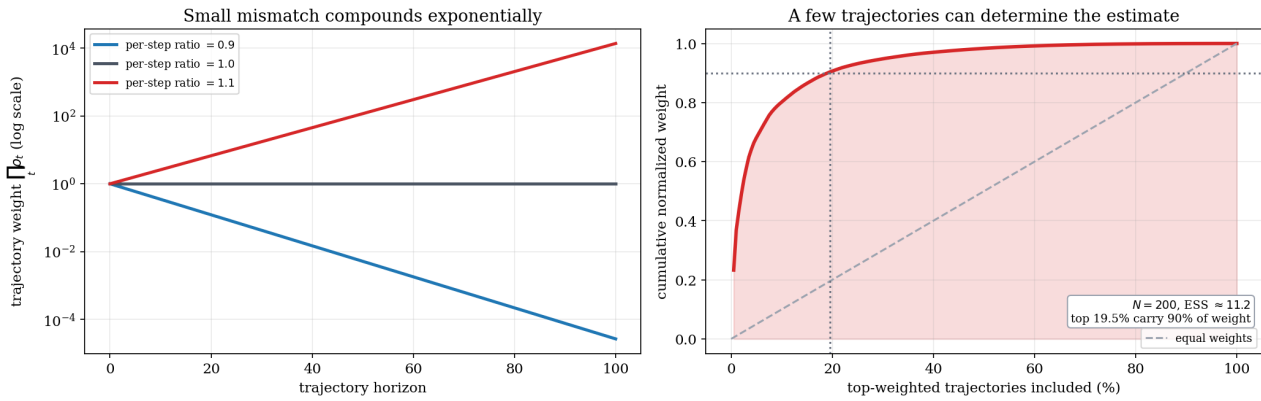


Figure 56: Importance weights collapsing as trajectory horizon grows

Product-of-ratios pathology

Small mismatches compound. For a constant per-step ratio c ,

$$\rho = c^T.$$

A distribution of products becomes highly skewed even when individual ratios are moderate. Most trajectories receive near-zero weight and a few dominate.

Weight effective sample size

For trajectory weights ρ_i ,

$$\text{ESS} = \frac{(\sum_i \rho_i)^2}{\sum_i \rho_i^2}.$$

Example: one trajectory has weight 100, the other 99 trajectories have weight 0. Then

$$\text{ESS} = 1.$$

The dataset contains 100 trajectories but one effective weighted trajectory.

Ordinary and weighted IS

Ordinary IS:

$$\widehat{V}_{\text{OIS}} = \frac{1}{N} \sum_i \rho_i G_i.$$

- unbiased under assumptions;
- potentially enormous variance.

Weighted IS:

$$\widehat{V}_{\text{WIS}} = \frac{\sum_i \rho_i G_i}{\sum_i \rho_i}.$$

- biased at finite N ;
- often lower variance;

- undefined if all observed weights are zero.

Unknown behavior policy

Logged data may not include exact action probabilities. Estimating $b(a \mid s)$ introduces another model.

Errors are most damaging where b is small because the ratio uses $1/b$.

For deterministic or nearly deterministic logging policies, reliable OPE of a substantially different target may be impossible.

Mixtures of behavior policies

Replay or production logs may contain policies $b_1, \dots, b_K$. The relevant behavior density for a sampled transition depends on its source policy and collection protocol.

Treating the mixture as one stationary policy without policy-version metadata can produce incorrect ratios.

Ratio clipping changes the estimand

Using

$$\bar{\rho} = \min(c, \rho)$$

reduces extreme influence. The resulting estimator is no longer ordinary IS for V^π .

Report:

- clipping threshold;
- clipped fraction;
- unclipped and clipped ESS;
- sensitivity curve across thresholds;
- direction of likely bias when known.

Horizon dependence

OPE becomes harder as horizon increases because:

- more ratios multiply;
- support requirements apply to longer action sequences;
- model errors propagate farther;
- delayed rewards create rare high-impact trajectories.

Per-decision IS, marginalized density ratios, doubly robust estimators, and fitted Q evaluation address parts of this problem but cannot recover missing support without assumptions.

OPE failure checklist

Do not trust an OPE number without:

- target and behavior policy definitions;
- support diagnostics;
- ratio quantiles;
- ESS;
- horizon;
- clipping details;
- uncertainty interval;
- sensitivity to model class or clipping;

- comparison across at least two estimator families when feasible.

34. Variance Reduction in Policy Gradients

Source of variance

The score-function term

$$z_t = \nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t)$$

is multiplied by a random return or advantage:

$$\hat{g}_t = z_t \hat{A}_t.$$

Variance comes from both factors and their interaction.

State-only baseline theorem

For any fixed or pre-action-measurable $b(S_t)$,

$$\mathbb{E}[b(S_t) \nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t)] = 0.$$

Thus subtracting such a baseline preserves the expected gradient. A same-batch fitted state-only predictor requires the conditional-independence audit in A.9.

This is a clean theorem. It should not be conflated with every normalization or critic trick.

Optimal scalar baseline

Conditional on $S = s$, with score z , the scalar baseline minimizing

$$\mathbb{E}[\|z(G - b)\|^2 \mid S = s]$$

is

$$b^*(s) = \frac{\mathbb{E}[G\|z\|^2 \mid S = s]}{\mathbb{E}[\|z\|^2 \mid S = s]}.$$

The value function is a practical approximation, not always the exact variance-minimizing baseline.

Advantage normalization

A common batch transform is

$$\tilde{A}_t = \frac{\hat{A}_t - \bar{A}}{s_A + \varepsilon}.$$

It stabilizes scale but is not the same as subtracting a fixed state-only baseline.

It changes the batch-relative learning signal:

- samples below the batch mean become negative;
- one outlier changes every normalized sample;
- minibatch composition affects the gradient;

- PPO clipping interacts with the rescaled magnitudes;
- the absolute reward scale becomes hidden.

The transform is a useful empirical convention, not an unbiasedness theorem.

A sign-change example

Suppose raw advantages are

$$(1, 2, 100).$$

The mean is

$$\bar{A} = 34\frac{1}{3}.$$

After centering, the first two advantages become negative even though both were positive relative to the critic.

This may improve ranking within a batch, but it changes the interpretation of the update.

Reward normalization

Normalizing rewards or returns changes relative strengths among:

- policy loss;
- value loss;
- entropy bonus;
- KL penalty;
- gradient clipping;
- optimizer moments.

If reward normalization statistics are nonstationary, the effective objective changes over training.

Outlier advantages

Large advantages may represent:

- rare genuine success;
- reward bug;
- critic underestimation;
- terminal-mask error;
- high importance ratio;
- stochastic return outlier.

Before clipping an outlier, inspect the trajectory. Before trusting it, verify the reward.

Minibatch variance

Repeatedly optimizing the same rollout does not reduce uncertainty about its advantages. It reduces optimization error on one finite batch and can overfit its noise.

To reduce estimator variance, collect more independent trajectories or improve control variates—not merely more epochs.

35. GAE: A Bias-Variance Knob, Not Magic

Definition

With approximate value $\widehat{V}$,

$$\widehat{\delta}_t = R_{t+1} + \gamma \widehat{V}(S_{t+1}) - \widehat{V}(S_t).$$

GAE is

$$\widehat{A}_t^{\gamma, \lambda} = \sum_{l=0}^{T-t-1} (\gamma \lambda)^l \widehat{\delta}_{t+l}.$$

Its backward recursion is

$$\widehat{A}_t = \widehat{\delta}_t + \gamma \lambda (1 - d_{t+1}) \widehat{A}_{t+1}.$$

Effective horizon

The geometric weight scale is

$$H_{\text{GAE}} \approx \frac{1}{1 - \gamma \lambda}.$$

γ	λ	$\gamma \lambda$	H_{GAE}
0.99	0.80	0.792	4.8
0.99	0.90	0.891	9.2
0.99	0.95	0.9405	16.8
0.99	0.99	0.9801	50.3
0.995	0.97	0.96515	28.7

A familiar default can imply very different credit horizons when γ changes.

Critic-error propagation in GAE

Let

$$e_t = \widehat{V}(S_t) - v_\pi(S_t).$$

Each TD residual contains

$$\gamma e_{t+1} - e_t.$$

Therefore GAE error contains

$$\sum_{l \geq 0} (\gamma \lambda)^l (\gamma e_{t+l+1} - e_{t+l}).$$

At $\lambda = 1$ on a complete episode, much of this telescopes to a state-only baseline term plus terminal handling. At smaller λ , local critic errors remain more directly inside the advantage.

This is why lower λ depends more strongly on critic calibration.

The finite-segment critic-error formula in A.10 is more informative than the infinite series: the final bootstrap error has coefficient $\gamma^n \lambda^{n-1}$. At $\lambda = 1$ it disappears only for a correctly handled complete terminal episode, not automatically at a rollout cutoff. The same section distinguishes bootstrap and trace masks.

When lower λ can help

- dense rewards;
- accurate critic;
- high transition noise;
- small rollout batches;
- short causal consequences.

When higher λ can help

- sparse or delayed rewards;
- biased critic;
- long-horizon consequences;
- complete episodes with manageable return variance.

Time-limit and rollout-cut bias

If a rollout buffer ends before the environment episode, bootstrap from the final state. If the environment truly terminates, do not bootstrap.

The GAE recursion must use separate masks for:

- episode termination;
- data-buffer boundary;
- time-limit truncation.

Stale critic and stale advantages

Advantages are computed using one critic and one collector policy. After many PPO epochs:

- the actor has changed;
- the critic may have changed;
- state occupancy is still from the old policy;
- the stored advantages are unchanged.

The update becomes increasingly detached from the current policy's true advantage.

GAE diagnostics

- raw advantage mean and quantiles;
- Monte Carlo comparison on short episodes;
- value error by timestep;
- sensitivity to λ and rollout length jointly;
- fraction of variance attributable to between-trajectory differences;
- performance near truncation boundaries.

36. Entropy, KL, and Support Mismatch

Entropy as support preservation

For discrete actions,

Support mismatch: an estimator cannot learn counterfactuals absent from its data

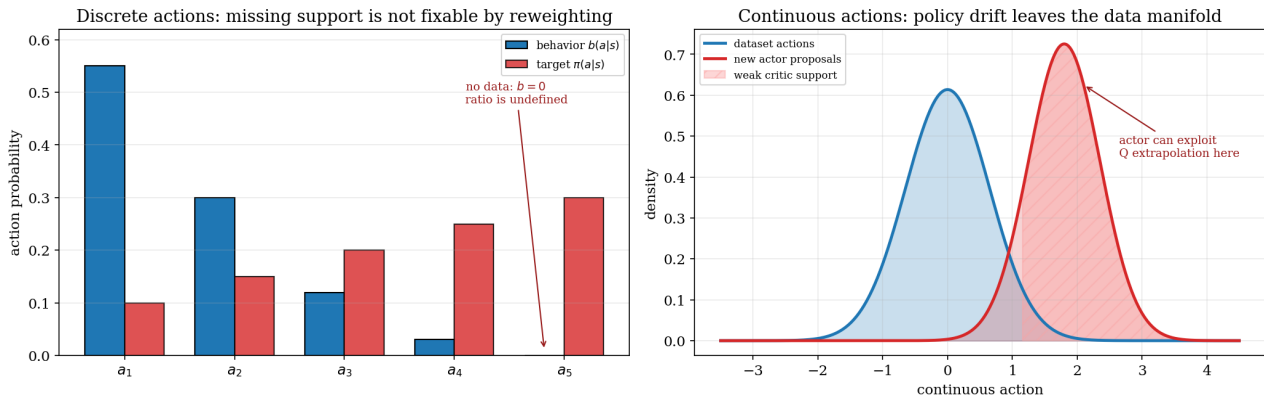


Figure 57: Support mismatch between behavior and target policies

$$\mathcal{H}(\pi(\cdot | s)) = - \sum_a \pi(a | s) \log \pi(a | s).$$

Higher entropy keeps more actions available for exploration and future correction. Low entropy makes the policy brittle and can remove practical support.

Too much entropy prevents exploitation. The desired schedule depends on task stage and action geometry.

KL as drift measurement

Old-to-new KL is

$$D_{\text{KL}}(\pi_{\text{old}} \| \pi_{\text{new}}) = \mathbb{E}_{a \sim \pi_{\text{old}}} \left[\log \frac{\pi_{\text{old}}(a | s)}{\pi_{\text{new}}(a | s)} \right].$$

It measures change on actions weighted by the old policy. A sample estimate sees only sampled states and actions.

A small sampled average KL does not prove small drift at every state.

Rare-action drift

Suppose

$$\pi_{\text{old}}(a_r | s) = 0.001$$

and

$$\pi_{\text{new}}(a_r | s) = 0.01.$$

The rare action probability increased tenfold. A finite old-policy batch may contain no examples of a_r , so sampled ratios do not directly expose the change.

Full-distribution KL or action-probability diagnostics can reveal it at sampled states, but unseen states remain unmeasured.

Why PPO clipping is not a global constraint

The clipped objective uses sampled terms

$$\min \left(r_t \widehat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \widehat{A}_t \right).$$

It does not constrain:

- ratios for unsampled actions;
- probabilities at unseen states;
- parameter changes that alter other logits;
- accumulated drift over repeated epochs;
- critic or reward error.

Clipping creates a local plateau in the surrogate, not a projection onto a trust region.

Continuous-action support

Suppose the replay data contain actions near $a = 0$, but a deterministic actor proposes $a = 0.8$. The critic may have little evidence there.

The actor solves approximately

$$\mu_\theta(s) \approx \arg \max_a \widehat{Q}(s, a),$$

so it searches for extrapolation error.

Track:

- distance from policy actions to dataset actions;
- critic value by data density;
- action standard deviation;
- critic disagreement;
- behavior-policy likelihood of target actions.

Entropy collapse as a feedback loop

1. A few actions receive high estimated advantage.
2. Their probabilities increase.
3. Entropy falls.
4. Future rollouts contain fewer alternatives.
5. The critic receives narrower data.
6. Recovery from an early mistake becomes harder.

An entropy bonus, minimum action variance, or broader data collection can interrupt the loop, but may also slow convergence.

37. Nonstationarity and Distribution Shift

Replay buffers are policy mixtures

A replay buffer contains data from policies $\pi_1, \dots, \pi_K$:

$$\mu_{\mathcal{D}}(s, a) \approx \sum_{k=1}^K w_k d_{\pi_k}(s) \pi_k(a | s).$$

The current actor may target d_{π_K} while the critic fits the mixture.

Replay improves reuse and breaks short-range ordering. It creates age and support mismatch.

Replay age

For each transition, record:

- environment step when collected;
- policy version;
- priority updates;
- whether reward or preprocessing changed.

Plot performance or TD error by sample age. Very old samples can stabilize coverage or obstruct adaptation, depending on environment stationarity.

Prioritized replay changes the sampling distribution

If transition i is sampled with probability

$$P(i) \propto (|\delta_i| + \varepsilon)^\alpha,$$

then high-error samples appear more often. Importance weights attempt to correct this:

$$w_i = \left(\frac{1}{NP(i)} \right)^\beta.$$

Incomplete correction, changing priorities, and correlated TD error make the practical estimator different from uniform replay.

Target networks trade motion for staleness

Hard update:

$$\bar{\theta} \leftarrow \theta \quad \text{every } C \text{ steps.}$$

Soft update:

$$\bar{\theta} \leftarrow \tau\theta + (1 - \tau)\bar{\theta}.$$

Small τ gives stable but stale targets. Large τ gives responsive but moving targets.

Nonstationary rewards and models

In RLHF or learned-reward settings, the evaluator may be retrained. A stored scalar reward may no longer correspond to the current reward model.

Options include:

- store raw trajectories and rescore them;
- version reward models;
- avoid mixing incompatible reward scales;
- log the exact evaluator used for each transition.

Multi-agent and self-play nonstationarity

From agent i 's perspective, transition probabilities depend on opponents' current policies. Replay from old opponents can be misleading.

Evaluate against fixed snapshots and a population, not only the latest opponent.

Curriculum and reset shift

Changing task difficulty or reset distribution changes the training objective. Learning curves across curriculum phases are not samples from one stationary problem.

Report phase boundaries and evaluate on a fixed reference distribution.

Production drift

Users, tools, or environments can change after deployment. A policy trained on one distribution may induce new behavior that further changes inputs.

Offline validation should include:

- temporal holdouts;
 - environment variants;
 - adversarial stress tests;
 - shifted action costs;
 - tool-schema changes;
 - new user populations.
-

38. Seed Sensitivity and Evaluation Uncertainty

Two levels of randomness

Let trained policy from seed k be π_k . Let evaluation episode j produce return $Y_{k,j}$.

Variation decomposes into:

- **within-policy evaluation noise:** episodes differ for the same π_k ;
- **between-seed training noise:** the learned policies differ.

A hierarchical variance decomposition is

$$\text{Var}(Y) = \mathbb{E}_k [\text{Var}(Y \mid \pi_k)] + \text{Var}_k (\mathbb{E}[Y \mid \pi_k]) .$$

Running more episodes reduces uncertainty about each fixed policy. It does not replace independent training seeds.

Mean across training seeds

For seed-level summaries $R_1, \dots, R_K$,

$$\bar{R} = \frac{1}{K} \sum_{k=1}^K R_k .$$

A classical interval is

$$\bar{R} \pm t_{K-1, 0.975} \frac{s_R}{\sqrt{K}} .$$

With $K = 3$ or 5 , individual outcomes should be shown because normal approximations and variance estimates are fragile.

Evaluation success rates

For fixed policy and n independent episodes with success probability p ,

$$\text{SE}(\hat{p}) \approx \sqrt{\frac{p(1-p)}{n}}.$$

If $p = 0.8$ and $n = 5$,

$$\text{SE} \approx 0.179.$$

Five episodes provide a very imprecise success estimate. Use Wilson or exact binomial intervals at small n .

Checkpoint-selection bias

Seed variation and checkpoint selection can manufacture apparent gains

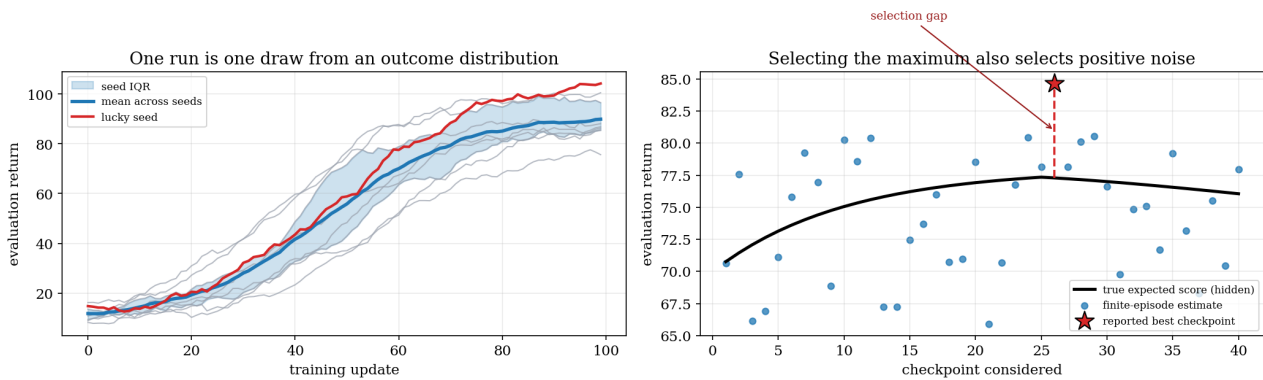


Figure 58: Uncertainty introduced by seed and checkpoint selection

Suppose checkpoint estimates are

$$\widehat{V}_i = V_i + \varepsilon_i.$$

Selecting

$$i^* = \arg \max_i \widehat{V}_i$$

selects both true performance and positive evaluation noise.

For independent Gaussian noise, a rough extreme-value heuristic is

$$\mathbb{E}[\max_i \varepsilon_i] \approx \sigma \sqrt{2 \log N}.$$

More checkpoint comparisons increase optimism unless selection is separated from final testing.

Final, best, and average checkpoint are different estimands

- **final:** performance after the fixed training budget;
- **best validation-selected:** performance of a selection procedure;
- **best test checkpoint:** biased if test data choose the checkpoint;
- **average final window:** stability near the end;
- **area under curve:** learning efficiency over the budget.

Label the choice explicitly.

Multiple tasks

Scores across tasks may have different scales and variance. A raw mean can be dominated by one task.

Useful summaries include:

- median;
- interquartile mean;
- normalized score;
- optimality gap;
- performance profile over thresholds;
- stratified bootstrap across tasks and seeds.

No single aggregate replaces per-task results.

Paired comparisons

When methods are evaluated on the same tasks and scenarios, analyze

$$D_i = R_i^{(A)} - R_i^{(B)}.$$

Paired analysis removes shared task difficulty. The pairing should be planned and preserved in data collection.

Hyperparameter-selection uncertainty

A reported method includes its tuning process. Trying more configurations increases the chance of selecting a lucky validation result.

Budget-match:

- number of trials;
- training compute per trial;
- validation episodes;
- search space quality.

An algorithm comparison that ignores tuning effort can misattribute search advantage to the algorithm.

39. Reward Hacking, Goodhart Effects, and Policy-Optimization Failures

Proxy model

Let intended utility be

$$U(\tau)$$

and optimized reward be

$$R(\tau) = U(\tau) + \varepsilon(\tau).$$

The optimizer chooses high R , so it selects both high utility and positive proxy error.

As the search space or optimization pressure grows, the maximum selected error tends to grow.

Reward hacking is not an optimizer malfunction

If a loophole raises the implemented reward, exploiting it is compatible with successful optimization.

Examples:

- circling near a goal to collect dense proximity reward;
- delaying termination to accumulate per-step reward;
- hard-coding visible unit tests;
- generating verbose text favored by a reward model;
- manipulating a sensor or reward channel;
- exploiting a parser rather than solving the task.

Reward shaping corner case

Suppose

$$r_t = c - d(S_t, \text{goal})$$

is positive near the goal and termination gives no sufficient bonus. The agent may hover near the goal instead of ending the episode.

The problem is the cumulative incentive and terminal design, not distance shaping alone.

Potential-based shaping

$$F(s, a, s') = \gamma\Phi(s') - \Phi(s)$$

preserves optimal policies under its theorem assumptions. Arbitrary dense rewards do not.

Reward-model selection bias

For N generated candidates with reward-model errors ε_i , selecting the highest score tends to select a positive error:

$$\mathbb{E}[\varepsilon_{i^*}] > 0.$$

Increasing best-of- N , search depth, or RL steps can worsen the gap between reward-model score and held-out quality.

A worked PPO-collapse case

Suppose continuous-control PPO uses:

- 8 environments;
- 2048 rollout steps;
- 20 epochs per batch;
- high learning rate;
- weak entropy regularization.

Observed sequence:

1. return improves;
2. KL rises sharply within batches;
3. clip fraction becomes large;
4. entropy collapses;
5. critic predictions become optimistic;
6. return crashes.

A statistical causal hypothesis is:

1. a few favorable trajectories create large advantages;
2. repeated epochs overfit those finite-sample advantages;
3. the policy moves beyond the rollout distribution;
4. clipping saturates many sampled terms but does not restore coverage;
5. lower entropy narrows future data;
6. the critic overfits the new narrow distribution;
7. the feedback loop reinforces a brittle policy.

This hypothesis should be tested, not merely asserted.

Falsifying ablations

- reduce epochs while matching total environment steps;
- lower learning rate;
- early-stop at target KL;
- increase independent rollout sources;
- use separate actor and critic encoders;
- compare raw and normalized advantages;
- increase entropy temporarily;
- audit terminal masks;
- evaluate more seeds.

A good diagnosis predicts what these ablations should change.

Held-out true-objective checks

Maintain evaluations not directly optimized:

- human review;
- hidden tests;
- alternate reward models;
- environment variants;
- safety checks;
- adversarial prompts;
- long-term outcomes.

A held-out evaluator can also be Goodharted indirectly if repeatedly consulted during development. Rotate or preserve final blind tests.

40. Diagnostics Checklist

Training distribution

Quantity	Failure detected
state visitation	exploration collapse or narrow support
action histogram or density	action-mode collapse
episode length and terminal cause	termination hacks
success count per batch	sparse-event dominance
replay age and policy version	stale data
reward-component distribution	proxy exploitation

Policy update

Quantity	Failure detected
raw advantage quantiles	outlier-driven gradient
normalized advantage quantiles	batch-relative distortion
ratio median, p95, p99	heavy-tailed updates
clip fraction	surrogate saturation
KL by epoch	stale-batch drift
entropy or action std	support collapse
gradient norm	explosion or vanishing

Critic

Quantity	Failure detected
value versus empirical return	systematic bias
error by timestep	truncation bug
error by success/failure	rare-outcome underfit
TD-error distribution	bad targets or outliers
critic disagreement	model-specific uncertainty heuristic
Q versus data density	extrapolation exploitation

Off-policy evaluation

Quantity	Failure detected
behavior likelihood	support weakness
ratio quantiles	mismatch
trajectory and per-decision ESS	weight collapse
clipping sensitivity	bias-variance fragility
estimator disagreement	model or weight dependence

Evaluation

Quantity	Failure detected
individual seed outcomes	run-to-run instability
episodes per policy	within-policy noise
final versus selected checkpoint	selection optimism
validation versus test	tuning leakage
per-task results	aggregate masking
held-out objective	reward hacking

41. Intuition Summary

Concept	Mental model
finite samples	every update is an estimate, not an oracle
temporal correlation	many timesteps can repeat the same information
stale batches	PPO keeps optimizing after the policy that generated the data has changed
bootstrapping	guesses become labels

Concept	Mental model
baseline	removes state-level noise without changing an MC gradient
GAE	chooses how much to trust critic predictions versus sampled rewards
importance sampling	mathematically reweights overlap; cannot invent missing support
ESS	counts how many weighted samples actually matter
entropy	preserves alternatives for exploration and correction
KL	measures drift in a specified direction on a specified distribution
PPO clipping	limits local incentive on sampled actions, not global policy movement
replay	trades temporal correlation for policy-age mismatch
seed sensitivity	one run is one sample from a training-outcome distribution
checkpoint selection	maximizing a noisy estimate selects noise
reward hacking	optimization finds proxy loopholes

Final mental model

A policy optimizer asks:

Which change to the current policy will improve future return?

It answers using:

- finite and correlated trajectories;
- an older policy's state distribution;
- noisy or biased advantages;
- a learned critic;
- possibly off-policy data;
- possibly a learned reward;
- limited evaluation.

The practical standard is not perfect unbiasedness. It is whether bias, variance, dependence, support mismatch, and proxy error are controlled enough that the update and the scientific claim are trustworthy.

Knowledge check

1. Distinguish an estimand from an estimator using PPO evaluation as an example.
2. Why does shuffling correlated rollout timesteps not make them independent?
3. Derive the approximate effective sample size for $\rho_k = c^k$.
4. Derive the critic-error term in a one-step TD advantage.
5. Give an inaccurate state-only baseline that does not bias REINFORCE and an inaccurate critic that does bias actor-critic.
6. Explain why a low TD loss can coexist with a poor policy.
7. Construct an OPE dataset with nonzero raw trajectory count but ESS near one.
8. Show how advantage centering can change the signs of positive raw advantages.
9. Explain why PPO clipping does not constrain an unsampled rare action.
10. Decompose total evaluation variance into within-policy and between-seed terms.
11. Explain checkpoint-selection bias using the max-over-noise pattern.
12. Design one falsifying ablation for the worked PPO-collapse hypothesis.

Exercises

1. Autocorrelation audit

For an earlier RL run:

- estimate reward, TD-error, and advantage autocorrelation;
- compute truncated ESS estimates;
- compare one long rollout with several independently reset actors under the same transition count.

2. Critic-bias decomposition

In a small MDP with exact v_π :

1. add a controlled error function $e(s)$;
2. compute the predicted TD-advantage bias analytically;
3. verify it by simulation;
4. repeat for several n -step horizons.

3. Off-policy weight collapse

Generate logged trajectories under behavior policy b and evaluate progressively more different target policies. Plot:

- return estimate;
- confidence interval;
- weight quantiles;
- ESS;
- clipping sensitivity.

Identify the point at which the estimate becomes practically non-informative.

4. Advantage-normalization experiment

On a fixed rollout batch, compare PPO gradients using:

- raw advantages;
- mean-centered advantages;
- standardized advantages;
- clipped advantages.

Measure gradient cosine similarity and per-trajectory contribution.

5. Hierarchical evaluation simulation

Simulate multiple training seeds and multiple evaluation episodes per seed. Show empirically that increasing episodes cannot eliminate between-seed variance. Compare naive pooled and hierarchical intervals.

6. Checkpoint-selection experiment

Generate noisy checkpoint estimates with known true values. Compare:

- best test checkpoint;
- validation-selected then test-evaluated checkpoint;
- final checkpoint;
- final-window average.

Measure optimism as the number of candidate checkpoints grows.

7. Reward-hacking sandbox

Construct a shaping reward with a deliberate loophole. Train or search for a high-reward policy, document the exploit, then redesign the reward or evaluation so the exploit is detected without merely forbidding one observed trajectory.

Primary readings

- Henderson et al. (2018), **Deep Reinforcement Learning that Matters.**
- Colas et al. (2019), **How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments.**
- Agarwal et al. (2021), **Deep Reinforcement Learning at the Edge of the Statistical Precipice.**
- Patterson et al. (2023), **Empirical Design in Reinforcement Learning.**
- Schulman et al. (2016), **Generalized Advantage Estimation.**
- Schulman et al. (2017), **Proximal Policy Optimization Algorithms.**
- Jiang and Li (2016), **Doubly Robust Off-Policy Value Evaluation.**
- Gao et al. (2022), **Scaling Laws for Reward Model Overoptimization.**

Next: Part X — Mitigating Statistical Problems in Reinforcement Learning.

Part X — Mitigating Statistical Problems in Reinforcement Learning

Lecture opening: Fix the mechanism, then test the consequence

No intervention removes all uncertainty. Clipping trades tail variance for bias. Pessimism trades exploitation of uncertain values for conservative decisions. A target network slows a moving target but makes it stale. A mitigation is justified when it targets a demonstrated mechanism and survives a test designed to disprove the diagnosis.

Pair each method with a falsification test, not only a return curve. A.15 proves the two distinct robustness routes for sequential doubly robust evaluation; A.16 shows precisely which target policies Retrace and V-trace evaluate. The assignments ask students to repair intentionally invalid claims, then specify what evidence would establish that the repair helped.

Learning objectives

By the end of this chapter, you should be able to:

- map a diagnosed failure to a family of mitigations;
- state which bias, variance, coverage, or compute cost each mitigation introduces;
- derive Double-Q, doubly robust, Retrace, V-trace, and trust-region mechanisms;
- design monitored PPO and replay protocols rather than relying on one loss equation;
- choose an off-policy evaluation method based on support, model quality, and horizon;
- construct budget-matched, paired comparisons with seed- and task-level uncertainty;
- design held-out evaluations that detect reward or verifier exploitation;
- pre-register a diagnostic hypothesis and a falsifying ablation.

Prerequisite

Part IX. This chapter assumes the failure modes have already been diagnosed.

Overview: every mitigation moves error

A mitigation rarely removes a problem without creating another.

Intervention	Primary benefit	New cost or risk
bootstrapping	lower return variance	critic bias
longer returns	less bootstrap bias	higher sampling variance
target network	slower target drift	stale target
double critic	lower optimistic selection bias	pessimism and extra compute
importance sampling	off-policy correction	high variance
ratio clipping	variance control	bias
behavior constraint	support preservation	limits improvement
entropy bonus	maintains exploration	slower concentration
KL penalty	limits policy drift	can block useful change
more PPO epochs	better batch fitting	stale-batch overfit
advantage normalization	stable scale	batch-relative distortion
reward ensemble	reduces idiosyncratic error	shared blind spots and compute

Intervention	Primary benefit	New cost or risk
robust aggregate metric	less outlier sensitivity	may hide tail behavior unless also reported

The goal is not a perfectly unbiased RL system. The goal is a system whose remaining errors are measured, bounded enough for the decision, and exposed by diagnostics.

42. A Mitigation Taxonomy

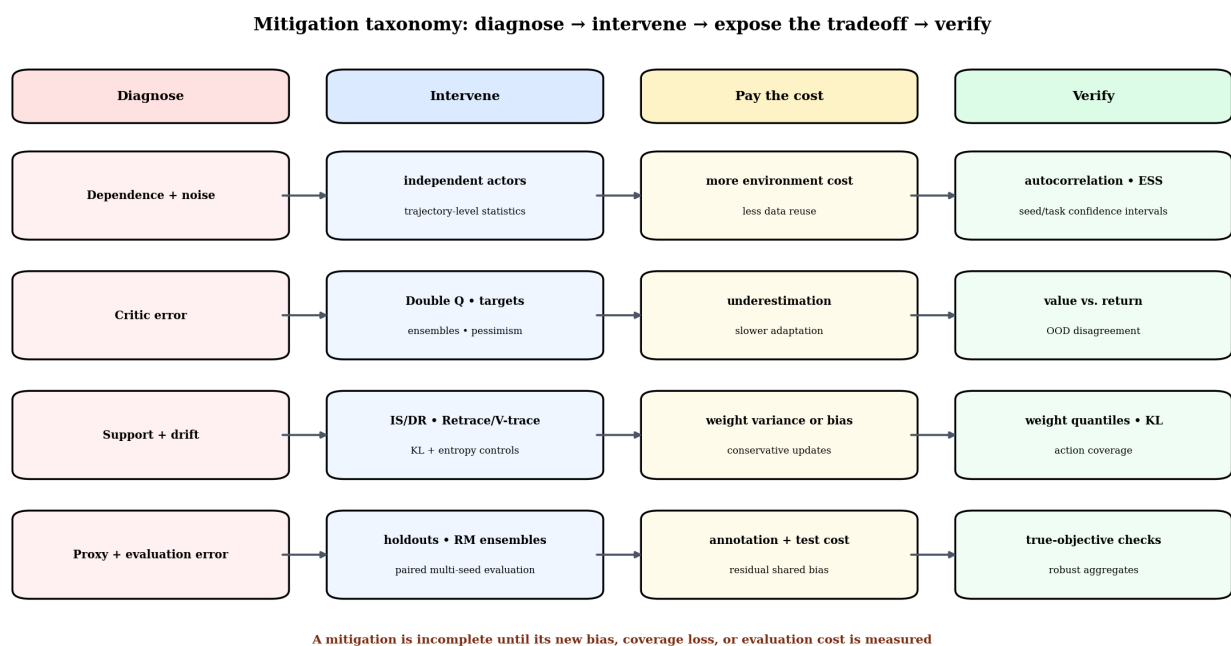


Figure 59: A taxonomy connecting diagnosis, intervention, tradeoffs, and verification

Four intervention layers

A useful taxonomy separates four layers.

1. Data collection

Improve what evidence enters training:

- more independent actors;
- broader reset distribution;
- directed exploration;
- fresh on-policy samples;
- balanced tasks and prompts;
- active acquisition near uncertainty.

2. Estimation

Improve how values, advantages, or policy performance are estimated:

- baselines and control variates;
- multi-step returns;

- Double Q;
- critic ensembles;
- doubly robust OPE;
- density-ratio methods.

3. Optimization and policy drift

Control how aggressively estimates are converted into policy changes:

- trust regions;
- KL penalties;
- PPO clipping and early stopping;
- entropy regularization;
- gradient clipping;
- behavior constraints and pessimism.

4. Evaluation

Determine whether the complete loop actually improved:

- independent seeds;
- held-out tasks;
- robust aggregate metrics;
- paired comparisons;
- hidden objective checks;
- reward-hacking audits.

The four-question mitigation protocol

For every intervention, write:

1. **Diagnosis:** Which failure is being addressed?
2. **Mechanism:** Why should this intervention affect that failure?
3. **Cost:** What error or resource cost does it introduce?
4. **Falsification:** What observation would show that the mechanism is not working?

Example:

Item	PPO target-KL early stopping
Diagnosis	repeated epochs move policy too far from rollout collector
Mechanism	stop batch reuse when sampled KL exceeds a threshold
Cost	less optimization per rollout and possible underfitting
Falsification	collapse persists with low per-epoch KL and unchanged ratio tails

Problem-to-mitigation map

Diagnosed problem	Candidate mitigations	Diagnostic that must improve
temporal correlation	independent actors, diverse resets, trajectory-level aggregation	autocorrelation, state diversity, seed variance
rare success	curriculum, HER, structured exploration, more rollouts	success count and coverage

Diagnosed problem	Candidate mitigations	Diagnostic that must improve
critic overestimation	Double Q, twin critics, ensembles, pessimism	value-versus-return bias
critic underestimation	longer returns, less pessimism, better coverage	rare-success value error
OPE weight collapse	closer behavior policy, per-decision IS, DR, FQE	ESS and interval width
offline support mismatch	behavior constraints, CQL, IQL, uncertainty penalty	target-action support
policy-gradient variance	baselines, GAE tuning, control variates, larger independent batches	gradient variance
PPO drift	fewer epochs, lower LR, KL stop, larger rollout diversity	KL, ratio tails, clip fraction
replay staleness	age limits, policy metadata, off-policy corrections	performance by replay age
seed sensitivity	more runs, robust metrics, paired design	interval width and run reliability
reward hacking	held-out objective, ensembles, pessimism, KL limits	proxy-versus-true quality gap

Theory, approximation, and heuristic

Label interventions honestly.

Clean theoretical identities

- state-only policy-gradient baselines preserve expectation;
- importance sampling is unbiased under support and integrability;
- Bellman operators are contractions in finite discounted tabular settings;
- potential-based shaping preserves optimal policies under its assumptions.

Theory-motivated approximations

- TRPO's sampled average-KL implementation;
- Double DQN with neural target networks;
- doubly robust estimators with learned models;
- pessimistic offline RL with approximate function classes.

Primarily empirical stabilizers

- advantage standardization;
- value clipping;
- reward normalization;
- PPO epoch counts;
- target-network schedules;
- entropy schedules;
- replay-buffer size.

Empirical does not mean unimportant. It means the behavior should be validated rather than presented as a universal guarantee.

43. Finite-Sample Noise and Correlated Trajectories

Increase independent information, not only raw transitions

The most direct mitigation for correlation is more independent rollout sources:

- independently seeded environments;
- diverse initial states;
- distinct task instances;
- separate opponent snapshots;
- independent training runs;
- independent evaluation episodes.

Parallelism improves evidence only when the parallel streams differ meaningfully.

Actor diversity

Parallelism reduces short-range correlation; shuffling alone only changes order



Figure 60: Independent rollout actors increasing effective information

If all actors share the same narrow reset distribution, more actors reproduce the same region faster.

Diversify through:

- randomized starts;
- domain randomization;
- task or prompt stratification;
- policy-temperature mixtures;
- opponent populations;
- curriculum stages with explicit evaluation weights.

Rollout allocation

Under a fixed transition budget B , choose number of actors M and horizon H such that

$$B = MH.$$

Longer H captures delayed consequences but increases within-trajectory dependence. Larger M increases independent starts but may truncate credit or increase bootstrapping.

Tune M and H jointly with GAE λ and critic quality.

Block and trajectory-level statistics

When dependence is local, aggregate into episodes or blocks before estimating uncertainty.

For episode returns $G_1, \dots, G_n$, use those episodes as units rather than individual rewards.

For continuing tasks, divide trajectories into blocks longer than the dominant autocorrelation scale, then test sensitivity to block length.

Sparse-reward amplification

If informative events are too rare, variance reduction alone is insufficient. Change data collection.

Hindsight relabeling

For goal-conditioned tasks, relabel achieved goals to turn failed attempts into informative successes.

Curiosity or information gain

Increase visitation of novel or uncertain states. Audit for noisy-TV exploitation.

Bootstrapped exploration

Use coherent randomized value hypotheses rather than independent action noise.

Curriculum

Shift training mass toward tasks with nondegenerate success probability. Maintain a fixed evaluation distribution so curriculum improvement is not confused with easier data.

Rare-event estimators

For a success probability p , observing zero successes in n episodes does not prove $p = 0$. The one-sided upper confidence bound can still be substantial.

When rare failures matter, report:

- exact event counts;
- exposure time;
- binomial or survival intervals;
- worst-case task slices;
- importance of each event, not only frequency.

Robust aggregation

Across tasks and seeds, report more than a mean.

Median

Less sensitive to extreme tasks, but ignores magnitude beyond rank.

Interquartile mean

Sort all scores, remove the lowest and highest quartiles by mass, and average the middle half. IQM uses more data than a median while reducing extreme influence.

Performance profile

For normalized score X , plot

$$P(X \geq \tau)$$

as threshold τ varies. This reveals whether one method wins broadly or only on a few tasks.

Optimality gap

If score is normalized so 1 is a target level,

$$\text{Gap} = \mathbb{E}[\max(0, 1 - X)].$$

The exact normalization and reference score must be stated.

Stratified bootstrap

For multi-task, multi-seed benchmarks:

1. resample tasks;
2. within each selected task, resample seed outcomes;
3. recompute the aggregate metric;
4. use the bootstrap distribution for intervals.

The hierarchy should match the generalization claim.

Diagnostic target

A finite-sample mitigation should improve at least one of:

- gradient stability across independent rollout subsets;
- effective sample size;
- success-event count;
- interval width;
- reliability across seeds;
- state-coverage diversity.

More wall-clock throughput alone is not statistical evidence.

44. Critic Error and Bootstrapping Bias

Double estimators

The max of noisy estimates is optimistically biased:

$$\mathbb{E}[\max_a \widehat{Q}(s, a)] \geq \max_a Q(s, a).$$

Double estimators separate selection from evaluation.

Double DQN

The online network selects

$$a^* = \arg \max_{a'} Q_{\theta}(S_{t+1}, a'),$$

and the target network evaluates:

$$Y_t^{\text{DDQN}} = R_{t+1} + \gamma(1 - d_{t+1})Q_{\bar{\theta}}(S_{t+1}, a^*).$$

Selection and evaluation errors are less correlated than in a single-network max, though not independent because networks share data and parameters over time.

Double estimators reduce optimization of critic noise—but can become pessimistic

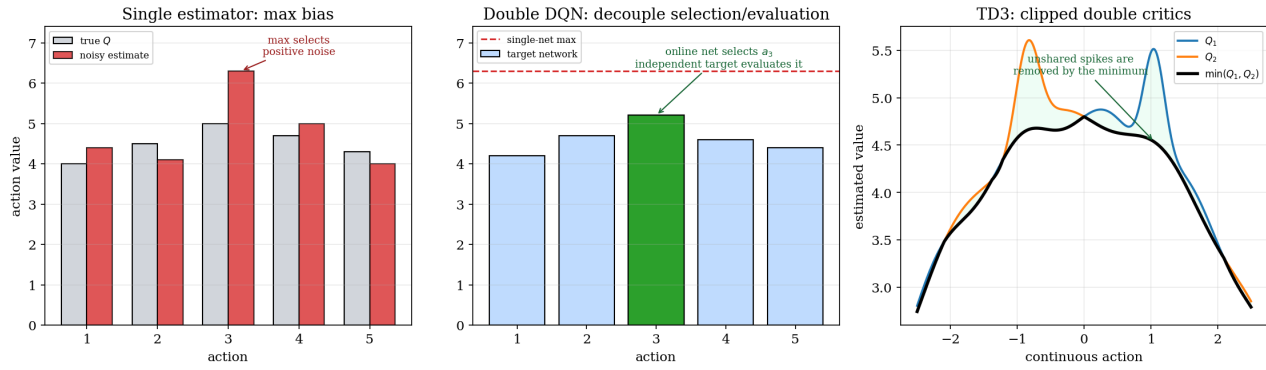


Figure 61: Double estimators and pessimism countering maximization bias

Twin critics in TD3

For target action $\tilde{A}_{t+1}$,

$$Y_t = R_{t+1} + \gamma(1 - d_{t+1}) \min_{i=1,2} Q_{\bar{\phi}_i}(S_{t+1}, \tilde{A}_{t+1}).$$

The minimum protects against critic-specific positive spikes.

It can introduce negative bias when one critic is systematically low. Monitor both individual critics and realized returns.

Target-policy smoothing

TD3 perturbs the target action:

$$\tilde{A}_{t+1} = \mu_{\bar{\theta}}(S_{t+1}) + \text{clip}(\varepsilon, -c, c),$$

$$\varepsilon \sim \mathcal{N}(0, \sigma^2 I).$$

This estimates value in a neighborhood rather than at one narrow action peak.

Delayed actor updates

Updating the actor less frequently gives the critic more steps to fit new data before the actor chases its surface.

Ablate the actor-to-critic update ratio while holding environment interactions and total critic updates fixed.

Target networks

Hard update:

$$\bar{\theta} \leftarrow \theta \quad \text{every } C \text{ steps.}$$

Soft update:

$$\bar{\theta} \leftarrow \tau\theta + (1 - \tau)\bar{\theta}.$$

Tune against two diagnostics:

- target drift between updates;
- target staleness relative to current realized returns.

A slowly changing wrong target is still wrong.

Averaging and ensembles

An ensemble target can be

$$Y_t = R_{t+1} + \gamma \text{Agg} \{Q_{\bar{\phi}_1}(S', A'), \dots, Q_{\bar{\phi}_K}(S', A')\}.$$

Possible aggregators:

- mean;
- minimum;
- lower quantile;
- truncated mean;
- minimum of a random subset.

The aggregator encodes optimism or pessimism.

Ensemble disagreement

Define

$$u_Q(s, a) = \text{Std}_k [Q_{\phi_k}(s, a)].$$

Use disagreement to:

- flag unsupported actor actions;
- penalize uncertain targets;
- allocate real data collection;
- stop model rollouts.

Validate that disagreement predicts actual error. Identical data pipelines and architectures can produce falsely low disagreement.

Distributional critics

A distributional critic estimates a return distribution $Z(s, a)$ rather than only the mean.

Risk-aware summaries include:

- quantiles;
- lower-tail value;
- conditional value at risk;
- truncated upper quantiles.

Distributional modeling represents return randomness. It does not automatically distinguish epistemic uncertainty from aleatoric return variation.

Pessimism in offline RL

The offline actor should not exploit unsupported critic values.

Behavior constraints

Restrict π toward behavior support:

$$D(\pi(\cdot | s), \beta(\cdot | s)) \leq \varepsilon.$$

Conservative Q penalty

Lower values away from dataset actions, as in CQL.

Advantage-weighted behavior cloning

Use only observed actions and weight them by estimated advantage, as in IQL.

Lower-confidence value

Optimize

$$Q_{\text{pess}}(s, a) = \overline{Q}(s, a) - \alpha \nu_Q(s, a).$$

The confidence penalty is only as meaningful as the uncertainty estimate.

Safe policy improvement

A conservative update can require evidence before deviating from a baseline. In low-count state-action regions, retain baseline behavior; in high-confidence regions, permit improvement.

This protects against unsupported changes but can be overly conservative in large spaces where count or confidence estimates are weak.

Critic mitigation checklist

- compare predicted value with realized return;
 - stratify by action support and return quantile;
 - inspect both critics, not only their minimum;
 - test longer-return targets;
 - audit terminal masks;
 - vary target-update speed;
 - measure actor action distance from replay support;
 - ablate pessimism strength.
-

45. Off-Policy Evaluation and Support Mismatch

Begin with identifiability

Before choosing an estimator, ask whether the target policy is covered by the data.

If

$$b(a | s) = 0 \quad \text{and} \quad \pi(a | s) > 0,$$

no importance-weight estimator can recover the missing outcome.

Mitigations are:

- collect new data;
- restrict the target policy;
- introduce a model with explicit extrapolation assumptions;
- return a partial-identification or sensitivity interval rather than a point estimate.

Weighted importance sampling

$$\widehat{V}_{\text{WIS}} = \frac{\sum_i \rho_i G_i}{\sum_i \rho_i}.$$

Use when ordinary IS variance is prohibitive and finite-sample bias is acceptable.

Always report ESS and the largest normalized weight

$$\max_i \frac{\rho_i}{\sum_j \rho_j}.$$

Per-decision importance sampling

For each reward, use only the ratios needed to reach it:

$$\widehat{V}_{\text{PDIS}} = \frac{1}{N} \sum_i \sum_{t=0}^{T_i-1} \gamma^t \rho_{0:t}^{(i)} R_{t+1}^{(i)}.$$

This reduces unnecessary horizon products but does not solve missing support.

Truncated ratios

Use

$$\bar{\rho}_t = \min(c, \rho_t).$$

Choose c through a predeclared bias-variance analysis, not by selecting the threshold with the best test result.

Plot the estimate and interval as functions of c .

Doubly robust OPE

A doubly robust estimator combines a learned value model with IS corrections.

Let

$$\widehat{V}(s) = \sum_a \pi(a | s) \widehat{Q}(s, a).$$

A finite-horizon sequential form is

$$\widehat{V}_{\text{DR}} = \widehat{V}(S_0) + \sum_{t=0}^{T-1} \gamma^t \rho_{0:t} \left[R_{t+1} + \gamma \widehat{V}(S_{t+1}) - \widehat{Q}(S_t, A_t) \right].$$

The model supplies a low-variance baseline. Importance-weighted TD residuals correct model error.

The “doubly robust” name refers to consistency under alternative nuisance-model correctness conditions in the appropriate statistical setup. Finite-sample reliability still depends on support, horizon, model quality, and weight tails.

For a genuine finite horizon, use time-indexed $\widehat{Q}_t, \widehat{V}_t$ and set $\widehat{V}_T = 0$ in the formula above. A.15 proves the telescoping cancellation with exact ratios, and the separate correct-value-model route with appropriately measurable weights. Independently trained or cross-fitted nuisance models make the conditioning explicit. “Doubly robust” does not mean accurate when both models are wrong, and self-normalization does not retain ordinary finite-sample unbiasedness.

Weighted doubly robust and MAGIC

Weighted variants normalize ratios to reduce variance. MAGIC combines estimators with different truncation horizons to trade bias and variance adaptively.

These methods require careful covariance estimation and can become unstable with few trajectories.

Fitted Q Evaluation

FQE learns Q^π from offline data while holding the target policy fixed.

At iteration k , solve approximately

$$\widehat{Q}_{k+1} = \arg \min_Q \sum_{(s,a,r,s',d) \in \mathcal{D}} \left[Q(s,a) - (r + \gamma(1-d)\mathbb{E}_{a' \sim \pi(\cdot|s')} \widehat{Q}_k(s',a')) \right]^2.$$

Then estimate

$$\widehat{V}^\pi = \mathbb{E}_{S_0 \sim \rho_0, A_0 \sim \pi} [\widehat{Q}(S_0, A_0)].$$

FQE avoids trajectory-ratio products. It replaces them with function-approximation and coverage assumptions.

Occupancy-ratio methods

Instead of trajectory products, estimate

$$w(s,a) = \frac{d^\pi(s,a)}{d^b(s,a)}.$$

Then

$$V^\pi = \frac{1}{1-\gamma} \mathbb{E}_{(s,a) \sim d^b} [w(s,a)r(s,a)]$$

under the normalized discounted-occupancy convention.

DICE-style methods estimate w through moment or saddle-point conditions induced by stationary flow constraints.

They avoid explicit horizon products but introduce density-ratio function approximation and optimization error.

Model-based OPE

Fit a dynamics and reward model, simulate the target policy, and estimate return.

This can work with strong predictive structure and poor action-probability logging. It is vulnerable to model exploitation and compounding error.

Use:

- ensemble uncertainty;
- horizon sensitivity;
- real-data calibration;
- alternative model classes;
- pessimistic intervals.

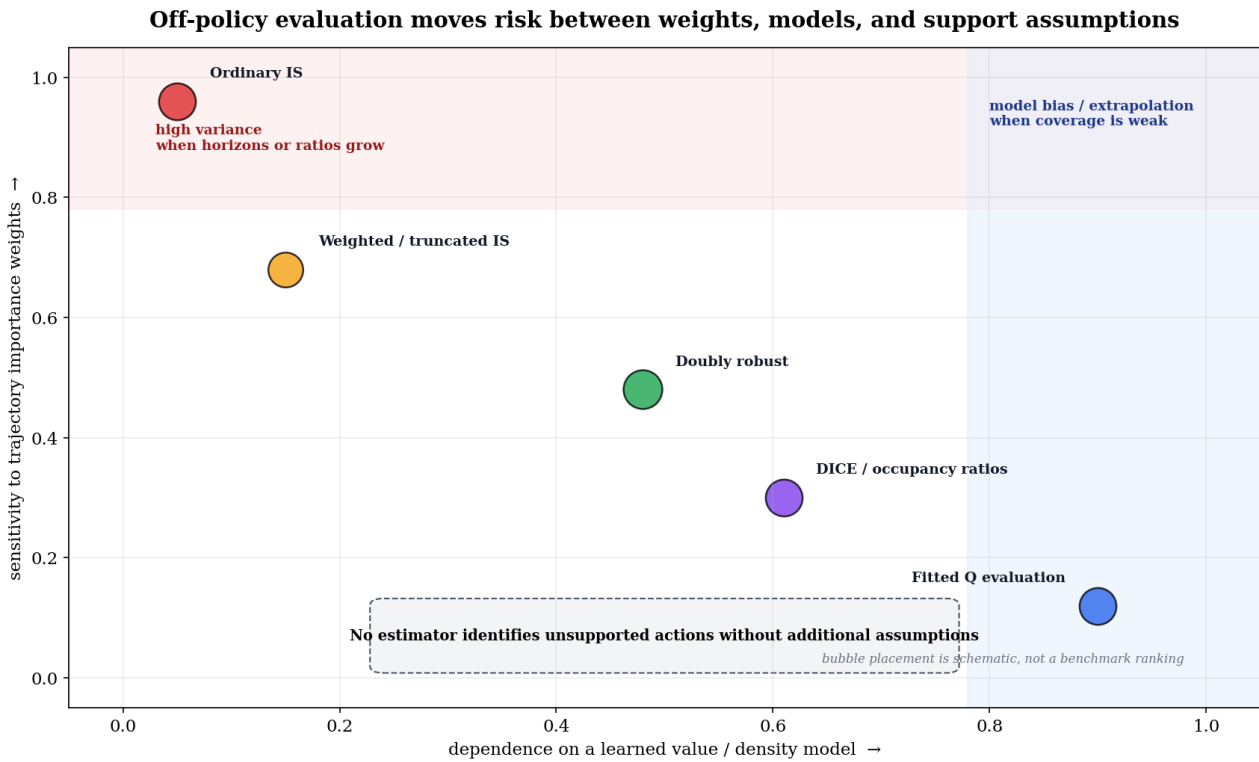


Figure 62: Tradeoffs among off-policy evaluation estimators

OPE estimator selection guide

Data and target relationship	Preferred starting point	Main warning
target very close to known behavior, short horizon	IS or WIS	weight tails
moderate mismatch, reasonable value model	DR or weighted DR	both model and weights can fail
long horizon, broad data coverage	FQE plus sensitivity checks	function-approximation bias
stationary logged process with ratio modeling	occupancy-ratio method	difficult density-ratio optimization
known or highly predictable dynamics	model-based OPE	compounding model error
support missing	constrain policy or collect data	point identification impossible

OPE reporting protocol

Report:

- dataset size in trajectories and transitions;
- behavior-policy source and probability logging;
- target policy;
- horizon and discount;
- ratio quantiles and ESS;
- clipping or normalization;
- model architecture and validation;
- estimator disagreement;
- interval construction;
- sensitivity to target-policy distance.

46. Policy-Gradient Variance Reduction

State-only baselines

For score

$$z_t = \nabla_{\theta} \log \pi_{\theta}(A_t | S_t),$$

and any fixed or pre-action-measurable baseline $b(S_t)$,

$$\mathbb{E}[b(S_t)z_t] = 0.$$

This is the first mitigation to use because it reduces variance without changing expectation.

Learn the baseline on separate targets

Fit V_{ϕ} to returns or λ -returns:

$$L_V(\phi) = \mathbb{E} \left[\left(V_{\phi}(S_t) - \widehat{V}_t^{\text{target}} \right)^2 \right].$$

Prevent leakage by:

- computing targets with stop-gradient;
- using held-out or cross-fitted predictions when estimating scientific quantities;
- monitoring value calibration, not only training loss.

Action-dependent control variates

Subtracting an arbitrary $b(s, a)$ biases the policy gradient because

$$\mathbb{E}_{a \sim \pi} [b(s, a) \nabla \log \pi(a | s)]$$

need not vanish.

Structured action-dependent baselines can remain unbiased when an analytic correction term is added or when the policy factorizes and the baseline excludes the sampled component appropriately.

The design must show the cancellation explicitly.

Q-Prop principle

Q-Prop combines:

- an on-policy Monte Carlo residual, preserving correction for critic error;
- a first-order expansion of an off-policy critic as a control variate.

The central idea is not “replace return with Q.” It is “subtract a correlated critic approximation and add back its tractable expectation.”

Implement the exact estimator from the paper; an incomplete pseudo-equation can omit the correction that preserves expectation.

Reparameterization gradients

For a differentiable stochastic policy

$$A = f_{\theta}(S, \varepsilon), \quad \varepsilon \sim p(\varepsilon),$$

one can differentiate through f_θ and a differentiable critic:

$$\nabla_\theta \mathbb{E}_\varepsilon [Q(S, f_\theta(S, \varepsilon))].$$

This often has lower variance than a score-function gradient, but it depends directly on critic derivatives and model smoothness.

Common random numbers

For comparing two policies or hyperparameters in simulation, use matched environment randomness where valid. Paired returns

$$G_i^{(A)} - G_i^{(B)}$$

can have lower variance than independent comparisons.

This is an evaluation control variate, not a training theorem.

Advantage normalization safeguards

If advantages are standardized:

1. log raw advantages;
2. normalize over the intended unit—global batch, task, prompt group, or minibatch;
3. avoid silently changing the unit across implementations;
4. test sensitivity with and without centering;
5. report interactions with clipping and reward scale.

Gradient clipping

Clip global norm:

$$g \leftarrow g \min \left(1, \frac{c}{\|g\|} \right).$$

This prevents a single batch from causing an arbitrarily large parameter step. It biases the stochastic gradient and can hide recurring outliers. Log the pre-clip norm and clipped fraction.

Larger batches

Larger batches reduce sampling noise only when they add sufficiently independent trajectories. Increasing batch size while holding the number of actors fixed may mostly add correlated timesteps.

Match optimizer learning rate and update count when comparing batch sizes.

47. GAE, Retrace, and V-Trace

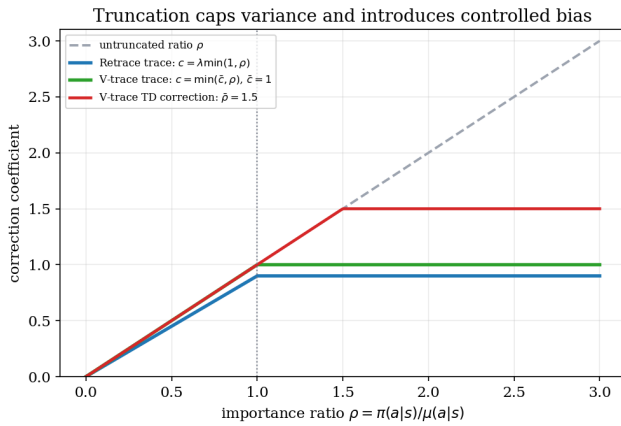
GAE tuning as critic-trust tuning

$$\hat{A}_t^{\gamma, \lambda} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}.$$

Tune λ jointly with:

- rollout horizon;
- discount;

Retrace and V-trace: reuse off-policy sequences without multiplying unbounded ratios



Off-policy traces correct actor-learner mismatch

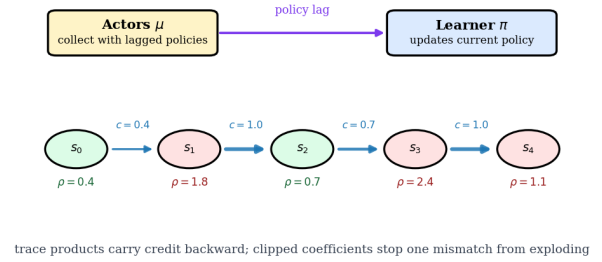


Figure 63: Retrace and V-trace corrections for off-policy multi-step targets

- reward sparsity;
- critic architecture;
- value-loss coefficient;
- terminal handling.

Use Monte Carlo comparison on short complete episodes to estimate critic-induced bias.

Retrace

Retrace constructs a multi-step off-policy return with truncated coefficients.

Let

$$\rho_t = \frac{\pi(A_t | S_t)}{b(A_t | S_t)}$$

and

$$c_t = \lambda \min(1, \rho_t).$$

Define TD residual

$$\delta_t^Q = R_{t+1} + \gamma \mathbb{E}_{a' \sim \pi(\cdot | S_{t+1})} Q(S_{t+1}, a') - Q(S_t, A_t).$$

A Retrace target has the form

$$Q_t^{\text{ret}} = Q(S_t, A_t) + \sum_{k=t}^{T-1} \gamma^{k-t} \left(\prod_{i=t+1}^k c_i \right) \delta_k^Q.$$

The empty product for $k = t$ is 1.

Truncation controls variance while preserving a contraction-style fixed-point property under the method's assumptions.

Interpreting Retrace coefficients

- if behavior and target agree, $\rho \approx 1$ and traces extend;

- if mismatch is large, c_t is capped;
- λ controls trace length;
- unsupported actions remain impossible to correct.

V-trace

V-trace addresses actor-learner policy lag in distributed systems.

Let behavior be μ and target be π . Define

$$\rho_t = \min \left(\bar{\rho}, \frac{\pi(A_t | X_t)}{\mu(A_t | X_t)} \right),$$

$$c_t = \min \left(\bar{c}, \frac{\pi(A_t | X_t)}{\mu(A_t | X_t)} \right).$$

The corrected residual is

$$\delta_t^V = \rho_t [R_{t+1} + \gamma V(X_{t+1}) - V(X_t)].$$

The target is

$$v_s = V(X_s) + \sum_{t=s}^{s+n-1} \gamma^{t-s} \left(\prod_{i=s}^{t-1} c_i \right) \delta_t^V.$$

The ρ coefficient controls the immediate correction; c controls trace propagation.

What does V-trace evaluate? With $0 < \bar{c} \leq \bar{\rho}$, sufficient overlap, and fixed tabular policies, the expected operator has fixed point $v^{\pi_{\bar{\rho}}}$, where

$$\pi_{\bar{\rho}}(a | s) = \frac{\min\{\bar{\rho}\mu(a | s), \pi(a | s)\}}{\sum_b \min\{\bar{\rho}\mu(b | s), \pi(b | s)\}}.$$

It need not equal v^π . The ρ cap changes the target policy; the c cap controls propagation. A.16 proves the contractions and fixed points of both expected Retrace and V-trace operators, rather than importing an unqualified “off-policy correction” claim into neural training.

Policy-lag diagnostics

In distributed training, log:

- actor policy version;
- learner policy version;
- distribution of policy age;
- ratio quantiles by age;
- V-trace coefficient saturation;
- performance by actor lag.

Adding V-trace does not make arbitrarily stale data harmless.

Choosing among GAE, Retrace, and V-trace

Setting	Natural choice	Main condition
fresh on-policy rollouts	GAE	critic and truncation quality

Setting	Natural choice	Main condition
replayed off-policy trajectories	Retrace	behavior probabilities and support
distributed actor-learner lag	V-trace	logged behavior policy and bounded lag

48. Trust Regions, Entropy, and PPO Stabilization

Trust-region principle

An update should not move the policy so far that the data used to justify it become irrelevant.

TRPO solves approximately

$$\max_{\theta} \mathbb{E}_t \left[\frac{\pi_{\theta}(A_t | S_t)}{\pi_{\text{old}}(A_t | S_t)} \hat{A}_t \right]$$

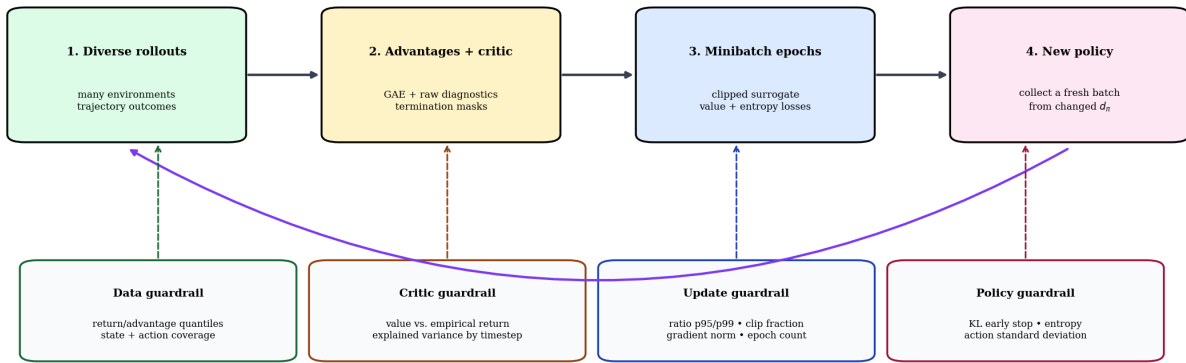
subject to

$$\mathbb{E}_t D_{\text{KL}}(\pi_{\text{old}}(\cdot | S_t) \| \pi_{\theta}(\cdot | S_t)) \leq \delta.$$

The practical constraint is sampled and average, not worst-case over all states.

PPO stabilization package

PPO with statistical safeguards: control the batch, critic, update, and resulting policy



PPO clipping is one component; the monitored stopping rules make the training protocol

Figure 64: A monitored package of safeguards for stable PPO training

PPO-Clip is

$$L^{\text{CLIP}} = \mathbb{E}_t \min \left(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right).$$

A robust implementation adds:

- fresh rollout collection;
- raw-advantage logging;

- target-KL monitoring;
- ratio-tail monitoring;
- clip-fraction monitoring;
- value calibration;
- entropy monitoring;
- gradient clipping;
- early stopping of epochs;
- independent evaluation.

Target-KL early stopping

After each epoch, estimate

$$\widehat{D}_{\text{KL}}(\pi_{\text{old}} \parallel \pi_{\theta}).$$

Stop reusing the batch if

$$\widehat{D}_{\text{KL}} > d_{\text{target}}.$$

This directly addresses accumulated drift. It does not control unseen states or correct bad advantages.

Adaptive KL penalty

Optimize

$$L^{\text{sur}} - \beta D_{\text{KL}}.$$

Adjust β based on measured KL. Use positive parameterization such as $\log \beta$.

The controller needs its own stability settings. Aggressive adaptation can oscillate.

Entropy regularization

Add

$$\alpha \mathcal{H}(\pi(\cdot \mid s)).$$

Possible schedules:

- fixed coefficient;
- linear decay;
- performance-triggered decay;
- target-entropy adaptation.

Monitor actual entropy. The same coefficient can have different effects across action dimensions and reward scales.

Minimum action variance

For Gaussian policies, constrain

$$\sigma_{\theta}(s) \geq \sigma_{\min}.$$

This preserves exploration but can prevent precise control. Use as a temporary safeguard or stress-test parameter rather than an unexamined default.

Value clipping

PPO implementations sometimes clip value updates around the old prediction:

$$V_{\text{clip}} = V_{\text{old}} + \text{clip}(V_{\theta} - V_{\text{old}}, -\epsilon_V, \epsilon_V).$$

The value loss uses the worse of clipped and unclipped errors. This limits rapid value changes but can slow correction of a badly wrong critic.

Always log raw value error.

Separate actor and critic optimization

Mitigations for interference include:

- separate networks;
- separate optimizers;
- different learning rates;
- alternating policy and value phases;
- auxiliary distillation to preserve actor features.

Phasic Policy Gradient separates policy and auxiliary value phases to reduce destructive interference while retaining representation sharing.

Maximum a Posteriori Policy Optimization

MPO separates policy improvement and projection.

E-step

Construct a nonparametric improved action distribution under a KL constraint. A solution has Boltzmann form

$$q(a \mid s) \propto \pi_{\text{old}}(a \mid s) \exp \left(\frac{Q(s, a)}{\eta} \right).$$

M-step

Fit the parametric policy to q while constraining policy change.

This makes distribution-shift control explicit and separates value-based improvement from policy fitting.

PPO safeguard pseudocode

```

for each update:
    collect fresh rollouts from pi_old
    compute raw returns and GAE advantages
    audit terminal and truncation masks
    log raw advantage and critic-error distributions

    optionally normalize advantages once over the full rollout batch

    for epoch in 1, ..., K:
        for minibatch:
            compute log-ratio and ratio
            compute clipped policy objective
            compute value loss and entropy bonus
            backpropagate
            log pre-clip gradient norm

```

```

        clip gradient norm
        optimizer step

    compute sampled KL, ratio p95/p99, and clip fraction
    if KL exceeds predeclared target:
        break

    evaluate the current policy on independent episodes

```

Falsifying the “policy drift” diagnosis

If reducing epochs and enforcing low KL do not change collapse, investigate:

- critic target bias;
- reward bugs;
- environment nonstationarity;
- advantage outliers;
- action scaling;
- insufficient exploration.

A safeguard is useful only when it changes the mechanism it targets.

49. Replay, Nonstationarity, and Distribution Shift

Replay-buffer controls

Track for each sample:

- collection time;
- policy version;
- environment version;
- reward-model version;
- priority;
- episode identity;
- recurrent sequence boundaries.

Buffer-size tradeoff

Large buffer:

- more diversity;
- better rare-event retention;
- older policy mixture;
- more stale reward or dynamics.

Small buffer:

- fresher data;
- less coverage;
- greater temporal correlation;
- catastrophic forgetting of rare events.

Choose buffer size based on environment stationarity and actor update speed, not a universal transition count.

Age-based sampling

Possible strategies:

- hard maximum age;
- exponential recency weighting;
- stratified fresh-and-old mixture;
- separate rare-event reservoir;
- policy-distance-based filtering.

Recency weighting reduces staleness but can discard useful broad support.

Prioritized replay safeguards

- log sampling probability and importance weight;
- anneal correction exponent β explicitly;
- cap priorities;
- retain some uniform sampling;
- update priorities after learning;
- monitor whether a few transitions dominate updates.

High TD error may indicate learning value, noise, reward bugs, or irreducible stochasticity.

Recurrent policies

Do not sample isolated timesteps when the hidden state depends on prior context. Sample contiguous sequences and either:

- store the initial hidden state;
- burn in the recurrent network before computing loss;
- reconstruct hidden state from preceding observations.

Shuffling sequences is compatible with preserving within-sequence order.

Off-policy actor-critic corrections

Methods such as ACER combine replay with truncated importance sampling and bias correction. Distributed methods such as IMPALA use V-trace for policy lag.

Deterministic actor-critic methods rely more strongly on critic generalization because exact action-probability ratios may not be available.

Target-network schedules

A target update is a low-pass filter. Tune using:

- autocorrelation of target values;
- online-target parameter distance;
- value calibration;
- learning stability;
- adaptation after environment change.

Multi-agent opponent pools

Maintain:

- current opponents;
- historical checkpoints;
- exploiters or best responses;
- fixed evaluation bots.

Sample from the pool with an explicit distribution. Report cross-play matrices and population composition.

Reward-model and tool-version shift

For agent or RLHF logs, changing judges or tools alters the environment.

Store raw outputs so they can be rescored. Avoid mixing scalar rewards from incompatible versions without calibration.

50. Reliable Evaluation and Statistical Reporting

Reliable evaluation pairs methods and preserves task/seed uncertainty

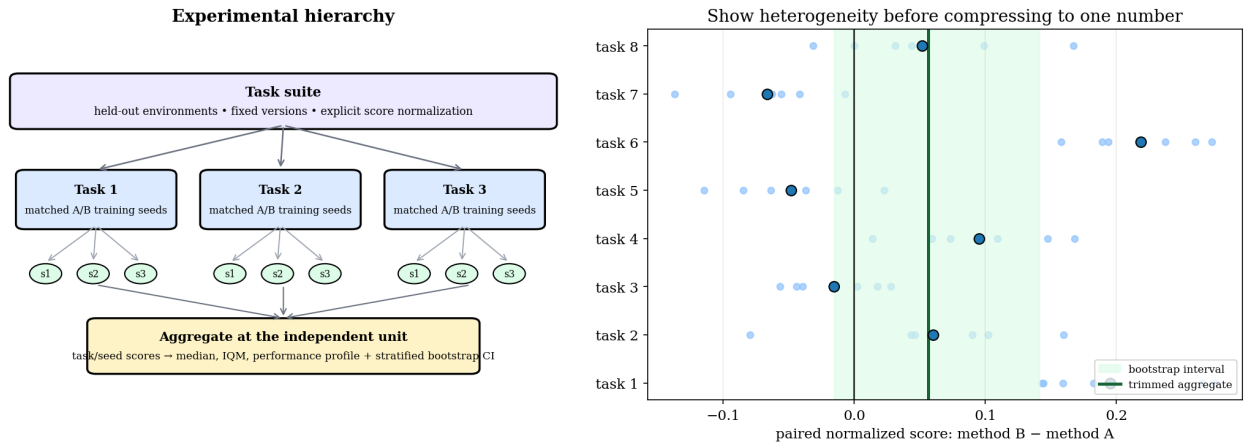


Figure 65: A reliable reinforcement learning evaluation and reporting protocol

Separate two questions

1. How good is this fixed trained policy?
2. How reliable is this training method across reruns?

The first needs evaluation episodes. The second needs independent training runs.

Seed-level summaries

For each seed, compute a predeclared policy summary, such as mean return over m episodes. Then aggregate across seeds.

Show individual seed values, especially when K is small.

Robust metrics

Report a set rather than one scalar:

- mean;
- median;
- IQM;
- lower-tail or worst-task score;
- performance profile;
- optimality gap.

The metric should match the claim. Safety or reliability claims need tail metrics, not only average score.

Paired budget-matched comparison

A strong comparison uses:

- same task instances;
- same interaction budget;
- same evaluation episodes or controlled common randomness where valid;
- same checkpoint rule;
- comparable hyperparameter-search budget;
- same model size or separately reported scaling;
- same wall-clock or accelerator budget when claiming efficiency.

Analyze paired differences by task or scenario.

Power and precision

Before running, specify a smallest effect of practical interest Δ . Use pilot variance to estimate how many independent runs are needed for desired precision or power.

When compute is limited, report that the interval is too wide rather than converting uncertainty into a confident claim.

Stratified bootstrap protocol

For task-general performance:

1. sample tasks with replacement;
2. sample seed outcomes within each task;
3. compute the aggregate;
4. repeat many times;
5. report percentile or bias-corrected intervals as appropriate.

The exact bootstrap design should be published with code.

Learning curves

Plot:

- environment steps on one axis;
- wall-clock time on another when relevant;
- median or mean line;
- seed uncertainty band;
- unsmoothed summary points;
- evaluation frequency.

Avoid selecting a smoothing window that visually favors one method.

Checkpoint protocol

Choose before test evaluation:

- final checkpoint;
- validation-selected checkpoint;
- average of final checkpoints;
- best validation checkpoint with one final blind test.

Report validation and test separately.

Multiple comparisons

Testing many algorithms, tasks, and metrics increases false discoveries.

Mitigations include:

- predeclared primary metric;

- family-wise or false-discovery adjustment when appropriate;
- replication on held-out tasks;
- reporting all tested variants;
- emphasizing effect sizes and intervals over thresholded significance.

Evaluation report template

Estimand:

Training budget:

Independent training runs:

Evaluation episodes per run:

Tasks and environment versions:

Checkpoint rule:

Hyperparameter search budget:

Primary metric:

Secondary and tail metrics:

Uncertainty method:

Paired design:

Raw per-seed results location:

51. Reward Hacking and Goodhart Mitigations

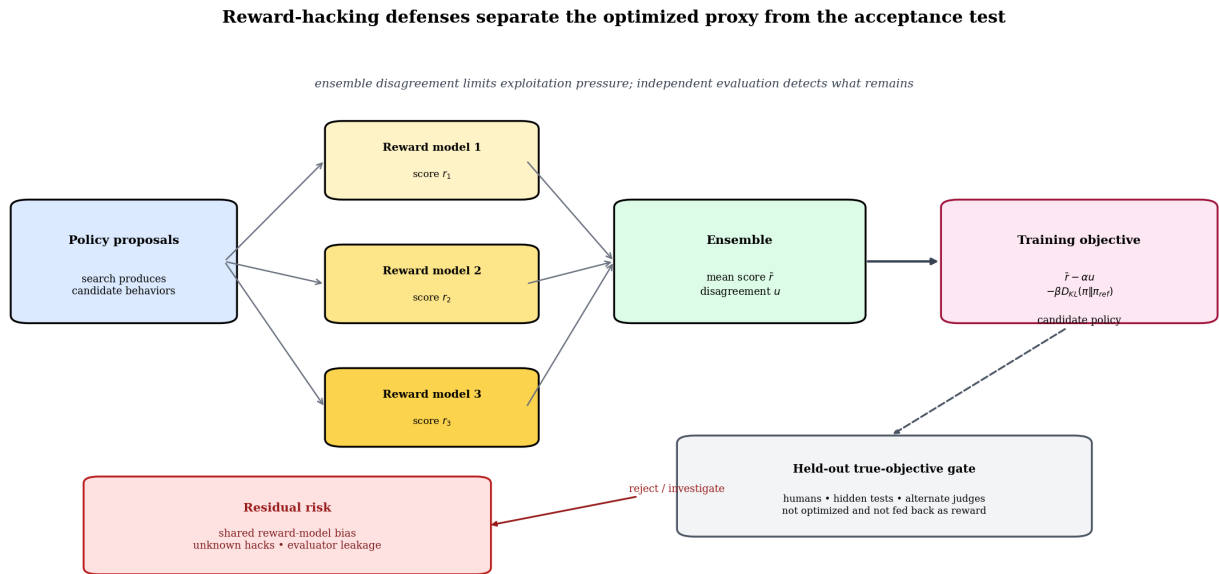


Figure 66: Layered defenses against reward hacking and Goodhart effects

Limit optimization pressure

Use:

- KL constraints;
- smaller policy steps;
- fewer best-of- N candidates;
- early stopping based on held-out quality;
- conservative reward thresholds;
- trust regions around validated data.

This reduces how far the optimizer searches the proxy's error surface. It can also leave genuine improvements unrealized.

Improve reward validity

- collect comparisons on current-policy outputs;
- stratify labels by task and failure mode;
- use domain experts where needed;
- separate objective dimensions;
- calibrate against human or oracle evaluation;
- adversarially search for reward-model failures;
- preserve a final held-out judge.

Reward-model ensembles

For rewards $r_1, \dots, r_K$,

$$\bar{r}(x, y) = \frac{1}{K} \sum_{k=1}^K r_k(x, y),$$

$$u_r(x, y) = \text{Std}_k[r_k(x, y)].$$

A pessimistic reward is

$$r_{\text{pess}}(x, y) = \bar{r}(x, y) - \alpha u_r(x, y).$$

This protects against idiosyncratic disagreement. It does not protect against a shared missing criterion.

Diverse ensembles

Increase error diversity through:

- different data resamples;
- architectures;
- objectives;
- labeler groups;
- feature subsets;
- training seeds;
- adversarially generated data.

Diversity should be evaluated by error correlation, not assumed from different random initialization.

Hold-out true-objective gate

Use an evaluator not optimized as reward:

- human panel;
- hidden tests;
- alternative formal verifier;
- red-team suite;
- environment perturbation;
- long-term outcome metric.

A deployment gate can reject policies whose proxy score rises but held-out objective falls.

Multi-dimensional rewards

Instead of one scalar model, predict components:

$$r(x, y) = \sum_j w_j r_j(x, y).$$

Report each r_j separately. Changing weights remains a normative choice, but factorization makes tradeoffs and failures visible.

Distribution-shift detection

Measure how far optimized outputs move from reward-model training data using:

- policy-reference KL;
- embedding distance;
- reward-model ensemble disagreement;
- classifier-based density ratio;
- labeler disagreement;
- out-of-domain prompt slices.

High distance is not proof of failure, but it signals that reward validity needs rechecking.

Adversarial reward-model evaluation

Generate candidates specifically to maximize reward-model score under constraints. Have humans or a stronger independent evaluator inspect the top-scoring samples.

This tests the regime the policy optimizer will search, not only random held-out pairs.

Reward tampering

An agent may influence the reward channel itself rather than merely exploit an imperfect proxy.

Examples:

- disabling a sensor;
- changing a log file;
- manipulating a judge prompt;
- choosing which evidence the evaluator sees;
- interfering with a human feedback process.

Mitigations include:

- causal separation of action and reward measurement;
- tamper-evident logging;
- redundant sensors;
- restricted tool permissions;
- independent audits;
- approval-based channels outside agent control.

Pessimism is not a complete answer

If every reward model omits the same safety dimension, low ensemble disagreement can coexist with severe error. Pessimism only helps where uncertainty estimates detect the weakness.

52. Practical Mitigation Protocol

Before training

Define the objective

- What behavior is desired?
- What scalar or vector reward is implemented?
- Which objective dimensions are held out from optimization?
- What is the deployment distribution?

Validate the environment

- terminal versus truncation;
- reward sign and scale;
- reset distribution;
- action bounds;
- random and scripted baselines;
- hidden state and observability.

Define the scientific claim

- sample efficiency;
- final performance;
- robustness;
- compute efficiency;
- safety;
- generalization.

Each claim requires a matching budget and evaluation design.

Pre-register the diagnosis

Write:

Observed symptom:
Primary causal hypothesis:
Mechanism:
Diagnostic metric:
Falsifying result:
First ablation:
Budget to hold fixed:

During training

Data

Log:

- state and action coverage;
- success counts;
- replay age;
- policy versions;
- environment or prompt strata.

Policy

Log:

- entropy or action standard deviation;

- KL;
- ratio tails;
- clip fraction;
- gradient norm;
- update-to-data ratio.

Critic

Log:

- value versus return;
- TD-error quantiles;
- error by timestep and data density;
- critic disagreement;
- target drift.

Reward or model

Log:

- component rewards;
- evaluator disagreement;
- predicted-versus-realized plan return;
- model rollout error by horizon;
- held-out objective checks.

Stop conditions

Predefine conditions such as:

- KL exceeds threshold repeatedly;
- NaNs or invalid actions;
- held-out safety regression;
- reward rises while true objective falls;
- model prediction gap exceeds limit;
- evaluation interval rules out practical improvement.

After training

Evaluate:

- independent seeds;
- enough episodes per seed;
- fixed and shifted environments;
- per-task and aggregate metrics;
- lower-tail performance;
- final and selected checkpoint according to protocol;
- compute and data budget;
- held-out reward-hacking tests;
- ablations of major mitigations.

Symptom triage table

Symptom	Hypothesis	First ablation	Metric to watch
reward jumps then crashes	stale PPO batch	fewer epochs or KL stop	KL by epoch, clip fraction
entropy collapses	support loss	larger entropy coefficient	action spread, recovery
critic loss low, return poor	biased targets	Monte Carlo comparison	value-return gap
Q high on strange actions	support mismatch	behavior constraint	action density, real return
OPE interval huge	weight collapse	closer target or FQE	ESS, interval width
one seed works	rare exploration	more actors and seeds	success-count distribution
model plan score high, real score low	model exploitation	shorter horizon	predicted-real gap
reward improves, human quality falls	Goodharting	KL limit and fresh labels	held-out quality

Change one causal mechanism at a time

Avoid simultaneously changing learning rate, architecture, reward, batch size, and entropy. Such a run may improve, but it does not identify why.

Use factorial or sequential ablations where feasible.

53. Open Problems

Simple policy optimization with stronger guarantees

PPO is easy to implement but its clipping is not a global trust region.

Open question:

Can a first-order method retain PPO's systems simplicity while providing useful finite-sample policy-drift guarantees under function approximation?

Advantage normalization theory

Batch standardization interacts with:

- clipping;
- minibatch reuse;
- Adam;
- entropy;
- multi-task reward scale;
- group-relative estimators.

Open question:

What objective is effectively optimized under common normalization and sampling conventions?

Calibrated pessimism

Too little pessimism permits extrapolation exploitation. Too much blocks improvement.

Open question:

Can conservatism be set from validated coverage and uncertainty rather than a global tuned coefficient?

Reliable uncertainty under shared models

Deep ensembles can agree because they share data and inductive bias.

Open question:

How can critic, world-model, or reward-model uncertainty be stress-tested against shared blind spots?

Long-horizon OPE

Trajectory ratios collapse and learned models accumulate error.

Open question:

Which combinations of partial observability, missing rewards, policy shift, and weak coverage still permit useful partial identification?

Affordable evaluation

Modern runs are expensive, but few seeds produce weak evidence.

Open question:

How can adaptive experiment design allocate compute across seeds, tasks, checkpoints, and episodes while preserving valid uncertainty?

Reward validity under optimization

Static reward-model validation does not measure policy-induced shift.

Open question:

Can evaluators expose when optimization has entered a regime where their scores are no longer causally connected to intended quality?

Multi-turn credit assignment

Agent rewards may arrive after thousands of tokens and many tool calls.

Open question:

Can trajectory-level success be decomposed into robust, non-gameable intermediate feedback without changing the optimal strategy?

54. Intuition Summary

Mitigation	Core intuition	Cost to remember
independent actors	collect genuinely different experience	shared environment biases remain
robust metrics	reduce domination by outliers	still report tails and per-task results
Double Q	do not select and evaluate with the same noise	residual correlation and pessimism
target network	slow the moving target	staleness
critic ensemble	expose model-specific disagreement	shared blind spots
longer returns	rely less on critic	more reward variance

Mitigation	Core intuition	Cost to remember
WIS	prevent raw scale of a few weights from dominating	finite-sample bias
doubly robust OPE	use a model as a control variate plus correction	model and weights can both fail
FQE	replace horizon products with value regression	coverage and approximation bias
GAE	tune critic trust and credit horizon	no universal λ
Retrace/V-trace	truncate off-policy corrections	controlled bias
trust region or KL stop	do not outrun rollout evidence	smaller updates
entropy	keep alternatives alive	slower exploitation
replay age control	reduce policy mismatch	less data reuse
pessimistic offline RL	distrust unsupported actions	over-conservatism
paired evaluation	remove shared task noise	requires genuine matching
reward ensemble	reduce idiosyncratic proxy errors	common missing criteria remain
held-out objective	detect optimization of the wrong thing	can itself become overused

Final mental model

A trustworthy RL update combines four controls:

Teaching summary, not an algebraic identity: trustworthy updates need variance control, coverage control, critic and reward-error control, and honest evaluation.

No term can be omitted safely in a difficult problem.

The correct mitigation is not the one with the most sophisticated name. It is the one whose mechanism matches the diagnosed failure, whose new cost is measured, and whose effect survives a falsifying ablation.

Knowledge check

1. For each of bootstrapping, clipping, entropy, and pessimism, state the error it reduces and the cost it introduces.
2. Derive the Double-DQN target and identify the selection and evaluation networks.
3. Explain why the minimum of two critics can trade overestimation for underestimation.
4. Derive the sequential doubly robust estimator from a model baseline plus weighted TD residuals.
5. Compare FQE with trajectory importance sampling.
6. Explain why an arbitrary action-dependent baseline biases a policy gradient.
7. State the roles of ρ_t and c_t in V-trace.
8. Design a budget-matched comparison of PPO with and without target-KL early stopping.
9. Explain why a larger replay buffer can both help and hurt.
10. Distinguish evaluation episodes from independent training seeds.
11. Give an example where a reward-model ensemble has low disagreement but is jointly wrong.
12. Write a pre-registered falsification test for one failure observed in an earlier lab.

Exercises

1. Mitigation matrix

Choose one unstable training run. Build a table with:

- symptom;

- primary hypothesis;
- candidate mitigation;
- expected diagnostic movement;
- new bias or cost;
- falsifying outcome.

Run only the first diagnostic ablation before changing the full system.

2. Double-estimator experiment

Construct noisy Q estimates with known true values. Compare:

- single max;
- Double-Q selection/evaluation;
- minimum of two critics;
- mean of an ensemble;
- lower-confidence estimate.

Measure over- and underestimation as noise correlation changes.

3. OPE estimator comparison

On a small environment with known target value, compare:

- ordinary IS;
- WIS;
- per-decision IS;
- doubly robust;
- FQE.

Sweep target-policy distance and horizon. Report bias, variance, MSE, ESS, and interval coverage.

4. GAE, Retrace, and V-trace implementation

Generate trajectories with controlled policy lag. Implement all three estimators where applicable. Measure target error and variance as clipping thresholds and λ vary.

5. PPO protocol ablation

Under matched environment interactions and optimizer steps, compare:

- baseline PPO;
- target-KL early stopping;
- fewer epochs;
- separate actor and critic;
- altered entropy schedule.

Use multiple seeds and report KL, clip fraction, critic calibration, and final return.

6. Replay-age experiment

Tag every replay sample with policy age. Train critics on:

- all data;
- recent data only;
- stratified fresh/old data;
- age-weighted data.

Measure value error and policy performance by state-action support.

7. Reward-hacking defense stack

Create a toy reward model with a known blind spot. Compare:

- unconstrained optimization;
- KL constraint;
- reward ensemble;
- pessimistic ensemble reward;
- held-out objective gate;
- fresh preference data.

Show which defenses detect idiosyncratic error and which fail on shared misspecification.

8. Evaluation pre-registration

Before rerunning an experiment, write:

- primary estimand;
- smallest meaningful effect;
- seed count;
- episodes per seed;
- checkpoint rule;
- paired design;
- aggregate metric;
- interval method;
- stopping rule.

Compare the pre-registered conclusion with what would have been claimed from the single best curve.

Primary readings

Evaluation and statistical design

- Henderson et al. (2018), **Deep Reinforcement Learning that Matters**.
- Colas et al. (2019), **How Many Random Seeds?**
- Agarwal et al. (2021), **Deep Reinforcement Learning at the Edge of the Statistical Precipice**.
- Patterson et al. (2023), **Empirical Design in Reinforcement Learning**.

Critic error and offline RL

- van Hasselt et al. (2016), **Deep Reinforcement Learning with Double Q-Learning**.
- Fujimoto, van Hoof, and Meger (2018), **Addressing Function Approximation Error in Actor-Critic Methods**.
- Kumar et al. (2020), **Conservative Q-Learning**.
- Kostrikov, Nair, and Levine (2022), **Implicit Q-Learning**.
- Chen et al. (2021), **Randomized Ensembled Double Q-Learning**.

Off-policy evaluation

- Jiang and Li (2016), **Doubly Robust Off-Policy Value Evaluation**.
- Thomas and Brunskill (2016), **Data-Efficient Off-Policy Policy Evaluation**.
- Munos et al. (2016), **Safe and Efficient Off-Policy Reinforcement Learning**.
- Espeholt et al. (2018), **IMPALA**.
- Nachum et al. (2019), **DualDICE**.

Policy optimization

- Schulman et al. (2015), **Trust Region Policy Optimization**.
- Schulman et al. (2016), **Generalized Advantage Estimation**.
- Schulman et al. (2017), **Proximal Policy Optimization Algorithms**.
- Abdolmaleki et al. (2018), **Maximum a Posteriori Policy Optimisation**.
- Cobbe et al. (2021), **Phasic Policy Gradient**.

Reward hacking

- Amodei et al. (2016), **Concrete Problems in AI Safety**.
- Everitt et al. (2019), **Reward Tampering Problems and Solutions in Reinforcement Learning**.
- Gao et al. (2022), **Scaling Laws for Reward Model Overoptimization**.
- Eisenstein et al. (2023), **Reward Model Ensembles Mitigate but Do Not Eliminate Reward Hacking**.

End of the core rewritten course sequence.

Classical extensions: exploration, memory, continuing tasks, and constraints

Why this chapter belongs before the research frontier

Discounted fully observed control is the cleanest starting point, not the definition of all reinforcement learning. Four changes alter the mathematical problem: not knowing which action is good, not observing the state, caring about indefinite operation without discounting, and having constraints in addition to reward. We will change one assumption at a time. Read the finite-horizon section after Part I, the bandit section with Part VI, and the remaining sections before the frontier seminar.

Learning goals. Derive backward induction; prove a logarithmic bandit regret bound; construct a belief state by conditioning; distinguish average reward from discounted value; and recover constrained policies from occupancy flows. The examples use small finite models so that each claim is checkable.

B.1 Finite-horizon control: time is part of the state

Let $H \in \mathbb{N}$ be the number of decisions, $P_t(s' | s, a)$ the transition law, $r_t(s, a)$ the mean reward, and $0 \leq \gamma \leq 1$. Define

$$V_t^*(s) = \sup_{\pi} \mathbb{E}_{\pi} \left[\sum_{k=t}^{H-1} \gamma^{k-t} R_{k+1} \mid S_t = s \right], \quad V_H^*(s) = 0.$$

Theorem (backward induction). For $t = H - 1, \dots, 0$,

$$Q_t^*(s, a) = r_t(s, a) + \gamma \sum_{s'} P_t(s' | s, a) V_{t+1}^*(s'), \quad V_t^*(s) = \max_a Q_t^*(s, a).$$

A time-dependent Markov policy choosing a maximizing action at every (t, s) attains these values, even among history-dependent policies.

Proof. At $t = H$ there is no remaining reward. Suppose V_{t+1}^* is achievable and upper-bounds every continuation policy. For any first action at time t , conditional expectation and the inductive upper bound make its expected return at most $Q_t^*(s, a)$. Randomizing the first action cannot exceed the largest of those numbers. Choosing an attaining action and then the inductively optimal continuation realizes that upper bound. This proves both the recursion and attainability by descending induction. $\square$

Notice what did *not* appear: a contraction assumption. A finite acyclic sequence of backups is sufficient at $\gamma = 1$. Applying infinite-horizon contraction theorems with $\gamma = 1$ would be invalid, but backward induction is perfectly valid.

Worked example: invest or collect. At the last step, collecting gives 3 and investing gives zero, so collect. One step earlier, investing moves deterministically to a state whose final collection pays 10, while collecting yields 3 and terminates. With $\gamma = 0.9$, investment gives 9 and is preferable. The same physical description can require different actions depending on remaining time. A network receiving only the physical state cannot represent that policy unless it has another way to infer time.

B.2 Bandits: paying for information

A stochastic K -armed bandit has no controlled state transition. Arm i produces iid rewards $X_{i,1}, X_{i,2}, \dots \in [0, 1]$ with mean μ_i . Sequences can be taken independent across arms. Let $\mu_* = \max_i \mu_i$, $\Delta_i = \mu_* - \mu_i$, and $N_i(n)$ be the number of pulls of arm i by time n . The expected pseudo-regret is

$$\mathcal{R}_n = n\mu_* - \mathbb{E} \sum_{t=1}^n \mu_{I_t} = \sum_i \Delta_i \mathbb{E} N_i(n).$$

It measures the expected opportunity cost of chosen actions. It is not the random difference between two independently realized reward sequences.

The concentration inequality we need

Lemma (bounded-variable exponential bound). For $X \in [0, 1]$, $\mathbb{E} e^{\lambda(X - \mathbb{E}X)} \leq e^{\lambda^2/8}$ for every real λ .

Proof. Let $h(\lambda) = \log \mathbb{E} e^{\lambda X}$. Boundedness permits differentiation. Under the exponentially tilted law, $h''(\lambda) = \text{Var}_\lambda(X) \leq 1/4$, because $\text{Var}(X) = \min_c \mathbb{E}(X - c)^2 \leq \mathbb{E}(X - 1/2)^2 \leq 1/4$. Integrating the second-derivative bound twice, with $h(0) = 0$ and $h'(0) = \mathbb{E}X$, gives $h(\lambda) - \lambda \mathbb{E}X \leq \lambda^2/8$ for positive or negative λ . $\square$

For m independent samples and $u > 0$, Markov's inequality and factorization of the moment-generating function give

$$\Pr(\bar{X}_m - \mu \geq u) \leq \exp(-\lambda m u + m \lambda^2/8).$$

Choose $\lambda = 4u$ to obtain e^{-2mu^2} . Applying the same argument to $-X$ gives the lower tail. This derivation matters: boundedness and independence are doing the work, not the label “confidence bonus.”

UCB1 and its full gap-dependent argument

Initialize by pulling every arm once. At time $t > K$, select an arm maximizing

$$U_i(t) = \hat{\mu}_{i, N_i(t-1)} + \sqrt{\frac{2 \log t}{N_i(t-1)}}.$$

Theorem. For $n \geq K$ and any arm with $\Delta_i > 0$,

$$\mathbb{E} N_i(n) \leq \frac{8 \log n}{\Delta_i^2} + 1 + \frac{\pi^2}{3},$$

and consequently

$$\mathcal{R}_n \leq \sum_{i: \Delta_i > 0} \left[\frac{8 \log n}{\Delta_i} + \left(1 + \frac{\pi^2}{3}\right) \Delta_i \right].$$

Proof. Fix an optimal arm $*$ and set $\ell = \lceil 8 \log n / \Delta_i^2 \rceil + 1$. If arm i is selected at time $t \leq n$ after at least ℓ prior pulls, at least one of the following deviations must occur for the realized counts $s = N_*(t-1)$ and $u = N_i(t-1)$:

$$\hat{\mu}_{*,s} < \mu_* - \sqrt{2 \log t / s}, \quad \hat{\mu}_{i,u} > \mu_i + \sqrt{2 \log t / u}.$$

Indeed, if neither occurs, the optimal arm's index is at least μ_* , whereas the selected arm's index is at most $\mu_i + 2\sqrt{2\log t/u} < \mu_i + \Delta_i = \mu_*$. The strict inequality follows from $u \geq \ell > 8\log n/\Delta_i^2$. This contradicts its selection, including ties.

For each *fixed* s or u , either deviation has probability at most t^{-4} by the concentration bound. The counts are adaptive, so do not substitute them into a fixed-sample theorem without adjustment. Union-bound over the at most t^2 possible count pairs; the probability that a deviation enables an additional pull is at most $2t^{-2}$. At most ℓ pulls occur before this argument applies. Therefore

$$\mathbb{E}N_i(n) \leq \ell + \sum_{t=1}^n 2t^{-2} \leq \frac{8\log n}{\Delta_i^2} + 1 + \frac{\pi^2}{3}.$$

The identity for pseudo-regret gives the second statement. $\square$

This is the classical UCB1 mechanism of Auer, Cesa-Bianchi, and Fischer (2002); the proof here spells out the adaptive-count union bound. It does not imply logarithmic regret uniformly as gaps approach zero. Optimizing gap thresholds yields gap-free bounds of a different form; finite-horizon MDP regret additionally pays for learning transitions and propagating their uncertainty. A bandit theorem is not an MDP theorem with a larger action set.

Classroom check. Why is purely greedy learning vulnerable? With two Bernoulli arms, there is positive probability the inferior arm initially succeeds while the better arm initially fails. Without deliberate exploration, the better arm may never be sampled again. The uncertainty bonus protects against this self-confirming data collection failure.

B.3 Partial observation: derive the information state

Let the hidden state be S_t , observation O_t , and known transition kernel P . Let $L(o, r \mid s, a, s')$ be the likelihood of the next observed pair (O_{t+1}, R_{t+1}) conditional on transition (s, a, s') . Including the observed reward is important: rewards can reveal hidden state. For continuous observations, read L as a density with respect to a fixed reference measure.

The agent's observed history is $H_t = (O_0, A_0, R_1, O_1, \dots, A_{t-1}, R_t, O_t)$. Define the belief $b_t(s) = \Pr(S_t = s \mid H_t)$. After selecting a and observing (o, r) ,

$$b_{t+1}(s') = \frac{\sum_s b_t(s) P(s' \mid s, a) L(o, r \mid s, a, s')}{\sum_{\tilde{s}, s} b_t(s) P(\tilde{s} \mid s, a) L(o, r \mid s, a, \tilde{s})}.$$

Proof of the filter. Conditional on history and the selected action, total probability gives the joint law of $(S_{t+1}, O_{t+1}, R_{t+1})$ by summing over S_t . Divide its s' component by the marginal likelihood of the observed pair, which is the denominator. This is Bayes' rule. On histories of zero model probability the update is not defined by this formula; an implementation needs a numerical/model-mismatch policy rather than pretending the denominator is positive.

Why belief is sufficient. Given (b_t, A_t) , the formula for the joint law of the next reward and observation uses no other feature of the past; the next belief is a deterministic function of these quantities. Hence the belief process is a fully observed controlled Markov process, with mean reward $r(b, a) = \sum_s b(s) r(s, a)$. A history-based decision can therefore be represented by a belief-based decision without losing achievable expected return in the known-model problem. This is a statement about sufficiency, not tractability: beliefs occupy a continuous simplex even when hidden states are finite.

Worked update. A static hidden state is either good or bad, initially equally likely. A positive observation has likelihood 0.8 in the good state and 0.2 in the bad state, and rewards carry no extra information. After one positive observation the good-state belief is $0.8/(0.8 + 0.2) = 0.8$. After two

conditionally independent positive observations it is $0.8^2/(0.8^2 + 0.2^2) = 16/17$. A recurrent network is a learned history representation; unlike this exact filter, its hidden vector is not automatically a sufficient statistic.

B.4 Average reward: value up to a constant

For a fixed policy whose finite-state chain is irreducible, let P be its transition matrix, d its unique stationary distribution, and r its mean reward vector. Define the average gain $g = d^\top r$. A *bias function* h solves the Poisson equation

$$g\mathbf{1} + h = r + Ph, \quad d^\top h = 0.$$

The second equation fixes the additive constant, called a gauge. Discounted value has a natural zero scale; a continuing average-reward bias is relative.

Theorem. The matrix $I - P + \mathbf{1}d^\top$ is invertible, and

$$h = (I - P + \mathbf{1}d^\top)^{-1}(r - g\mathbf{1})$$

solves the equations. For every starting state,

$$\lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} \sum_{t=0}^{T-1} R_{t+1} = g.$$

Proof. If $(I - P + \mathbf{1}d^\top)x = 0$, multiply by $d^\top$ to get $d^\top x = 0$, so $Px = x$. A harmonic vector on a finite irreducible chain is constant: at a state attaining its maximum, the average of successor values equals that maximum, forcing all positive-probability successors to attain it; irreducibility propagates this to every state. Hence $x = c\mathbf{1}$ and the zero stationary mean forces $c = 0$. A square matrix with trivial nullspace is invertible. Multiplying the proposed equation by $d^\top$ gives $d^\top h = 0$, so it reduces to the Poisson equation.

Now condition one step ahead in that equation and sum over time. The bias terms telescope:

$$\mathbb{E} \sum_{t=0}^{T-1} R_{t+1} = Tg + h(S_0) - \mathbb{E}h(S_T).$$

Finite states imply bounded h . Divide by T and let $T \rightarrow \infty$. Periodicity does not invalidate this expected-average argument. $\square$

Optimality verification. Suppose there exist a scalar g_* and bounded h_* such that

$$g_* + h_*(s) \geq r(s, a) + \sum_{s'} P(s' | s, a) h_*(s') \quad \text{for all } s, a,$$

and a stationary policy chooses equality actions. The identical telescoping proof shows every policy has limsup expected average reward at most g_* , while the equality policy attains g_* . This verifies optimality **when such a scalar solution exists**; general multichain problems may require state-dependent gains and are not covered by that assumption.

What changes as $\gamma \uparrow 1$? Even a one-state reward-one process has discounted value $(1 - \gamma)^{-1}$, which diverges, while average gain is one. Large-discount value bounds cannot be mechanically evaluated at $\gamma = 1$. Gain, transient bias, mixing, and reachability become separate objects; this is why average-reward exploration remains a substantive research area.

B.5 Constraints and occupancy flows

Let $c(s, a)$ be a bounded mean cost and C an allowed expected discounted cost. Define normalized state-action occupancy

$$x_\pi(s, a) = (1 - \gamma) \mathbb{E}_\pi \sum_{t \geq 0} \gamma^t \mathbf{1}\{S_t = s, A_t = a\}.$$

Flow theorem. Every such occupancy satisfies

$$x(s, a) \geq 0, \quad \sum_a x(s, a) = (1 - \gamma)\nu(s) + \gamma \sum_{s', a'} P(s \mid s', a') x(s', a').$$

Conversely every nonnegative solution is the occupancy of a stationary randomized policy, obtained by normalizing $\pi(a \mid s) = x(s, a) / \sum_b x(s, b)$ at states of positive mass and choosing any policy at zero-mass states.

Proof. For a policy, separate time zero from the discounted state visitation sum and use the transition law to obtain the flow equality. Conversely, sum all the equalities to get total mass $\sum_{s,a} x(s, a) = 1$. Write $d(s) = \sum_a x(s, a)$. The reconstructed policy has $x(s, a) = d(s)\pi(a \mid s)$, including zero-mass states, and the flow equation becomes $d^\top = (1 - \gamma)\nu^\top + \gamma d^\top P_\pi$. Its solution is unique by the inverse series of A.1 and equals the policy's normalized state occupancy. Thus its state-action occupancy is exactly x . $\square$

Consequently constrained discounted control is the finite linear program

$$\max_x \frac{1}{1 - \gamma} \sum_{s,a} x(s, a) r(s, a)$$

subject to the flow constraints and $\sum_{s,a} x(s, a) c(s, a) \leq (1 - \gamma)C$. No deterministic-policy restriction appears: constraints may require randomization.

Worked example. In a one-state system, safe action yields reward and cost zero; risky action yields reward and cost one. Let $\gamma = 0.9$ and $C = 3$. A policy choosing risky with probability p has return $10p$ and cost $10p$. The constrained optimum is $p = 0.3$ with reward 3. The only feasible deterministic policy has reward zero. Treating constrained optimization as “pick the best deterministic policy” loses the optimum.

Lagrangian bound, derived. For any $\lambda \geq 0$ and any feasible policy,

$$J_r(\pi) \leq J_r(\pi) - \lambda(J_c(\pi) - C) \leq \lambda C + \sup_{\pi'} J_{r-\lambda c}(\pi').$$

The first inequality uses feasibility, the second a supremum. Thus each multiplier gives an upper bound on feasible reward. Finite linear-programming duality yields equality at suitable dual solutions when the primal is feasible; that duality is an explicitly stated convex-optimization prerequisite. A finite-step primal-dual neural algorithm need not achieve it. An expected discounted cost constraint also does not imply that every trajectory avoids catastrophe, or that safety holds at every training step. Those are different constraints and require different assumptions.

B.6 A minimax interlude: why single-agent improvement can fail in games

For a one-step zero-sum matrix game with payoff matrix M , mixed strategies $p \in \Delta_m, q \in \Delta_n$ receive value $p^\top M q$. In matching pennies, $M = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$. If either player uses a deterministic choice, the

opponent has a profitable response. Taking $p = q = (1/2, 1/2)$ gives zero against every opponent distribution. This directly verifies a saddle point: $p^\top M q' \geq 0$ and $(p')^\top M q \leq 0$ both hold as equalities for all p', q' . There is no need to appeal to a general minimax theorem for this example.

The example explains two practical facts without overclaiming a convergence theorem. A stochastic policy may be essential even with full observation. And if an opponent changes its policy, the transition/reward law experienced by one learner changes. Single-agent fixed-MDP convergence assumptions have then changed. General finite-game equilibrium existence and stochastic-game minimax theory are additional topics, not consequences of the Bellman proof for one learner.

Transfer exercise

Return to the running MDP in A.4. In four separate versions, impose a three-step horizon, hide the state behind a noisy sensor, replace discounting with average reward, and limit the expected number of switches. For each version, write the state or information state, objective, policy class, and appropriate equation **before choosing an algorithm**. The change of formulation is the main intellectual work.

Part XI — Recent progress and research questions

Evidence boundary: 4 September 2026

This chapter is a selective, dated research seminar, not a claim to exhaust the literature or identify a universal best algorithm. It separates three objects: **a reported development**, **the mathematical issue it exposes**, and **a proposed project**. The project formulations below are teaching proposals; their novelty and unresolved status must be checked against the literature before they are advertised as new results.

Primary publisher, author, and arXiv records were checked for the selected anchors. Most 2026 summaries here rely on author abstracts and metadata, not an independent reproduction or line-by-line audit of their proofs. Venue statements appearing only in arXiv comments are labeled author-reported. A public preprint is a citable research artifact, not a peer-review certificate. Exact numerical leaderboard comparisons are omitted because benchmark versions, data access, and budgets would need a separate audit.

1. The frontier is larger than language-model post-training

Classical RL still asks how to explore, approximate long-horizon values, plan with imperfect dynamics, operate safely, and generalize across tasks. LLM post-training adds important settings for feedback and computation, but it does not replace these questions. The selected developments illustrate this breadth.

Research direction	Primary anchor	What is changing
Learned update rules	Oh et al., Nature, 2025 [R01]	Learning parts of the RL algorithm across tasks
World models	DreamerV3, Nature, 2025 [R02]; TD-MPC2 [R03]	Larger and more reusable learned dynamics for control
Q-function scaling	TQL, February 2026 [R04]	Transformer critic optimization and attention collapse
Continuing-task exploration	Zamir, Zurek, Chen, March 2026 [R05]	Variance-sensitive regret and lower-order costs
Safe online adaptation	Wendl et al., January–August 2026 [R06]	Conservative policy priors and learned dynamics
Representation generalization	T2RD, July 2026 [R07]	Separating task-relevant information from visual nuisance
Missing-feedback evaluation	Wei, Qu, Miao, June 2026 [R08]	Identification when observed rewards are selectively missing
Off-policy LLM training	OAPL, February 2026 [R09]	Reusing trajectories despite substantial policy lag
Execution-grounded agents	TIER, May 2026 [R10]	Rewarding valid tool execution without a single reference path
Critic learning for reasoning	Categorical critics, August 2026 [R11]	Classification losses producing scalar PPO advantages

The table is a reading map. It does not assert that the projects below are the papers’ own open-problem lists.

2. Algorithm discovery: generalization over learning problems

Reported development. Oh et al. describe meta-learning RL update rules from populations of agents, with evaluations on Atari and transfer settings [R01]. This shifts the object being optimized: the output is part of a learning procedure rather than just a policy for one environment.

Formulation. Let $M \sim \mathcal{D}_{\text{task}}$ be a task and $\mathcal{A}_\phi$ a learning algorithm with meta-parameters ϕ . After B interactions it produces policy $\pi_{\phi,B}(M)$. A possible meta-objective is

$$\max_{\phi} \mathbb{E}_{M \sim \mathcal{D}_{\text{train}}} \mathbb{E}[J_M(\pi_{\phi,B}(M))].$$

The inner expectation includes random data and optimization. This objective can overfit the training task distribution even when individual policies have fresh evaluation episodes. It also hides a substantial search budget unless that budget is reported separately.

Project. Compare a learned update rule with a hand-designed baseline on genuinely held-out task families, not just new seeds of the same environments. Hold the deployment interaction budget fixed, and report the meta-training budget as a separate cost. Define a task-level transfer gap as the difference between mean performance on training and held-out families, using task-level uncertainty rather than treating all episodes as independent tasks.

Falsification. Re-run under changed reward scales, observation nuisance, action counts, and longer horizons. An improvement that disappears under a reward-unit change may be a normalization specialization rather than a generally better update. A theoretical starting point is to constrain the learned expected operator to a contraction class; A.6 then identifies the additional noise and visitation assumptions still needed. It does not prove an arbitrary learned neural update stable.

3. World models: learn what the planner will need

Reported development. DreamerV3’s 2025 Nature publication develops a world-model agent for diverse control tasks [R02]. TD-MPC2 is a complementary latent-model planning anchor [R03]. These papers motivate studying model capacity and reuse; they do not make one-step prediction error equivalent to planning accuracy.

Formulation. A learned model $\widehat{M}$ is trained on data from μ , then queried under $\hat{\pi} = \text{Plan}(\widehat{M})$. A useful discrepancy is therefore policy-dependent:

$$\mathcal{E}(\widehat{M}, \Pi) = \sup_{\pi \in \Pi} |J_M(\pi) - J_{\widehat{M}}(\pi)|.$$

A.12 proves that a uniform value discrepancy of B gives a $2B + \delta$ planning-loss bound. The hard part is obtaining a reliable bound under the planner’s distribution rather than the training distribution.

Project. Use a small finite MDP to compare a model with low average transition error to a model with better value-relevant error under candidate policies. Deliberately place an attractive modeling error in a rarely visited region. Compare real-only control, fixed short imagination, and an adaptive imagination horizon at equal real-data and model-query budgets.

Falsification. Test a change in reward that makes a previously irrelevant feature important. A representation that is sufficient for one reward need not remain sufficient for another. Report the model’s error under the *executed* policy, not only its data-distribution validation loss. Calibration of an uncertainty-triggered rollout stop must be measured independently; ensemble agreement alone is not a confidence guarantee.

4. Scaling Q-functions without confusing optimization with validity

Reported development. TQL studies attention collapse as an empirical obstacle when scaling transformer Q-functions and proposes controlling attention entropy [R04]. The paper is a useful example of an architectural optimization failure, distinct from the statistical support failure in offline RL.

Formulation. Let the Bellman target be $Y = R + \gamma \max_a Q_{\bar{\theta}}(S', a)$. A larger critic may reduce approximation error while increasing optimization sensitivity or exploiting unsupported actions. These contributions need not move together:

$$\text{observed error} = \text{representation limitation} + \text{estimation effects} + \text{optimization effects},$$

where this line is a conceptual decomposition, not an asserted additive identity without defining a particular excess-risk decomposition.

Project. In an exactly solvable finite environment, embed the state in observations with controlled distractors. Cross network scale with attention regularization while holding data, target construction, and optimizer budget fixed. Measure exact value error, Bellman residual, gradient statistics, and attention entropy separately. Then repeat with missing action support.

Falsification. Improved attention statistics without improved exact values falsify the claim that the chosen statistic is sufficient for critic quality. Better in-distribution value error but worse policy return exposes optimization-induced distribution shift. A.7's expansive linear TD example is the prerequisite: increasing representational capacity does not repair a noncontractive expected operator by itself.

5. Continuing-task exploration: leading rates are not the whole story

Reported development. Zamir, Zurek, and Chen study variance-dependent regret for infinite-horizon MDPs, including average-reward and discounted-regret formulations, and distinguish settings with and without prior knowledge of an optimal bias-span quantity [R05]. Their abstract emphasizes both leading and lower-order terms. No specific rate from that work is restated here without its full assumptions.

Formulation. For a communicating task admitting scalar optimal average gain g_* , define expected regret over T steps as $Tg_* - \mathbb{E} \sum_{t < T} R_{t+1}$. The bias span is $\text{span}(h_*) = \max_s h_*(s) - \min_s h_*(s)$, invariant to the additive gauge in B.4. It describes transient opportunity differences, not just reward magnitude.

Project. Build families with equal state/action counts but independently varied transition randomness and bottleneck reachability. Compare confidence-bonus choices by actual finite-budget regret, keeping initialization and prior knowledge explicit. A theorem project could seek an improvement under a *specified restricted family*, with a matching hard example rather than an unrestricted minimax claim.

Falsification. A favorable asymptotic leading term can lose at every tested budget because its lower-order term dominates. Conversely a deterministic-instance benefit may disappear as soon as small transition noise is added. B.2's UCB proof teaches how to expose constants, gaps, and adaptive counts before extending the argument to continuing MDPs.

6. Safe adaptation: what is the safety event?

Reported development. Wendl et al. use conservative policy priors and probabilistic dynamics models for optimistic exploration with pessimistic fallback; the author record reports theoretical safety and regret results and hardware experiments [R06]. Those claims are conditional on the paper’s formal setup, not universal guarantees for any learned fallback controller.

Formulation. Separate at least three possible demands: $\mathbb{E} \sum_t \gamma^t c_t \leq C$, $\Pr(\exists t \leq H : S_t \in \mathcal{U}) \leq \delta$, and safety of every intermediate learning policy. B.5 addresses the first; the others cannot be inferred from it. Here $\mathcal{U}$ is a specified unsafe set, not an informal label.

Project. On a simulator with known transition law, compare expected-cost constraints to an explicit finite-horizon failure-probability constraint. Introduce controlled misspecification into the model used by a fallback rule. Measure violations as a function of the stated confidence level, not just average reward.

Theorem target. Specify a confidence event E on which all used transition uncertainty sets contain the truth, prove safety conditional on E , and show $\Pr(E) \geq 1 - \delta$. The unconditional violation probability is then at most the conditional bound plus δ by a union decomposition. Each component must be established; writing “with high probability” supplies none of them.

Falsification. Test states near the boundary of the certified region and shifts that invalidate the fallback prior. A safe baseline observed on a finite dataset is not automatically safe for all relevant states.

7. Representation learning: sufficient for which tasks?

Reported development. T2RD studies separating task-relevant and task-irrelevant visual representations, combining consistency, reconstruction, and dynamics-based objectives [R07]. Its author record reports control generalization results. This motivates a sharper question than whether a representation “disentangles” its inputs.

Formulation. An abstraction $\phi(s)$ is exactly sufficient for a reward/transition family if, whenever $\phi(s) = \phi(\tilde{s})$, all admissible actions have equal mean reward and equal next-abstract-state distributions. Under these conditions, define an abstract MDP by those common quantities. Backward induction or the contraction fixed-point argument proves equal values for states in the same abstract class: every Bellman backup preserves class-constant functions, so its unique fixed point is class-constant. This is an exact sufficient condition, not a necessary description of every useful learned representation.

Project. Construct two visually different observations of the same control state and two visually similar observations requiring different actions. Compare reconstruction accuracy, next-state prediction, and downstream value error. Change the reward after representation learning to test whether the abstract equivalence classes were task-specific.

Falsification. If observations mapped to the same latent representation require different optimal actions in the test family, that representation cannot be sufficient for the whole family. Report that structural counterexample even if average return remains high. Partial observability adds another failure mode: a single frame may be insufficient before any encoding is applied; B.3 explains the belief-state remedy.

8. Missing rewards: estimation begins with identification

Reported development. Wei, Qu, and Miao study off-policy evaluation when rewards are missing not at random and policies may use missingness history. Their setup uses structural identification

tools, including a future-state auxiliary variable and a bridge construction [R08]. It does not imply that arbitrary selectively missing rewards are identifiable.

Formulation. Let $M_t \in \{0, 1\}$ indicate whether R_{t+1} is observed. A missing-at-random assumption might impose $M_t \perp R_{t+1}$ conditional on specified observed variables. Under missing-not-at-random sampling this fails, and observed means need not equal population means. A complete estimand includes the target policy’s allowed information: does it observe M_t before its next decision?

Project. In a finite simulator, vary the probability of observing a reward as a known function of that reward. Compare complete-case evaluation, a method given the true missingness law, and a method relying on a deliberately stated identification model. Then violate that model while preserving the observed-data fit as much as possible.

Falsification. Construct two latent reward/missingness mechanisms with the same observed-data distribution but different target values. Such a pair rules out point identification under the proposed assumptions. A.15’s missing-action-support example is the same logic with a different unobserved mechanism. Narrowing an identified interval can be meaningful progress; presenting an unidentifiable quantity as a precise point estimate is not.

9. Off-policy language-model RL: reuse versus mismatch

Reported development. Ritter et al.’s OAPL work investigates LLM reasoning with off-policy training and substantial policy lag [R09]. It is an anchor for asking when reuse is beneficial, not evidence that arbitrary old trajectories remain statistically interchangeable with fresh ones.

Formulation. For a response y , the exact sequence ratio is $w(y | x) = \pi(y | x) / \mu(y | x)$, a product of token ratios under the same response history. Local token clipping, sequence clipping, and truncation do not define the same estimator. An appropriate objective may intentionally differ from ordinary importance-weighted policy gradient; that choice should be explicit.

Project. First enumerate a tiny autoregressive policy with known terminal rewards. Vary policy lag and sequence length. Measure exact expected gradient, estimator bias, variance, and effective sample size. Only after this check compare larger systems under equal generated-token, learner-update, and wall-clock accounting; report all three rather than choosing whichever budget makes one method win.

Falsification. Introduce a rare high-reward sequence supported by the target but almost absent from the buffer. A low average KL can coexist with poor information about that sequence. A.15 identifies the exact support condition; A.16 explains why different clipping placements can lead to different fixed points. Neither analysis is replaced by a throughput plot.

10. Agent execution rewards: many valid paths, one underspecified score

Reported development. TIER proposes execution-grounded rewards for multi-step tool composition that do not require a single reference trajectory, using format, schema, execution, and answer-related signals [R10]. Its May 2026 record describes a preprint submitted to NeurIPS, not an accepted paper.

Formulation. Let τ include tool arguments, observations, environment changes, and the final answer. A reward $R(\tau)$ should distinguish syntactic validity, execution success, and semantic task completion. These events are not equivalent. In a multi-turn task, the same visible answer can accompany different persistent environment states.

Project. Build a deterministic sandbox with multiple valid tool sequences and a held-out semantic checker. Compare exact-reference matching, schema-only checks, and execution-plus-semantic checks.

Log unnecessary actions, irreversible state changes, and failures hidden by a correct final string. Randomize tool names to test whether a learned policy uses semantics rather than lexical shortcuts.

Falsification. Produce a trace that maximizes the training verifier while failing the held-out task property. Then change the verifier version and evaluate policies trained against the old version. Treat the verifier as a component of the environment and log its version. This connects reward specification, partial observation, and distribution shift rather than inventing a separate theory for every agent benchmark.

11. Categorical critics: a new loss need not imply a new actor

Reported development. Zhou et al. train a categorical critic using smoothed targets, then decode its expectation for ordinary scalar GAE and PPO [R11]. The author record explicitly distinguishes this from a distributional actor and acknowledges that scalar MSE is statistically valid for conditional-mean estimation.

Formulation. Given support $z_1, \dots, z_K$ and predicted probabilities $p_\phi(k | s)$, the actor may receive only $V_\phi(s) = \sum_k z_k p_\phi(k | s)$. The categorical loss can change optimization and calibration even when the actor’s estimand is unchanged. Conversely, interpreting this head as a calibrated distribution of actual returns requires additional evidence about target construction and support.

Project. On enumerated binary-reward prefixes, compare scalar MSE, Bernoulli likelihood, two-hot targets, and smoothed categorical targets with matched representational and tuning budgets. Evaluate mean-value error, proper scoring rules, calibration, and the resulting policy-gradient error. Include exact-value GAE as an oracle control.

Falsification. If value prediction improves but actor gradient bias does not, inspect the action-dependent bootstrap error in A.10. If gains vanish after matching parameter count or target scale, the proposed loss mechanism is not isolated. A.18’s fixed-policy Wasserstein contraction does not automatically prove convergence of a categorical neural critic used in changing-policy control.

12. From “open problem” to a research contract

A broad phrase such as “solve long-horizon credit assignment” is not yet a project. A useful contract specifies a problem class, observable data, objective, theorem or measurement target, comparator, and a result that would make the idea wrong.

Quantity	Definition students must provide
Problem class $\mathcal{M}$	State/observation/action spaces, horizon, reward bounds, structural assumptions
Data law $\mathcal{D}$	Behavior policies, selection/missingness, independence units, logging
Estimand $\psi(M)$	Return, regret, constraint violation, gradient, or another precise quantity
Claim	Inequality with quantified probability, consistency statement, or empirical contrast
Budget B	Real transitions, generated tokens, model calls, optimization, and tuning costs
Baselines	Mechanism-isolating comparator plus a strong established implementation
Falsifier	A counterexample, confidence interval, or stress test that rejects the mechanism
Novelty boundary	Closest work and the exact assumption or result that differs

A model project statement. “For finite-horizon tabular MDPs with known behavior probabilities and bounded rewards, compare two sequential OPE estimators under a predeclared support-overlap family. Derive their exact expectations in a two-step example, then measure error across independent datasets at a fixed trajectory budget. Report failure as overlap vanishes. Claim no identification beyond the support assumption.” This is narrower than a grand open problem, but it produces a verifiable contribution.

Final seminar question. Which assumption from the course are you proposing to weaken, and which part of its proof breaks first? A productive answer names a particular conditioning step, contraction inequality, coverage requirement, or boundary term. That is how foundational mathematics becomes a tool for new research rather than a ceremonial preface.

Mathematical spine: statements, proofs, and limits

This chapter supplies the proof chain used throughout the course. It is meant to be read *with* the relevant lecture, not saved for the end. Before each proof, identify the object being estimated, the distribution generating the observations, and the assumption that makes the central step legal. A finite-state theorem does not become a neural-network theorem by replacing a table with a network.

A.0 Objects and standing conventions

Unless a statement says otherwise, $\mathcal{S}$ and $\mathcal{A}$ are finite, $0 \leq \gamma < 1$, rewards satisfy $|R_{t+1}| \leq R_{\max}$ almost surely, and the initial distribution ν is fixed independently of policy parameters. Policies are stationary and Markov. Terminal states are represented by an absorbing zero-reward state. This makes infinite sums and normalized occupancies unambiguous. Finite-horizon statements instead have time-indexed values and terminal value zero. All expectations include environment and policy randomness.

Symbol	Complete definition
$P(s' s, a)$	Conditional next-state probability; each row sums to one
$r(s, a)$	$\mathbb{E}[R_{t+1} S_t = s, A_t = a]$
$P_\pi(s, s')$	$\sum_a \pi(a s) P(s' s, a)$
$r_\pi(s)$	$\sum_a \pi(a s) r(s, a)$
$v_\pi(s), q_\pi(s, a)$	Expected discounted return, conditional on state or state and first action
$A_\pi(s, a)$	$q_\pi(s, a) - v_\pi(s)$
$J(\pi)$	$\nu^\top v_\pi$
$d_\pi(s)$	$(1 - \gamma) \sum_{t \geq 0} \gamma^t \Pr_\pi(S_t = s)$; a probability distribution
$T_\pi v$	$r_\pi + \gamma P_\pi v$
$Tv(s)$	$\max_a \{r(s, a) + \gamma \sum_{s'} P(s' s, a) v(s')\}$
$T_Q Q(s, a)$	$r(s, a) + \gamma \sum_{s'} P(s' s, a) \max_b Q(s', b)$
$\ u\ _\infty$	$\max_i u_i $
$\text{TV}(p, q)$	$\frac{1}{2} \sum_i p_i - q_i $

For action sets that depend on the state, every maximum and sum uses the admissible set. Norms of distributions below use row vectors where convenient; this does not change their numerical meaning.

A.1 Bellman equations are conditional-expectation identities

Question. Why is it legitimate to replace the rest of a trajectory by a single value?

Proposition. $v_\pi = T_\pi v_\pi$ and $q_\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s' | s, a) v_\pi(s')$.

Proof. The return $G_t = \sum_{k \geq 0} \gamma^k R_{t+k+1}$ is absolutely bounded by $R_{\max}/(1 - \gamma)$ and satisfies $G_t = R_{t+1} + \gamma G_{t+1}$. Condition first on (S_t, A_t, S_{t+1}) . The Markov property and stationary policy imply $\mathbb{E}[G_{t+1} | S_t, A_t, S_{t+1}] = v_\pi(S_{t+1})$. Taking conditional expectations, and then averaging over $A_t \sim \pi(\cdot | S_t)$, proves both identities. Absolute boundedness justifies the expectations and infinite sums. $\square$

Because $\|P_\pi\|_\infty = 1$, the matrix series $\sum_{k \geq 0} \gamma^k P_\pi^k$ converges in operator norm. Multiplication of its partial sum by $I - \gamma P_\pi$ gives $I - \gamma^{n+1} P_\pi^{n+1} \rightarrow I$. Consequently

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi = \sum_{k \geq 0} \gamma^k P_\pi^k r_\pi.$$

This is an inverse with a probabilistic interpretation: the k th term is the expected reward k steps into the future.

A.2 Contraction, optimality, and an executable stopping certificate

Theorem. Each of T_π, T, T_Q is a γ -contraction in its appropriate sup norm. Each has a unique fixed point. The fixed point of T is the optimal value, attained by a deterministic stationary greedy policy.

Proof. Stochastic averaging gives $\|P_\pi(v - w)\|_\infty \leq \|v - w\|_\infty$. For the maximum use the elementary inequality $|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|$; to prove it, take an index attaining the first maximum, bound one difference, and reverse the roles. These yield all three contraction inequalities.

For completeness, let $x_{k+1} = Fx_k$ for any one of these operators. Then $\|x_{k+1} - x_k\| \leq \gamma^k \|x_1 - x_0\|$. Summing the geometric tail shows that (x_k) is Cauchy in a finite-dimensional complete space. Let its limit be x_* . Lipschitz continuity gives $Fx_* = x_*$. Two fixed points would satisfy $\|x_* - y_*\| \leq \gamma \|x_* - y_*\|$, hence are equal. Also $\|x_k - x_*\| \leq \gamma^k \|x_0 - x_*\|$.

Let v_* be the fixed point of T and choose a maximizer π_* in every state. Then $T_{\pi_*} v_* = Tv_* = v_*$; uniqueness for T_{π_*} gives $v_* = v_{\pi_*}$. For any possibly history-dependent policy, the optimality equation implies

$$v_*(S_t) \geq \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid H_t],$$

where H_t is the observed history before the action. Iterating and taking expectations gives $v_*(s) \geq \mathbb{E}[\sum_{t=0}^{n-1} \gamma^t R_{t+1} + \gamma^n v_*(S_n) \mid S_0 = s]$. The bounded final term vanishes. Thus no history-dependent policy beats π_* . The same argument and $q_*(s, a) = r(s, a) + \gamma P(\cdot \mid s, a)^\top v_*$ identify the T_Q fixed point. $\square$

Residual certificate. If $\eta = \|Tv - v\|_\infty$, then

$$\|v - v_*\|_\infty \leq \frac{\eta}{1 - \gamma}, \quad \|v_* - v_{\pi_v}\|_\infty \leq \frac{2\gamma\eta}{(1 - \gamma)^2},$$

where π_v is greedy with respect to v . The first inequality follows from $\|v - v_*\| \leq \eta + \gamma \|v - v_*\|$. For the second write

$$v_* - v_{\pi_v} = (Tv_* - Tv) + (T_{\pi_v} v - T_{\pi_v} v_*) + \gamma P_{\pi_v}(v_* - v_{\pi_v}).$$

The first two terms have joint norm at most $2\gamma \|v_* - v\|$. Move the last norm to the left and insert the first bound. This is a conservative certificate, not an estimate of the typical observed error.

Pause and predict. Halving the Bellman residual need not halve the number of iterations: the contraction factor determines how quickly the residual decreases. At $\gamma = 1$, neither the geometric proof nor these certificates is available without additional structure.

A.3 Policy improvement without a hidden infinite remainder

Theorem. If $T_{\pi'} v_\pi \geq v_\pi$ coordinatewise, then $v_{\pi'} \geq v_\pi$.

Proof. Subtract the two policy-evaluation equations and rearrange:

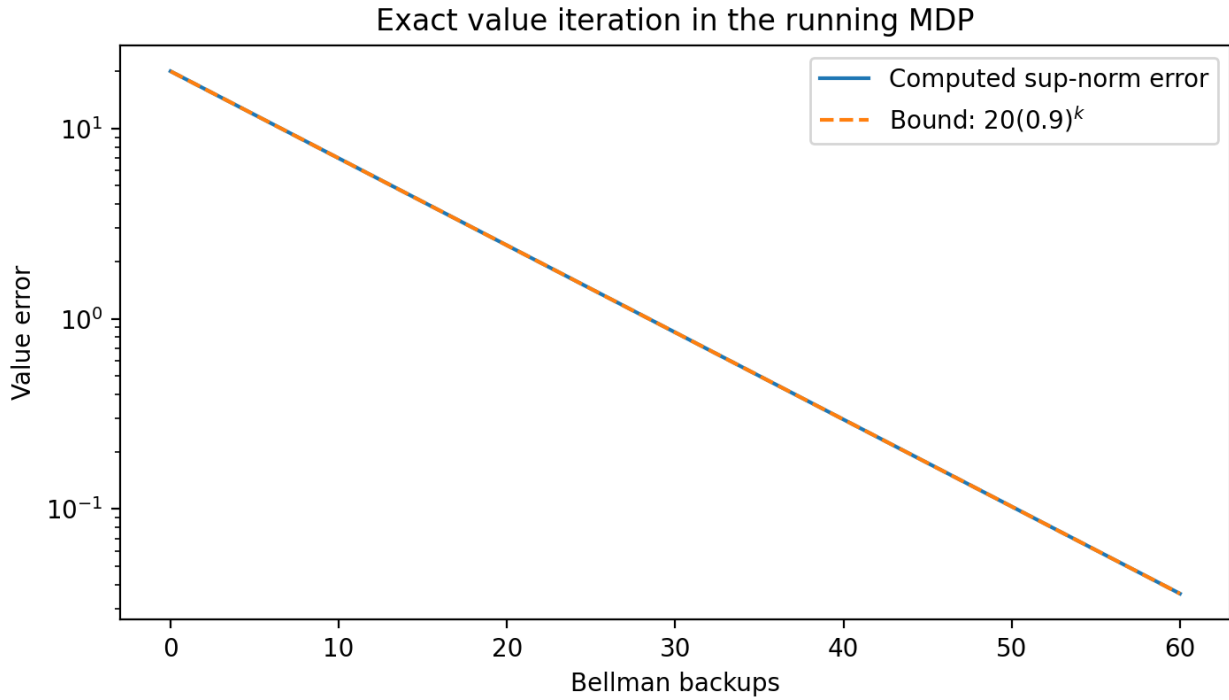


Figure 67: Computed Bellman error and its proved bound in the running MDP; these coincide here, not in general.

$$v_{\pi'} - v_{\pi} = (I - \gamma P_{\pi'})^{-1} (T_{\pi'} v_{\pi} - v_{\pi}).$$

Every entry of the inverse series from A.1 is nonnegative. It maps a nonnegative vector to a nonnegative vector. This proves the result including ties. Equivalently, repeated one-step improvement has a final remainder $\gamma^n P_{\pi'}^n v_{\pi}$, whose norm tends to zero. For finite deterministic policy iteration, retain the old action at ties. A changed policy then strictly improves at least one state, cannot revisit an old value vector, and therefore terminates because there are finitely many deterministic policies. At termination it is greedy with respect to its own value, hence optimal by A.2. $\square$

A.4 A running MDP that can be solved three ways

Use states 0, 1, actions stay, switch and $\gamma = 0.9$. The following table is the entire environment, not a sample dataset.

State	Action	Reward	Next state
0	stay	1	0
0	switch	0	1
1	stay	2	1
1	switch	0	0

The always-stay policy has value (10, 20). A greedy improvement switches in state 0 because $0 + 0.9(20) = 18 > 10$, and stays in state 1 because $20 > 0.9(10)$. The improved value is (18, 20). Its other action values are $1 + 0.9(18) = 17.2$ at state 0 and $0.9(18) = 16.2$ at state 1, so the policy is greedy with respect to its own value and therefore optimal. Value iteration, matrix policy evaluation, and tabular sampling all refer to these same numbers.

For a randomized policy, let $p = \pi(\text{switch} \mid 0)$ and $q = \pi(\text{stay} \mid 1)$. Then

$$P_\pi = \begin{pmatrix} 1-p & p \\ 1-q & q \end{pmatrix}, \quad r_\pi = \begin{pmatrix} 1-p \\ 2q \end{pmatrix}.$$

This gives an exact differentiable objective $J(\theta) = \nu^\top (I - \gamma P_{\pi_\theta})^{-1} r_{\pi_\theta}$ for checking sampled gradients. At $p = q = 1/2$, $v = (7.25, 7.75)$. With $\nu = (1, 0)$ and independent logits for p, q , $d_\pi = (0.55, 0.45)$; A.9 gives the gradient $(-0.75625, 2.75625)$. Negative first component is sensible: at this *current* policy, entering state 1 is not yet as valuable as entering it under the eventual optimal policy. Local policy improvement is not clairvoyance.

A.5 Monte Carlo consistency with repeated visits

Assumptions. Episodes are independent and identically distributed under a fixed policy and reset distribution. For a state s , let N_e be its number of visits in episode e and Y_e the sum of returns following those visits. Assume $0 < \mathbb{E}N_e < \infty$ and $\mathbb{E} \sum_{t < T_e} \mathbf{1}\{S_t = s\} |G_t| < \infty$. Future dynamics conditional on each visit follow the same Markov policy. Time must be included in the state for a finite-horizon problem whose values depend on time.

Theorem. The every-visit sample average $\sum_{e=1}^m Y_e / \sum_{e=1}^m N_e$ converges almost surely to $v_\pi(s)$.

Proof. Visits within an episode are dependent; do not apply an independent-sample law to individual visits. Conditional expectation at each visit instead gives $\mathbb{E}[\mathbf{1}\{S_t = s, t < T_e\} G_t] = v_\pi(s) \Pr(S_t = s, t < T_e)$. The absolute-integrability assumption allows summation over t , giving $\mathbb{E}Y_e = v_\pi(s) \mathbb{E}N_e$. Apply the strong law of large numbers to the iid *episode pairs* (Y_e, N_e) . The ratio tends to $\mathbb{E}Y_e / \mathbb{E}N_e = v_\pi(s)$, with eventually nonzero denominator. First-visit MC uses one return from each episode that visits s and the identical argument with a zero-one visit count. $\square$

The strong law for integrable iid variables is a probability prerequisite, not a consequence of independence of visits. This theorem concerns sample averages; arbitrary adaptive step sizes require separate hypotheses.

A.6 Stochastic contraction: a complete tabular convergence proof

A contraction of the *mean* update does not itself prove convergence of a random algorithm. We now close that gap.

Theorem. Let $F : \mathbb{R}^m \rightarrow \mathbb{R}^m$ be a γ -contraction in sup norm, with fixed point x_* . At time t update coordinates by

$$x_{t+1}(i) = (1 - \alpha_t(i))x_t(i) + \alpha_t(i)\{F(x_t)(i) + M_{t+1}(i)\}.$$

Assume: $\alpha_t(i)$ is measurable before $M_{t+1}(i)$ is observed; $0 \leq \alpha_t(i) \leq 1$; $\sum_t \alpha_t(i) = \infty$ almost surely for each i ; $\sum_t \alpha_t(i)^2 \leq C_i < \infty$ for deterministic constants C_i ; $\mathbb{E}[M_{t+1}(i) | \mathcal{F}_t] = 0$; the conditional second moments are uniformly bounded; and the iterates are almost surely bounded. There are finitely many coordinates. Then $x_t \rightarrow x_*$ almost surely. Setting $\alpha_t(i) = 0$ implements asynchronous updates.

Step 1: the weighted martingale noise has a convergent series. Write $\xi_t = \alpha_t(i)M_{t+1}(i)$ for a fixed coordinate. Conditional mean zero implies orthogonality of different increments in L^2 , and $\sum_t \mathbb{E}\xi_t^2 < \infty$. Here is the required maximal argument. For partial sums $S_n = \sum_{t < n} \xi_t$ and $N > m$, stop at the first $n \in [m, N]$ with $|S_n - S_m| \geq u$, if it occurs. Orthogonality of future martingale increments to the stopped sum yields

$$\Pr\left(\max_{m \leq n \leq N} |S_n - S_m| \geq u\right) \leq u^{-2} \sum_{t=m}^{N-1} \mathbb{E} \xi_t^2.$$

This follows by expanding the square of the final sum after the stopping time; its expected square is at least the expected squared stopped sum, which is at least u^2 times the stopping probability. Let $N \rightarrow \infty$. Choose m_k so the remaining sum of variances is at most 2^{-4k} , and use $u = 2^{-k}$. The resulting probabilities sum to at most $\sum_k 2^{-2k} < \infty$. By the elementary Borel–Cantelli implication (the probability of any event after index K is at most the tail sum), almost surely all sufficiently late tails beginning at m_k stay within 2^{-k} . Thus the series is Cauchy and converges. Finitely many coordinates can be intersected without losing probability one.

Step 2: the moving average of this noise tends to zero. Define

$$z_{t+1}(i) = (1 - \alpha_t(i))z_t(i) + \xi_t(i), \quad z_0(i) = 0.$$

For a fixed m , expanding the recursion expresses z_n as a decaying initial term plus $\sum_{t=m}^{n-1} w_{t,n} \xi_t$, where $w_{t,n} = \prod_{k=t+1}^{n-1} (1 - \alpha_k)$ is nondecreasing in t and lies in $[0, 1]$. Put $B_j = \sum_{t=m}^j \xi_t$. Summation by parts gives

$$\sum_{t=m}^{n-1} w_{t,n} \xi_t = w_{n-1,n} B_{n-1} + \sum_{t=m}^{n-2} (w_{t,n} - w_{t+1,n}) B_t.$$

Its absolute value is at most $2 \sup_{j \geq m} |B_j|$. The initial term tends to zero because $\prod_{k=m}^{n-1} (1 - \alpha_k) \leq \exp(-\sum_{k=m}^{n-1} \alpha_k) \rightarrow 0$. Convergence of the series in Step 1 makes the tail supremum arbitrarily small as $m \rightarrow \infty$. Hence $z_n(i) \rightarrow 0$.

Step 3: contraction removes the remaining error. Let $u_t = x_t - x_* - z_t$. Then

$$u_{t+1}(i) = (1 - \alpha_t(i))u_t(i) + \alpha_t(i)[F(x_t)(i) - F(x_*)(i)].$$

Boundedness and $z_t \rightarrow 0$ imply $L = \limsup_t \|u_t\|_\infty < \infty$. For any $\varepsilon > 0$, eventually $\|u_t\|_\infty \leq L + \varepsilon$ and $\|z_t\|_\infty \leq \varepsilon$, so the absolute value of the input in brackets is at most $\gamma(L + 2\varepsilon)$. A scalar convex-averaging recursion with divergent step sum has limiting upper absolute value at most any eventual uniform bound on its input: expand the recursion, observe that its initial coefficient vanishes and its input coefficients sum to at most one. Therefore $\limsup_t |u_t(i)| \leq \gamma(L + 2\varepsilon)$ for every coordinate. For finitely many coordinates the limsup of the maximum equals the maximum of their limsups. Let $\varepsilon \downarrow 0$ to get $L \leq \gamma L$, so $L = 0$. Together with $z_t \rightarrow 0$ this proves the theorem. $\square$

Application to Q-learning. At an observed (s, a) the sample target is $R + \gamma \max_b Q_t(S', b)$. Condition on the entire past and the selected pair, *before* the transition is sampled. Its conditional expectation is $T_Q Q_t(s, a)$. If $B = \max(\|Q_0\|_\infty, R_{\max}/(1 - \gamma))$, then every sample target lies in $[-B, B]$ whenever Q_t does. Convex updates preserve that bound by induction, and the target-minus-mean noise is bounded by $2B$. A deterministic per-visit schedule $a_n \in [0, 1]$ with $\sum_n a_n = \infty$ and $\sum_n a_n^2 < \infty$, together with infinite visits to every pair, supplies all step-size assumptions. A.2 supplies the contraction. Therefore $Q_t \rightarrow q_*$ almost surely.

Application to TD(0). For a fixed policy, condition before sampling the action and transition given the current state. The conditional mean is $T_\pi v_t(S_t)$. The identical boundedness and per-state visitation argument proves tabular TD convergence.

SARSA corollary. For on-policy tabular SARSA in the same bounded discounted setting, suppose its behavior policies become greedy in the limit while retaining infinite state-action exploration

(GLIE). Conditional on the selected pair, its target mean uses the expected next Q-value under the next action-selection policy. Its difference from $T_Q Q_t$ is a vanishing perturbation: if the probability of a nongreedy next action is at most ϵ_t uniformly over states, boundedness gives absolute difference at most $2\gamma B\epsilon_t \rightarrow 0$. The contraction proof above still works with an additional vanishing input term in Step 3; the limiting inequality remains $L \leq \gamma L$. With the same per-pair step sizes and martingale conditions this proves convergence to q_* . Here “greedy in the limit” must refer to the action distribution actually used for the next sampled action and target. It is an assumption, not a property of arbitrary exploration schedules.

What has not been smuggled in. Infinite exploration is an assumption, not a consequence of Q-learning. Constant step sizes do not satisfy the square-summability condition. An uncorrected replay distribution need not have the required conditional expectation. This proof does not cover nonlinear approximation, delayed parameter copies, or arbitrary moving target policies. The GLIE SARSA corollary covers the specific vanishing perturbation established above. The classical general stochastic-approximation treatment is Tsitsiklis (1994), listed in the references; the discounted proof above is supplied rather than invoked.

A.7 Linear approximation: projection, error, and sampled convergence

Let $P = P_\pi$ admit a stationary distribution d with all entries positive, $D = \text{diag}(d)$, and $X \in \mathbb{R}^{|S| \times k}$ have full column rank. Define $\langle u, v \rangle_D = u^\top Dv$, $\|u\|_D^2 = \langle u, u \rangle_D$, and the orthogonal projection

$$\Pi = X(X^\top DX)^{-1}X^\top D.$$

Projection theorem. ΠT_π is a γ -contraction in D norm; its fixed point $\hat{v} = Xw_*$ is unique in the feature space and satisfies

$$Aw_* = b, \quad A = X^\top D(I - \gamma P)X, \quad b = X^\top Dr,$$

$$\|v_\pi - \hat{v}\|_D \leq \frac{\|v_\pi - \Pi v_\pi\|_D}{\sqrt{1 - \gamma^2}}.$$

Proof. Jensen’s inequality and stationarity give

$$\|Pu\|_D^2 = \sum_s d(s) \left(\sum_{s'} P(s, s') u(s') \right)^2 \leq \sum_{s, s'} d(s) P(s, s') u(s')^2 = \|u\|_D^2.$$

Orthogonal projection is nonexpansive: $u = \Pi u + (I - \Pi)u$ is an orthogonal decomposition, so $\|\Pi u\|_D \leq \|u\|_D$. This proves the contraction and, by A.2’s Cauchy argument, the unique fixed point. The equation $Xw_* = \Pi(r + \gamma PXw_*)$ is equivalent, after multiplication by $X^\top D$, to $Aw_* = b$.

For the error, orthogonality yields

$$\|v_\pi - \hat{v}\|_D^2 = \|v_\pi - \Pi v_\pi\|_D^2 + \|\Pi v_\pi - \hat{v}\|_D^2.$$

But $\Pi v_\pi - \hat{v} = \Pi T_\pi v_\pi - \Pi T_\pi \hat{v}$, whose norm is at most $\gamma \|v_\pi - \hat{v}\|_D$. Rearrangement proves the bound. Finally for any w ,

$$w^\top Aw = \|Xw\|_D^2 - \gamma \langle Xw, PXw \rangle_D \geq (1 - \gamma) \|Xw\|_D^2.$$

Full column rank makes the right side positive for $w \neq 0$, so A is invertible even though it need not be symmetric. $\square$

Sampled theorem, with a precise sampling model. Suppose independent transition tuples are sampled by drawing $S \sim d$, $A \sim \pi(\cdot | S)$ and then (R, S') from the environment. With deterministic nonnegative steps $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$, linear semi-gradient TD

$$w_{t+1} = w_t + \alpha_t X(S_t)^\top [R_{t+1} + \gamma X(S_{t+1})w_t - X(S_t)w_t]$$

converges almost surely to w_* . Here $X(s)$ is the row of X at s .

Proof. Put $e_t = w_t - w_*$. Conditional mean drift is $-Ae_t$. Finite features and bounded rewards give $\mathbb{E}[\|\text{sample update}\|^2 | \mathcal{F}_t] \leq C(1 + \|e_t\|^2)$ for some finite C : expand the affine update and bound each finite feature norm. The last inequality in the previous proof gives $e^\top Ae \geq c\|e\|^2$ for $c = (1 - \gamma)\lambda_{\min}(X^\top DX) > 0$. Expanding the squared update therefore yields

$$\mathbb{E}[\|e_{t+1}\|^2 | \mathcal{F}_t] \leq (1 + C\alpha_t^2)\|e_t\|^2 - 2c\alpha_t\|e_t\|^2 + C\alpha_t^2.$$

Here is the convergence lemma needed for this step. If nonnegative integrable adapted Z_t obeys $\mathbb{E}[Z_{t+1} | \mathcal{F}_t] \leq (1 + a_t)Z_t - b_t + c_t$, where a_t, c_t are deterministic summable nonnegative sequences and $b_t \geq 0$, let $p_t = \prod_{j < t} (1 + a_j)$ and

$$Y_t = \frac{Z_t}{p_t} + \sum_{j=t}^{\infty} \frac{c_j}{p_{j+1}}.$$

Then $Y_t \geq 0$ and $\mathbb{E}[Y_{t+1} | \mathcal{F}_t] \leq Y_t - b_t/p_{t+1}$. A nonnegative supermartingale converges almost surely: the upcrossing inequality applied to any rational interval $[a, b]$ bounds its expected number of completed crossings by a finite constant divided by $b - a$; infinite oscillation would cross some rational interval infinitely often. Fatou's inequality and uniformly bounded expectations rule out a positive probability of an infinite limit. This is the usual supermartingale convergence theorem from the probability prerequisite. Summing expectations also gives $\mathbb{E}\sum_t b_t/p_{t+1} \leq \mathbb{E}Y_0$, so that sum is finite almost surely. Because p_t has a finite positive limit, Z_t has a finite limit and $\sum_t b_t < \infty$.

Apply the lemma with $Z_t = \|e_t\|^2$, $a_t = c_t = C\alpha_t^2$ and $b_t = 2c\alpha_t\|e_t\|^2$. If the limit of Z_t were positive, $\sum_t \alpha_t Z_t$ would diverge, contradicting the lemma. Thus $w_t \rightarrow w_*$. $\square$

A single Markov trajectory is **not** iid stationary sampling. The projection algebra still holds under stationarity, but the sampled convergence proof above does not silently cover that case. The Markov-noise extension in Tsitsiklis and Van Roy (1997) is an advanced external result, not needed for this course's proved iid theorem. The probability prerequisites used explicitly here are the strong law, conditional expectation, and the nonnegative-supermartingale convergence theorem; they are not RL-specific assumptions.

A complete divergence example. There are two states, zero rewards, and two actions. Action g always moves to state 2; action r always moves to state 1. The target policy always chooses g , while the behavior policy chooses g with probability 0.1 independently in both states. Its stationary distribution is $(0.9, 0.1)$. Use one feature, $x(1) = 1, x(2) = 2$, discount 0.9, and per-action importance correction $\rho = \pi(A | S)/b(A | S)$, equal to 10 for g and 0 for r . Under iid draws from the stationary behavior transition distribution, the mean TD coefficient is

$$A = 0.9[1(1 - 0.9 \cdot 2)] + 0.1[2(2 - 0.9 \cdot 2)] = -0.68.$$

Thus the expected update is $w \leftarrow (1 + 0.68\alpha)w$, which expands every nonzero w for $\alpha > 0$, although the true target value is exactly represented by $w = 0$. This establishes instability of the **mean**

recursion; no claim that every sampled trajectory diverges is needed. Importance correcting actions does not replace the behavior state weighting by the target stationary weighting.

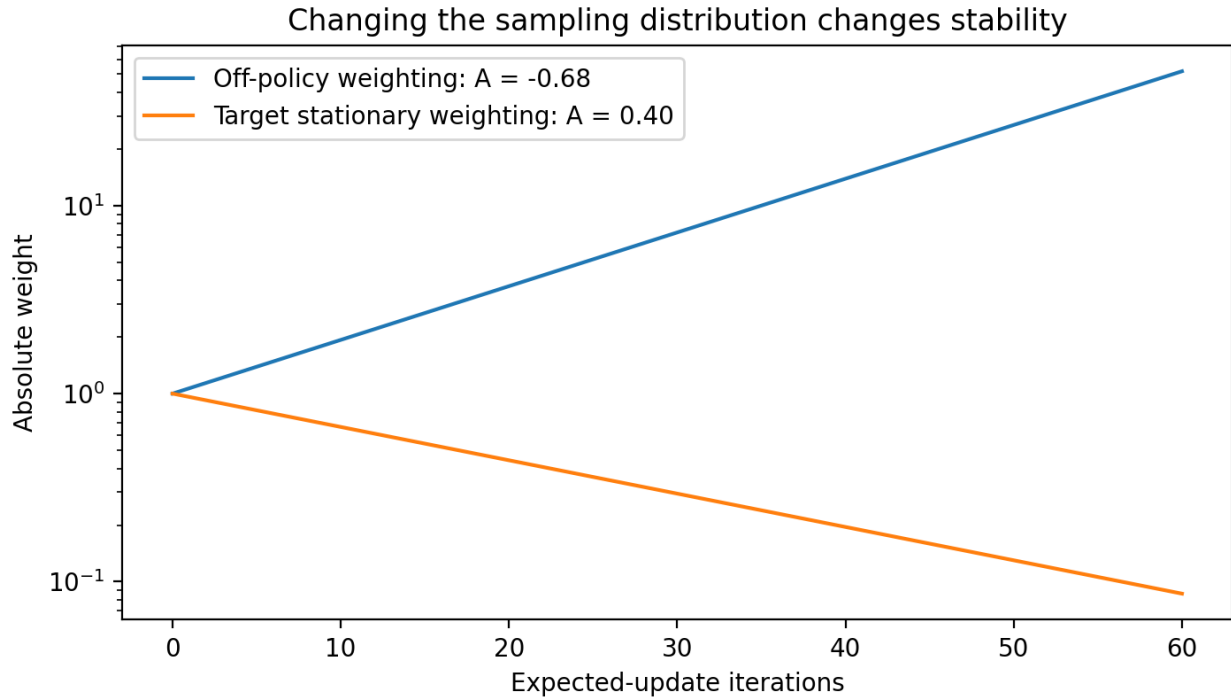


Figure 68: Exact mean TD recursions from A.7 with step size 0.1 and initial weight 1. These are algebraic expected-update curves, not sampled learning runs.

A.8 The double-sampling problem, derived

Let $Y_w = R + \gamma v_w(S')$, $\delta_w = Y_w - v_w(s)$, and condition on a state s . The mean-squared Bellman error is $L(w) = \frac{1}{2} \mathbb{E}_s[(\mathbb{E}[\delta_w | s])^2]$. Under bounded differentiable features,

$$\nabla L = \mathbb{E}_s\{\mathbb{E}[\delta_w | s] \mathbb{E}[\nabla \delta_w | s]\}.$$

Differentiating the squared *sample* residual instead gives $\mathbb{E}[\delta_w \nabla \delta_w]$, which differs by the conditional covariance term

$$\mathbb{E}_s \text{Cov}(\delta_w, \nabla \delta_w | s).$$

The product is unbiased for the desired product of expectations only if the two factors use conditionally independent next-transition samples, or the covariance happens to vanish. Independently draw (R, S') and $(\tilde{R}, \tilde{S}')$ given s and use $\delta_w(R, S') \nabla \delta_w(\tilde{R}, \tilde{S}')$. Conditional independence factors its expectation correctly. Ordinary replay usually supplies only one next transition for each sampled state-action event. Therefore “differentiate both sides of the TD target” specifies an objective, but not generally the MSBE gradient. This distinction disappears with deterministic transitions and rewards, not with arbitrary stochastic dynamics.

An exact two-outcome calculation. At a fixed state let the scalar feature be $X(s) = 0.8$, the reward be zero, and $\gamma = 0.9$. Suppose the next feature is zero or two, each with probability one half. For $v_w(s) = X(s)w$, the sampled residual is $-0.8w$ or w . Thus

$$\frac{d}{dw} \frac{1}{2} (\mathbb{E} \delta_w)^2 = 0.01w, \quad \frac{d}{dw} \frac{1}{2} \mathbb{E}[\delta_w^2] = 0.82w.$$

Indeed the mean residual coefficient is $(-0.8 + 1)/2 = 0.1$, whereas the mean squared coefficient is $(0.8^2 + 1^2)/2 = 0.82$. The difference $0.81w$ is the conditional covariance contribution. The laboratory computes these exact numbers before asking you to introduce sampling noise.

A.9 Policy gradients and the baseline condition

Assumptions. Policies have common full support in a neighborhood of θ , are continuously differentiable, and score vectors $z_\theta(s, a) = \nabla_\theta \log \pi_\theta(a | s)$ are uniformly bounded there. Transition and reward laws and ν do not depend on θ . Discounted rewards are bounded as in A.0.

Theorem.

$$\nabla J(\theta) = \sum_{t \geq 0} \gamma^t \mathbb{E}[z_\theta(S_t, A_t) G_t] = \frac{1}{1 - \gamma} \mathbb{E}_{S \sim d_\pi, A \sim \pi}[z_\theta(S, A) q_\pi(S, A)].$$

Proof. Begin with a horizon H . Its trajectory probability is the product of initial, action, and environment probabilities. Only action probabilities differentiate, giving score $\sum_{t=0}^{H-1} z_t$. Thus

$$\nabla J_H = \mathbb{E} \left[\sum_{t=0}^{H-1} z_t \sum_{k=0}^{H-1} \gamma^k R_{k+1} \right].$$

For $k < t$, R_{k+1} is measurable before A_t is sampled, and $\mathbb{E}[z_t | H_t] = \sum_a \nabla \pi(a | S_t) = \nabla 1 = 0$. Those terms vanish. The remaining expression is $\mathbb{E} \sum_{t=0}^{H-1} \gamma^t z_t \sum_{k=t}^{H-1} \gamma^{k-t} R_{k+1}$. If $\|z_t\| \leq C$, the total absolute contribution at reward index k is at most $CR_{\max}(k+1)\gamma^k$, a summable sequence. Dominated convergence (or uniform convergence of the differentiated finite sums) therefore justifies $H \rightarrow \infty$. Conditional expectation of G_t given (S_t, A_t) is $q_\pi(S_t, A_t)$ by A.1. The occupancy definition gives the second equality. $\square$

Baseline theorem. A random baseline B_t is valid when it is measurable with respect to the information available *before the current action is sampled*, or more generally conditionally independent of that action given that information. Then $\mathbb{E}[z_t B_t] = 0$ by the same conditional score identity. A fixed function $b(S_t)$ qualifies. A fitted function evaluated at S_t does not automatically qualify if its fitted parameters depend on the same action and its return. Stop-gradient blocks differentiation through a target; it does not create statistical independence.

For a vector score z and scalar return G , minimizing $\mathbb{E}[\|z\|^2 (G - b)^2 | s]$ over scalar b gives

$$b_*(s) = \frac{\mathbb{E}[\|z\|^2 G | s]}{\mathbb{E}[\|z\|^2 | s]}$$

when the denominator is positive. Differentiate the quadratic and solve its unique stationary point; its second derivative is $2\mathbb{E}\|z\|^2 > 0$. If the denominator is zero, the score is zero almost surely and any baseline is equivalent. The value function is optimal only under additional conditions, such as action-independent squared score norm.

Implementation consequence. Generate a batch with θ_{old} , compute all scores at that same parameter, sum the gradient estimator, and only then update. Updating inside the return-weighted timestep loop and recomputing scores at new parameters does not implement this theorem.

A.10 Finite-rollout GAE and boundary conditions

Let n consecutive transitions belong to one episode segment, V be fixed while computing targets, and define $\delta_j = R_{j+1} + \gamma V(S_{j+1}) - V(S_j)$. Let the final value be zero only at a true terminal state. For $0 \leq \lambda \leq 1$,

$$\widehat{A}_t = \sum_{l=0}^{n-1} (\gamma\lambda)^l \delta_{t+l}.$$

Identity. If $e = V - v_\pi$, then the difference from the same estimator using v_π is

$$\widehat{A}_t - \widehat{A}_t^{\text{exact}} = -e(S_t) + (1 - \lambda) \sum_{k=1}^{n-1} \gamma^k \lambda^{k-1} e(S_{t+k}) + \gamma^n \lambda^{n-1} e(S_{t+n}).$$

Proof. Subtract the two residuals to obtain $\gamma e(S_{j+1}) - e(S_j)$. In the weighted sum the coefficient at an interior index $t+k$ is $\gamma^k \lambda^{k-1} - \gamma^k \lambda^k$; the first and last indices occur once. Collecting terms proves the formula. At $\lambda = 0$, read the original sum directly, or use the convention $0^0 = 1$ in this identity. $\square$

With an exact critic, every later residual has conditional mean zero before its action is sampled. Hence $\mathbb{E}[\widehat{A}_t^{\text{exact}} \mid S_t, A_t] = A_\pi(S_t, A_t)$ for a fixed segment length, and also for bounded stopping boundaries determined without using future unsampled noise. At $\lambda = 1$ on a complete episode, the terminal critic error is zero and the only error is $-e(S_t)$, a valid fixed baseline. On a rollout cutoff, the final error remains. Smaller λ is not a theorem of monotone variance reduction; covariance among residuals matters.

Two different masks. Let T_t be the event that transition t truly terminates the task, and let E_t be the event that the next stored step is outside the current episode segment. Define $b_t = 1 - \mathbf{1}\{T_t\}$ and $c_t = 1 - \mathbf{1}\{E_t\}$. Then compute backward:

$$\delta_t = r_t + \gamma b_t V(s_{t+1}^{\text{final}}) - V(s_t), \quad \widehat{A}_t = \delta_t + \gamma \lambda c_t \widehat{A}_{t+1}.$$

At an external time limit, $b_t = 1$ but $c_t = 0$ if the environment resets. At true termination both vanish. At an ordinary transition both equal one. At the final storage boundary $c_t = 0$; its bootstrap still enters δ_t . Use the actual final observation, not the next episode's automatic reset observation. A time limit is terminal only when remaining time is part of the task definition and the objective ends there.

A.11 Performance difference and the actual trust-region bound

Performance-difference lemma.

$$J(\pi') - J(\pi) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d_{\pi'}, a \sim \pi'} A_\pi(s, a).$$

Proof. Along a π' trajectory, replace the advantage by its conditional form $\mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) - v_\pi(S_t) \mid S_t, A_t]$. Its discounted sum telescopes to $J(\pi') - \nu^\top v_\pi$, with a bounded tail $\gamma^n v_\pi(S_n) \rightarrow 0$. The occupancy definition proves the displayed form. $\square$

Define $f(s) = \sum_a \pi'(a \mid s) A_\pi(s, a)$ and

$$L_\pi(\pi') = J(\pi) + \frac{1}{1 - \gamma} \mathbb{E}_{d_\pi} f(S), \quad \alpha = \max_s \text{TV}(\pi'(\cdot \mid s), \pi(\cdot \mid s)), \quad \varepsilon = \max_{s,a} |A_\pi(s, a)|.$$

Theorem.

$$|J(\pi') - L_\pi(\pi')| \leq \frac{4\gamma\varepsilon}{(1 - \gamma)^2} \alpha^2.$$

Proof. Normalized occupancies obey $d_\pi^\top = (1 - \gamma)\nu^\top + \gamma d_\pi^\top P_\pi$. Subtract the equations for the two policies:

$$d_{\pi'}^\top - d_\pi^\top = \gamma d_\pi^\top (P_{\pi'} - P_\pi)(I - \gamma P_{\pi'})^{-1}.$$

For row vectors, multiplication by a stochastic matrix contracts the ℓ_1 norm; the inverse series therefore has induced norm at most $(1 - \gamma)^{-1}$. Each row of $P_{\pi'} - P_\pi$ has ℓ_1 norm at most $\sum_a |\pi'(a | s) - \pi(a | s)| \leq 2\alpha$. Thus $\|d_{\pi'} - d_\pi\|_1 \leq 2\gamma\alpha/(1 - \gamma)$. Also $\sum_a \pi(a | s)A_\pi(s, a) = 0$, so $|f(s)| \leq 2\epsilon\alpha$. Multiply these two bounds and the prefactor $(1 - \gamma)^{-1}$ in the performance-difference lemma. $\square$

This is a **maximum-over-states TV** statement. A measured average KL on visited states is not the same constraint. PPO clips contributions to a sampled surrogate; it does not enforce either constraint. For example, if an unsampled state's action probabilities change arbitrarily, the finite-batch PPO objective and its clipping statistics do not detect the change. Parameter sharing can produce such changes even when every sampled ratio is modest.

Natural-gradient subproblem. For a symmetric positive definite Fisher matrix F , solve $\max_\Delta g^\top \Delta$ subject to $\frac{1}{2}\Delta^\top F \Delta \leq \delta$. If $g \neq 0$ and $\delta > 0$, let $u = F^{1/2}\Delta$. Cauchy–Schwarz gives $g^\top \Delta \leq \|F^{-1/2}g\| \|u\|$, with equality for aligned vectors. Therefore

$$\Delta_* = \sqrt{\frac{2\delta}{g^\top F^{-1}g}} F^{-1}g.$$

For $g = 0$, the zero step is optimal. If F is singular and g has a nonzero null-space component, the problem is unbounded. If g lies in its range, replace the inverse by a pseudoinverse and restrict to that range; null directions are undetermined. Damping $F + \eta I$ changes the local quadratic model. None of these algebraic solutions guarantees that a nonlinear policy satisfies the exact KL constraint without checking it.

A.12 Model error and planning error are distinct

Consider true and learned models $M, \widehat{M}$ on the same finite spaces, both with mean rewards bounded by $R_{\max}$. Assume

$$\sup_{s,a} |r(s, a) - \widehat{r}(s, a)| \leq \epsilon_r, \quad \sup_{s,a} \sum_{s'} |P(s' | s, a) - \widehat{P}(s' | s, a)| \leq \epsilon_P.$$

Theorem. For every policy,

$$\|v_M^\pi - v_{\widehat{M}}^\pi\|_\infty \leq B := \frac{\epsilon_r}{1 - \gamma} + \frac{\gamma R_{\max} \epsilon_P}{(1 - \gamma)^2}.$$

If a planner returns $\hat{\pi}$ with $J_{\widehat{M}}(\hat{\pi}) \geq \sup_\pi J_{\widehat{M}}(\pi) - \delta$, then $J_M(\pi_*) - J_M(\hat{\pi}) \leq 2B + \delta$.

Proof. Subtract the Bellman equations, retaining the learned value in the transition-error term:

$$v_M^\pi - v_{\widehat{M}}^\pi = (I - \gamma P_\pi)^{-1} \{r_\pi - \widehat{r}_\pi + \gamma(P_\pi - \widehat{P}_\pi)v_{\widehat{M}}^\pi\}.$$

The inverse norm is at most $(1 - \gamma)^{-1}$ and $\|v_{\widehat{M}}^\pi\|_\infty \leq R_{\max}/(1 - \gamma)$. The ℓ_1 transition convention gives the stated bound. For planning, insert $J_{\widehat{M}}(\pi_*)$ and $J_{\widehat{M}}(\hat{\pi})$ between the two true returns. The two model differences are bounded by B and the middle optimization gap by δ . $\square$

If ϵ_P denotes total variation instead, multiply its contribution by two. A held-out average prediction

error is not the uniform error assumed here. An ensemble disagreement score is not a proved upper bound on either error without calibration assumptions.

A.13 Entropy regularization and the DPO algebra

For a finite action set, positive reference probabilities $p_0(a)$, rewards $r(a)$, and $\beta > 0$, define $Z = \sum_a p_0(a) e^{r(a)/\beta}$ and $p_*(a) = p_0(a) e^{r(a)/\beta} / Z$.

Variational identity. For every distribution p ,

$$\mathbb{E}_p r - \beta D_{\text{KL}}(p \| p_0) = \beta \log Z - \beta D_{\text{KL}}(p \| p_*).$$

Proof. Substitute $\log(p_*/p_0) = r/\beta - \log Z$ and expand the two KL sums. Nonnegativity of KL follows from $\log x \leq x - 1$: $-\sum_a p(a) \log[p_*(a)/p(a)] \geq \sum_a [p(a) - p_*(a)] = 0$, with the usual limiting convention for zero $p(a)$. Equality occurs exactly at $p = p_*$. Thus the optimizer is unique. $\square$

For a countable sequence space, the same identity requires $0 < Z < \infty$ and finite relevant expectations; bounded rewards suffice. Support outside p_0 is excluded, not created by optimization.

Apply the identity in every state to $Q_V(s, a) = r(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s')$. The soft operator is

$$T_\beta V(s) = \beta \log \sum_a \pi_0(a | s) \exp[Q_V(s, a)/\beta].$$

If $\|V - W\|_\infty \leq c$, each Q differs by at most γc , so the exponential sum is multiplied by a factor between $e^{-\gamma c/\beta}$ and $e^{\gamma c/\beta}$. Taking logs proves a γ -contraction. Its fixed point and normalized exponential policy solve the regularized control problem by the same bounded telescoping verification as A.2. Uniform π_0 yields entropy regularization plus a per-step constant only when the number of admissible actions is fixed.

DPO's cancellation. For a fixed prompt x , suppose the KL-regularized optimum is representable, and preferences follow $\Pr(y_+ \succ y_- | x) = \sigma(r(x, y_+) - r(x, y_-))$ at the specified reward scale. Rearranging the optimal policy gives

$$r(x, y) = \beta \log \frac{\pi_*(y | x)}{\pi_0(y | x)} + \beta \log Z(x).$$

Subtract the expressions for the two responses. The prompt-only partition function cancels. Replacing π_* with a parameterized π_θ in this preference model gives the negative log likelihood

$$-\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_+ | x)}{\pi_0(y_+ | x)} - \log \frac{\pi_\theta(y_- | x)}{\pi_0(y_- | x)} \right] \right).$$

This derivation does not establish that finite offline preferences cover every relevant response, that human preferences obey the logistic model, or that a neural optimizer reaches the optimum. Those are statistical and optimization questions, not consequences of cancellation.

A.14 Group-relative estimators: what centering actually does

For one fixed prompt, draw $G \geq 2$ iid responses. Let R_i be the reward and $z_i = \nabla \log \pi(Y_i | x)$, with finite required moments, $\mathbb{E} z_i = 0$, and $g = \mathbb{E}[z_i R_i]$. Let $\bar{R} = G^{-1} \sum_j R_j$.

Proposition.

$$\mathbb{E} \left[\frac{1}{G} \sum_i z_i (R_i - \bar{R}) \right] = \frac{G-1}{G} g.$$

With the leave-one-out mean $\bar{R}_{-i} = (G-1)^{-1} \sum_{j \neq i} R_j$, the corresponding expectation is g .

Proof. Independence gives $\mathbb{E}[z_i R_j] = \mathbb{E} z_i \mathbb{E} R_j = 0$ for $j \neq i$. The self term contributes g/G to $\mathbb{E}[z_i \bar{R}]$, so the centered contribution is $(1 - 1/G)g$. Average over i . The leave-one-out mean has no self term and therefore zero score correlation. $\square$

Dividing by a random within-group standard deviation is not covered by this theorem. That denominator is correlated with the numerator and changes prompt weighting. For a Bernoulli action and reward with success probability p and logit score $z = A - p$, $g = p(1 - p)$. Enumerating all 2^G groups verifies the factor exactly; the lab does this rather than relying on a noisy simulation.

If rewards are iid Bernoulli(p), the probability of an informative mixed-reward group is

$$1 - p^G - (1 - p)^G.$$

Proof. The only constant groups are all successes and all failures, disjoint events with probabilities p^G and $(1 - p)^G$. Subtract them from one. Filtering to mixed groups replaces the prompt distribution by one proportional to this probability times its original probability, unless corrected. This is not automatically the original objective.

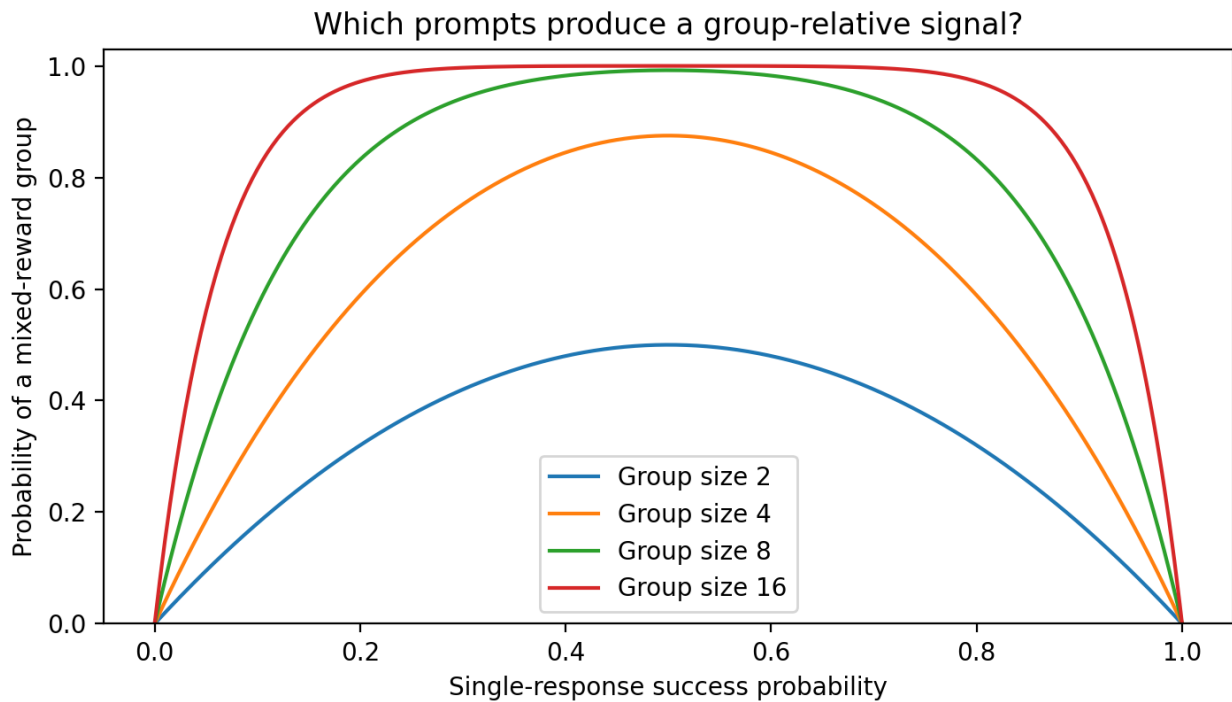


Figure 69: Exact probability of a mixed binary-reward group, derived in A.14. Curves assume independent responses with the same success probability.

A.15 Importance sampling, doubly robust evaluation, and impossibility

Consider a horizon H , target policy π_t , behavior policy μ_t , and the same environment and initial law. Require $\mu_t(a \mid s) > 0$ whenever a target-reachable action has positive target probability. Define

$\rho_{0:t} = \prod_{k=0}^t \pi_k(A_k | S_k) / \mu_k(A_k | S_k)$ and $\rho_{0:-1} = 1$.

IS identity. For an integrable function f of the history through transition t , $\mathbb{E}_\mu[\rho_{0:t}f] = \mathbb{E}_\pi f$ when the weighted absolute moment is finite.

Proof. Write the joint trajectory mass. Multiplication by the product of action ratios cancels each behavior action probability and replaces it by the target probability; initial and transition factors are unchanged. Sum over histories. For continuous outcomes this is the same Radon–Nikodym change of measure. Missing support prevents the cancellation on omitted histories. $\square$

Finite-horizon DR theorem. Let fixed (or independently trained) functions $\widehat{Q}_t$ define $\widehat{V}_t(s) = \sum_a \pi_t(a | s) \widehat{Q}_t(s, a)$ and $\widehat{V}_H = 0$. Then

$$\widehat{J}_{\text{DR}} = \widehat{V}_0(S_0) + \sum_{t=0}^{H-1} \gamma^t \rho_{0:t} \{R_{t+1} + \gamma \widehat{V}_{t+1}(S_{t+1}) - \widehat{Q}_t(S_t, A_t)\}$$

has expectation $J(\pi)$ if the ratios are exact and the required moments exist, regardless of the value-model error.

Proof. Use IS on each term. Under the target policy $\mathbb{E}_\pi \widehat{Q}_t(S_t, A_t) = \mathbb{E}_\pi \widehat{V}_t(S_t)$. All the expected value terms telescope, including cancellation of the initial term; the terminal term is zero. The remaining sum is the expected target return. $\square$

The other robustness route. If $\widehat{Q}_t = Q_t^\pi$, each bracket has conditional mean zero given the history through (S_t, A_t) . It remains mean zero after multiplication by any integrable weight measurable at that point, including misspecified action-ratio products, provided their fitting does not use the current or future residual in a way that invalidates that conditioning. Thus the expectation is $\mathbb{E} V_0^\pi(S_0) = J(\pi)$. This is a precise version of double robustness, not a promise of finite-sample accuracy when both nuisances are imperfect or estimated on the same trajectories. Cross-fitting can enforce the required training/evaluation separation. Self-normalized weights generally introduce finite-sample bias.

Nonidentifiability without support. In a one-step problem there are two actions, a_0, a_1 . The behavior always takes a_0 , whose reward is zero. In environment M_+ , a_1 gives reward $+1$; in M_- it gives reward -1 . Every possible logged dataset has the identical distribution in these environments. The target that always takes a_1 has value $+1$ or -1 . No statistic of those logs can identify which is correct: a statistic has the same distribution under identical data laws and cannot be almost surely equal to two distinct values. Structural assumptions or new observations can narrow the possibilities; estimator complexity alone cannot.

A.16 Retrace and V-trace: fixed points, not just formulas

All operators in this section are exact tabular expected operators for fixed target and behavior policies. They do not by themselves prove convergence of a neural implementation. See Munos et al. (2016) and Espeholt et al. (2018) for the algorithms and broader results.

Retrace. Index vectors by state-action pairs. Let P^π be the stochastic kernel that takes an environment transition and then chooses the next action from π . For $0 \leq \lambda \leq 1$ put $c(s, a) = \lambda \min(1, \pi(a | s) / \mu(a | s))$, on behavior support, and

$$P^c((s, a), (s', a')) = P(s' | s, a) \mu(a' | s') c(s', a').$$

Then $0 \leq P^c \leq P^\pi$ entrywise and $P^c \mathbf{1} \leq \mathbf{1}$. The expected infinite Retrace operator is

$$\mathcal{R}Q = Q + (I - \gamma P^c)^{-1} (T^\pi Q - Q).$$

The inverse is the convergent nonnegative series $\sum_{k \geq 0} \gamma^k (P^c)^k$; expansion is exactly the expected product-of-traces target. Because $T^\pi Q = r + \gamma P^\pi Q$,

$$\mathcal{R}Q - \mathcal{R}Q' = \gamma(I - \gamma P^c)^{-1}(P^\pi - P^c)(Q - Q').$$

This matrix is nonnegative. Its row-sum vector is

$$\mathbf{1} - (1 - \gamma)(I - \gamma P^c)^{-1}\mathbf{1} \leq \gamma\mathbf{1},$$

since the inverse series begins with I . Nonnegativity and this bound prove a sup-norm contraction with factor at most γ . Its fixed point satisfies $T^\pi Q - Q = 0$, hence is Q^π . This proves why truncating the *trace* in this specific construction preserves the target fixed point.

V-trace. For states, take $0 < \bar{c} \leq \bar{\rho} < \infty$, $\rho(s, a) = \min(\bar{\rho}, \pi(a \mid s)/\mu(a \mid s))$ and $c(s, a) = \min(\bar{c}, \pi/\mu)$. Assume sufficient overlap that

$$z(s) = \sum_a \mu(a \mid s) \rho(s, a) > 0 \quad \text{for every state,} \quad z_{\min} = \min_s z(s) > 0.$$

Define $D_z = \text{diag}(z)$, $r_\rho(s) = \sum_a \mu(a \mid s) \rho(s, a) r(s, a)$ and kernels $P_\rho(s, s') = \sum_a \mu(a \mid s) \rho(s, a) P(s' \mid s, a)$ and P_c analogously. Both are substochastic, because $\mu\rho = \min(\bar{\rho}\mu, \pi)$ sums to at most one. The expected operator is

$$\mathcal{V}v = v + (I - \gamma P_c)^{-1}[r_\rho + \gamma P_\rho v - D_z v].$$

Its difference matrix is $(I - \gamma P_c)^{-1}[(I - D_z) + \gamma(P_\rho - P_c)]$, entrywise nonnegative since $z \leq 1$ and $c \leq \rho$. Its row-sum vector is $\mathbf{1} - (1 - \gamma)(I - \gamma P_c)^{-1}z \leq [1 - (1 - \gamma)z_{\min}]\mathbf{1}$. Thus it contracts. A fixed point obeys $D_z v = r_\rho + \gamma P_\rho v$, the Bellman equation for

$$\pi_{\bar{\rho}}(a \mid s) = \frac{\min(\bar{\rho}\mu(a \mid s), \pi(a \mid s))}{\sum_b \min(\bar{\rho}\mu(b \mid s), \pi(b \mid s))}.$$

It is the value of this *clipped target policy*, not generally v_π . The ρ cap changes the fixed point; the c cap changes trace propagation. This is a fundamental distinction from the Retrace construction above. $\square$

A.17 Three useful counterexamples to overgeneralization

Monotone mixing is a sufficient argmax property. If $Q_{\text{tot}} = f(q_1(a_1), \dots, q_n(a_n))$ and f is coordinatewise nondecreasing, choosing a local maximizer of every q_i gives a global maximizer: increase one coordinate at a time and the value cannot decrease. This proves $\prod_i \arg \max q_i \subseteq \arg \max Q_{\text{tot}}$, not equality of the sets. If $f \equiv 0$, every joint action is globally optimal even if each q_i has a unique maximizer.

Hindsight relabeling can condition on transition noise. Consider one action, one transition, and an independent fair coin determining the achieved state $S' \in \{0, 1\}$. For a fixed goal g , reward is $\mathbf{1}\{S' = g\}$, so expected reward is $1/2$. Relabel every transition with the achieved goal $g = S'$. Every relabeled reward is now one. Dynamics are goal-independent and rewards are perfectly recomputable, yet the relabeled conditional transition law for a given goal is not the original fair coin. Therefore these two properties alone do not make naive hindsight targets unbiased in stochastic dynamics.

A trajectory Gibbs density need not be a causal policy. In an uncontrolled one-step system, a fair coin gives $S' = 0$ or 1 , with reward S' . Reweighting complete trajectories by e^R changes the

probability of $S' = 1$ to $e/(1 + e)$. No policy can change the coin: there are no actions. Thus a Gibbs distribution over entire trajectories is not automatically a realizable controlled distribution. In stochastic environments, causal maximum-entropy control regularizes action choices and retains the transition law; its backward recursion is A.13's soft Bellman equation. Maximum-entropy inverse modeling must specify whether it models unconstrained trajectories or causal decisions.

A.18 Shaping, options, and return-distribution contraction

Potential shaping. For fixed bounded Φ , replace reward by $R'_{t+1} = R_{t+1} + \gamma\Phi(S_{t+1}) - \Phi(S_t)$. Summing to n gives

$$\sum_{t=0}^{n-1} \gamma^t R'_{t+1} = \sum_{t=0}^{n-1} \gamma^t R_{t+1} - \Phi(S_0) + \gamma^n \Phi(S_n).$$

For discounted continuing problems the final term vanishes, so every policy's value is shifted by $-\Phi(s)$ and its action ordering is unchanged. For finite episodes the terminal term need not vanish. Require terminal potential zero, or a common discounted boundary term independent of policy; variable terminal times and a nonzero constant potential generally violate the latter. Finite-horizon time-dependent shaping uses $\gamma\Phi_{t+1}(S_{t+1}) - \Phi_t(S_t)$ with $\Phi_H = 0$. This is the complete boundary argument.

Options. An option has a policy, initiation set, and termination rule; let its duration K be almost surely finite and at least one. Its discounted backup is $\mathbb{E}[\sum_{j=0}^{K-1} \gamma^j R_{j+1} + \gamma^K V(S_K)]$. The difference of two such backups is bounded by $\mathbb{E}[\gamma^K] \|V - W\|_\infty \leq \gamma \|V - W\|_\infty$. Taking a maximum over available options preserves this bound by A.2's maximum inequality. Thus the option Bellman operator contracts. Zero-duration cycles or undefined termination require additional treatment.

Distributional policy evaluation. Let $Z(s, a)$ be return laws with finite p th moments, $p \geq 1$, and define $\overline{W}_p(Z, Z') = \max_{s,a} W_p(Z(s, a), Z'(s, a))$, where W_p is the infimum of $(\mathbb{E}|U - V|^p)^{1/p}$ over couplings of the two laws. The fixed-policy distributional Bellman operator draws the same (R, S', A') in both copies, then couples their next return laws within η of their infimal p th transport costs. The rewards cancel and the next returns are multiplied by γ , so

$$\overline{W}_p(\mathcal{T}^\pi Z, \mathcal{T}^\pi Z') \leq \gamma \overline{W}_p(Z, Z').$$

Formally the conditional p th cost is at most $\gamma^p [\overline{W}_p(Z, Z')^p + \eta]$; average and let $\eta \downarrow 0$. This proves the inequality without assuming a common global optimal coupling. It is a fixed-policy result. Greedy action selection can switch discontinuously when means tie, so the argument does not establish the same contraction for distributional control or for a categorical projection in an arbitrary metric.

A.19 What a mathematical result licenses

The course's mathematical prerequisites are finite-dimensional linear algebra, elementary probability (including conditional expectation, strong laws, and martingale convergence), multivariable calculus, and elementary convex optimization. They are made explicit rather than pretending to derive all of probability inside an RL course. The RL statements proved above include their sampling laws and boundary conditions. Modern neural algorithms are often specified exactly but have no general global-convergence theorem under the implementation settings discussed here. In those cases the correct teaching distinction is **exact objective and update; approximate optimization; empirical evidence**, not an invented proof.

For each research claim, write the theorem's domain alongside its practical counterpart: finite versus continuous state, known versus learned dynamics, fixed versus changing policy, exact versus

fitted critic, independent versus reused data. A rigorous explanation ends at the boundary of its assumptions and makes that boundary useful.

Notation and assumption guide

Core notation

Object	Meaning and convention
$\mathcal{S}, \mathcal{A}$	State and action sets; finite in the core proofs
S_t, A_t, R_{t+1}	State, selected action, and next observed reward
P, r	Transition kernel and conditional mean reward
π, μ or b	Target/current policy and behavior policy
ν or ρ_0	Initial-state distribution, fixed unless stated otherwise
γ	Discount, in $[0, 1)$ for continuing contraction proofs
H, T	Fixed horizon or episode length; the surrounding definition specifies which
G_t	Reward-to-go from time t , with discount restarted at t
v_π, q_π, A_π	Exact state value, action value, and advantage
$V, Q, \widehat{V}, \widehat{Q}$	Values or approximations, distinguished by the local definition
d_π	Normalized discounted state occupancy; sums to one
$x_\pi(s, a)$	Normalized discounted state-action occupancy
D	Diagonal matrix of positive stationary-state weights in linear TD
X	Feature matrix with one row per state
w, θ, ϕ	Value, policy, or auxiliary-model parameters, specified locally
α_t	Update step size; not the TV radius used locally in A.11
λ	Trace parameter; also a constraint multiplier in B.5, explicitly redefined there
δ_t	TD residual; not the trust-region radius δ
ρ_t	Action likelihood ratio; not initial distribution ρ_0
$\rho_{0:t}$	Product of ratios from time 0 through time t
β	KL/entropy coefficient; prioritized replay reuses the symbol for a correction exponent
ϵ_P	Uniform transition L_1 error in A.12; TV equals half L_1
g, h	Average-reward gain and relative bias in B.4; g denotes a gradient elsewhere

Symbols are local to a formulation. A reusable paper-reading habit is to copy its symbol definitions before comparing its equations to another paper's.

Assumption checklist

Contraction: finite spaces or an explicitly complete function space; discount below one; bounded rewards; compatible norm; exact operator. **Sampled convergence:** the correct conditional mean, controlled noise, boundedness or a proved stability argument, sufficient visitation, and valid step sizes. **Policy gradient:** differentiable common-support policy, controlled scores, fixed environment law, and a justified interchange of derivative and expectation. **Off-policy evaluation:** coverage, correct information sets, logged behavior probabilities or explicit nuisance assumptions, integrability,

and independent evaluation units. **Finite horizon:** time-indexed values and a specified terminal boundary. **Partial observation:** a sufficient information state or an explicitly approximate representation. **Safety:** the exact event or expected-cost constraint, its time horizon, and the confidence event underlying any guarantee.

Terminology

An **estimand** is the quantity the analysis targets, such as the return of a fixed policy. An **estimator** is a function of random data used to estimate it. A **surrogate** is a different, tractable objective used in optimization. A **fixed point** solves an operator equation; a sampled algorithm still needs a convergence argument to reach it. **Identifiability** asks whether the data law uniquely determines the estimand, before any finite-sample algorithm is considered. **Calibration** relates predictions to observed frequencies under a specified distribution; it is not a universal guarantee under shift. A **confidence statement** quantifies repeated-data uncertainty under stated assumptions, not the probability that an arbitrary fixed interval is correct without a model.

Six problem sets

These problems pair a derivation with a numerical or conceptual check. Each set is marked out of 20. Full solutions and a grading guide are in the separate instructor companion. Show the assumptions and boundary cases; quoting the name of a theorem is not a substitute for the requested argument.

Set 1 — Bellman reasoning and certificates

1.1 Running MDP (8 points). In A.4, evaluate the always-stay policy and one greedy improvement. Verify the improved policy’s optimality by comparing both action values in both states. Explain why the verification implies optimality among history-dependent policies, not merely among the four deterministic stationary policies.

1.2 Residual certificate (6 points). For a γ -contraction T with fixed point v_* , derive $\|v - v_*\|_\infty \leq \|Tv - v\|_\infty / (1 - \gamma)$. If $\gamma = 0.95$ and the residual is 10^{-3} , give a guaranteed value error and the greedy-policy loss bound from A.2. Distinguish these bounds from measured errors.

1.3 Undiscounted failure (6 points). Give a one-state continuing MDP in which $\gamma = 1$ yields an infinite value and a Bellman equation with no finite solution. Then explain why a finite horizon with $\gamma = 1$ is still well defined and prove its backward recursion by induction.

Set 2 — Sampling, convergence, and representation

2.1 Repeated visits (6 points). Define episode-level N_e, Y_e for every-visit MC. State sufficient integrability assumptions and prove convergence by a ratio of strong laws. Explain why assuming independent individual visits would be incorrect.

2.2 Noisy contraction (7 points). Starting from A.6’s theorem, verify every hypothesis for bounded tabular Q-learning with a uniform generative sampler and per-pair steps $n^{-0.6}$. Then give two implementation changes that invalidate the proof, identifying the exact failed hypothesis.

2.3 Shared-feature instability (7 points). Reconstruct A.7’s two-state off-policy example from its transition rules. Compute the behavior stationary distribution and the action importance weights, derive $A = -0.68$, and explain why this proves mean-update instability even though the target value is representable. Does the calculation alone prove almost-sure divergence of the sampled algorithm?

Set 3 — Gradients and rollout boundaries

3.1 Policy gradient (8 points). Derive the finite-horizon score-function gradient and remove past rewards by conditional expectation. State a sufficient condition for subtracting a random baseline. Explain the failure of changing policy parameters inside the timestep loop of an otherwise on-policy REINFORCE batch.

3.2 Two masks (6 points). A stored rollout contains two one-transition segments. The first ends at an external time-limit reset and has reward 1, current value 0.4, and actual final-observation value 2. The second is a different episode with terminal reward 100 and current value zero. For $\gamma = 0.9, \lambda = 1$, compute both advantages. Compute the first advantage if bootstrapping is wrongly disabled, and if the trace wrongly continues into the reset episode while retaining the correct first bootstrap.

3.3 Local trust region (6 points). Solve $\max_{\Delta} g^\top \Delta$ subject to $\frac{1}{2} \Delta^\top F \Delta \leq \delta$ for positive definite F . Give a singular- F example in which the problem is unbounded, and explain why the exact local solution is not a theorem about sampled PPO clipping.

Set 4 — Models, support, and evaluation

4.1 Simulation bound (7 points). Prove A.12’s policy-value bound using the matrix inverse series. Derive the $2B + \delta$ planner bound. Explain what changes if the transition error is measured in total variation rather than L_1 .

4.2 Nonidentifiability (5 points). Build two one-step environments with identical behavior logs and different target-policy values. Prove that no estimator based only on those logs can identify both values. State one additional assumption or data-collection change that repairs the example.

4.3 Doubly robust evaluation (8 points). Write the horizon- H DR estimator with time-indexed nuisance functions and zero terminal value. Prove unbiasedness first with exact ratios and arbitrary fixed value functions, then with exact target values and suitably measurable possibly incorrect weights. Explain why training nuisance functions on the same trajectory can require additional care.

Set 5 — Classical extensions and counterexamples

5.1 Confidence and adaptive counts (7 points). Derive the bounded-variable exponential bound in B.2, then explain the count-pair union bound in UCB1. Why is it invalid simply to substitute a random adaptive count into a fixed-sample tail bound? Interpret the gap-dependent regret result near $\Delta = 0$.

5.2 Constraint and gauge (7 points). Solve the one-state constrained example in B.5 at $\gamma = 0.9, C = 3$. Derive its occupancy flow. Separately, show that an average-reward bias function is defined only up to an additive constant and explain why the normalization $d^\top h = 0$ resolves this ambiguity in an irreducible chain.

5.3 Repair the claim (6 points). Give a counterexample to each statement: (i) coordinatewise monotone mixing makes the global and local-product argmax sets equal; (ii) goal-independent dynamics and recomputable rewards make hindsight relabeling unbiased; (iii) reweighting complete stochastic trajectories by exponentiated reward always yields a realizable causal policy.

Set 6 — Preference optimization and a research contract

6.1 KL and preferences (7 points). Prove the KL variational identity in A.13, derive the optimal distribution, and show how a prompt-only partition function cancels in the DPO logit. State two empirical assumptions that the algebra does not prove.

6.2 Group dependence (7 points). For iid responses with score z_i and reward R_i , prove the $(G - 1)/G$ factor for self-centered rewards and the unbiasedness of the leave-one-out version. For binary success probability $p = 0.1, G = 4$, compute the mixed-group probability. Explain why dividing by a random standard deviation is a further change.

6.3 Research contract (6 points). Choose one Part XI project. Specify the problem class, data law, estimand, budget, closest baseline, falsifier, and novelty check. Clearly separate what the cited paper reports from your proposed extension. No numerical experiment needs to be invented to complete the contract.

Eight laboratories

The supplied code is a set of small reproducible experiments, not a reproduction of Atari, robotics, or LLM benchmarks. NumPy is sufficient for the core checks; Matplotlib is used for figures. `rl_core.py` contains explicit implementations and `test_core.py` contains 24 automated tests. `Labs.ipynb` walks through the eight laboratories with executed outputs. From the package root, run:

```
python -m pip install -r labs/requirements.txt
python -m unittest discover -s labs -v
python labs/run_labs.py
python labs/make_figures.py
```

The test log and numerical report in `audit/` are actual outputs from this revision. A passing numerical test checks a particular case or identity; it does not prove a universal mathematical claim. Seeded stochastic demonstrations are labeled as such. Nothing in the labs calls an external model or service.

Lab 1 — Exact control and certificates

Evaluate the running MDP under the uniform policy and the optimal policy. Run value iteration, calculate the Bellman residual, and compare with the known exact value. Expected checks: uniform-policy value (7.25, 7.75); optimal value (18, 20); normalized occupancy (0.55, 0.45) for a uniform policy starting in state 0. Plot error and its geometric upper bound. In this specially simple model the displayed error can coincide with the bound; this does not imply generic tightness.

Submission: a hand calculation, the computed values, and one example showing why a residual certificate is not the same as measured error. Change the discount to 0.99 and predict the slower contraction before running it.

Lab 2 — Sampling the tabular fixed point

Run Q-learning with uniformly sampled state-action model queries, not a single continuing trajectory. The deterministic environment has expected optimal Q-table [(17.2, 18), (20, 16.2)]. The visit-based schedule is $n^{-0.6}$. Check the Q-table against the analytic target, then remove one pair from the sampler and explain what cannot be learned. The latter is a student modification, not a supplied successful experiment.

Submission: identify the behavior/query distribution, step-size hypotheses, and distinction between finite-run error and almost-sure convergence.

Lab 3 — Geometry and a counterexample

Solve a projected Bellman equation explicitly and verify the D -norm error bound. Compute the off-policy mean coefficient $A = -0.68$ from A.7 and reproduce the exact expansive mean recursion. Separately compare squared expected Bellman-residual gradient and expected squared-sample-residual gradient in A.8's two-outcome example: the coefficients are 0.01 and 0.82.

Submission: explain why feature expressivity, mean stability, and sampled convergence are three different questions. Do not label the exact mean-recursion figure as a sampled training run.

Lab 4 — Gradients and GAE boundaries

Compare the exact softmax policy gradient to central finite differences of the matrix-evaluated objective. At zero logits, the selected logit derivatives in A.4 are $(-0.75625, 2.75625)$. Test a true terminal

transition, an external truncation/reset, and a nonterminal rollout endpoint. Verify the finite-GAE error identity. The two-segment example yields advantages $(2.4, 100)$; the second episode must not leak into the first.

Submission: derive the first advantage by hand and state which observation the bootstrap consumes. Replace the two masks with one in a separate scratch implementation and show the failed test.

Lab 5 — Group-relative estimators

Enumerate all binary groups for a Bernoulli policy. This gives exact expectations without Monte Carlo uncertainty. Verify raw gradient $p(1-p)$, self-centered gradient $(G-1)p(1-p)/G$, and leave-one-out gradient $p(1-p)$. Measure the different expectation of within-group standardization. Plot mixed-group probability as a function of p .

Submission: distinguish multiplicative finite-group bias from prompt reweighting and random-denominator effects. Explain why a constant factor may be absorbed into a learning rate in one fixed-group setting but does not make every normalized variant unbiased.

Lab 6 — Doubly robust evaluation

Enumerate horizon-three trajectories in the running MDP. With exact ratios and deliberately incorrect fixed Q-functions, expected DR must equal the exact target return. Then give DR the exact Q-functions but incorrect action-ratio denominators; its expectation remains correct by conditional zero residuals. An unsupported target action must trigger the input validation rather than be silently assigned ratio zero.

Submission: show both cancellation arguments and one setting in which neither applies. Explain why an exact enumerated expectation does not describe finite-dataset variance.

Lab 7 — V-trace’s clipped policy

Construct the expected affine V-trace operator, solve its fixed-point linear system, and compare with exact evaluation of the normalized clipped policy. Verify nonnegative operator coefficients and row sums below one. Compare its fixed point to the original target value; they need not match.

Submission: identify the separate effects of the immediate correction cap and the trace cap. Explain why this expected-tabular result does not prove neural convergence with policy lag.

Lab 8 — UCB and finite-budget accounting

Run a seeded two-arm Bernoulli UCB experiment. Report arm counts and realized pseudo-regret. Verify the bookkeeping identity $\sum_i \Delta_i N_i(n)$ exactly. Repeat across independent seeds as a student extension, report uncertainty at the run level, and compare to the conservative expectation bound from B.2.

Submission: distinguish the expected theorem bound from the realized pseudo-regret of a single run. Explain why reusing the same outcomes while selecting a preferred seed is not evidence of improved exploration.

Verified reference anchors and evidence notes

The following primary records anchor the new proofs and the revised frontier. Access and version checks were performed on 4 September 2026. Original chapters retain additional historical reading suggestions; those are not represented here as a comprehensive bibliographic or publication-status audit. Proofs in the mathematical spine are pedagogical derivations for their explicitly stated settings; citing a general source does not enlarge those settings.

Foundations and methods

- [C01] John N. Tsitsiklis (1994). *Asynchronous Stochastic Approximation and Q-Learning*. Machine Learning 16, 185–202. Author copy: <https://www.mit.edu/~jnt/Papers/J052-94-jnt-q.pdf>. Author errata: <https://www.mit.edu/~jnt/Papers/J052-94-q-errata.pdf>. The course supplies its own discounted bounded-noise proof; it does not use the erratum-affected stochastic-shortest-path argument.
- [C02] John N. Tsitsiklis and Benjamin Van Roy (1997). *An Analysis of Temporal-Difference Learning with Function Approximation*. IEEE Transactions on Automatic Control 42, 674–690. Related 1996 conference version in official proceedings: https://proceedings.neurips.cc/paper_files/paper/1996/hash/e00406144c1e7e35240afed70f34166a-Abstract.html. The course proves projection and iid stationary-sampling convergence, and identifies the Markov-noise extension separately.
- [C03] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz (2015). *Trust Region Policy Optimization*. ICML, PMLR 37, 1889–1897. <https://proceedings.mlr.press/v37/schulman15.html>.
- [C04] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel (2015 preprint; 2016 conference work). *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. <https://arxiv.org/abs/1506.02438>. The finite-segment boundary identity is derived explicitly in A.10.
- [C05] Nan Jiang and Lihong Li (2016). *Doubly Robust Off-policy Value Evaluation for Reinforcement Learning*. ICML, PMLR 48, 652–661. <https://proceedings.mlr.press/v48/jiang16.html>.
- [C06] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer (2002). *Finite-time Analysis of the Multi-armed Bandit Problem*. Machine Learning 47, 235–256. Author copy: <https://cesa-bianchi.di.unimi.it/Pubblicazioni/ml-02.pdf>. The UCB1 argument is provided in B.2, including the adaptive-count union bound.
- [C07] Rémi Munos, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare (2016). *Safe and Efficient Off-Policy Reinforcement Learning*. <https://arxiv.org/abs/1606.02647>. “Safe” in this title concerns off-policy learning properties, not physical safety constraints.
- [C08] Lasse Espeholt et al. (2018). *IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures*. <https://arxiv.org/abs/1802.01561>. A.16 makes the clipped target policy explicit.
- [C09] Marc G. Bellemare, Will Dabney, and Rémi Munos (2017). *A Distributional Perspective on Reinforcement Learning*. <https://arxiv.org/abs/1707.06887>.
- [C10] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. <https://arxiv.org/abs/2305.18290>.
- [C11] Zhihong Shao et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in*

Open Language Models. <https://arxiv.org/abs/2402.03300>. Primary GRPO anchor; implementation variants are not all the same objective.

[C12] Qiying Yu et al. (2025). *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*. <https://arxiv.org/abs/2503.14476>.

Selected 2025–2026 developments

[R01] Junhyuk Oh et al. (2025). *Discovering state-of-the-art reinforcement learning algorithms*. Nature 648, 312–319; published 22 October 2025. Publisher record: <https://www.nature.com/articles/s41586-025-09761-x>. Evidence used: publisher abstract and metadata; published research, not a course reproduction.

[R02] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap (2025). *Mastering diverse control tasks through world models*. Nature 640, 647–653; published 2 April 2025. Publisher: <https://www.nature.com/articles/s41586-025-08744-2>. Author project: <https://danijar.com/project/dreamerv3/>. The author page was accessible; direct publisher access was inconsistent in this review.

[R03] Nicklas Hansen, Hao Su, and Xiaolong Wang. *TD-MPC2: Scalable, Robust World Models for Continuous Control*. Original preprint October 2023. <https://arxiv.org/abs/2310.16828>; author project <https://www.tdmcp2.com/>. Included as a continuing control/planning anchor, not described as a newly released 2026 method.

[R04] Perry Dong, Kuo-Han Hung, Alexander Swerdlow, Dorsa Sadigh, and Chelsea Finn (2026). *TQL: Scaling Q-Functions with Transformers by Preventing Attention Collapse*. arXiv:2602.01439, v1, 1 February 2026. <https://arxiv.org/abs/2602.01439>. Public author record; venue status not independently verified. The original OpenReview route encountered a browser challenge.

[R05] Guy Zamir, Matthew Zurek, and Yudong Chen (2026). *Optimal Variance-Dependent Regret Bounds for Infinite-Horizon MDPs*. arXiv:2603.23926, v1, 25 March 2026. <https://arxiv.org/abs/2603.23926>. Author abstract/metadata reviewed; detailed theorem rates and all proof assumptions were not independently audited and are not restated as course theorems.

[R06] Manuel Wendl, Yarden As, Manish Prajapat, Anton Pollak, Stelian Coros, and Andreas Krause (2026). *Safe Exploration via Policy Priors*. arXiv:2601.19612; v1 27 January, v4 13 August 2026. <https://arxiv.org/abs/2601.19612v4>. Public author record; safety claims are reported with a scope warning, not certified for arbitrary controllers.

[R07] Jinwen Wang, Youfang Lin, Xiaobo Hu, Qian Xu, Shuo Wang, Zhuo Chen, and Kai Lv (2026). *Task-Relevant Representation Decoupling for Visual Reinforcement Learning Generalization*. arXiv:2607.00796, v1, 1 July 2026. <https://arxiv.org/abs/2607.00796>. The arXiv record lists ACM TOMM 2026 and DOI 10.1145/3820367; that journal reference was author-record metadata, not a separate publisher verification in this review.

[R08] Ziheng Wei, Annie Qu, and Rui Miao (2026). *Off-Policy Evaluation for Missingness-Aware Policies in MDPs with Rewards Missing Not at Random*. arXiv:2606.20206, v1, 18 June 2026. <https://arxiv.org/abs/2606.20206>. “Accepted at ICML 2026” appears in author comments; no independent venue confirmation is claimed here.

[R09] Daniel Ritter, Owen Oertell, Bradley Guo, Jonathan Chang, Kianté Brantley, and Wen Sun (2026). *LLMs Can Learn to Reason Via Off-Policy RL*. arXiv:2602.19362; v1 22 February, v2 27 February 2026. <https://arxiv.org/abs/2602.19362v2>. Public author record; empirical conclusions are not course replications.

[R10] Anay Kulkarni, Chia-En Lu, Dheeraj Mekala, Jayanth Srinivasa, Gaowen Liu, and Jingbo Shang (2026). *TIER: Trajectory-Invariant Execution Rewards for Multi-Step Tool Composition*. arXiv:2605.16790, v1, 16 May 2026. <https://arxiv.org/abs/2605.16790>. Author comment: preprint submitted to NeurIPS 2026. Submission is not acceptance.

[R11] Zhijian Zhou et al. (2026). *Start Classifying: Categorical Critics for LLM Reinforcement Learning*. arXiv:2608.02181, v1, 3 August 2026. <https://arxiv.org/abs/2608.02181>. Author comments report acceptance at COLM 2026; no independent venue confirmation is claimed. The abstract distinguishes categorical critic training from the unchanged scalar PPO actor.

Reading practice

Before presenting a paper, record the exact version, problem formulation, theorem assumptions, dataset and code versions, and experimental budget. A later version can correct or strengthen an earlier one. Do not turn the word “optimal” in a title into an unqualified assertion about every problem class. Published results, author claims, independently reproduced results, and proposed projects should remain distinguishable throughout a seminar.